

CLEYROP · HEMERA V2 · 2 OCTOBRE 2026

Dossier des services et des fonctionnalités

Existant, cible Rust sans JVM, intégration Icegres/Eidos, contrats, migration et recette.

22

services du monorepo
examinés

216

fiches de capacités, avec
cible et acceptation

56

dossiers recensés,
services et support

744

déclarations d'interfaces
indexées

Le recensement porte sur les arbres locaux indiqués dans les sources. Une capacité décrite dans le code n'est pas une preuve de déploiement ou de fonctionnement. Les propositions cible et leurs critères d'acceptation ne sont pas annoncés comme livrés. Les déclarations d'interfaces comprennent des préfixes et registrations partielles ; elles ne forment pas un inventaire de routes actives.

Le recensement porte sur les arbres locaux indiqués dans les sources. Une capacité décrite dans le code n'est pas une preuve de déploiement ou de fonctionnement. Les propositions cible et leurs critères d'acceptation ne sont pas annoncés comme livrés. Les déclarations d'interfaces comprennent des préfixes et registrations partielles ; elles ne forment pas un inventaire de routes actives.

Sommaire du dossier

1. [iam](#) : Identités métier, autorisations et administration
2. [keycloak](#) : Authentification OIDC et configuration du realm
3. [whitelister](#) : Règles réseau entrantes et découverte des IP de sortie
4. [data-collect](#) : Connexions, dépôts et collectes de datasets
5. [files](#) : Fichiers, dossiers et traitements documentaires
6. [data-model](#) : Catalogue, contrats et cycle de vie des datasets
7. [data-dev](#) : Développement et versionnement des flows
8. [process-data](#) : Exécution des flows, planification et ressources de calcul
9. [data-serve](#) : Consultation, SQL, exposition et qualité des données
10. [app-management](#) : Applications, images et espaces de développement
11. [devspace-agent](#) : Agent sortant de tunnel des devspaces
12. [ai-gen-proxy](#) : Proxy de modèles et contrôle des fournisseurs
13. [ai-gen-assistant](#) : Assistants, contextes et corpus RAG
14. [mcp-server-dataset](#) : Outils MCP d'analyse des datasets
15. [mcp-server-corpus](#) : Outils MCP de recherche de corpus
16. [mcp-server-internet](#) : Outils MCP de recherche internet
17. [datasphere-cleyrop](#) : Labels Datasphere, clearance et exports chiffrés
18. [datasphere-front](#) : Interface de composition des labels SPIF
19. [cleyrop-front-end](#) : Interface de conception, catalogue et administration
20. [business-front-end](#) : Interface métier de conversations, agents et applications
21. [notification-center](#) : Notifications, abonnements et diffusion temps réel
22. [migration-orchestrator](#) : Format et validation des migrations transverses
23. [Périmètre fonctionnel et règles de lecture](#)
24. [Les 34 composants de support](#) : bibliothèques, charts et images
25. [Icegres](#) : bibliothèque de calcul, protocoles et writer de confiance
26. [Eidos](#) : sémantique gouvernée et actions préparées
27. [Architecture cible](#) : domaines Rust, contrats et états
28. [Exploitation, restauration et retrait progressif des JVM](#)
29. [Parcours utilisateurs et recette de compatibilité](#)
30. [Comparaison avec iceberg-data-platform et intégration dans Cleyrop](#)

ÉTUDE DE DOMAINE

Périmètre fonctionnel et règles de lecture

Ce dossier descend au niveau des fonctionnalités et des contrats de chaque service. Il décrit les règles présentes dans le code, les données possédées, les appels et événements, le comportement cible et les cas qui devront autoriser la migration. Une fiche de capacité couvre un comportement cohérent ; elle ne correspond pas nécessairement à une seule route HTTP ou à un seul ticket.

Un périmètre vérifiable dans les dépôts locaux

- Les 22 dossiers de services du monorepo forment la base de couverture obligatoire. La lecture inclut les contrôleurs ou routeurs, DTO et modèles, handlers métier, persistance, événements, configuration de runtime et charts pertinents. Les interfaces, serveurs MCP et petits services d'exploitation ont leurs propres fiches. Le recensement inclut aussi les 12 packages, 12 charts et 10 dossiers sous hub et images. Les workflows de CI, release, installation et migration sont traités dans les chapitres d'exploitation.
- Les sources d'Icgres et d'Eidos sont lues dans leurs arbres locaux, avec leurs modifications utilisateur. Le HEAD Git ne suffit donc pas à identifier tout le contenu observé. L'index des preuves conserve les chemins, lignes et empreintes des fichiers référencés. Aucun de ces deux projets n'est modifié par la rédaction du dossier.
- La comparaison avec iceberg-data-platform porte sur le clone épinglé à 4506d8a82833b3cd8edd2d613a2285352521b3. Il s'agit d'un laboratoire de référence à comparer aux fonctions de Cleyrop. Sa composition ne devient pas automatiquement une architecture de production Cleyrop.
- L'index automatisé d'interfaces conserve les déclarations rencontrées dans le code. Les préfixes de route, réécritures de gateway, flags de déploiement et registrations dynamiques peuvent changer leur exposition effective. Les fiches manuelles décrivent la sémantique des appels. La mise en service réelle d'une installation cliente et son état de données ne sont pas observés par cette revue de sources.

Références de l'analyse

[monorepo](#) · [ci/components.json:1](#) Racines de composants, dépendances et adaptateurs de build déclarés.

[monorepo](#) · [AGENTS.md:1](#) Organisation du monorepo et conventions de services, packages, charts et exploitation.

Les exigences qui encadrent toutes les fiches

- La cible finale exclut les JVM, même lorsqu'elles sont cachées derrière un protocole client Python, un writer tiers, un notebook, un outil JDBC, un opérateur ou une image de maintenance. Spark, Trino, Keycloak, Kafka/Strimzi et le writer Java OLake doivent donc avoir une trajectoire de retrait explicite. La présence temporaire d'une JVM dans un banc de comparaison ne qualifie pas son remplacement.
- Les cores métier des services cible sont écrits en Rust. Les interfaces restent en TypeScript lorsque leur travail est la présentation et l'interaction. Le code utilisateur Python et SQL reste pris en charge dans des exécutants isolés. Des composants tiers Go ou C++ peuvent être encapsulés, à condition de ne pas posséder l'autorité de publication, les droits métier ou le checkpoint de source.
- Les domaines cible décrivent des responsabilités et des frontières de confiance. Ils ne prescrivent pas tous un déploiement Kubernetes distinct dès le premier lot. Une implémentation peut regrouper des modules de contrôle, tout en séparant les privilèges du writer, du publisher et des exécutants. Les interfaces et propriétaires d'état doivent rester explicites pour permettre une séparation ultérieure.
- La compatibilité se mesure par comportement. Traduire une API Kotlin en Rust sans conserver son contrat de nommage, ses permissions, ses transitions ou ses erreurs ne ferme pas la migration. Inversement, conserver un comportement dangereux observé dans le code n'est pas une obligation : le dossier l'identifie, propose la correction et précise le scénario de non-régression du parcours utilisateur.

Exigence	Conséquence pour une fiche de service
Destination autoritaire	Un statut SQL, un cache ou un state file ne valide pas une ingestion. L'édition publiée porte le checkpoint et la provenance.
Publication avant ack	Le service retrouve la publication approuvée couvrant la plage avant d'acquitter la source.
WAP sans exception	Toute sortie gouvernée passe par un candidat privé et ses audits. Un veto n'est pas contourné par un bouton d'administration.
Journal rejouable	Une entrée éphémère doit être capturée durablement avant de prétendre offrir reprise et audit.
Outils tiers encapsulés	Un adaptateur produit des fichiers, des plans ou une préparation ; il ne publie pas main de façon autonome.
Runs bornés	Les workers sont remplaçables. Un ancien worker ne conserve pas le droit de publier après perte d'autorité.

Références de l'analyse

study · [HANDOVER.md:94](#) Invariants du programme et distinction entre données publiées et autres journaux durables.

Qui possède l'état et comment le reconstruire

- Une refonte complète doit traiter séparément données publiées, commandes, identités, configurations et effets externes. Le principe d'édition autoritaire ne permet pas d'effacer les bases de contrôle en supposant qu'Iceberg reconstruira tous les choix humains et toutes les opérations déjà effectuées.
- Les objets audités doivent rester immuables face au worker. Le droit natif Lakekeeper modify comprend les commits et la suppression de table dans le modèle inspecté. Le worker générique ne reçoit donc pas un tel credential ; un writer de confiance médie déjà les commits de staging. Le publisher vérifie ensuite la preuve et les préconditions de publication. Cette séparation applicative reste à implémenter et qualifier.
- Une preuve d'audit est produite après création du candidat. Un déplacement de référence Iceberg ne permet pas de modifier rétroactivement le summary audité. Le lien de preuve doit être immuable, récupérable et retenu avec la publication. Une recherche d'operation_id parmi tous les snapshots ne prouve pas qu'une édition a été approuvée.

État	Autorité cible	Projection ou reprise
Données, checkpoint, origine	Snapshot de l'édition approuvée et son summary origin.*	Caches Catalog/Flows reconstruits depuis les publications autorisées.
Sorties atomiques d'un groupe	Édition Iceberg de release, membres et checkpoint commun	Les lecteurs résolvent la release une fois puis épinglent ses membres.
Draft de flow ou modèle	Dépôt versionné ou stockage de contrôle durable	Ne pas le déduire du dernier résultat publié ; sauvegarder les brouillons et leur historique.
Commandes et annulations	Journal de contrôle durable	Reconstruire les files et les tentatives sans réexécuter un effet déjà confirmé.
Identités, clés et politiques	IAM, IdP et stockage de secrets avec sauvegardes	Réconcilier OpenFGA et gateways ; bloquer les accès si la restauration des politiques est incohérente.

État	Autorité cible	Projection ou reprise
Déploiements d'applications	Spécification désirée et journal de contrôle	Kubernetes fournit l'état observé ; une édition de modèle ne suffit pas à reconstruire les choix de routage.
Exports et notifications	Intentions, clés d'idempotence et reçus d'effets	Une reconstruction de cache ne doit pas réenvoyer une notification ou une livraison.
Index de fichiers, vecteurs, features	Édition Iceberg de données ou de manifeste	Index de recherche, collections vectorielles et stores online restent des projections versionnées.
Expiration et purge physique	Plan approuvé, racines de rétention et journal d'application	Une branche ou un rollback ne restaure pas un objet physiquement supprimé.

Références de l'analyse

[study · docs/hemera-v2/08-plateforme-rust-sans-jvm.html:81](#) Contrat writer/publisher, preuve d'audit et registre de publications.

[study · docs/hemera-v2/09-decision-el4-wap.html:25](#) Préparation, publication conditionnelle, reprise et acquittement.

Lire les résultats sans transformer une observation en garantie

- Le préflight de cette rédaction a réinstallé le prototype dans son environnement isolé, exécuté `demo.py` puis `python -m pytest -q`. La démo réussit et les dix tests passent en 0,88 seconde. Le code produit n'a pas été modifié pour rédiger les fiches.
- Le banc historique contient 33 entrées qui se recoupent. Son bilan brut est 18 passed, 5 failed et 10 not_executed. Il ne s'agit ni de 33 tests indépendants ni d'un pourcentage d'acceptation. I4 prouve un aller-retour de summary sur le même handle, sans reprise de processus. I8 peut être passed tout en rapportant une divergence SQL, car il n'affirme pas de parité. Le dossier conserve ces limites.
- Les critères d'acceptation de chaque fiche sont un travail à exécuter. Leur rédaction ne ferme aucun gate. Les soupçons issus d'une branche conditionnelle du code restent des hypothèses tant que le scénario correspondant n'a pas été reproduit. La présence d'un test dans un dépôt ne signifie pas que ce test a été lancé pendant cette revue.
- La couverture complète des 22 services signifie qu'aucun dossier de services recensé n'est omis. Elle ne vaut pas certification exhaustive de tous les chemins d'exécution, de toutes les configurations possibles ou de tous les déploiements clients. La matrice, l'index des interfaces et les preuves permettent de retrouver les limites du périmètre et de préparer la recette réelle.

Statut	Ce que le lecteur peut en déduire
Observé dans le code	Le chemin et sa condition sont localisés. Son comportement en déploiement reste à vérifier lorsque cela compte pour la migration.
Reproduit dans le banc historique	Le résultat est établi pour l'environnement, les versions et les données synthétiques archivés.
Cible proposée	Le contrat définit ce qu'il faut implémenter ou sélectionner ; il n'annonce pas une fonction déjà disponible.
Acceptation à exécuter	Le scénario et le résultat attendu servent à la recette et au découpage des tickets.
Décision ouverte	Le mécanisme ou le candidat n'est pas qualifié. La migration ne doit pas en dépendre implicitement.

Références de l'analyse

[study](#) · [docs/hemera-v2/10-preuves-et-campagnes.html](#):20 Lecture du registre par neuf familles de contrats et limites des sondes.

Correspondance des responsabilités

Service actuel	Capacités détaillées	Domaines cible proposés
iam	8	IAM, Audit, Catalog, Query, Files, MCP, Applications
keycloak	6	IAM, Notifications, Frontends, Audit
whitelister	6	Egress, IAM, Audit, Maintenance
data-collect	27	Connectors, Datasets, Flows, Writer, Publisher, Files, IAM, Audit, Egress, Maintenance, Notifications, Embeddings
files	8	Files, Writer, Publisher, Audit, Maintenance, IAM, Catalog, Semantics, Embeddings
data-model	21	Catalog, Datasets, IAM, Flows, Connectors, Writer, Publisher, Audit, Maintenance, Notifications, Egress, Migration
data-dev	23	Flows, IAM, Datasets, Connectors, Writer, Publisher, Query, Runtimes, Applications, Notifications, Audit, Migration, Maintenance, Frontends
process-data	22	Runtimes, Flows, IAM, Writer, Publisher, Query, Applications, Notifications, Audit, Migration, Maintenance, Egress
data-serve	18	Query, Catalog, Datasets, Audit, Egress, IAM, Flows, Applications, MCP, Maintenance, Migration
app-management	10	Applications, Runtimes, IAM, Egress, Audit, Migration, Maintenance
devspace-agent	3	Runtimes, Applications, IAM, Audit
ai-gen-proxy	5	AI, Egress, IAM, Audit, Embeddings
ai-gen-assistant	6	AI, Embeddings, MCP, Flows, Writer, Publisher, Files, IAM, Audit, Runtimes
mcp-server-dataset	5	MCP, Query, Catalog, IAM, AI, Audit
mcp-server-corpus	4	MCP, AI, Embeddings, IAM, Audit
mcp-server-internet	4	MCP, Egress, AI, Audit
datasphere-cleyrop	4	IAM, Audit, Files, Query, Egress, Flows
datasphere-front	4	Frontends, IAM, Catalog, Audit

Service actuel	Capacités détaillées	Domaines cible proposés
cleyrop-front-end	8	Frontends, IAM, Catalog, Datasets, Flows, Query, Connectors, Runtimes, Applications, AI, Notifications
business-front-end	6	Frontends, IAM, AI, Embeddings, MCP, Applications, Files, Query
notification-center	13	Notifications, IAM, Audit, Egress, Flows, Datasets, Runtimes, Frontends, Migration
migration-orchestrator	5	Migration, Audit, Maintenance, IAM

SERVICE ACTUEL · IAM

Identités métier, autorisations et administration

8 capacités analysées

services/iam · Python, FastAPI et SQLAlchemy asynchrone ; PostgreSQL, OpenFGA, APISIX, Keycloak et Kafka dans les intégrations actuelles.

Gérer les personnes, groupes, projets, comptes techniques et décisions d'accès, puis projeter ces droits dans OpenFGA et APISIX. Les utilisateurs disposent de leur profil, sessions et partages ; les administrateurs pilotent les identités et la sécurité.

Lecture de code

Pas de qualification runtime nouvelle

Domaines cible : IAM, Audit, Catalog, Query, Files, MCP, Applications.

Périmètre de lecture

- AGENTS.md du composant ; rapport identité-gouvernance existant.
- Routes utilisateurs, organisations, équipes, membres, projets, partages, brokers, sessions, tokens et administration.
- Handlers de partage et transfert, services de tokens et comptes techniques, délégation, APISIX et outbox. Lecture ciblée, pas audit intégral de chaque branche.

Objets et cycle de vie

- User garde identité métier, sujet de connexion, état et statut administrateur ; le sujet JWT ne doit pas être confondu avec la clé interne.
- Organisations, équipes, memberships, projets et espaces personnels portent la hiérarchie et les rôles.
- ServiceAccount et ExpositionApiToken décrivent des identités et credentials d'intégration, pas des éditions de données.
- Routes APISIX, audits, état break-glass et outbox ont une autorité PostgreSQL ; les tuples sont des projections et relations de décision.

Dépendances du service

- Keycloak assure actuellement connexion, clients, sessions et fédérations.
- OpenFGA évalue les relations ; APISIX applique le routage et les contrôles gateway.
- Kafka distribue les faits aux miroirs ; Kubernetes reçoit certains secrets et objets d'intégration.

Fonctionnalités et compatibilité à préserver

IAM-01 Cible non qualifiée

Administrer les personnes et organiser leur départ

COMPORTEMENT OBSERVÉ

Les routes couvrent profil courant, langue, annuaire, création, mise à jour, désactivation et suppression. Le drapeau local `is_platform_admin` est l'autorité métier pour les administrateurs de plateforme ; `set_platform_admin` écrit aussi projection de droit, audit et événement. Retirer le dernier administrateur actif est refusé. L'offboarding désigne des héritiers par famille de ressources puis désactive ou supprime la personne, avec un événement `iam.user.offboarded` ordonné avant le fait utilisateur courant dans la même transaction. Une simple suppression d'utilisateur n'a pas ces sémantiques de réaffectation.

UTILISATEURS ET RÔLES

Utilisateur pour son profil ; administrateur pour le cycle de vie et le départ.

INTERFACES DANS LE CODE

GET/PUT `/api/v1/users/me` ; PUT `/api/v1/users/me/lang` ; CRUD `/api/v1/users` ; POST `/api/v1/admin/users/{user_id}/offboard` ; `UserService.set_platform_admin`.

DONNÉES ET ÉTATS

User, statut actif/supprimé, identifiant métier et sujet Keycloak, drapeau administrateur, audit et événements outbox.

DÉPENDANCES ET ÉCHANGES

Keycloak pour identité de connexion ; services propriétaires pour réaffectation ; OpenFGA et Kafka pour projections.

COMPORTEMENT CIBLE PROPOSÉ

IAM Rust conserve une identité métier stable et un mapping explicite `issuer/sub`. Le départ devient une commande idempotente avec héritiers validés et suivi des accusés des services propriétaires. Le dernier administrateur reste protégé sous concurrence.

MIGRATION DE CETTE CAPACITÉ

Importer IDs, sujets, statuts et correspondances sans fusion par seul courriel. Porter les codes d'erreur et publier les mêmes faits via un transport sans JVM. Faire converger les réaffectations avant retrait des anciens consommateurs.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Deux retraits concurrents ne peuvent supprimer le dernier administrateur. Rejouer une commande d'offboarding ne transfère pas deux fois les ressources et conserve le même événement métier.

Références de l'analyse

[monorepo · services/iam/src/iam_service/services/user_service.py:3587](#) Écriture centralisée du statut administrateur et garde du dernier actif.

[monorepo · services/iam/src/iam_service/api/v1/admin/admin_offboarding.py:75](#) Contrat de départ avec réaffectations et ordre des événements.

[monorepo · services/iam/src/iam_service/api/v1/users.py:322](#) Édition de langue du profil courant.

IAM-02 Cible non qualifiée

Structurer organisations, équipes, projets et memberships

COMPOTEMENT OBSERVÉ

Les API gèrent organisations, équipes et appartenances avec rôles, ainsi que projets, membres et espaces personnels. Les projets gardent un ID interne et des alias hérités. Le transfert de propriétaire prend le verrou du projet, modifie memberships, audit et outbox dans la transaction, puis utilise une écriture OpenFGA contrôlée sur les alias. Le propriétaire sortant devient manager par défaut ; le départ peut retirer son membership. PostgreSQL et OpenFGA n'ont pas de transaction distribuée commune.

UTILISATEURS ET RÔLES

Propriétaires, responsables de projets et équipes, administrateurs.

INTERFACES DANS LE CODE

CRUD `/api/v1/organizations`, `/api/v1/teams`, `/api/v1/projects` ; `/api/v1/memberships` ;
`ProjectService.transfer_owner` ; `OrganizationService.transfer_ownership`.

DONNÉES ET ÉTATS

Organization, Team, Membership, Project et ProjectMember ; correspondances `legacy_id` ; relations OpenFGA.

DÉPENDANCES ET ÉCHANGES

OpenFGA, Keycloak pour les sujets, miroirs utilisateurs/projets des autres services.

COMPOTEMENT CIBLE PROPOSÉ

IAM Rust porte les invariants de hiérarchie, les espaces personnels et transferts. Une intention durable identifie les mutations externes et permet de réconcilier les tuples sans transformer un échec de lecture en graphe vide.

MIGRATION DE CETTE CAPACITÉ

Importer la hiérarchie et tous les alias avant de changer le routage. Comparer les relations effectives et les listings pour des profils réels ; garder un seul auteur des memberships par cohorte.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Deux transferts concurrents se sérialisent. Une panne du graphe ne laisse pas un ancien propriétaire rétabli par le prochain reconciler. L'espace personnel ne devient pas partageable par accident.

Références de l'analyse

[monorepo · services/iam/src/iam_service/services/project_service.py:1361](#) Verrouillage, propriétaire sortant et limite transactionnelle.

[monorepo · services/iam/src/iam_service/services/organization_service.py:633](#) Transfert au niveau organisation.

[monorepo · services/iam/src/iam_service/api/v1/memberships.py:166](#) Contrat des rôles de membership.

IAM-03 **Cible non qualifiée**

Décider les droits de lecture et d'action à partir de faits contrôlés

COMPOTEMENT OBSERVÉ

Les brokers internes authentifient le client appelant avec JWT vérifié, émetteur, audience configurée et azp attendu. Ils résolvent un utilisateur actif puis demandent une décision OpenFGA avec cohérence renforcée. Le broker dataset accepte de 1 à 100 contrôles et au plus 12 projets liés par contrôle. Il distingue consultation, téléchargement, accès projet et token. Un compte technique ne télécharge jamais un dataset RESTRICTED ; un rôle AGENT stocké ne contourne pas cette règle. Une décision indisponible provoque 503, sans tableau partiellement autorisé.

UTILISATEURS ET RÔLES

Clients de service de confiance agissant pour une personne ou un compte technique.

INTERFACES DANS LE CODE

DatasetAuthorizationRequest ; POST /data-serve/authorize dans le routeur dataset_authorization ; decision_broker.verify_broker_identity et evaluate_subject_checks ; autres brokers typés files, ingestion, development, application, model et runner.

DONNÉES ET ÉTATS

Utilisateur actif et comptes techniques PostgreSQL ; relations OpenFGA ; faits contextuels de requête non persistés.

DÉPENDANCES ET ÉCHANGES

OpenFGA ; identité OIDC ; faits résolus par le service propriétaire, sans rôles arbitraires fournis par le client.

COMPORTEMENT CIBLE PROPOSÉ

IAM Rust expose un contrat de décision typé liant sujet, acteur, tenant, audience, opération, ressource ou édition et version de politique. Query et Files restent les points de médiation des lignes, colonnes et octets ; un droit Lakekeeper table.modify ne devient pas un droit générique de worker.

MIGRATION DE CETTE CAPACITÉ

Porter les matrices de décision et les routes sans assouplir les refus. Comparer les décisions ancienne/nouvelle en mode observation avant bascule, avec invalidation bornée et cas de panne.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un mauvais azp, un utilisateur inactif et un export RESTRICTED par compte technique sont refusés. Une panne sur la dernière décision d'un lot n'autorise pas les premières par une réponse partielle.

Références de l'analyse

[monorepo · services/iam/src/iam_service/api/v1/decision_broker.py:18](#) Identité des brokers et vérification JWT obligatoire.

[monorepo · services/iam/src/iam_service/api/v1/dataset_authorization.py:32](#) Champs et limites du contrat.

[monorepo · services/iam/src/iam_service/api/v1/dataset_authorization.py:123](#) Règles humaines/techniques, états actifs et refus en cas de panne.

IAM-04 **Cible non qualifiée****Partager une ressource et remplacer réellement le rôle accordé****COMPORTEMENT OBSERVÉ**

Les routes partagent applications, fichiers et dossiers avec un utilisateur ou une équipe, révoquent le partage, listent les permissions et les ressources partagées avec soi. SharingService traite aussi les projets. Un partage fixe un rôle ; il ne cumule pas un nouveau rôle avec un rôle plus fort déjà détenu. Le service recense les rôles, accorde le rôle voulu puis retire ceux remplacés sur les alias. Une personne sans sujet d'identité exploitable est refusée avant le premier tuple. Les restrictions de propriétaire et de représentation des memberships restent distinctes d'un partage simple.

UTILISATEURS ET RÔLES

Propriétaire ou gestionnaire de partages ; bénéficiaire utilisateur ou équipe.

INTERFACES DANS LE CODE

POST/DELETE /api/v1/sharing/{applications|files|folders}/{id}/share ; GET permissions ; GET /api/v1/sharing/shared-with-me ; SharingService.share_resource.

DONNÉES ET ÉTATS

Tuples utilisateur ou équipe, aliases de ressource, memberships projet et audit.

DÉPENDANCES ET ÉCHANGES

OpenFGA et résolution des sujets ; miroir des memberships projet.

COMPORTEMENT CIBLE PROPOSÉ

IAM Rust garde une commande set-role explicite et une commande revoke. Les résultats portent l'état effectif et la version de décision. Catalog collections et EXPOSURE réutilisent ces identités sans conférer implicitement des droits sur les données.

MIGRATION DE CETTE CAPACITÉ

Recenser droits directs, hérités et alias. Rejouer promotions et rétrogradations sur une copie du graphe, puis migrer les écritures avec reprise des opérations incomplètes.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Passer de admin à viewer retire le droit admin sur tous les alias. Une indisponibilité du recensement ne peut répondre succès. Un partage de collection ne permet pas de lire un dataset interdit.

Références de l'analyse

[monorepo · services/iam/src/iam_service/services/sharing_service.py:261](#) Sémantique set-role, recensement, grant puis revoke.

[monorepo · services/iam/src/iam_service/api/v1/sharing.py:326](#) Liste des ressources partagées au sujet.

IAM-05 **Cible non qualifiée****Administrer comptes techniques, tokens d'exposition et délégations MCP****COMPORTEMENT OBSERVÉ**

IAM découvre, enregistre, met à jour, synchronise et retire des comptes techniques avec portée de projet ou organisation. Les tokens d'exposition ont propriétaire, périmètre, routes et date d'expiration. Créer un token déjà expiré conserve la ligne mais ne provisionne pas de consommateur actif dans APISIX ; modifier sa date conserve sa clé, et passer dans le passé déclenche la révocation. Le reconciler de délégation configure l'échange standard uniquement pour les clients prévus ; la délégation de l'assistant reste dans la session utilisateur. Les outils dataset/corpus passent par read-identity avec leurs comptes dédiés.

UTILISATEURS ET RÔLES

Administrateurs, propriétaires de projets, intégrations et assistant délégué.

INTERFACES DANS LE CODE

ServiceAccountService.register/update/unregister/sync ; ExpositionTokenService.create_token/patch_token ; exposition_tokens router ; POST /mcp/read-identity dans le routeur dédié ; reconcile_keycloak_delegation.

DONNÉES ET ÉTATS

ServiceAccount, User technique, ExpositionApiToken, consommateurs/routes APISIX et configuration de clients OIDC.

DÉPENDANCES ET ÉCHANGES

Keycloak, APISIX, Kubernetes Secrets et broker IAM de résolution d'acteur.

COMPORTEMENT CIBLE PROPOSÉ

IAM Rust distingue personne, acteur et identité de service. Les capacités courtes sont liées au run et à une opération précise. Les tokens d'accès aux API restent des credentials ; EXPOSURE devient séparément un consommateur déclaré d'éditions.

MIGRATION DE CETTE CAPACITÉ

Importer portées et propriétaires, renouveler les secrets au changement d'IdP et mesurer la révocation. Ne pas présumer portables les sessions ni refresh tokens. Préserver les intégrations par une façade d'authentification bornée.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un token créé expiré ne donne jamais accès. Fermer la session utilisateur invalide la délégation associée. Un compte limité à un projet ne gagne aucun accès par substitution de X-User-Id.

Références de l'analyse

[monorepo · services/iam/src/iam_service/services/service_account_service.py:206](#) Enregistrement et portée des comptes techniques.

[monorepo · services/iam/src/iam_service/services/exposition_token_service.py:400](#) Traitement des credentials déjà expirés et provisioning APISIX.

[monorepo · services/iam/src/iam_service/services/keycloak_delegation.py:2](#) Contrat de délégation et distinction des outils dataset/corpus.

IAM-06 **Cible non qualifiée****Gérer sessions, fédérations et politiques de connexion****COMPORTEMENT OBSERVÉ**

Une personne liste ses sessions et révoque une session ou les autres sessions ; le sujet vient de l'identité authentifiée. Les API d'administration lisent, modifient, réinitialisent et audient politiques de mots de passe et sessions. La fédération est stockée dans la représentation d'IdP Keycloak, avec extensions de configuration ; le mode de sécurité global est en base IAM. Passer en SSO_ONLY exige un fournisseur actif. Supprimer ou désactiver le dernier fournisseur rétablit le mode qui permet les credentials, pour éviter un blocage du login.

UTILISATEURS ET RÔLES

Utilisateurs pour leurs sessions ; administrateurs pour fédérations et politiques.

INTERFACES DANS LE CODE

GET/DELETE /api/v1/users/me/sessions ; DELETE /api/v1/users/me/sessions/{session_id} ; /api/v1/admin/identity-providers ; admin_security_policies router.

DONNÉES ET ÉTATS

Sessions et représentations de fournisseurs dans Keycloak ; system_config et audits de politique dans PostgreSQL.

DÉPENDANCES ET ÉCHANGES

API Admin Keycloak, system_config, audit et frontend de connexion.

COMPORTEMENT CIBLE PROPOSÉ

IAM Rust conserve les décisions métier et délègue l'authentification à un fournisseur sans JVM qualifié. Les politiques et fédérations utilisent un modèle versionné avec audit ; la suppression du dernier moyen de connexion est refusée ou compensée explicitement.

MIGRATION DE CETTE CAPACITÉ

Inventorier fédérations, paramètres de mots de passe, MFA et lifespans sans exporter de secrets dans le dossier. Tester le transfert des credentials ; réenrôler les moyens non portables et imposer une réauthentification lors de la coupure.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un utilisateur ne peut révoquer la session d'un autre. Retirer le dernier IdP ne laisse pas une page SSO sans voie de connexion. Une révocation arrête aussi les accès délégués dans la fenêtre contractuelle.

Références de l'analyse

[monorepo · services/iam/src/iam_service/api/v1/user_sessions.py:283](#) Révocation des sessions du sujet courant.

[monorepo · services/iam/src/iam_service/api/v1/admin/admin_identity_providers.py:23](#) Autorité de stockage et garde anti-blocage.

[monorepo · services/iam/src/iam_service/services/security_policy_service.py:322](#) Écriture des politiques de session et audit.

IAM-07 **Cible non qualifiée****Réconcilier le routage et les barrières d'accès de la gateway****COMPOTEMENT OBSERVÉ**

IAM possède la table de routes APISIX de la plateforme. Au démarrage, `ensure_seed_data` rapproche le seed des lignes stockées, puis les synchronisations rendent routes, upstreams et plugins. Le service propose synchronisation et diff. Les types `oidc` et `oidc_openfga` séparent authentification et décision ; une route `oidc_openfga` sans configuration de droit est refusée. Des règles protègent les routes gérées par les instances et empêchent les rétrogradations d'authentification non permises. Retirer des lignes du seed entraîne un nettoyage ciblé des routes obsolètes.

UTILISATEURS ET RÔLES

Exploitation, administrateurs et services créateurs de routes d'instances.

INTERFACES DANS LE CODE

`authz/seed_routes.py` ; `ApisixService.ensure_seed_data/sync_all/diff` ; `admin_routes` et `instance_routes`.

DONNÉES ET ÉTATS

`ApisixRoute`, `Upstream`, `PluginConfig` et `Consumer PostgreSQL` ; rendu externe dans APISIX.

DÉPENDANCES ET ÉCHANGES

APISIX Admin, OIDC, OpenFGA et services amont.

COMPOTEMENT CIBLE PROPOSÉ

IAM Rust compile une politique de routage versionnée avec inventaire des exceptions publiques. Les identités et décisions sont propagées aux services sans permettre à un header utilisateur non vérifié de remplacer le sujet.

MIGRATION DE CETTE CAPACITÉ

Exporter le seed et comparer le rendu réel, y compris routes plates, WebSocket et endpoints publics. Basculer les routes par famille ; conserver un diff et une procédure de retour qui ne réactive pas une route retirée.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une route privée ajoutée sans politique est refusée avant déploiement. Une route retirée du seed disparaît sans supprimer les routes d'instances encore actives. Une tentative de passage à `auth_type none` est bloquée.

Références de l'analyse

[monorepo · services/iam/src/iam_service/services/apisix_service.py:1655](#) Rapprochement du seed et nettoyage associé.

[monorepo · services/iam/src/iam_service/services/apisix_service.py:1291](#) Validation de la configuration de droit.

[monorepo](#) · [services/iam/src/iam_service/api/v1/admin/admin_routes.py:62](#) Protection des objets gérés par les instances.

IAM-08 **Cible non qualifiée**

Auditer, réconcilier les droits et ouvrir un accès de secours borné

COMPORTEMENT OBSERVÉ

Les mutations produisent audits et faits relayés par une outbox. Le réconcilier compare projections et relations, avec un traitement particulier des administrateurs de plateforme. Le break-glass s'active depuis une identité déjà administrateur actif en base locale, sans créer de sujet inconnu pendant la panne. Il a une portée et une expiration ; son rendu assouplit certaines décisions gateway tout en conservant l'authentification. Les écritures IAM et routes avec contrôle system:global restent protégées. La sortie manuelle ou l'expiration déclenche une réconciliation lorsque le service requis est disponible.

UTILISATEURS ET RÔLES

Administrateurs de plateforme, exploitation et auditeurs.

INTERFACES DANS LE CODE

/api/v1/admin/audit ; admin_breakglass router ; BreakGlassService.set_break_glass/expire_if_due ; OpenFGAReconciler ; EventOutboxDispatcher.

DONNÉES ET ÉTATS

AuditLog, AuthzSystemState, EventOutbox et projections OpenFGA.

DÉPENDANCES ET ÉCHANGES

PostgreSQL, OpenFGA, APISIX et transport actuel Kafka.

COMPORTEMENT CIBLE PROPOSÉ

IAM et Audit Rust gardent l'intention, le motif, la portée, l'échéance et les preuves d'application. La lecture d'une politique dégradée reste explicite. Le transport sans JVM conserve ordre par ressource et reprise, sans transformer l'audit IAM en preuve de publication de données.

MIGRATION DE CETTE CAPACITÉ

Importer historique utile et état de secours, puis qualifier panne et restauration. Remplacer Kafka avec maintien des clés et identifiants d'événement. Garder les mécanismes de rattrapage jusqu'à convergence des projections.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une activation expirée n'assouplit plus aucune route même si la boucle de nettoyage tarde. Une personne inactive ne peut activer le secours. Après panne puis reprise, l'audit et le graphe concordent avec les mutations durables.

Références de l'analyse

[monorepo](#) · [services/iam/src/iam_service/api/v1/admin/admin_breakglass.py:54](#) Autorité locale et refus de provisionner un sujet inconnu.

[monorepo](#) · [services/iam/src/iam_service/services/breakglass_service.py:367](#) Expiration de l'activation.

[monorepo](#) · [services/iam/src/iam_service/services/event_outbox_dispatcher.py:312](#) Prise en charge durable des événements à relayer.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- Identités, relations, délégations, politiques, sessions métier, audit des mutations et réconciliation versionnée.
- Décisions sur ressources et éditions avec sujet et acteur distincts ; autorité des commandes en PostgreSQL.

Plan de données proposé

- IAM ne transporte pas les lignes ni les blobs. Query, Files et Writer consomment des décisions liées à un périmètre précis.
- Le Publisher contrôle séparément la publication ; les workers ne reçoivent pas de permission REST générique table.modify.

Découpage du core Rust

- Modules proposés identity, membership, policy, delegation, route-policy et reconciliation, avec adaptateurs OIDC/OpenFGA/APISIX.
- Outbox Rust transactionnelle et interfaces de provisioning idempotentes pour les dépendances externes.

Identité, autorisation et isolation

- Préserver toutes les restrictions propres aux comptes techniques, la garde du dernier administrateur et les décisions indisponibles.
- Mesurer révocation et cache jusqu'aux flux ouverts ; ne pas supposer que la durée d'un JWT suffit à prouver une révocation.

Exploitation et reprise

- Sauvegarder PostgreSQL et les représentations d'identité nécessaires à la reprise ; restaurer puis réconcilier avec OpenFGA et gateway.
- Superviser le retard d'outbox, les décisions dégradées, les divergences et l'échéance de secours. La cible retire Keycloak et Kafka, y compris leurs tâches d'exploitation JVM.

Séquence de migration

- Figurer les contrats HTTP et les matrices de décision à partir des tests existants.
- Importer les IDs et mappings issuer/sub ; comparer sans écrire les droits rendus par Rust.
- Basculer les mutations par famille vers un auteur unique ; qualifier départ, révocation, panne et restauration.

- Retirer les composants JVM lorsque les parcours de connexion et délégation ont leur preuve de remplacement.

Écarts et décisions encore ouverts

- Le fournisseur OIDC sans JVM reste à qualifier sur les fédérations et moyens MFA réellement utilisés. Le rapport original cite un candidat, sans preuve d'intégration.
- Cette lecture ciblée ne couvre pas chaque branche des nombreux contrôleurs administratifs. Les contrats détaillés des ressources spécialisées doivent conserver leurs tests existants.
- Aucun test, appel à un IdP ou scénario de panne n'a été exécuté pour ce lot.

Références de l'analyse

[monorepo](#) · [services/iam/src/iam_service/main.py:2542](#) Assemblage des familles de routes.

[monorepo](#) · [services/iam/src/iam_service/services/apisix_service.py:1942](#) Démarrage de la configuration gateway.

SERVICE ACTUEL · KEYCLOAK

Authentification OIDC et configuration du realm

6 capacités analysées

services/keycloak · Keycloak 26.7.4 dans une image JVM, PostgreSQL, opérateur Keycloak Kubernetes, thème et plugin JAR ; Helm, scripts shell/jq et keycloak-config-cli pour la configuration.

Fournir connexion, tokens, sessions et page de login, puis réconcilier la configuration d'authentification livrée avec la plateforme.

Lecture de code

Pas de qualification runtime nouvelle

Domaines cible : IAM, Notifications, Frontends, Audit.

Périmètre de lecture

- Rapport identité-gouvernance existant ; Dockerfile ; clients OIDC ; définition du realm et flows.
- CR Keycloak, Job realm-sync, CronJob de drift et scripts d'invariants. Aucun realm de client interrogé.

Objets et cycle de vie

- Le realm contient clients, scopes, rôles, flows, sessions et politiques ; IAM garde les identités et droits métier.
- Le chart gère seulement les objets explicitement déclarés et certains invariants ; les fournisseurs fédérés et objets propres à une installation ne sont pas tous possédés par le chart.

Dépendances du service

- PostgreSQL pour Keycloak ; opérateur Kubernetes via la CR k8s.keycloak.org/v2alpha1.
- JAR de thème et plugin ; keycloak-config-cli pour import ; IAM pour clients dynamiques et délégations.

Fonctionnalités et compatibilité à préserver

KC-01 Cible non qualifiée

Connecter les interfaces Web par authorization code et PKCE

COMPORTEMENT OBSERVÉ

Le chart déclare le client public de frontend avec standardFlowEnabled, désactive implicitFlow et exige PKCE S256 côté serveur. Les origines et redirections viennent du rendu de la configuration. Le password grant est un opt-in de test partagé ; il n'est pas implicitement activé par le mode développement. Les scopes incluent identité et audience nécessaire au contrôle de la gateway.

UTILISATEURS ET RÔLES

Utilisateurs des interfaces Web.

INTERFACES DANS LE CODE

Template realm_clients/_api_client_front.tpl ; endpoints OIDC du realm fournis par Keycloak.

DONNÉES ET ÉTATS

Realm, client frontend, scopes, audiences et sessions Keycloak.

DÉPENDANCES ET ÉCHANGES

Frontend, IAM et gateway APISIX.

COMPORTEMENT CIBLE PROPOSÉ

Conserver authorization code avec PKCE, validation d'audience et logout dans un fournisseur sans JVM. IAM Rust garde les identités métier, et les interfaces TypeScript consomment le protocole OIDC sans dépendre de claims propriétaires non documentés.

MIGRATION DE CETTE CAPACITÉ

Inventorier redirections et claims contractuels, configurer le nouvel issuer puis effectuer une période bornée de double validation. Réauthentifier les utilisateurs à la coupure ; ne pas importer aveuglément des sessions.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Le nouveau login refuse une redirection non déclarée et un code sans vérificateur PKCE. Les appels API ont la bonne audience et les mêmes profils métier après connexion.

Références de l'analyse

[monorepo · services/keycloak/chart/templates/realm_clients/_api_client_front.tpl:50](#) Flux navigateur déclaré.

[monorepo · services/keycloak/chart/templates/realm_clients/_api_client_front.tpl:105](#) PKCE obligatoire côté IdP.

KC-02 **Cible non qualifiée****Authentifier CLI et SDK par device authorization****COMPORTEMENT OBSERVÉ**

Le client public cleyrop-cli utilise le device grant, sans redirection ni origine Web et sans standard ou implicit flow. Il délivre des refresh tokens selon les politiques du realm et propose offline_access en scope optionnel. Le PKCE du device grant est piloté par un paramètre dédié ; le template documente que le SDK livré n'envoie pas encore de challenge. Le password grant reste lié à l'opt-in de test.

UTILISATEURS ET RÔLES

Utilisateurs CLI, SDK Python et outils interactifs.

INTERFACES DANS LE CODE

Template realm_clients/_cli.tpl ; device authorization et token endpoints du fournisseur.

DONNÉES ET ÉTATS

Client CLI, demande device, session utilisateur et refresh tokens.

DÉPENDANCES ET ÉCHANGES

SDK, IdP et API de la plateforme.

COMPORTEMENT CIBLE PROPOSÉ

Qualifier device grant, polling, expiration et stockage des credentials avec le fournisseur sans JVM. Préserver le lien entre SDK et session utilisateur, en ajoutant PKCE seulement avec une version de client compatible.

MIGRATION DE CETTE CAPACITÉ

Tester l'ancien et le nouveau SDK contre le fournisseur choisi. Versionner le changement PKCE et invalider les anciens refresh tokens lors du retrait d'issuer.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une CLI peut terminer le login sur un autre appareil, puis renouveler l'accès. Une demande device expirée échoue et la révocation de session rend le renouvellement impossible.

Références de l'analyse

[monorepo](#) · [services/keycloak/chart/templates/realm_clients/_cli.tpl:13](#) Absence de flux navigateur.

[monorepo](#) · [services/keycloak/chart/templates/realm_clients/_cli.tpl:50](#) Bascule PKCE et device grant.

KC-03 **Cible non qualifiée**

Fournir des clients confidentiels et des audiences de service**COMPORTEMENT OBSERVÉ**

Le client API est confidentiel, active les service accounts et déclare scopes et protocol mappers. Le chart définit aussi les clients nécessaires aux intégrations retenues. Les invariants ajoutent l'audience gateway aux clients non déclarés qui doivent appeler les API et le scope profile aux comptes techniques. La configuration de délégation MCP appartient au reconciler IAM, ce qui est une dépendance à migrer en plus des templates du chart.

UTILISATEURS ET RÔLES

Services, intégrations et réconciliateurs IAM.

INTERFACES DANS LE CODE

Templates `_realm_clients.tpl`, `realm_clients/_api_client.tpl` et `_realm_client_scopes.tpl` ; `invariants.sh` pour clients non déclarés.

DONNÉES ET ÉTATS

Clients, secrets référencés, scopes, mappers et comptes techniques du realm.

DÉPENDANCES ET ÉCHANGES

IAM, APISIX et Kubernetes Secrets.

COMPORTEMENT CIBLE PROPOSÉ

IAM Rust provisionne des identités de service à portée explicite auprès d'un IdP sans JVM. Un token technique doit conserver une audience limitée et l'acteur lorsqu'il agit par délégation.

MIGRATION DE CETTE CAPACITÉ

Recenser les clients créés par chart et ceux créés par IAM, puis renouveler secrets et audiences. Remplacer les constructions d'URL Keycloak dans chaque client avant retrait.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une intégration continue de recevoir un token attendu par sa seule API. Un token sans audience gateway est refusé et une intégration ne peut échanger pour une audience non autorisée.

Références de l'analyse

[monorepo](#) · [services/keycloak/chart/templates/realm_clients/_api_client.tpl:60](#) Compte technique du client confidentiel.

[monorepo](#) · [services/keycloak/chart/files/realm-sync/invariants.sh:228](#) Audience et scope des clients non déclarés par le chart.

KC-04 Cible non qualifiée**Appliquer politiques de session, MFA et protections du login****COMPORTEMENT OBSERVÉ**

La définition du realm déclare politique de mot de passe, lifespans de tokens et sessions, rotation des refresh tokens, protections brute force et headers navigateur. Le flow Jupyterhub peut être conditionné par un rôle. Les invariants rendent le sous-flow MFA du navigateur obligatoire lorsque demandé, et ne relâchent pas un realm déjà renforcé. La définition enregistre les événements de connexion, échec, logout, mot de passe et MFA pour l'exploitation.

UTILISATEURS ET RÔLES

Utilisateurs et responsables sécurité.

INTERFACES DANS LE CODE

`_realm_definition.tpl` ; `_realm_auth_flows.tpl` ; `_realm_events.tpl` ; `invariants.sh` section MFA.

DONNÉES ET ÉTATS

Politiques et flows du realm, credentials MFA, sessions et événements Keycloak.

DÉPENDANCES ET ÉCHANGES

IAM pour les écrans d'administration ; Jupyterhub ; IdP.

COMPORTEMENT CIBLE PROPOSÉ

Traduire chaque politique dans un modèle de configuration IAM versionné, puis qualifier son effet avec le fournisseur sans JVM. Les facteurs et passkeys dont l'export n'est pas compatible nécessitent un réenrôlement explicite.

MIGRATION DE CETTE CAPACITÉ

Comparer les politiques réellement actives, y compris celles que le chart ne possède pas. Tester MFA, expiration, logout et anti-brute-force ; ne pas annoncer une portabilité automatique des credentials.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un compte soumis à MFA ne peut terminer le login avec le seul mot de passe. Un déploiement ne relâche pas la politique appliquée. Les bornes de session arrêtent les délégations associées.

Références de l'analyse

[monorepo](#) · [services/keycloak/chart/templates/_realm_definition.tpl:90](#) Politiques et paramètres déclarés du realm.

[monorepo](#) · [services/keycloak/chart/files/realm-sync/invariants.sh:268](#) Renforcement MFA à sens unique.

[monorepo](#) · [services/keycloak/chart/templates/_realm_auth_flows.tpl:9](#) Flow de login conditionné par rôle.

KC-05 Cible non qualifiée**Réconcilier le realm à chaque déploiement et détecter la dérive****COMPORTEMENT OBSERVÉ**

Le Job post-install/post-upgrade exécute wait, import, invariants puis realm-diff dans un ordre imposé par les init containers. Un échec bloque le Job et le déploiement. L'import met à jour les objets déclarés sans supprimer ceux laissés hors configuration. Les invariants traitent notamment profil admin-only, bootstrap, marqueurs de bienvenue, clients à retirer par nom et vérification des credentials. Le CronJob de drift utilise un client en lecture seule ; il ne couvre pas les mappings de rôles des comptes techniques ni le realm master.

UTILISATEURS ET RÔLES

Exploitation et automatisation de déploiement.

INTERFACES DANS LE CODE

Job realm-sync ; CronJob realm-drift ; scripts wait.sh, invariants.sh et realm-diff.sh.

DONNÉES ET ÉTATS

Définition Helm, objets effectifs du realm et résultats de Job ; configuration hors propriété du chart préservée.

DÉPENDANCES ET ÉCHANGES

Kubernetes, Helm, API Admin Keycloak et keycloak-config-cli JVM.

COMPORTEMENT CIBLE PROPOSÉ

Un reconciler Rust et des jobs bornés remplacent keycloak-config-cli, l'opérateur et les scripts dépendant de Keycloak. Chaque propriété gérée a une comparaison avant/après et une preuve de convergence ; les extensions clientes restent hors propriété explicite.

MIGRATION DE CETTE CAPACITÉ

Séparer objets possédés, objets seulement lus et exceptions clientes. Porter invariants et drift avant la bascule d'IdP ; supprimer ensuite toutes les images JVM de sync et d'exploitation.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un écart dans une propriété gérée fait échouer la vérification. Un fournisseur fédéré hors propriété du chart reste intact. Deux applications successives de la même configuration produisent le même état.

Références de l'analyse

[monorepo · services/keycloak/chart/templates/realm-sync.yaml:12](#) Ordonnancement et blocage sur échec.

[monorepo · services/keycloak/chart/templates/realm-sync.yaml:101](#) Protection no-delete des types gérés.

[monorepo · services/keycloak/chart/templates/realm-drift.yaml:6](#) Limites du contrôle périodique en lecture seule.

KC-06 **Cible non qualifiée**

Personnaliser le login, les emails et les premiers accès**COMPORTEMENT OBSERVÉ**

L'image assemble un thème et un plugin JAR. Le realm choisit thème de login et d'email, langues, locale et configuration SMTP via une référence de coffre. Le listener welcome-email est branché aux événements. Lors de la première adoption par le sync, les utilisateurs existants reçoivent un marqueur qui évite un envoi de bienvenue collectif ; les nouvelles identités suivent ensuite le listener. Les invariants protègent aussi les attributs de politique et de bienvenue dans le profil utilisateur.

UTILISATEURS ET RÔLES

Utilisateurs, administrateurs de branding et exploitation.

INTERFACES DANS LE CODE

Dockerfile de l'image ; _realm_definition.tpl ; invariants.sh section welcome-email.

DONNÉES ET ÉTATS

Thèmes, locale, configuration SMTP, attributs de profil et marqueurs de bienvenue.

DÉPENDANCES ET ÉCHANGES

Thème et plugin partagés, SMTP, IAM et futur Notifications.

COMPORTEMENT CIBLE PROPOSÉ

Conserver les écrans en TypeScript ou dans un thème supporté par l'IdP sans JVM. Notifications Rust porte l'envoi métier de bienvenue avec déduplication ; le mécanisme de sécurité du mot de passe reste au fournisseur d'identité.

MIGRATION DE CETTE CAPACITÉ

Inventorier traductions et parcours email sans recopier les paramètres SMTP. Importer les marqueurs de bienvenue et qualifier liens, expiration et changements de langue.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Migrer des comptes existants n'envoie pas de bienvenue en masse. Un nouveau compte reçoit au plus un message métier et le lien de sécurité est valide uniquement dans le contexte attendu.

Références de l'analyse

[monorepo · services/keycloak/app/Dockerfile:3](#) Assemblage des extensions JAR.

[monorepo · services/keycloak/chart/templates/_realm_definition.tpl:81](#) Listener branché et thèmes du realm.

[monorepo · services/keycloak/chart/files/realm-sync/invariants.sh:151](#) Marquage initial des comptes déjà présents.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- Configuration OIDC, identités de service, fédérations et politiques versionnées par IAM Rust.
- Réconciliation explicite et inventaire des propriétés gérées sans suppression générale du realm.

Plan de données proposé

- Émission et validation de tokens par un fournisseur sans JVM ; aucun accès direct aux tables ou blobs conféré par le seul login.

Découpage du core Rust

- Adaptateur IdP, réconciliateur de clients/politiques et mapping issuer/sub dans IAM.
- Événements de bienvenue idempotents confiés à Notifications. Le serveur OIDC peut être un tiers sans JVM qualifié.

Identité, autorisation et isolation

- Ne pas relâcher PKCE, MFA, restriction des audiences ou lifespans pour faciliter la migration.
- Ne pas fusionner les personnes par courriel ni supposer portables les sessions, refresh tokens et facteurs MFA.

Exploitation et reprise

- Le chart installe une CR Keycloak adossée à PostgreSQL et un Job de convergence bloquant.
- La sortie zéro JVM couvre serveur, opérateur, JAR, image de sync et outils de runbook. Sauvegarde, restauration et fonctionnement sans Internet restent à qualifier pour le remplaçant.

Séquence de migration

- Inventorier clients, fédérations, facteurs et claims utilisés.
- Qualifier un IdP sans JVM et le reconciler sur un environnement isolé.
- Importer IDs et credentials compatibles, imposer réauthentification ou réenrôlement pour le reste.
- Basculer les audiences et clients, mesurer révocation, puis retirer l'ensemble des dépendances Keycloak.

Écarts et décisions encore ouverts

- Aucun fournisseur sans JVM n'a été testé dans cette mission.
- Le chart ne suffit pas à établir les fédérations, facteurs et politiques effectifs de chaque cluster.
- Les plugins et thèmes partagés sont cités comme dépendances ; ce lot ne relit pas intégralement leur code.

Références de l'analyse

[monorepo](#) · [services/keycloak/chart/templates/keycloak.yaml:1](#) Déploiement par l'opérateur Keycloak.

[monorepo](#) · [services/keycloak/app/Dockerfile:1](#) Version et runtime de l'image actuelle.

SERVICE ACTUEL · WHITELISTER

Règles réseau entrantes et découverte des IP de sortie

6 capacités analysées

services/whitelister · Python, FastAPI, SQLAlchemy asynchrone et PostgreSQL ; client Kubernetes, ConfigMaps et agent de découverte ; JWT/JWKS et limitation de débit.

Permettre à l'exploitation de gérer les CIDR admis par l'ingress et d'identifier les adresses vues par les services externes. La découverte d'IP de sortie n'est pas une politique d'autorisation des destinations sortantes.

Lecture de code

Pas de qualification runtime nouvelle

Domaines cible : Egress, IAM, Audit, Maintenance.

Périmètre de lecture

- Routes, schémas CIDR et egress ; services whitelist, Kubernetes, egress et audit ; middleware et boucle de démarrage.
- Agent de découverte et contrats chart repérés ; aucune opération Kubernetes exécutée.

Objets et cycle de vie

- WhitelistRule porte CIDR, description et flags protected/init. AuditLog enregistre action et attribution.
- Les observations EgressEntry sont horodatées par nœud, distinctes des règles de destinations sortantes.

Dépendances du service

- PostgreSQL et Kubernetes Core API pour les ConfigMaps.
- Ingress NGINX, agents par nœud, service d'écho et issuer/JWKS configuré.

Fonctionnalités et compatibilité à préserver

WL-01 Cible non qualifiée

Créer et éditer une liste de CIDR admis

COMPORTEMENT OBSERVÉ

L'API normalise les CIDR, crée une règle ou un lot d'au plus cent, puis expose liste paginée et détail. Les règles manuelles ont init=false ; protected peut être déclaré à la création. Le lot écrit ses règles dans une transaction et refuse une collision d'unicité. PUT et PATCH modifient CIDR ou description ; une règle protégée ne peut être modifiée. Les écritures déclenchent ensuite la synchronisation Kubernetes.

UTILISATEURS ET RÔLES

Administrateurs réseau.

INTERFACES DANS LE CODE

POST/GET /api/v1/whitelist ; POST /api/v1/whitelist/bulk ; GET/PUT/PATCH /api/v1/whitelist/{rule_id}.

DONNÉES ET ÉTATS

WhitelistRule avec CIDR normalisés, description, protected, init et dates en PostgreSQL.

DÉPENDANCES ET ÉCHANGES

PostgreSQL et contrôleur de ConfigMap ingress.

COMPORTEMENT CIBLE PROPOSÉ

Un module de politique réseau Rust conserve les règles désirées, leur version et leur provenance. Le contrôleur Kubernetes applique une génération distincte, de sorte que l'API distingue accepté et appliqué.

MIGRATION DE CETTE CAPACITÉ

Importer les règles manuelles et celles de configuration sans changer leur protection. Conserver normalisation, limites de lot et erreurs, puis basculer l'unique contrôleur de ConfigMap.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un lot contenant un doublon n'écrit aucune règle. Une règle protected ne peut changer par PATCH. La réponse indique la génération effective lorsque la projection réseau est en retard.

Références de l'analyse

[monorepo](#) · [services/whitelister/src/whitelister/schemas/whitelist.py:13](#) Normalisation du contrat.

[monorepo](#) · [services/whitelister/src/whitelister/services/whitelist_service.py:157](#) Transaction de lot et collision.

[monorepo](#) · [services/whitelister/src/whitelister/services/whitelist_service.py:305](#) Protection et mise à jour.

WL-02 **Cible non qualifiée**

Refuser le retrait de sa propre plage et des règles protégées**COMPORTEMENT OBSERVÉ**

La suppression distingue introuvable, protégé et auto-exclusion. Une règle protégée donne un refus. Si l'adresse cliente se trouve dans l'un des CIDR de la règle, la suppression est aussi refusée et auditée. L'adresse vient de l'extraction du contexte réseau/proxy. Le code ignore une adresse impossible à parser dans ce contrôle ; le résultat dépend donc aussi de la confiance accordée aux headers proxy.

UTILISATEURS ET RÔLES

Administrateurs modifiant la liste d'accès réseau.

INTERFACES DANS LE CODE

DELETE /api/v1/whitelist/{rule_id} ; WhitelistService.delete_rule.

DONNÉES ET ÉTATS

Règle et audit de tentative bloquée ; aucune suppression pour les deux refus métier.

DÉPENDANCES ET ÉCHANGES

Extraction de l'IP par le proxy et journal d'audit.

COMPORTEMENT CIBLE PROPOSÉ

Le contrôleur Rust valide l'identité réseau de l'opérateur et la projection simulée avant retrait. Une procédure de secours bornée peut traiter un changement volontaire sans supprimer les garanties de l'API ordinaire.

MIGRATION DE CETTE CAPACITÉ

Reprendre le contrat de refus et qualifier le calcul d'adresse derrière l'ingress réel. Ajouter un plan de changement qui évalue l'accès d'administration restant.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un opérateur ne peut retirer la plage qui contient son adresse vérifiée. Un header de provenance forgé ne contourne pas ce contrôle. La tentative refusée reste visible dans l'audit.

Références de l'analyse

[monorepo · services/whitelister/src/whitelister/services/whitelist_service.py:226](#) Protections et résultats de suppression.

[monorepo · services/whitelister/src/whitelister/api/routes.py:346](#) Correspondance des refus vers les réponses HTTP.

WL-03 Cible non qualifiée**Projeter les règles vers l'ingress et réinitialiser la configuration de démarrage****COMPOTEMENT OBSERVÉ**

Le service lit toutes les règles, joint les CIDR puis met à jour une clé d'une ConfigMap Kubernetes. La synchronisation réessaie trois fois avec attente croissante ; après le dernier échec, elle journalise et ne remonte pas l'exception. Une création peut donc être enregistrée en base sans preuve que l'ingress l'a appliquée. POST sync relance cette projection. Au démarrage, les règles init sont supprimées et recréées depuis la configuration, y compris si elles étaient protégées.

UTILISATEURS ET RÔLES

Exploitation et automatisation du déploiement.

INTERFACES DANS LE CODE

POST /api/v1/whitelist/sync ; sync_to_kubernetes ; reset_init_rules ; KubernetesService.update_configmap.

DONNÉES ET ÉTATS

État désiré PostgreSQL et ConfigMap appliquée ; règles de démarrage distinctes des règles manuelles.

DÉPENDANCES ET ÉCHANGES

API Kubernetes et ingress NGINX consommateur de la ConfigMap.

COMPOTEMENT CIBLE PROPOSÉ

Un reconciler Rust garde génération désirée, génération observée et erreur durable. L'initialisation calcule un diff et ne traverse pas un état intermédiaire non voulu. Le statut appliqué exige la lecture de la projection effective.

MIGRATION DE CETTE CAPACITÉ

Capturer la ConfigMap et comparer les CIDR normalisés à la base avant transfert. Désactiver l'ancien sync lorsque le nouveau contrôleur devient auteur. Préserver les règles manuelles lors d'un changement de configuration.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une panne Kubernetes laisse l'opération pending ou failed, sans succès d'application trompeur. Un redémarrage avec la même configuration ne retire pas temporairement les règles attendues.

Références de l'analyse

[monorepo · services/whitelister/src/whitelister/services/whitelist_service.py:57](#) Retries et erreur finale absorbée.

[monorepo · services/whitelister/src/whitelister/services/whitelist_service.py:87](#) Cycle de réinitialisation des règles init.

[monorepo](#) · [services/whitelister/src/whitelister/services/kubernetes_service.py:28](#) Écriture de la clé de ConfigMap.

WL-04 Cible non qualifiée

Découvrir les IP de sortie et maintenir leur règle

COMPOTEMENT OBSERVÉ

Des agents par nœud interrogent un service d'écho et publient leurs observations horodatées en ConfigMap. Le service agrège les IP, ajoute les overrides valides et signale les observations périmées, mais les conserve dans l'ensemble. GET egress-ips déduplique les IP et renvoie origine, nœud, date et warnings. Une boucle peut maintenir une règle protégée pour ces IP. Le rendu CIDR ajoute /32 à chaque IP ; la prise en charge IPv6 nécessite donc une vérification spécifique.

UTILISATEURS ET RÔLES

Exploitation et administrateurs d'intégrations externes.

INTERFACES DANS LE CODE

GET /api/v1/egress-ips ; egress_agent ; EgressService.get_all_egress_ips/upsert_egress_rule.

DONNÉES ET ÉTATS

Observations de nœuds en ConfigMap, overrides et règle protégée PostgreSQL.

DÉPENDANCES ET ÉCHANGES

Agents Kubernetes, service d'écho, ConfigMap et boucle de synchronisation.

COMPOTEMENT CIBLE PROPOSÉ

Egress Rust sépare observation réseau et politique des destinations autorisées. Le discovery garde fraîcheur, famille IP et preuve d'observation ; les connecteurs consomment une politique de sortie avec DNS/TLS et borne de périmètre, pas une simple liste d'IP vues.

MIGRATION DE CETTE CAPACITÉ

Porter l'agent sans JVM ou le garder encapsulé, puis publier un contrat IPv4/IPv6 et une politique explicite de péremption. Maintenir les overrides nécessaires pendant le changement de topologie.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une observation périmée est marquée et son effet suit une politique explicite. Une IPv6 produit un préfixe adapté. Une panne du service d'écho ne transforme pas une ancienne IP en observation fraîche.

Références de l'analyse

[monorepo](#) · [services/whitelister/src/whitelister/egress_agent.py:28](#) Découverte et validation des réponses d'écho.

[monorepo](#) · [services/whitelister/src/whitelister/services/egress_service.py:87](#) Agrégation, overrides et données périmées.

[monorepo](#) · [services/whitelister/src/whitelister/services/egress_service.py:145](#) Règle protégée et construction de CIDR.

WL-05 Cible non qualifiée

Authentifier les administrateurs réseau et limiter les requêtes

COMPOTEMENT OBSERVÉ

Quand l'authentification est activée, le middleware exige un bearer vérifié par JWKS, signature, dates, issuer et audience ou azp configurés. Le contrôle de groupes accepte un nom de groupe ou un chemin complet. Les routes exclues et preflights suivent un parcours séparé. Lorsque l'authentification est désactivée, un token

peut être vérifié seulement pour attribution d'audit ; l'absence de token n'est alors pas un refus. Les écritures ont une limite de débit et la découverte d'egress une borne spécifique.

UTILISATEURS ET RÔLES

Administrateurs réseau authentifiés.

INTERFACES DANS LE CODE

JWTAuthMiddleware.dispatch ; verify_access_token ; décorateurs limiter des routes.

DONNÉES ET ÉTATS

Utilisateur vérifié dans request.state ; cache de groupes requis et validation JWKS.

DÉPENDANCES ET ÉCHANGES

OIDC/JWKS, proxy et futur broker IAM.

COMPORTEMENT CIBLE PROPOSÉ

IAM Rust fournit une permission d'administration réseau explicite. Le service vérifie issuer, audience, sujet actif et décision ; le mode sans authentification reste une configuration isolée et déclarée, jamais une dégradation automatique.

MIGRATION DE CETTE CAPACITÉ

Remplacer les groupes Keycloak par des relations métier sans perdre l'accès de secours prévu. Mettre à jour l'issuer/JWKS avec les autres services et qualifier les chemins de santé exclus.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un JWT avec mauvaise signature, issuer ou audience est refusé. Un membre d'un groupe de nom voisin n'obtient pas le droit. Une panne du fournisseur ne désactive pas l'authentification.

Références de l'analyse

[monorepo · services/whitelister/src/whitelister/middleware.py:155](#) Contrat de vérification, groupes et mode désactivé.

[monorepo · services/whitelister/src/whitelister/api/routes.py:437](#) Borne spécifique de découverte.

WL-06 Cible non qualifiée

Consulter les audits et distinguer santé de base et état réseau

COMPORTEMENT OBSERVÉ

Les opérations et refus alimentent un journal avec action, IP cliente, utilisateur et règle. Les routes listent avec filtres et pagination puis rendent un détail. Le helper général d'audit journalise et absorbe les erreurs d'écriture ; une action peut donc réussir sans audit durable confirmé. Le health contrôle une requête PostgreSQL, pas l'état effectif de l'ingress. Les métriques Prometheus sont exposées séparément.

UTILISATEURS ET RÔLES

Exploitation, support et auditeurs.

INTERFACES DANS LE CODE

GET /api/v1/audit ; GET /api/v1/audit/{log_id} ; GET /api/v1/health ; /metrics.

DONNÉES ET ÉTATS

AuditLog PostgreSQL et métriques de processus ; pas de génération réseau acquittée dans le contrat lu.

DÉPENDANCES ET ÉCHANGES

PostgreSQL, métriques Prometheus et contrôleur Kubernetes.

COMPORTEMENT CIBLE PROPOSÉ

Audit Rust conserve intention et résultat dans une transaction durable ; le contrôleur rapporte séparément application Kubernetes. La santé distingue API, base, retard de projection et découverte réseau.

MIGRATION DE CETTE CAPACITÉ

Importer l'audit existant avec sa complétude déclarée. Ajouter identifiants de changement et reçus d'application avant de couper l'ancien service.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une panne de journal n'est pas présentée comme une action intégralement auditée. Une base disponible et un ingress non synchronisé produisent deux états distincts. Un filtre ne révèle pas de secrets réseau ou credentials.

Références de l'analyse

[monorepo](#) · [services/whitelister/src/whitelister/services/whitelist_service.py:31](#) Écriture d'audit sans remontée d'erreur.

[monorepo](#) · [services/whitelister/src/whitelister/api/routes.py:499](#) Filtres et pagination.

[monorepo](#) · [services/whitelister/src/whitelister/api/routes.py:486](#) Santé limitée à la connexion base.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- Gestion des règles désirées, approbation, générations, état de projection et audit durable.
- Politique d'autorisation des destinations pour connecteurs à concevoir séparément de la découverte d'IP.

Plan de données proposé

- Le service actuel ne relaie pas le trafic applicatif. L'ingress applique les CIDR ; les agents observent la sortie réseau.

Découpage du core Rust

- Normaliseur CIDR, planificateur de diff, reconciler Kubernetes et modèle d'observation réseau.
- Adaptateur IAM et journal transactionnel pour les changements.

Identité, autorisation et isolation

- La confiance dans l'IP cliente dépend du proxy ; conserver une attribution vérifiée.
- La découverte ne constitue pas une barrière anti-exfiltration. La cible doit contrôler les destinations et les capacités de chaque run.

Exploitation et reprise

- Surveiller divergence base/ConfigMap, âge des observations et erreurs d'audit.
- Le startup remplace actuellement les règles init ; tester les changements de configuration et les échecs Kubernetes avant remplacement.

Séquence de migration

- Comparer règles en base et projection ingress.
- Déployer un reconciler Rust en observation puis transférer l'écriture de ConfigMap.
- Porter authz, audit et discovery en gardant une procédure de secours bornée.

Écarts et décisions encore ouverts

- Le traitement IPv6 et la conservation des observations périmées doivent être contractualisés.
- Le lot n'a pas exécuté de mutation réseau ni de test d'auto-exclusion.
- Le succès du sync actuel n'est pas une preuve que tous les contrôleurs ingress ont chargé la règle.

Références de l'analyse

[monorepo](#) · [services/whitelister/src/whitelister/main.py:43](#) Boucle de synchronisation des IP de sortie.

[monorepo](#) · [services/whitelister/src/whitelister/main.py:70](#) Initialisation et tâches de fond.

SERVICE ACTUEL · DATA-COLLECT

Connexions, dépôts et collectes de datasets

27 capacités analysées

services/data-collect · Kotlin 2.4.10 et Spring Boot 4.1.1 observés dans le catalogue Gradle ; modules domaine et adaptateurs Spring ; PostgreSQL/JPA/Flyway, Quartz JDBC, Kafka/Protobuf, SSH, client S3, JDBC et Parquet/Hadoop sur JVM.

Le service administre les sources et dépôts, aide à choisir et vérifier une lecture, collecte les octets dans le stockage d'import, planifie les reprises et échange les états avec data-dev, data-model et process-data. Il sert aussi les fichiers destinés aux corpus IA et le catalogue de sources du SDK Python.

Lecture de code

Pas de qualification runtime nouvelle

Domaines cible : Connectors, Datasets, Flows, Writer, Publisher, Files, IAM, Audit, Egress, Maintenance, Notifications, Embeddings.

Périmètre de lecture

- AGENTS.md du service et règle Helm commune.
- Tous les contrôleurs REST : datasource, fichiers, ingestion IA, permission, historique, KPI, config dataset, propriétaires, SFTP et administration.
- Registres et variantes de DTO de connexion/configuration/format ; agrégats datasource, dataset, permission, fichiers et schedules.
- Handlers de création, modification, suppression, partage, preview, contrôle de schéma, import, refresh, finalisation, dépôt et transfert de propriété.
- Factory et managers JDBC/API/S3/SFTP/local, stratégies API de pagination/découverte/authentification, stockage S3, cache de listing, chiffrement et reprise.
- Consumers data-model/process-data/Depot/IAM, droits résiduels, scheduler et démarrage ; chart Deployment, NetworkPolicy et HPA.
- Inventaire des tests de contrats concernés ; aucun build, test applicatif, cluster ou connecteur distant exécuté.

Objets et cycle de vie

- DataSource identifie une connexion et son propriétaire. La configuration de lecture du dataset est séparée de la connexion et peut figer une version API.
- ResourceDescriptor se spécialise en FileDescriptor ou TableDescriptor. Les identifiants sont réconciliés à la redécouverte ; les propriétés de navigation ne constituent pas un snapshot garanti de la source distante.
- DataSourceProjectPermission conserve demande, décision, justification et acteurs ; la projection IAM matérialise le droit d'usage accepté.
- DatasetCollect conserve nom, propriétaire, projet optionnel, configuration courante/ancienne et refreshConfig. CollectScheduled encode une étape asynchrone et son budget de reprise.
- SftpAccess et SftpAccessAuthorization reflètent un compte distant et ses clés publiques ; SftpExpireTime décrit une rétention distincte de l'expiration d'accès.

- FolderListingGeneration et ses entrées forment un cache jetable ; historique, UsageRecord, maps d'identité, files différées et outbox ne doivent pas être confondus avec les checkpoints de données publiées.

Dépendances du service

- Data-dev produit les créations de collectes et les anciens miroirs projet/utilisateur. Data-model produit changements de dataset, de schéma, de schedule et vidages.
- Process-data retourne les événements de fin d'exécution. data-collect publie l'état ingestion/process et la demande de suite de traitement par Kafka/Protobuf.
- Notification-center reçoit les échecs de collecte, demandes/décisions de permission, abonnements automatiques et événements synchrones de contrôle de fichier.
- IAM fournit visibilité, autorisations déléguées et événements de cycle de vie. APISIX contrôle les routes exposées ; IAM écrit OpenFGA à partir des faits data-collect.
- Les octets transitent par connecteurs externes, cache local et bucket S3 d'import. Le SDK Python consomme la liste autorisée de sources. ai-gen-assistant consomme listing et contenu sous délégation.

Fonctionnalités et compatibilité à préserver

DC-01 Cible non qualifiée

Enregistrer et faire évoluer une datasource

COMPOTEMENT OBSERVÉ

Une datasource porte un UUID, un nom, une description, un propriétaire, une sensibilité, un connecteur, un état et un historique. Le DTO de création permet un propriétaire explicite et applique ses propres valeurs par défaut. Le manager du connecteur construit les descripteurs initiaux. La modification conserve les secrets laissés vides et interdit de changer de type de connecteur. La suppression retire les liens projet et les configurations de collecte, demande la suppression des imports, puis émet les événements de suppression. La création publie un fait de propriété et attend sa visibilité dans IAM. Les erreurs métier ne suivent pas toutes la même convention HTTP, certains handlers renvoient null puis 400.

UTILISATEURS ET RÔLES

Gestionnaires de sources, propriétaires, administrateurs et créations automatiques depuis un dépôt.

INTERFACES DANS LE CODE

POST /datasources ; GET /datasources/{id} ; PUT /datasources/{id} ; DELETE /datasources/{id}.

DONNÉES ET ÉTATS

DataSource, Connector, ResourceDescriptor, DataSourceHistory et UsageRecord en PostgreSQL. SUCCESS, WARNING et FAILURE décrivent la source.

DÉPENDANCES ET ÉCHANGES

ConnectorFactory ; stockage objet des imports ; événements DataSourceCreateEvent, DataSourceUpdateEvent, DataSourceDeleteEvent ; IAM pour la visibilité du propriétaire.

COMPOTEMENT CIBLE PROPOSÉ

Connectors devient propriétaire des définitions de connexion en Rust. Datasets référence leur UUID et une révision immuable de configuration. Toute opération possède un identifiant idempotent, un acteur et un tenant.

MIGRATION DE CETTE CAPACITÉ

Importer les UUID et les références existantes. Séparer la désactivation de source de la suppression physique des données. Fournir un adaptateur de routes et expliciter les erreurs de création et de modification.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Créer une source, modifier sa description sans renvoyer son secret, puis relire ses capacités. Une modification de type est refusée. Supprimer une source liée produit un état de retrait traçable sans supprimer une édition publiée.

Références de l'analyse

[monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/controller/DataSourceController.kt:190](#) Création, visibilité IAM et routes de modification/suppression.

[monorepo · services/data-collect/app/data-collect-domain/src/main/kotlin/com/cleyrop/data/collect/domain/datasource/aggregate/DataSource.kt:101](#) Immutabilité du type et calcul des changements.

[monorepo · services/data-collect/app/data-collect-domain/src/main/kotlin/com/cleyrop/data/collect/domain/datasource/command/DeleteDataSourceCommandHandler.kt:42](#) Effets de la suppression sur liens, collectes et imports.

DC-02 **Cible non qualifiée****Chercher des sources accessibles et les exposer à la bibliothèque Python****COMPOTEMENT OBSERVÉ**

La liste combine recherche textuelle, tri, pagination, projet, types de connecteurs, sensibilités et statuts des demandes. Elle récupère les UUID candidats puis interroge IAM avant la pagination visible. Le mode notRequested suit un parcours distinct où visibleIds reste null. Les UUID de projet entrants sont traduits vers les anciennes clés de stockage. La bibliothèque Python dispose d'une route propre qui ne retient que les sources acceptées dans le projet et applique la même visibilité IAM. Une ancienne variante sélectionnée par User-Agent pylib subsiste et est dépréciée.

UTILISATEURS ET RÔLES

Utilisateurs de catalogue, membres de projets, notebooks et bibliothèque Python.

INTERFACES DANS LE CODE

GET /datasources ; GET /lib/datasources?projectId=... ; ancienne variante GET /datasources sélectionnée par User-Agent.

DONNÉES ET ÉTATS

Permissions projet, miroirs de projets, correspondances IAM/legacy et pagination calculée sur les candidats autorisés.

DÉPENDANCES ET ÉCHANGES

IamDataSourceVisibilityClient et ProjectIdMap ; contrats consommés par les frontends et les bibliothèques.

COMPOTEMENT CIBLE PROPOSÉ

Catalog et Connectors servent une recherche filtrée par IAM avec pagination stable. Le SDK Python consomme les mêmes identifiants canoniques et ne reçoit aucune capacité de connexion permanente.

MIGRATION DE CETTE CAPACITÉ

Conserver les filtres et formes de réponse du SDK pendant la transition. Retirer la sélection par User-Agent après migration des clients. Définir le contrat exact de notRequested avant sa reprise.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un membre voit seulement les sources autorisées sur chaque page et dans le SDK. Les totaux reflètent ce filtrage. Un même projet envoyé sous son ancien ou son nouvel UUID ne crée pas deux partitions de droits.

Références de l'analyse

[monorepo](#) · [services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/controller/DataSourceController.kt:90](#) Filtres et vérification des candidats par IAM.

[monorepo](#) · [services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/controller/DataSourceForLibController.kt:45](#) Route SDK, sources acceptées et dépréciation.

DC-03 **Cible non qualifiée**

Demander et décider un accès projet à une source**COMPORTEMENT OBSERVÉ**

Un membre demande l'activation d'une source dans un projet avec un commentaire. Le handler réutilise une demande déjà acceptée ou en attente ; une demande refusée permet une nouvelle demande. Il stocke le demandeur et le projet, émet un événement de permission, notifie la demande et abonne le demandeur aux réponses INAPP. Le décideur ne peut fixer que ACCEPTED ou REFUSED et ajoute justification, date et validateur. L'acceptation publie un fait de lien, le refus un fait de révocation. Une route liste les demandes filtrées et paginées.

UTILISATEURS ET RÔLES

Membres de projet, responsables de sources et administrateurs selon les routes IAM.

INTERFACES DANS LE CODE

POST /datasources/{id}/projects/{projectId} ; POST /datasources/permissions/{permissionId}/{permission} ; GET /datasources/permissions.

DONNÉES ET ÉTATS

DataSourceProjectPermission : REQUESTED, ACCEPTED ou REFUSED ; auteur, commentaire, validateur, justification et horodatages.

DÉPENDANCES ET ÉCHANGES

Notifications de demande/réponse ; abonnement automatique ; IAM reçoit les faits de lien via outbox.

COMPORTEMENT CIBLE PROPOSÉ

IAM tranche les droits, Connectors garde le workflow de demande et Audit son historique. Le lien approuvé donne un droit d'usage, sans exposer les credentials au projet.

MIGRATION DE CETTE CAPACITÉ

Reprendre les demandes actives et les décisions passées en conservant les identifiants. Dédupliquer par source/projet/demande et reconstruire les faits IAM avant bascule.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Deux envois d'une demande active retournent le même dossier. Un refus motivé laisse une trace et permet une nouvelle demande. Une acceptation devient visible dans le SDK puis une révocation retire l'accès.

Références de l'analyse

[monorepo](#) · [services/data-collect/app/data-collect-](#)

[domain/src/main/kotlin/com/cleyrop/data/collect/domain/datasource/command/CreateDataSourceLinkToProjectCommandHandler.kt:43](#) Création, notifications et abonnements de réponse.

monorepo · services/data-collect/app/data-collect-domain/src/main/kotlin/com/cleyrop/data/collect/domain/datasource/command/AcceptDataSourceLinkToProjectCommandHandler.kt:43 Décision et faits IAM.

monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/controller/DataSourcePermissionController.kt:65 Routes et traduction des UUID projet.

DC-04 Cible non qualifiée

Auditer les changements et compter les sources et propriétaires

COMPORTEMENT OBSERVÉ

L'historique présente les créations et modifications, de la plus récente à la plus ancienne. Les changements conservent les valeurs avant/après des champs non secrets ; un changement de secret conserve seulement le nom du champ concerné. La recherche peut porter sur les champs modifiés et leurs valeurs. Les routes KPI de source et de dépôt renvoient des comptages. Le contrôle de propriétaire consulte séparément les sources et les datasets publics pour préparer les transferts de propriété.

UTILISATEURS ET RÔLES

Propriétaires, administrateurs, parcours de départ utilisateur et support.

INTERFACES DANS LE CODE

GET /datasources/{id}/history ; GET /kpi/datasources ; GET /sftp-accesses/kpi ; GET /users/{userId}/check-owner.

DONNÉES ET ÉTATS

DataSourceHistory et DataSourceFieldChange ; journaux UsageRecord ; relations de propriété.

DÉPENDANCES ET ÉCHANGES

Repositories de sources, collectes et dépôts ; miroirs des auteurs.

COMPORTEMENT CIBLE PROPOSÉ

Audit conserve des événements signés logiquement par acteur, commande et révision. Catalog fournit les compteurs avec une définition de périmètre tenant. IAM utilise un inventaire de propriété pour les départs.

MIGRATION DE CETTE CAPACITÉ

Convertir les historiques existants sans réintroduire les anciennes valeurs secrètes. Versionner les noms de champs recherchables et conserver les références d'auteur même après désactivation.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une rotation de credential apparaît dans l'historique sans valeur avant/après. Une recherche sur connector.host retrouve les modifications attendues. Les compteurs restent limités au tenant demandé.

Références de l'analyse

monorepo · services/data-collect/app/data-collect-domain/src/main/kotlin/com/cleyrop/data/collect/domain/datasource/aggregate/DataSource.kt:135 Diff des champs ordinaires et SecretChange sans contenu.

monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/controller/DataSourceHistoryController.kt:21 Filtres de l'historique.

monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/dataset/controller/UserController.kt:21 Contrôle des propriétés de source et dataset public.

DC-05 **Cible non qualifiée**

Tester une connexion et réutiliser un secret masqué

COMPORTEMENT OBSERVÉ

Le test d'une nouvelle connexion reçoit un ConnectorDTO. Le test d'une source persistée utilise une route portant l'UUID ; le paramètre historique datasourced sur la route générale est refusé en 422. Pour une source existante, les secrets vides sont remplacés par les secrets stockés, puis le manager vérifie la connexion. Le résultat expose un booléen et une catégorie d'erreur, sans renvoyer le message brut d'exception. Les DTO sortants passent par maskSecrets. Les secrets vides lors d'une modification ont aussi une sémantique de conservation.

UTILISATEURS ET RÔLES

Créateurs et gestionnaires de connexions.

INTERFACES DANS LE CODE

POST /datasources/check ; POST /datasources/{id}/check ; Connector.toDTO ; Connector.withStoredSecrets.

DONNÉES ET ÉTATS

Configuration soumise et éventuelle configuration persistée ; absence de mutation de la définition requise pour un simple test.

DÉPENDANCES ET ÉCHANGES

Managers JDBC, SFTP, S3 et HTTP ; catalogue ConnectionErrorType ; credentials déchiffrés côté service.

COMPORTEMENT CIBLE PROPOSÉ

Connectors exécute un test borné avec la révision et la référence de secret autorisées. Egress décide les destinations et Audit consigne la catégorie d'échec sans credential.

MIGRATION DE CETTE CAPACITÉ

Conserver le contrat de masquage et le comportement des champs vides. Remplacer ce mécanisme ambigu à terme par une commande explicite preserve/replace/clear.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un test sur une source masquée réussit avec son secret stocké ; un nouvel endpoint arbitraire ne peut pas réutiliser un secret non autorisé. Un refus distant retourne une catégorie stable sans texte confidentiel.

Références de l'analyse

[monorepo · services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/controller/DataSourceController.kt:155](#) Deux routes de test et rejet du paramètre historique.

[monorepo · services/data-collect/app/data-collect-](#)

[domain/src/main/kotlin/com/cleyrop/data/collect/domain/datasource/query/DataSourceCheckQueryHandler.kt:38](#) Fusion des secrets et catégories d'erreur.

[monorepo · services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/controller/dto/connector/ConnectorDTO.kt:107](#) Masquage systématique des DTO sortants.

DC-06 **Cible non qualifiée**

Chiffrer et faire tourner les credentials de connexion

COMPORTEMENT OBSERVÉ

Les nouveaux credentials utilisent une enveloppe AES-GCM versionnée avec identifiant de génération, nonce aléatoire et données associées. Le décodeur sait encore lire l'ancien format CBC. Une génération précédente peut coexister avec l'actuelle ; la sélection de clé legacy devient explicite en présence des deux. La rotation vérifie même une enveloppe déjà courante. Le ré-encrypteur parcourt les champs de secrets et les entrées API headers/params/body dans le JSON persistant. Une donnée illisible lève une erreur contextualisée par chemin, sans la convertir en chaîne vide.

UTILISATEURS ET RÔLES

Opérateurs de secrets et migration de connexions.

INTERFACES DANS LE CODE

AttributeEncryptor.encoder/decoder/rotate ; DatasourceCredentialReEncryptor.reEncrypt ; runner de ré-encryption.

DONNÉES ET ÉTATS

Payload JSON datasource chiffré, générations actuelle et précédente ; l'enveloppe porte la version et la clé logique.

DÉPENDANCES ET ÉCHANGES

Secrets de déploiement ; partage historique de la clé avec data-dev signalé dans AGENTS.md.

COMPORTEMENT CIBLE PROPOSÉ

Connectors et IAM utilisent des références de secrets. Un module Rust de compatibilité déchiffre les enveloppes existantes, puis les migre vers un stockage de secrets avec génération, droits et journaux de rotation.

MIGRATION DE CETTE CAPACITÉ

Lire le guide de rotation avant toute opération. Migrer par lots vérifiables et ré-exécutables, conserver l'ancienne génération jusqu'à preuve que tous les lecteurs ont basculé, puis révoquer.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une enveloppe de chaque génération connue reste lisible pendant la transition. Une altération est rejetée. Relancer un lot ne modifie pas une enveloppe déjà validée et aucun journal ne contient le secret.

Références de l'analyse

[monorepo · services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/utils/AttributeEncryptor.kt:65](#) Formats de lecture et rotation authentifiée.

[monorepo · services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/utils/AttributeEncryptor.kt:128](#) Nonce, chiffrement GCM et données associées.

[monorepo · services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/utils/DatasourceCredentialReEncryptor.kt:41](#) Réécriture du payload et champs couverts.

DC-07 **Cible non qualifiée**

Découvrir et extraire les tables SQL

COMPORTEMENT OBSERVÉ

Le manager JDBC découvre schémas, tables et vues, normalise les types de table et conserve les UUID de descripteurs lors d'une redécouverte. Il applique des règles propres à PostgreSQL, MySQL, SQL Server, Oracle et Snowflake, y compris certaines vues matérialisées ou tables externes. Le preview lit un nombre limité de lignes. L'import lit un flux JDBC, écrit un fichier Parquet local compressé puis l'envoie au bucket d'import. Les comptes de lignes et colonnes sont recalculés séparément. BIGQUERY est déclaré mais sa sélection de driver est vide dans le code audité ; son fonctionnement ne peut pas être affirmé.

UTILISATEURS ET RÔLES

Analystes qui importent des tables, gestionnaires de connexions SQL et workers de collecte.

INTERFACES DANS LE CODE

GET /datasources/{id}/resources ; GET /datasources/{id}/datasets/query ;
JdbcConnectorManager.list/read/import.

DONNÉES ET ÉTATS

TableDescriptor avec table, schéma, type et identifiant stable ; schémas inférés ; Parquet temporaire et objet d'import.

DÉPENDANCES ET ÉCHANGES

Drivers JDBC et bibliothèques Parquet/Hadoop JVM ; bases externes ; stockage compatible S3.

COMPORTEMENT CIBLE PROPOSÉ

Connectors fournit des lecteurs Rust ou des tiers Go/C++ encapsulés et qualifiés par moteur. Writer transforme des lots bornés en fichiers immuables, sans droit REST Iceberg générique. Publisher publie les éditions TABLE/INCREMENTAL.

MIGRATION DE CETTE CAPACITÉ

Construire une matrice de qualification par moteur, type de colonne, pagination et stratégie snapshot. Remplacer les drivers JVM un par un ; ne pas déclarer BIGQUERY pris en charge sans un adaptateur exécutable.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Pour chaque moteur retenu, importer une table avec décimaux, valeurs nulles, fuseaux et noms de colonnes particuliers. Comparer lignes, types et valeurs à une extraction de référence et vérifier une reprise sans doublon.

Références de l'analyse

[monorepo](#) · [services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/manager/connector/JdbcConnectorManager.kt:103](#)

Découverte et conservation des descripteurs.

[monorepo](#) · [services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/manager/connector/JdbcConnectorManager.kt:630](#) Flux

JDBC, Parquet local puis S3.

[monorepo](#) · [services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/manager/connector/JdbcConnectorManager.kt:736](#)

Drivers associés aux types dont BIGQUERY vide.

DC-08 Cible non qualifiée

Valider une requête SQL personnalisée sur une table

COMPORTEMENT OBSERVÉ

Une configuration SQL peut fournir customQuery. Le validateur parse exactement une instruction SELECT simple. Il refuse les JOIN, les CTE et les sous-requêtes et exige que la table, et le schéma quand il est connu, correspondent au descripteur sélectionné. Le preview et la création appellent cette validation. Les colonnes peuvent être inférées à partir des métadonnées du résultat, puis l'import utilise la requête. Ce contrat n'est donc pas un moteur SQL général ni une garantie de lecture sans effet propre au compte distant.

UTILISATEURS ET RÔLES

Analystes et auteurs de configurations de collecte SQL.

INTERFACES DANS LE CODE

Champ customQuery des configurations SQL ; routes datasets/preview et datasets/headers ; CustomQueryValidator.validate.

DONNÉES ET ÉTATS

Requête rattachée au dataset et schéma calculé sur son résultat, avec descripteur de la table source.

DÉPENDANCES ET ÉCHANGES

JSQLParser, métadonnées JDBC et compte distant.

COMPORTEMENT CIBLE PROPOSÉ

Connectors possède un sous-langage de lecture explicite et validé. Les transformations SQL générales appartiennent à Flows/Query. Le compte externe est en lecture seule et Egress borne la durée et les volumes.

MIGRATION DE CETTE CAPACITÉ

Versionner la grammaire et reprendre les requêtes acceptées sans élargir silencieusement leurs droits. Prévoir une migration explicite vers Flows pour les demandes de jointure.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une projection filtrée de la table sélectionnée est acceptée. Une seconde instruction, un JOIN, une CTE ou une table différente est refusé avant toute exécution distante.

Références de l'analyse

[monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/CustomQueryValidatorImpl.kt:21](#) Validation syntaxique et table cible.

[monorepo · services/data-collect/app/data-collect-domain/src/main/kotlin/com/cleyrop/data/collect/domain/dataset/query/PreviewDatasetConfigQueryHandler.kt:115](#) Utilisation du validateur dans le preview.

[monorepo · services/data-collect/app/data-collect-domain/src/main/kotlin/com/cleyrop/data/collect/domain/dataset/command/CreateCollectFromDatasourceCommandHandler.kt:287](#) Inférence du schéma de customQuery.

DC-09 **Cible non qualifiée**

Appeler des API avec authentification et reprises HTTP

COMPORTEMENT OBSERVÉ

Le connecteur API combine URL de base, endpoint, paramètres, en-têtes et méthode. L'authentification peut être absente, Basic ou fondée sur un endpoint de jeton. Ce dernier accepte GET/POST et injecte le token dans un header ou un paramètre selon un motif ; le cache tient compte d'une durée de vie fournie. Le transport distingue refus d'identité, refus d'accès, absence d'endpoint, timeout et autres statuts. Il réessaie selon un budget de temps et une politique de backoff, interprète Retry-After et peut invalider un token sur 401. Les téléchargements consomment un flux et suppriment un fichier partiel lors d'un échec.

UTILISATEURS ET RÔLES

Gestionnaires de connexions HTTP et collectes de données SaaS.

INTERFACES DANS LE CODE

APIConnectorManager ; ApiAuthenticator ; HttpGateway.exchange/download ; routes de test et preview.

DONNÉES ET ÉTATS

Définition de connexion, configuration par dataset, cache de jetons et fichier temporaire pour les octets.

DÉPENDANCES ET ÉCHANGES

API distante, endpoint de token et client HTTP mutualisé.

COMPORTEMENT CIBLE PROPOSÉ

Connectors implémente les mêmes modes avec références de secrets, requêtes bornées et catégories d'erreur stables. Egress autorise URL, redirections et hôtes de token avant chaque connexion.

MIGRATION DE CETTE CAPACITÉ

Conserver les options effectivement utilisées et établir des tests de contrat par méthode et fournisseur. Séparer les paramètres secrets des paramètres de données pour le masquage et l'audit.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une réponse 401 provoque au plus le renouvellement prévu, un 429 respecte son délai dans le budget, et un flux interrompu ne laisse aucun fichier réutilisable comme un résultat complet.

Références de l'analyse

[monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/manager/connector/api/auth/ApiAuthenticator.kt:55](#)
Modes d'authentification.

[monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/manager/connector/api/auth/ApiAuthenticator.kt:159](#)
Endpoint token et durée de vie.

[monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/manager/connector/api/http/RestClientHttpGateway.kt:61](#)
Flux et enveloppe de reprise.

DC-10 Cible non qualifiée

Parcourir les pages API sans perdre le contrat de source

COMPORTEMENT OBSERVÉ

Le résolveur propose absence de pagination, numéros de page, offset, curseur, lien de corps, Link HTTP et OData. SAP OData v2 possède sa stratégie propre ; les autres usages OData passent par l'autre implémentation. L'URL effective provient de la configuration du dataset, du template versionné ou de la

source. Le résolveur de liens accepte aussi des liens absolus. Il faut donc distinguer les protections de certaines découvertes fournisseur du contrôle général des destinations, qui n'est pas complet dans le code lu.

UTILISATEURS ET RÔLES

Collectes HTTP volumineuses, créateurs de connecteurs et opérateurs de reprise.

INTERFACES DANS LE CODE

PaginationConfig ; PaginationStrategyResolver.resolve ; stratégies PageBased, OffsetBased, CursorBased, LinkBased, HeaderLinkBased, OData et ODataV2.

DONNÉES ET ÉTATS

Paramètres de pagination dans ConfigConnector ; pages assemblées en fichier local de collecte.

DÉPENDANCES ET ÉCHANGES

API fournisseur, headers de réponse et curseurs distants.

COMPORTEMENT CIBLE PROPOSÉ

Connectors décrit un checkpoint source/epoch/range lié au run. Chaque run borne pages, octets et temps. Publisher publie l'édition et son checkpoint avant l'acquittement ; un curseur de commande PostgreSQL ne devient jamais le checkpoint publié autoritaire.

MIGRATION DE CETTE CAPACITÉ

Rejouer les configurations de pagination existantes avec fixtures déterministes. Ajouter la détection de boucle, une limite de pages et une politique explicite des liens absolus et changements d'origine.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un lien suivant cyclique interrompt la collecte avec une erreur exploitable. Une reprise après publication reconnaît le checkpoint publié ; une interruption avant publication ne fait pas avancer la source.

Références de l'analyse

monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/manager/connector/api/pagination/PaginationStrategyResolver.kt:40 Types de pagination et URL effective.

monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/manager/connector/api/pagination/util/UriResolver.kt:18 Acceptation des liens absolus.

DC-11 Cible non qualifiée

Découvrir des API métier et figer leur version

COMPORTEMENT OBSERVÉ

Salesforce, Business Central, Dynamics, SAP et SharePoint ont des versions connues et des templates dédiés. Les routes servent versions, parents, entités et colonnes. Les parents représentent notamment des sociétés ou services selon le fournisseur. La création et certains previews figent la version de lecture. Une incompatibilité fournisseur/configuration typée lève une erreur. Le catalogue seul ne prouve pas l'activation : CreateDataSourceCommandHandler interdit MS_DYNAMICS et SAP_S4HANA. Jira utilise le chemin API générique avec Basic à partir de ses champs ; CLEYROP accepte un token historique ou client_credentials.

UTILISATEURS ET RÔLES

Créateurs de sources SaaS et auteurs de configurations de lecture.

INTERFACES DANS LE CODE

GET /connectors/{connectorType}/versions ; GET /datasources/{id}/parents ; GET /datasources/{id}/entities ; GET /datasources/{id}/entities/{entityName}/columns.

DONNÉES ET ÉTATS

Version API de connexion et de lecture ; noms d'entités, parents et colonnes ; configurations typées propres à chaque fournisseur.

DÉPENDANCES ET ÉCHANGES

Templates versionnés, API de découverte, token endpoints et parseur EDMX.

COMPORTEMENT CIBLE PROPOSÉ

Connectors expose un registre versionné de capacités, avec état implémenté/activé/qualifié et contraintes fournisseur. Le domaine Rust conserve des contrats de lecture distincts de la découverte.

MIGRATION DE CETTE CAPACITÉ

Reprendre chaque configuration avec sa version explicite. Maintenir les connecteurs interdits hors du catalogue utilisable tant qu'un ticket de qualification ne les rouvre. Ne pas transformer le support de DTO en promesse produit.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une source Salesforce choisie dans une version reconnue conserve cette version après changement du défaut du registre. Une configuration SAP ne peut pas s'exécuter sur un connecteur d'un autre fournisseur.

Références de l'analyse

`monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/controller/DataSourceDataController.kt:118` Routes de découverte et de versions.

`monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/manager/connector/api/version/ConnectorVersionTemplateRegistry.kt:67` Templates par fournisseur/version.

`monorepo · services/data-collect/app/data-collect-domain/src/main/kotlin/com/cleyrop/data/collect/domain/datasource/command/CreateDataSourceCommandHandler.kt:72` Interdiction de création Dynamics/SAP.

`monorepo · services/data-collect/app/data-collect-domain/src/main/kotlin/com/cleyrop/data/collect/domain/datasource/aggregate/connector/CleyropConnector.kt:75` Mode service account CLEYROP.

DC-12 **Cible non qualifiée****Parcourir et lire des documents SharePoint****COMPORTEMENT OBSERVÉ**

SharePoint utilise OAuth client_credentials et un template Graph v1.0. Le résolveur découvre les drives du site. La stratégie fichier distingue la navigation des dossiers, les métadonnées et le téléchargement binaire ; elle fournit des informations de modification et de validation telles que quickXorHash/cTag. Les nextLink Graph doivent rester sous la racine configurée. Les URL de téléchargement pré-signées passent dans un transport séparé sans en-têtes d'authentification de l'API. Les mêmes documents peuvent servir à une collecte tabulaire ou au contenu de corpus IA.

UTILISATEURS ET RÔLES

Gestionnaires de documents, collecte et ingestion de corpus.

INTERFACES DANS LE CODE

Routes de listing et de contenu ingestion ; SharepointV1FileStrategy ; SharepointV1DriveResolver.

DONNÉES ET ÉTATS

Drives et chemins SharePoint, métadonnées enrichies, génération de listing et fichier téléchargé temporaire.

DÉPENDANCES ET ÉCHANGES

Microsoft Graph, fournisseur OAuth et hôte de téléchargement pré-signé.

COMPORTEMENT CIBLE PROPOSÉ

Connectors fournit un adaptateur SharePoint Rust avec policy Egress distincte pour Graph et les URL de contenu. Files produit un FILESET immuable ou Writer une TABLE selon le parcours ; Embeddings épingle le FILESET publié.

MIGRATION DE CETTE CAPACITÉ

Conserver les identifiants de drives, les chemins et les validateurs. Encapsuler les credentials et qualifier les renommages, suppressions, permissions retirées et téléchargements expirés.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un document non modifié évite son transfert ; un document changé pendant le téléchargement porte l'empreinte des octets servis. Une redirection vers une destination non autorisée échoue sans transmettre de token Graph.

Références de l'analyse

`monorepo · services/data-collect/app/data-collect-domain/src/main/kotlin/com/cleyrop/data/collect/domain/datasource/aggregate/connector/SharepointConnector.kt:36`
Authentification et version Graph.

`monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/manager/connector/api/version/sharepoint/v1/SharepointV1DriveResolver.kt:20` Découverte des drives.

`monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/manager/connector/api/version/sharepoint/v1/SharepointV1FileStrategy.kt:271` Contrôle de racine des nextLink.

`monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/manager/connector/APIConnectorManager.kt:606`
Téléchargement pré-signé sans headers.

DC-13 **Cible non qualifiée****Collecter les fichiers SFTP, dépôt et S3****COMPORTEMENT OBSERVÉ**

La factory instancie des managers fichier dédiés à SFTP/dépôt et S3. SFTP emploie une clé du service et sa configuration de source ; dépôt partage ce transport. S3 porte endpoint, région, bucket, préfixe et credentials. Les managers exposent découverte, lecture, enrichissement des métadonnées et import. Le listing de navigation traite un niveau de dossier et peut résoudre un chemin de fichier. Les descripteurs sont réconciliés pour conserver leurs identifiants et états. La configuration de source garde une règle de sélection des fichiers distincte du format de lecture.

UTILISATEURS ET RÔLES

Producteurs de fichiers, partenaires SFTP et analystes.

INTERFACES DANS LE CODE

POST /datasources/{id}/files ; SftpConnectorManager ; S3ConnectorManager ; FileConnectorManager.

DONNÉES ET ÉTATS

FileDescriptor, chemin relatif, taille, date, statut, checksum ; ressources persistées et cache temporaire distant.

DÉPENDANCES ET ÉCHANGES

Serveurs SFTP, S3 compatible, stockage import interne et clé SSH de service.

COMPORTEMENT CIBLE PROPOSÉ

Connectors implémente la découverte et la lecture bornée ; Files conserve des objets immuables. Une collecte ne supprime ni ne renomme un fichier externe avant que l'édition gouvernée et son checkpoint soient publiés.

MIGRATION DE CETTE CAPACITÉ

Qualifier séparément les chemins, encodages, host keys SSH, S3 compatible et la reprise d'un transfert. Remplacer les connexions en cache propres au processus par des workers fongibles capables de reprendre depuis un journal.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une interruption au milieu d'un fichier ne le marque pas traité. Un fichier externe renommé ou modifié ne réutilise pas un résultat audité appartenant à d'autres octets. Le worker de remplacement termine avec le même identifiant de run.

Références de l'analyse

[monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/ConnectorFactoryImpl.kt:189](#) Dispatch des managers fichiers.

[monorepo · services/data-collect/app/data-collect-domain/src/main/kotlin/com/cleyrop/data/collect/domain/datasource/aggregate/connector/S3Connector.kt:6](#) Paramètres de source S3.

[monorepo · services/data-collect/app/data-collect-domain/src/main/kotlin/com/cleyrop/data/collect/domain/datasource/aggregate/connector/Connector.kt:195](#) Réconciliation des descripteurs fichiers.

[monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/manager/connector/SftpConnectorManager.kt:274](#) Navigation au niveau de dossier.

DC-14 **Cible non qualifiée****Téléverser un fichier local par morceaux****COMPORTEMENT OBSERVÉ**

Le endpoint multipart reçoit un morceau et les positions Content-Range-Start, End et Max. Le service place les octets dans un répertoire temporaire propre à la source avec nom normalisé. Il exige que la longueur actuelle corresponde au début du morceau et que la taille après ajout soit strictement inférieure à la limite. Le descripteur n'est produit qu'à la fin. Une suppression retire un fichier ou tous les fichiers de la source. Un contrôle asynchrone compare ensuite le fichier au schéma du premier dataset associé et émet un message de conflit, de changement de schéma ou de succès.

UTILISATEURS ET RÔLES

Utilisateurs du frontend important un fichier local.

INTERFACES DANS LE CODE

POST /datasources/{id}/uploadFiles ; DELETE /datasources/{id}/files ; CheckFileAsynchCommandHandler.

DONNÉES ET ÉTATS

Fichier partiel dans le stockage temporaire, puis FileDescriptor persistant ; positions des morceaux fournies par le client.

DÉPENDANCES ET ÉCHANGES

Volume temporaire du pod ; normalisation de nom ; notification Socket.IO via événements de contrôle fichier.

COMPORTEMENT CIBLE PROPOSÉ

Files possède une session d'upload durable avec identifiant, taille attendue, empreintes des morceaux, reprise et finalisation atomique. Connectors/Writer consomment seulement un objet finalisé immuable.

MIGRATION DE CETTE CAPACITÉ

Conserver une façade compatible pour le frontend, mais enregistrer les sessions dans le plan de contrôle et les morceaux dans le stockage objet. Ne plus dépendre du pod initial pour terminer le transfert.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un morceau manquant ou répété avec d'autres octets est refusé sans corruption. Après remplacement du worker, l'upload reprend. Le résultat final porte une empreinte et le contrôle asynchrone correspond à la bonne session.

Références de l'analyse

[monorepo · services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/controller/DataSourceLocalFileController.kt:32](#) En-têtes et suppression des uploads.

[monorepo · services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/ChunkedFileUploadServiceImpl.kt:94](#) Append, ordre et limite stricte.

[monorepo · services/data-collect/app/data-collect-](#)

[domain/src/main/kotlin/com/cleyrop/data/collect/domain/datasource/command/CheckFileAsynchCommandHandler.kt:149](#)

Contrôle du premier dataset associé.

DC-15 **Cible non qualifiée****Paginer une arborescence et mettre en cache sa génération****COMPORTEMENT OBSERVÉ**

La navigation se distingue de l'inventaire utilisé pour la lecture d'un tableur. La requête valide skip non négatif et limit positif. La navigation génère un scope par source, chemin normalisé et filtres. Le cache PostgreSQL stocke une génération ordonnée, un total et des métadonnées enrichies. Le rafraîchissement prend un verrou de scope et remplace la génération ; les pages non initiales réutilisent la génération courante. Une tâche supprime les générations expirées/supplantées avec délai de garde. Les sources LOCAL n'acceptent pas le parcours générique de navigation sans format.

UTILISATEURS ET RÔLES

Explorateurs de fichiers, frontend et ingestion de corpus.

INTERFACES DANS LE CODE

POST /datasources/{id}/files ; RequestFilesDTO ; FolderListingService.serveFromCache/listFromSource/sweep.

DONNÉES ET ÉTATS

FolderListingScope, génération, entrées ordinales, total, expiration et remplacement ; cache distinct des descripteurs métier.

DÉPENDANCES ET ÉCHANGES

PostgreSQL pour cache/verrou et manager du connecteur pour parcourir le dossier.

COMPORTEMENT CIBLE PROPOSÉ

Connectors sert des pages attachées à un identifiant explicite de génération et un budget d'entrées. Les clients peuvent terminer une génération même si une autre a été reconstruite.

MIGRATION DE CETTE CAPACITÉ

Préserver tri, total et enrichissements, puis remplacer skip/limit par un curseur versionné sans ambiguïté. Documenter qu'un listing n'est pas un snapshot transactionnel de la source distante.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Le lecteur qui commence une pagination puis observe un rafraîchissement concurrent ne mélange pas deux générations. Un dossier dépassant le budget retourne une erreur définie et ne bloque pas les autres sources.

Références de l'analyse

[monorepo · services/data-collect/app/data-collect-](#)

[domain/src/main/kotlin/com/cleyrop/data/collect/domain/datasource/query/DatasourceListResourcesQueryHandler.kt:95](#)

Séparation navigation/tableur et validation de type.

[monorepo · services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/listing/FolderListingService.kt:92](#) Scope, pages et rafraîchissement.

[monorepo · services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/listing/FolderListingService.kt:146](#) Verrou et remplacement de génération.

DC-16 **Cible non qualifiée****Lire les formats et appliquer les préparations d'import****COMPORTEMENT OBSERVÉ**

La factory couvre CSV, tableur XLSX, JSON, XML et Parquet. Les DTO configurent encodage, suffixes/MIME, limite de preview et, selon le format, compression, séparateur CSV, feuille, présence d'en-tête, lignes et colonnes ignorées, trim ou chemin JSON/XML. Les préparations comprennent transposition et ajout de nom de fichier, URL, date ou horodatage d'import. Le domaine expose NONE et GZIP pour la compression.

withStandardizeColumns apparaît dans le DTO mais n'est pas converti en préparation dans AdvancedOptionsDTO.toPreparations. Les valeurs par défaut MIME/suffixes de Parquet/XML pointent vers la configuration JSON dans les DTO lus.

UTILISATEURS ET RÔLES

Analystes configurant une collecte et lecteurs de fichiers.

INTERFACES DANS LE CODE

FormatConfigDTO et ses variantes ; GET /datasources/format/default ; AdvancedOptionsDTO ; FormatFactoryImpl.

DONNÉES ET ÉTATS

FileFormat, ConfigConnector, Preparations ordonnées et TargetSchema.

DÉPENDANCES ET ÉCHANGES

Parseurs CSV, XLSX, JSON, XML, Parquet ; cache de fichiers et détection de types.

COMPORTEMENT CIBLE PROPOSÉ

Writer et Connectors partagent des codecs Rust bornés en mémoire et en octets décompressés. La configuration de lecture et les préparations deviennent une révision immuable liée à l'audit du candidat.

MIGRATION DE CETTE CAPACITÉ

Constituer des fixtures par format et option, y compris fichiers malformés et limites de décompression. Décider explicitement du comportement de standardisation et corriger les defaults suspects après mesure de compatibilité.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Chaque fixture produit les mêmes colonnes et valeurs utiles dans preview et import. Un séparateur invalide, une feuille absente, un chemin JSON invalide et une expansion démesurée échouent avec des erreurs stables.

Références de l'analyse

monorepo · services/data-collect/app/data-collect-

app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/FormatFactoryImpl.kt:19 Formats réellement instanciés.

monorepo · services/data-collect/app/data-collect-

app/src/main/kotlin/com/cleyrop/data/collect/app/dataset/controller/dto/AdvancedOptionsDTO.kt:11 Options et conversion des préparations.

monorepo · services/data-collect/app/data-collect-

app/src/main/kotlin/com/cleyrop/data/collect/app/dataset/controller/dto/format/ParquetConfigDTO.kt:17 Defaults de format Parquet.

monorepo · services/data-collect/app/data-collect-

domain/src/main/kotlin/com/cleyrop/data/collect/domain/datasource/aggregate/fileFormat/FileFormat.kt:15 Compression déclarée.

DC-17 **Cible non qualifiée****Prévisualiser les données et contrôler les colonnes****COMPORTEMENT OBSERVÉ**

Le preview distingue SQL, fichier et API. SQL redécouvre la table, valide une éventuelle customQuery et applique les préparations. Le fichier se résout par chemin de référence, peut imposer un schéma et fournir un arbre JSON. L'API actualise ses descripteurs puis renvoie frame, arbre JSON et chemin de base. Le endpoint headers accepte SQL ou API, avec métadonnées SQL ou arbre JSON. Le contrôle d'en-têtes rejette les doublons sans tenir compte de la casse, les noms trop longs, les types incompatibles et les erreurs de lecture ; il renvoie un RestError en 400.

UTILISATEURS ET RÔLES

Analystes pendant la configuration d'un dataset.

INTERFACES DANS LE CODE

POST /datasources/{id}/datasets/preview ; POST /datasources/{id}/datasets/headers ; POST /datasources/{id}/datasets/check-headers ; POST /datasources/{id}/datasets/sheetNames.

DONNÉES ET ÉTATS

DataFrame, TabularSchema, JsonDatasetHeaders, préparations et descripteurs actualisés lors de certains previews.

DÉPENDANCES ET ÉCHANGES

Managers et parseurs ; source externe ; paramètres de limite d'en-tête et de transposition.

COMPORTEMENT CIBLE PROPOSÉ

Connectors exécute des previews bornés et Writer utilise les mêmes conversions. Datasets conserve le schéma proposé et les écarts, sans confondre un échantillon valide avec la validation du candidat complet.

MIGRATION DE CETTE CAPACITÉ

Préserver les discriminants de réponse SQL/JSON et les noms d'erreurs frontend. Séparer l'inférence d'échantillon de la validation de publication.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un échantillon avec colonnes A et a échoue comme doublon. Un type compatible dans le preview mais invalide plus loin dans le fichier empêche la publication et laisse un rapport lié au candidat.

Références de l'analyse

monorepo · services/data-collect/app/data-collect-

domain/src/main/kotlin/com/cleyrop/data/collect/domain/dataset/query/PreviewDatasetConfigQueryHandler.kt:75 Trois parcours de preview.

monorepo · services/data-collect/app/data-collect-

domain/src/main/kotlin/com/cleyrop/data/collect/domain/dataset/query/HeadersDatasetConfigQueryHandler.kt:47 Headers SQL ou API seulement.

monorepo · services/data-collect/app/data-collect-

domain/src/main/kotlin/com/cleyrop/data/collect/domain/dataset/query/CheckTargetSchemaQueryHandler.kt:35 Ordre des contrôles et erreurs.

monorepo · services/data-collect/app/data-collect-

app/src/main/kotlin/com/cleyrop/data/collect/app/dataset/controller/DatasetConfigController.kt:113 Traduction HTTP du contrôle d'en-tête.

DC-18 **Cible non qualifiée****Créer une collecte et suivre son exécution distribuée****COMPOTEMENT OBSERVÉ**

La création de collecte arrive par les contrats internes. Elle enregistre d'abord un DatasetCollect avec des opérations WAITING, puis construit la configuration fichier, API ou SQL. Elle infère le schéma si nécessaire et planifie un import asynchrone, sauf en mode PLANNED. Les empreintes initiales persistées sont vidées tant que l'import n'est pas acquis. L'import met l'ingestion en RUNNING, prépare les octets et publie CollectUpdatedEvent pour la suite du traitement. La fin d'exécution reçue depuis process-data déclenche la finalisation et, en échec, la notification de collecte.

UTILISATEURS ET RÔLES

Auteurs de datasets, commandes data-dev, data-model et process-data.

INTERFACES DANS LE CODE

CreateCollectFromDatasourceCommandHandler ; CollectInitCommandHandler ; CollectUpdatedEvent ; ProcessDataEventHandlerKafkaConsumer.

DONNÉES ET ÉTATS

DatasetCollect, ancienne configuration, schéma cible, CollectScheduled et états des opérations ingestion/process.

DÉPENDANCES ET ÉCHANGES

Kafka/Protobuf, data-dev, data-model, process-data, stockage import et notification-center.

COMPOTEMENT CIBLE PROPOSÉ

Flows ordonne un run borné ; Connectors lit, Writer prépare un candidat immuable, Audit le vérifie et Publisher publie. Les événements d'avancement ne remplacent pas le reçu de publication. Les sorties sont une édition Iceberg autoritaire.

MIGRATION DE CETTE CAPACITÉ

Reconstituer la machine d'états et ses transitions avant de déplacer les producteurs. Maintenir un adaptateur événementiel temporaire, puis remplacer la chaîne JVM par des commandes et reçus Rust idempotents.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une panne entre upload et lancement du traitement laisse un run reprenable, sans fausse réussite. Une panne après publication et avant ack reconnaît la même édition et n'en publie pas une seconde.

Références de l'analyse

monorepo · services/data-collect/app/data-collect-domain/src/main/kotlin/com/cleyrop/data/collect/domain/dataset/command/CreateCollectFromDatasourceCommandHandler.kt:3
65 Enregistrement initial WAITING et préparation.

monorepo · services/data-collect/app/data-collect-domain/src/main/kotlin/com/cleyrop/data/collect/domain/dataset/command/CreateCollectFromDatasourceCommandHandler.kt:4
08 Empreintes en attente et planification.

monorepo · services/data-collect/app/data-collect-domain/src/main/kotlin/com/cleyrop/data/collect/domain/dataset/command/CollectInitCommandHandler.kt:65 Import/finalisation des opérations.

monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/common/event/consumer/ProcessDataEventHandlerKafkaConsumer.kt:18
Retour de fin d'exécution.

DC-19 **Cible non qualifiée****Rafraîchir un dataset avec règles de schéma et de doublons****COMPORTEMENT OBSERVÉ**

Le rafraîchissement peut changer fréquence et type APPEND/OVERRIDE. Les fichiers ont contrôles de format, schéma et checksum ; APPEND peut importer les fichiers valides en émettant WARNING si certains sont ignorés, alors que OVERRIDE échoue si un fichier a un schéma invalide. Les API JSON ne passent pas par le même contrôle de checksum. Le SQL relit les colonnes vivantes : STRICT_FIXED détecte colonnes absentes, nouvelles ou de type différent ; PERMISSIVE_FIXED tolère certaines différences et peut autoriser les casts implicites ; MERGE et REPLACE ne produisent pas ces conflits préalables. Une source vide ou déjà traitée possède une issue distincte.

UTILISATEURS ET RÔLES

Utilisateurs du rafraîchissement manuel et déclencheurs de collecte.

INTERFACES DANS LE CODE

POST /datasets/{id}/refresh ; UpdateCollectCommandHandler ; TriggerOperationCommandHandlerUpdate.

DONNÉES ET ÉTATS

RefreshType, SchemaMode, sourceConfig courante et ancienne, checksums et états
WARNING/FAILURE/SKIPPED.

DÉPENDANCES ET ÉCHANGES

Data-model pour les changements de schéma et datasets vidés ; process-data pour matérialisation ; notifications de conflit.

COMPORTEMENT CIBLE PROPOSÉ

Datasets versionne le contrat de schéma ; Writer produit un candidat APPEND ou remplacement sous WAP. Publisher avance la référence publiée seulement après Audit. Les empreintes deviennent des preuves sur octets immuables et les checkpoints publiés décrivent les entrées acquises.

MIGRATION DE CETTE CAPACITÉ

Reprendre les quatre modes de schéma avec une table de compatibilité explicite. Remplacer la notion de fichier traité stockée dans une ancienne config par des participants/checkpoints de publication, en conservant l'historique de provenance.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une collecte APPEND contenant un fichier déjà traité et un fichier neuf n'ajoute que le second. Une incompatibilité STRICT_FIXED bloque la publication ; une évolution MERGE suit la politique de gouvernance et garde l'édition précédente lisible.

Références de l'analyse

[monorepo · services/data-collect/app/data-collect-](#)

[domain/src/main/kotlin/com/cleyrop/data/collect/domain/dataset/command/TriggerOperationCommandHandler.kt:289](#) Contrôles fichiers et exception du parcours API.

[monorepo · services/data-collect/app/data-collect-](#)

[domain/src/main/kotlin/com/cleyrop/data/collect/domain/dataset/command/TriggerOperationCommandHandler.kt:316](#) Quatre modes de schéma SQL.

[monorepo · services/data-collect/app/data-collect-](#)

[domain/src/main/kotlin/com/cleyrop/data/collect/domain/dataset/command/TriggerOperationCommandHandler.kt:418](#) Comportements APPEND et OVERRIDE.

[monorepo · services/data-collect/app/data-collect-](#)

[domain/src/main/kotlin/com/cleyrop/data/collect/domain/dataset/aggregate/IngestionStatus.kt:24](#) Issues réglées, vide et déjà traité.

DC-20 **Cible non qualifiée**

Planifier les collectes et maîtriser leurs tentatives**COMPOTEMENT OBSERVÉ**

Les fréquences sont MANUAL, PLANNED et RECURRENT ; SchedulerCollect porte cron, fuseau et politique de reprise. Quartz utilise un store JDBC clusterisé et un trigger de retry persistant propre au service. Les opérations ont un UUID différent de celui du dataset, demandent la récupération après panne et exécutent un job non concurrent pour la même clé. Le résultat décide d'arrêter le trigger, de réessayer ou de finaliser à épuisement. Une garde d'admission en mémoire évite pendant cinq secondes deux soumissions du même dataset, sans verrou partagé entre pods. Une nouvelle demande annule les opérations pendantes du dataset.

UTILISATEURS ET RÔLES

Utilisateurs de schedules, déclencheurs et opérateurs.

INTERFACES DANS LE CODE

Événements ScheduleEvent ; SchedulerServiceQuartzImpl ; DatasetScheduledJob ; DatasetServiceAsyncJob ; IngestionRetryTrigger.

DONNÉES ET ÉTATS

CollectScheduled et tables Quartz PostgreSQL ; cron/fuseau/politique ; compteur de tentatives persistant ; garde locale éphémère.

DÉPENDANCES ET ÉCHANGES

PostgreSQL, Quartz, data-model et planification Spring au démarrage.

COMPOTEMENT CIBLE PROPOSÉ

Flows conserve en PostgreSQL les schedules et commandes de run, avec lease, deadline, retry budget et fencing applicatif. Les workers restent fongibles. Une occurrence planifiée a une clé idempotente distincte de l'édition publiée.

MIGRATION DE CETTE CAPACITÉ

Exporter les schedules et leurs fuseaux, comparer les prochaines occurrences, drainer les tentatives Quartz puis activer le scheduler Rust sur une frontière de temps. Ne pas compter la garde locale comme garantie de concurrence cible.

CRITÈRE D'ACCEPTATION À EXÉCUTER

À deux schedulers concurrents, une seule commande existe pour une occurrence. Un worker mort peut être remplacé sans double publication. Les changements d'heure et les retries conservent le nombre et l'ordre d'occurrences attendus.

Références de l'analyse

[monorepo · services/data-collect/app/data-collect-](#)

[domain/src/main/kotlin/com/cleyrop/data/collect/domain/dataset/aggregate/DatasetCollectRefreshConfig.kt:10](#) Fréquences, cron, fuseau et reprise.

[monorepo · services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/scheduler/service/SchedulerServiceQuartzImpl.kt:119](#) Opérations et annulation des tentatives pendantes.

[monorepo · services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/scheduler/service/quartzjob/DatasetServiceAsyncJob.kt:20](#) Décision de reprise/finalisation.

[monorepo · services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/dataset/service/InMemoryCollectAdmissionGuard.kt:24](#) Debounce local au processus.

[monorepo · services/data-collect/app/data-collect-app/src/main/resources/application.properties:251](#) Store Quartz JDBC clusterisé.

DC-21 **Cible non qualifiée****Préparer les objets d'import et les nettoyer après exécution****COMPOTEMENT OBSERVÉ**

Les imports s'écrivent dans un bucket compatible S3 sous un nom d'objet de dataset. Le writer courant supprime le BOM UTF-8 quand il est présent. À la fin de la collecte, le nettoyage parcourt les objets du préfixe et supprime ceux antérieurs au seuil temporel ; il garde ceux déposés après ce seuil pour éviter d'effacer une nouvelle tentative. Les fichiers locaux d'une source peuvent aussi être retirés. Le nettoyage est asynchrone et journalise les échecs sans mécanisme de publication Iceberg dans ce writer.

UTILISATEURS ET RÔLES

Workers de collecte et maintenance des imports temporaires.

INTERFACES DANS LE CODE

`S3ObjectWriter.import/deleteAllForDataset` ; `ConnectorFactoryImpl.deleteImport` ; `CollectFinishCommandHandler`.

DONNÉES ET ÉTATS

Objets temporaires S3 et fichiers locaux ; cutoff stagedBefore et configurations de collecte.

DÉPENDANCES ET ÉCHANGES

Bucket d'import, API S3 et retour process-data.

COMPOTEMENT CIBLE PROPOSÉ

Writer de confiance crée uniquement des fichiers immuables et médie le staging. Publisher médie les commits et la publication. Le worker n'obtient pas `table.modify REST`, qui inclut aussi `drop`. Audit lie sa preuve au candidat sans réécrire `origin.*` par `fast-forward`. Maintenance supprime sur plan audité avec garde de rétention.

MIGRATION DE CETTE CAPACITÉ

Remplacer les préfixes partagés et cutoffs temporels par des identifiants `run/attempt`, `manifests` et `reçus`. Conserver l'ancien nettoyage pour ses objets `legacy` seulement jusqu'au drainage.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une ancienne tentative terminée ne peut effacer les fichiers d'une nouvelle. Une preuve d'audit reste liée aux mêmes octets. Un worker ne peut pas commit/drop une table via le catalogue ; seul Publisher peut rendre l'édition visible.

Références de l'analyse

monorepo · services/data-collect/app/data-collect-

app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/repository/S3ObjectWriter.kt:80 Écriture et BOM.

monorepo · services/data-collect/app/data-collect-

app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/repository/S3ObjectWriter.kt:121 Suppression par préfixe et ancienneté.

monorepo · services/data-collect/app/data-collect-

domain/src/main/kotlin/com/cleyrop/data/collect/domain/dataset/command/CollectFinishCommandHandler.kt:104 Persistance et déclenchement de nettoyage à la fin.

DC-22 **Cible non qualifiée**

Fournir les fichiers à une ingestion de corpus IA**COMPOTEMENT OBSERVÉ**

Deux routes spécialisées listent les fichiers et téléchargent un fichier au nom d'un utilisateur et d'un projet. Elles appellent un contrôle IAM résiduel avant tout accès aux octets. Le téléchargement exige un connecteur FILE et un manager FileContentFetcher. Il consulte les métadonnées avant de décider un 304 sur If-None-Match. Sinon il télécharge dans un fichier temporaire, calcule quickXor sur les octets et relit les métadonnées si le validateur annoncé ne correspond pas. La réponse expose contenu, taille, MIME, date, ETag et digest selon disponibilité.

UTILISATEURS ET RÔLES

ai-gen-assistant en délégation d'un utilisateur, projets et ingestion de documents.

INTERFACES DANS LE CODE

POST /datasources/{id}/ingestion/files?userId=...&projectId=... ; GET /datasources/{id}/ingestion/files/content avec path et If-None-Match.

DONNÉES ET ÉTATS

Métadonnées de source, validateur conditionnel, fichier temporaire et empreinte réelle des octets servis.

DÉPENDANCES ET ÉCHANGES

IAM on-behalf-of, source S3/SFTP/SharePoint selon capacité du manager, corpus IA consommateur.

COMPOTEMENT CIBLE PROPOSÉ

Connectors et Files servent une délégation bornée à une ressource et un run. Le corpus référence une édition FILESET publiée, puis Embeddings une édition EMBEDDINGS avec provenance vers ses fichiers. Les credentials ne traversent pas vers l'assistant.

MIGRATION DE CETTE CAPACITÉ

Conserver le protocole conditionnel pour réduire les transferts, mais rattacher chaque document et son digest au manifest immuable. Passer les paramètres de délégation dans une autorisation signée vérifiée par IAM.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un utilisateur retiré du projet ne peut plus lister ni lire les octets par le service IA. Un fichier changé pendant le transfert ne reçoit pas un ETag trompeur ; la provenance des embeddings pointe sur les octets réellement ingérés.

Références de l'analyse

`monorepo · services/data-collect/app/data-collect-`

`app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/controller/DataSourceIngestionController.kt:45` Routes de délégation et contrôle préalable.

`monorepo · services/data-collect/app/data-collect-`

`domain/src/main/kotlin/com/cleyrop/data/collect/domain/datasource/query/DatasourceFileContentQueryHandler.kt:78 304,` téléchargement et vérification des octets.

`monorepo · services/data-collect/app/data-collect-`

`app/src/main/kotlin/com/cleyrop/data/collect/app/common/authz/ResidualAuthzChecks.kt:35` Délégation vers le broker IAM.

DC-23 **Cible non qualifiée**

Administrer les accès de dépôt SFTP et leurs clés

COMPORTEMENT OBSERVÉ

Un accès de dépôt comporte nom de ressource, nom affiché, répertoire, login, propriétaire, statut, expiration et clés SSH. La création vérifie l'unicité login/répertoire, crée le compte par commande SSH distante, ajoute les clés puis persiste. Elle compense les effets partiels tant que la persistance complète n'a pas été atteinte. La modification diffère les clés à ajouter/retirer ; une liste vide n'entre pas dans ce parcours. La suppression retire d'abord le compte distant puis les enregistrements et publie l'événement. La création/suppression de dépôt entraîne ensuite celle de la datasource DEPOT.

UTILISATEURS ET RÔLES

Administrateurs et propriétaires des dépôts de partenaires.

INTERFACES DANS LE CODE

GET/POST /sftp-accesses ; GET/PATCH/DELETE /sftp-accesses/{id} ; GET /sftp-accesses/generate-directory-name ; GET /datasources/sftp/key.

DONNÉES ET ÉTATS

SftpAccess, SftpAccessAuthorization et compte distant ; faits IAM et datasource DEPOT dérivée.

DÉPENDANCES ET ÉCHANGES

Serveur SFTP administré par SSH et commandes système ; base de miroirs utilisateurs ; événements DepotEvent.

COMPORTEMENT CIBLE PROPOSÉ

Files administre les dépôts avec un adaptateur SFTP non JVM. Egress exécute les effets d'administration sous identifiant d'opération et IAM autorise les objets sftp_access. Un reçu permet de réconcilier l'état distant et PostgreSQL.

MIGRATION DE CETTE CAPACITÉ

Reprendre comptes, clés publiques et propriétaires sans les recréer. Remplacer la composition de commandes shell par un protocole d'administration structuré ou des arguments strictement validés. Documenter vider/révoquer toutes les clés.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une panne après création distante puis avant persistance se résout par reprise ou compensation traçable. Retirer toutes les clés a un comportement explicite. Supprimer un accès absent reste idempotent et ne supprime aucun autre compte.

Références de l'analyse

monorepo · services/data-collect/app/data-collect-

domain/src/main/kotlin/com/cleyrop/data/collect/domain/depot/command/CreateSftpAccessCommandHandler.kt:41 Unicité, effets distants et compensation.

monorepo · services/data-collect/app/data-collect-

domain/src/main/kotlin/com/cleyrop/data/collect/domain/depot/command/UpdateSftpAccessCommandHandler.kt:40 Diff des clés seulement si liste non vide.

monorepo · services/data-collect/app/data-collect-

app/src/main/kotlin/com/cleyrop/data/collect/app/depot/service/SftpServiceImpl.kt:31 Administration SSH distante.

monorepo · services/data-collect/app/data-collect-

app/src/main/kotlin/com/cleyrop/data/collect/app/common/event/consumer/DataCollectEventHandlerKafkaConsumer.kt:31 Dépôt vers datasource.

DC-24 **Cible non qualifiée**

Lister les dépôts et appliquer la rétention des fichiers

COMPORTEMENT OBSERVÉ

Le service liste les accès de dépôt, leurs fichiers et un KPI de total. La liste de fichiers enlève un répertoire protégé et restitue des chemins relatifs. La configuration conserve une durée de rétention commune, modifiable par la route d'administration. Un job cherche récursivement les fichiers plus anciens que le cutoff et les supprime sur le serveur. Le démarrage n'active ce job que si la fonction et un hôte SFTP sont configurés, et annule un ancien job sinon. L'expiration portée par un accès n'établit pas, dans ces handlers, une révocation automatique du compte distant.

UTILISATEURS ET RÔLES

Administrateurs de dépôts et opérateurs de rétention.

INTERFACES DANS LE CODE

GET /sftp-accesses/files ; GET/PATCH /sftp-accesses/configuration ; GET /sftp-accesses/kpi ; CleanDepotJob.

DONNÉES ET ÉTATS

SftpExpireTime global, horodatages distants et jobs de nettoyage ; champ expiredAt d'accès séparé.

DÉPENDANCES ET ÉCHANGES

SFTP/SSH et Quartz ; dossiers protégés configurés.

COMPORTEMENT CIBLE PROPOSÉ

Files et Maintenance appliquent une politique de rétention par tenant et dépôt. Chaque suppression est un effet d'un plan audité qui vérifie leases, rétention et éditions consommatrices. IAM définit le contrat d'expiration d'accès.

MIGRATION DE CETTE CAPACITÉ

Importer la durée actuelle et inventorier les dossiers protégés sans publier leurs valeurs. Remplacer le balayage destructif direct par des candidats de suppression et des reçus. Décider l'effet de expiredAt sur les comptes.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un fichier encore nécessaire à une collecte ou retenu par politique n'est pas supprimé. Le plan exclut les dossiers protégés et les chemins hors racine. La reprise d'un plan n'efface pas un fichier nouveau au même nom.

Références de l'analyse

monorepo · services/data-collect/app/data-collect-

app/src/main/kotlin/com/cleyrop/data/collect/app/depot/controller/SftpAccessController.kt:203 Listing, rétention et KPI.

[monorepo](#) · [services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/depot/service/SftpServiceImpl.kt:141](#) Suppression des fichiers anciens hors répertoire protégé.

[monorepo](#) · [services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/configuration/RunAfterStartup.kt:108](#) Activation conditionnelle et annulation du job.

DC-25 **Cible non qualifiée**

Converger les identités, transférer les propriétés et reconstruire les droits

COMPORTEMENT OBSERVÉ

Les consumers IAM maintiennent des correspondances IAM/ancien UUID de projet et IAM/subject utilisateur. Ils dédupliquent les événements, diffèrent les memberships dont l'utilisateur n'est pas encore mappé, puis les rejouent dans l'ordre lors de la convergence. La suppression désactive le miroir et retire les memberships ; l'offboarding réversible peut les conserver. La réaffectation des sources et datasets est derrière un flag distinct. Le endpoint reconcile-tuples réenonce les faits détenus par data-collect et retourne 202 si activé, 409 sinon. IAM reste le seul écrivain OpenFGA. Malgré la note générale de l'AGENTS, le code conserve des vérifications broker pour ingestion et SFTP pendant la transition.

UTILISATEURS ET RÔLES

IAM, procédures de départ utilisateur et opérateurs de bascule des droits.

INTERFACES DANS LE CODE

IamEventConsumer ; POST /admin/reconcile-tuples ; GET /users/{userId}/check-owner ; ResidualAuthzChecks.

DONNÉES ET ÉTATS

Miroirs User/Project, maps d'UUID, événements différés, processed events, propriétaires et authz facts/outbox.

DÉPENDANCES ET ÉCHANGES

IAM, ancien data-dev/Benutzer, APISIX et OpenFGA via IAM.

COMPORTEMENT CIBLE PROPOSÉ

IAM porte les identités canoniques et les droits. Connectors/Files réenoncent seulement leurs faits métier et appliquent les transferts idempotents. Audit conserve l'ancien et le nouveau propriétaire, le mode de départ et la commande.

MIGRATION DE CETTE CAPACITÉ

Drainer les maps et les memberships différés avant suppression des anciens topics. Comparer l'inventaire des propriétés et la réconciliation des faits ; retirer les doubles contrôles seulement avec le chemin réseau imposé.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un membership arrivé avant l'identité est appliqué une seule fois après mapping. Un départ réversible n'efface pas les liens à rétablir ; un départ définitif retire les droits et transfère toutes les propriétés autorisées.

Références de l'analyse

[monorepo](#) · [services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/common/event/consumer/IamEventConsumer.kt:278](#) Membres différés et résolution de clés.

[monorepo](#) · [services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/common/event/consumer/IamEventConsumer.kt:394](#) Départ utilisateur et transfert sous flag.

[monorepo](#) · [services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/admin/controller/AdminReconcileTuplesController.kt:60](#) 202/409 de réconciliation.

[monorepo](#) · [services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/common/authz/ResidualAuthzChecks.kt:23](#) Contrôles résiduels réellement présents.

DC-26 **Cible non qualifiée**

Encadrer le réseau sortant et remplacer les dépendances opérationnelles JVM

COMPORTEMENT OBSERVÉ

Le chart restreint l'entrée au gateway et aux pairs déclarés, mais laisse volontairement la sortie réseau non restreinte pour les connecteurs. La validation API contrôle schéma HTTP, ports et syntaxe de chemin/paramètres, sans validation d'IP privée, loopback ou re-résolution DNS dans ce validateur. Le client HTTP utilise des timeouts et masque certains headers ; une politique SSRF de bout en bout ne ressort pas des fichiers lus. Le déploiement utilise probes, PostgreSQL/Flyway, Kafka, volume temporaire optionnel et clés SSH ; le HPA n'est rendu que sans persistance. Ces éléments décrivent les manifests, pas l'état d'un cluster exécuté.

UTILISATEURS ET RÔLES

Opérateurs, sécurité des connecteurs et plateforme d'exécution.

INTERFACES DANS LE CODE

ApiRequestValidator.validateUrl ; RestTemplateConfig ; chart Deployment/NetworkPolicy/HPA ; consumer/producer Kafka.

DONNÉES ET ÉTATS

Configuration, fichiers temporaires, tables Flyway/Quartz et outbox ; connexions et caches en mémoire.

DÉPENDANCES ET ÉCHANGES

JVM/Spring, Kafka JVM actuel, drivers JDBC/Hadoop, PostgreSQL, S3, IAM et services externes.

COMPORTEMENT CIBLE PROPOSÉ

Egress applique destination, port, résolution DNS, redirection, secrets transmissibles, quotas et délai par run. Les cores Rust remplacent Spring/Quartz/JDBC et le bus final exclut Kafka JVM. Les workers n'ont aucune autorité générale de catalogue et doivent obtenir des accès bornés aux octets.

MIGRATION DE CETTE CAPACITÉ

Déployer le contrôle Egress avant ouverture de connecteurs nouveaux. Tester les destinations internes explicitement autorisées et bloquer les autres. Découpler API de contrôle, scheduling et workers de lecture ; migrer l'outbox avec consommateurs compatibles.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une URL de métadonnées cloud, un changement DNS vers loopback et une redirection vers un hôte non autorisé sont refusés avant transfert de secrets. Les redémarrages laissent les commandes récupérables et les probes distinguent dépendance indisponible et processus vivant.

Références de l'analyse

[monorepo](#) · [services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/manager/connector/api/request/ApiRequestValidator.kt:81](#) Validation syntaxique, ports et limites sans contrôle d'adresse.

[monorepo](#) · [services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/configuration/RestTemplateConfig.kt:61](#) Configuration du transport HTTP.

[monorepo](#) · [services/data-collect/chart/templates/networkpolicy.yaml:27](#) Absence volontaire de politique egress.

[monorepo](#) · [services/data-collect/chart/templates/hpa.yaml:1](#) Autoscaling conditionné par la persistance.

DC-27 **Cible non qualifiée**

Conserver les événements sortants et isoler les échecs de consommation

COMPOTEMENT OBSERVÉ

Les producteurs métier et de faits IAM déposent leurs payloads dans une outbox PostgreSQL. Le relay prend un lot verrouillé, attend la confirmation Kafka puis renseigne sentAt ; une erreur augmente attempts et conserve la ligne. Les événements historiques sans clé restent sans clé de partition, alors que les faits IAM peuvent être ordonnés par agrégat. Les lignes envoyées sont purgées après une rétention configurable. Les consommateurs rejettent les messages en échec vers une DLT ; le consommateur terminal persiste payload, coordonnées originales et erreur en PostgreSQL, puis laisse l'offset non acquitté si cette persistance échoue. Les enregistrements DLT de data-collect reçoivent un UUID aléatoire à chaque persistance, ce qui ne constitue pas une déduplication du replay.

UTILISATEURS ET RÔLES

Opérateurs, consommateurs IAM/data-dev/process-data et automatisations de reprise.

INTERFACES DANS LE CODE

OutboxPublisher ; OutboxRelay.drain/cleanupSent ; DeadLetterEventConsumer.consume ; KafkaConfiguration.

DONNÉES ET ÉTATS

outbox_event, dead_letter_event et suivi des messages entrants utilisé par certains consumers ; sentAt, attempts et lastError.

DÉPENDANCES ET ÉCHANGES

PostgreSQL et Kafka ; consommateurs de contrats Protobuf et faits IAM.

COMPOTEMENT CIBLE PROPOSÉ

Un relais Rust conserve le même contrat de persistance avant envoi vers un bus sans JVM. Audit garde les coordonnées et causes de rejet. Chaque effet possède une clé d'idempotence stable ; les checkpoints de collecte publiés restent dans l'édition Iceberg et ne se confondent pas avec les offsets de transport.

MIGRATION DE CETTE CAPACITÉ

Inventorier les producteurs et les clés de partition, reprendre l'outbox non envoyée et les quarantaines, puis basculer le transport par consommateur. Fournir une commande de replay audité qui conserve l'identité du message.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Couper le transport avant et après son acquittement puis redémarrer. Tout événement reste disponible et les doublons éventuels sont absorbés par le consommateur. Un échec PostgreSQL pendant la mise en quarantaine ne fait pas avancer l'offset terminal.

Références de l'analyse

[monorepo](#) · [services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/common/outbox/OutboxRelay.kt:23](#) Confirmation d'envoi, reprises et purge des lignes envoyées.

[monorepo](#) · [services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/common/deadletter/DeadLetterEventConsumer.kt:26](#) Persistance terminale et réémission d'erreur.

monorepo · services/data-collect/app/data-collect-

app/src/main/kotlin/com/cleyrop/data/collect/app/common/deadletter/DeadLetterEventConsumer.kt:54 Coordonnées originales et UUID de stockage aléatoire.

Les 19 types de connecteurs déclarés

Les 19 noms de ConnectorType ne constituent pas 19 connecteurs qualifiés. Le remplacement JDBC/JVM se fait par moteur et protocole avec des tests de contrat. Un extracteur OLake Go éventuellement retenu ne couvre pas implicitement ces familles ni leurs authentifications, previews, transformations, versions et règles de refresh. Aucun composant OLake Java ne peut rester dans la cible.

Type / famille	Comportement observé	Cible et écart	Acceptation
LOCAL / Fichiers	Manager local et upload par chunks, descripteurs persistés, formats et préparations.	Files/Connectors Rust : préserver reprise Content-Range, noms normalisés, previews et évolution de schéma.	Upload interrompu/repris puis lecture des mêmes octets ; refus des offsets incohérents et des formats invalides.
SFTP / Fichiers distants	Manager SFTP, navigation/cache et lecture des fichiers avec credential ; distinct des comptes de dépôt administrés.	Adaptateur SFTP sans JVM qualifié pour auth, clés, listing et flux ; Egress et Writer bornent connexions et octets.	Même arborescence filtrée et fichiers sélectionnés ; coupure en lecture puis reprise sans publier un fichier partiel.
DEPOT / Dépôt administré	Manager de fichier partagé avec SFTP ; création/suppression d'accès distant, clés publiques, règles de rétention.	Distinguer gestion des comptes et lecteur de fichiers ; préserver compensation, protections de chemins et revue de la révocation.	Créer, modifier et retirer un accès ; vérifier la révocation réelle, les répertoires protégés et la rétention.
MYSQL / SQL/JDBC	SQLConnector et driver JDBC ; métadonnées, requête restreinte, Parquet d'import.	Lecteur sans JVM à qualifier : types, décimaux, timezone, caractères, vues, cohérence snapshot et streaming.	Extraction d'un corpus MySQL de référence avec égalité des valeurs/types et reprise bornée.
POSTGRESQL / SQL/JDBC	SQLConnector et driver JDBC ; découverte spécifique des objets et import Parquet.	Lecteur Rust ou tiers encapsulé ; conserver types, vues et identifiants, puis définir snapshot/CDC explicitement.	Corpus PostgreSQL incluant types particuliers et vue ; démontrer cohérence de l'édition et reprise sans perte.
ORACLE / SQL/JDBC	SQLConnector et driver JDBC Oracle, métadonnées et lecture SQL.	Choisir un adaptateur autorisé sans JVM ; qualifier connectivité, précision NUMBER, dates, schémas et limites du compte.	Comparer un corpus Oracle et son schéma à l'extraction historique ; timeout et refus de droits restent identifiables.
MSSQL / SQL/JDBC	SQLConnector et driver SQL Server ; schémas/tables et flux vers Parquet.	Lecteur SQL Server sans JVM, avec équivalences de types et stratégie de lecture cohérente.	Valider décimaux, dates, Unicode, noms qualifiés et comportement sous modification concurrente.
BIGQUERY / SQL/JDBC déclaré	Modèle SQL présent mais entrée de driver vide dans JdbcConnectorManager ; exécution non qualifiée.	Construire ou sélectionner un lecteur BigQuery sans JVM, qualifier l'API de jobs/lecture et les coûts ; ne pas annoncer une parité héritée.	Une authentification, découverte, preview et extraction bornée doivent fonctionner dans un environnement de test avant support déclaré.

Type / famille	Comportement observé	Cible et écart	Acceptation
SNOWFLAKE / SQL/JDBC	Modèle SQL et driver JDBC ; requêtes et métadonnées via le chemin commun.	Adaptateur sans JVM à qualifier pour authentification, contexte de schéma et types Snowflake ; contrat de snapshot explicite.	Comparer lignes et types d'un corpus, puis vérifier limites, annulation et reprise d'un export interrompu.
API / HTTP générique	Manager HTTP ; modes d'auth, cache token, pagination, retries, découverte et téléchargement.	Lecteur Rust conservant les stratégies connues, sous politique Egress et budgets pages/octets/temps.	Fixtures de chaque pagination, 401/429, Retry-After, lien absolu et téléchargement coupé.
JIRA / SaaS HTTP	Sous-type API ; endpoint de recherche JQL et Basic utilisateur/token.	Adaptateur Jira sans JVM avec contrat de version, champs et pagination réellement utilisés.	Même recherche et projection sur plusieurs pages ; erreurs d'auth et limites distantes traitées explicitement.
SALESFORCE / SaaS HTTP versionné	Template Salesforce V6 et versions déclarées ; découverte d'entités/colonnes.	Maintenir le pin de version et qualifier OAuth, métadonnées, pagination et types ; évolution versionnée.	Configuration versionnée reproduit entités, schéma et résultat après changement de version par défaut.
MS_DYNAMICS / SaaS déclaré mais bloqué en création	Modèle/template API v9.2 présents ; CreateDataSourceCommand Handler interdit ce type.	Décider maintien de configurations historiques ou activation future ; qualifier entièrement avant ouverture.	Une matrice de tests de connexion, découverte, preview et collecte valide le choix produit avant support annoncé.
SAP_S4HANA / SaaS déclaré mais bloqué en création	Modèle et templates OData v2/v4 ; création interdite par le handler.	Qualifier les deux variantes OData et le schéma SAP retenu ; ne pas assimiler présence de template et disponibilité produit.	Corpus OData v2 et v4 avec pagination et erreurs ; création reste interdite jusqu'à validation explicite.
BUSINESS_CENTRAL / SaaS HTTP versionné	Template API v2.0, contexte tenant/environnement et authentification token.	Adaptateur sans JVM conservant contexte, sélection d'entités, version et pagination.	Deux environnements ne mélangent ni tokens ni entités ; résultat multi-page identique à la référence.
GOOGLESHEET / AirByte déclaré incomplet	GoogleSheetConnector dérive d'AirByteConnector, sans manager dans la factory ; TriggerOperation contient NotImplementedError pour AIRBYTE.	Construire un adaptateur Sheets sans JVM ou retirer explicitement le type ; aucune couverture implicite par OLake.	Avant activation, démontrer auth, sélection de feuille, entêtes, types, preview et collecte ; sinon réponse unsupported explicite.
S3 / Objet cloud	Manager S3 avec cache distant, navigation et import ; credentials et configuration de bucket côté connexion.	Adaptateur S3 Rust séparant bucket externe, staging Writer et stockage des éditions ; préserver sélection et formats.	Lister/importer des clés avec pagination et noms particuliers ; tester changement d'objet en cours de lecture et accès refusé.
CLEYROP / API de plateforme	Sous-type API ; token historique ou OAuth client_credentials selon configuration.	Conserver un client de plateforme versionné, références d'édition et mandat inter-tenant explicite ; retirer les credentials permanents des workers.	Importer sous l'identité autorisée sans élargissement de droits et vérifier le pin de l'édition lue.

Type / famille	Comportement observé	Cible et écart	Acceptation
SHAREPOINT / Documents cloud HTTP	Template Graph v1.0, découverte de drives, dossiers/fichiers, métadonnées, ETag et téléchargement pré-signé.	Adaptateur Graph sans JVM : qualifier drives, versionnement, liens de téléchargement, rotation des tokens et Egress à chaque destination.	Pagination de dossiers, 304, fichier modifié entre métadonnées et téléchargement et URL de téléchargement refusée hors politique.

Références de l'analyse

`monorepo · services/data-collect/app/data-collect-domain/src/main/kotlin/com/cleyrop/data/collect/domain/datasource/aggregate/connector/Connector.kt:35` Liste des 19 types.

`monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/ConnectorFactoryImpl.kt:189` Branches effectivement câblées et cas unsupported.

`monorepo · services/data-collect/app/data-collect-domain/src/main/kotlin/com/cleyrop/data/collect/domain/datasource/command/CreateDataSourceCommandHandler.kt:72` Types interdits en création.

`monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/manager/connector/JdbcConnectorManager.kt:736` Drivers JDBC dont BIGQUERY vide.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- Connectors possède catalogue des types réellement qualifiés, définitions de connexion, versions, références de secrets, droits d'usage et workflows de demande.
- Flows gère schedules, occurrences, commandes idempotentes, tentatives bornées, leases et échéances ; PostgreSQL reste autoritaire pour ces commandes et les drafts.
- Datasets porte schéma et contrat d'édition ; la référence Iceberg publiée et ses origin.* constituent la preuve autoritaire de données. Les checkpoints publiés ne résident pas uniquement dans PostgreSQL.
- IAM et Audit enregistrent délégations, propriété et décisions. Une identité de worker ne possède jamais un droit générique table.modify au catalogue.

Plan de données proposé

- Lecteurs SQL, SFTP, S3 et HTTP produisent des batches ou fichiers immuables avec compteurs, limites et provenance. Chaque adaptateur déclare sa stratégie de snapshot et de checkpoint.
- Writer médie le staging de confiance et remet un candidat immuable à Audit. Publisher médie publication et ack source après commit publié. Tous les kinds, y compris STREAM et la compaction, suivent WAP.
- Un flux éphémère doit être journalisé durablement avant sa gouvernance. La reprise s'appuie sur source/epoch/range/participants. Une release multi-sorties est elle-même une édition Iceberg et ses lecteurs restent épinglés.
- Files expose FILESET et contenu de corpus ; Maintenance réalise GC/expiration par plan audité et garde de rétention. EXPORT reste un effet et EXPOSURE un consommateur déclaré.

Découpage du core Rust

- Proposition : modules `connector_registry`, `credential_ref`, `connection_check`, `source_discovery`, `source_reader` et `pagination_state`, chacun avec contrats de version et d'erreur.
- Proposition : codecs tabulaires communs au preview et au Writer, adaptateurs par moteur SQL sans JDBC, adaptateurs HTTP/S3/SFTP et lecteur SharePoint avec métadonnées conditionnelles.
- Proposition : scheduler et run coordinator Rust sur PostgreSQL pour commandes, avec fencing applicatif indépendant des seuls CAS UUID/refs/schema/spec et de l'ascendance Iceberg.
- Proposition : interfaces `trusted_writer.stage`, `auditor.attest`, `publisher.publish` et `maintenance.plan`. Elles lient le reçu de publication, les checkpoints et les octets audités.

Identité, autorisation et isolation

- L'AGENTS dit que l'autorisation est déléguée au gateway ; le code montre encore des brokers résiduels pour ingestion et SFTP. Reprendre les règles effectivement exécutées et leur flag de bascule.
- Le périmètre réseau entrant est une condition de confiance de X-Userinfo. L'egress actuel reste ouvert ; valider URL syntaxiquement ne constitue pas une défense SSRF complète.
- Masquer les secrets dans DTO, audit et logs ; porter les secrets par référence et réduire leur durée d'accès. Le mécanisme de conservation sur champ vide mérite un contrat explicite.
- Séparer tenant, source, fichier, run et édition. Valider les chemins relatifs, les host keys SSH, les autorisations on-behalf-of et les URL pré-signées à chaque frontière.
- Les fichiers audités ne doivent jamais être réécrits ; un transfert ou une suppression a une commande et un reçu. Les contrôles de schéma ne remplacent pas les contrôles d'accès aux octets.

Exploitation et reprise

- Le chart décrit un Deployment non root avec ports applicatif et management, probes liveness/readiness, configuration et secrets injectés, certificats et volume temporaire optionnel. Aucun état déployé n'a été inspecté.
- Quartz persiste jobs/triggers dans PostgreSQL et reconstruit les schedules au démarrage. Les caches de connexion et le debounce d'admission restent dans le processus.
- Le HPA est conditionné à l'absence de persistance. Le stockage temporaire et les sessions d'upload imposent de vérifier la topologie réelle avant toute promesse de scale horizontal.
- Flyway accepte des migrations out-of-order et le framework de migrations de code peut arrêter le processus avec code 0. La cible doit rendre les migrations verrouillées, reprenables et leur échec visible.
- Prévoir sauvegarde cohérente des définitions, maps IAM, schedules et outbox, ainsi que des clés requises pour déchiffrer l'historique. Les caches restent reconstructibles ; les objets de données suivent leur politique de rétention.
- Aucune qualification runtime n'est revendiquée. Les tests identifiés couvrent notamment uploads ordonnés, taille, isolation par source et téléchargements conditionnels d'ingestion.

Séquence de migration

- 1. Figer l'inventaire par type de connecteur, configuration réellement utilisée, source/projet/propriétaire et version API ; qualifier les types déclarés mais inactifs.
- 2. Importer UUID, historique masqué, permissions, schedules et références de secrets dans le contrôle Rust ; convertir les identifiants legacy avec preuves de correspondance.
- 3. Qualifier codecs et lecteurs sur fixtures puis sources de test ; comparer previews, schémas, métadonnées et octets, sans doubler les effets de collecte sur les sources.
- 4. Introduire Writer/Audit/Publisher et le journal durable d'entrées ; déplacer les checkpoints vers les éditions publiées et conserver un historique séparé des publications approuvées.
- 5. Basculer les schedules à une frontière d'occurrence, drainer Quartz et les événements existants, puis migrer les comptes SFTP et uploads encore actifs.
- 6. Retirer Spring, Quartz, drivers JDBC/Hadoop et Kafka JVM seulement après drainage et preuve que le plan de contrôle, les workers et les outils d'exploitation n'en dépendent plus.

Écarts et décisions encore ouverts

- Le code expose 19 types mais n'en qualifie pas 19 : GoogleSheet n'a pas de manager Airbyte ; BigQuery a un driver vide ; Dynamics et SAP sont interdits en création. Quelle compatibilité produit est requise pour chacun ?
- UpdateDataSourceCommandHandler.checkDepotDisplayName retourne true quand un dépôt de ce nom existe, alors que son appel et son log suggèrent un contrôle d'unicité. Le comportement réel doit être caractérisé avant correction.
- withStandardizeColumns n'apparaît pas dans la conversion des préparations. ParquetConfigDTO et XMLConfigDTO prennent des defaults JSON. Décider ce qui est un défaut historique à corriger et ce qui doit rester compatible.
- Le debounce d'admission local n'assure pas un verrou par dataset entre plusieurs pods. Définir la politique cible pour rafraîchissements concurrents, annulation et remplacement.
- Les protections de destination doivent couvrir JDBC, SFTP, S3, token endpoints, pagination et téléchargements pré-signés. La lecture statique n'établit pas une protection globale contre SSRF ou DNS rebinding.
- Le statut et expiredAt d'un accès SFTP ne prouvent pas sa révocation distante automatique. Quel comportement est attendu pour désactivation, expiration et suppression de toutes les clés ?
- Identifier les garanties de snapshot des sources SQL et API et l'écart entre comptage séparé et flux lu. Une simple comparaison de nombre de lignes ne suffit pas à prouver l'édition source.
- Le schéma Flyway de data-collect n'entre pas dans le linter Alembic du monorepo. Planifier Expand/Migrate/Contract et rollback sans retirer la compatibilité de lecture au même tag.

Références de l'analyse

[monorepo](#) · [services/data-collect/AGENTS.md:1](#) Rôle, organisation et obligations opérationnelles du composant.

[monorepo](#) · [services/data-collect/app/gradle/libs.versions.toml:22](#) Version Kotlin ; Spring Boot déclaré plus bas.

[monorepo](#) · [services/data-collect/chart/templates/deployment.yaml:50](#) Conteneur, probes, secrets et volumes.

[monorepo](#) · [services/data-collect/chart/templates/networkpolicy.yaml:23](#) Réseau entrant et absence d'egress restrictif.

SERVICE ACTUEL · FILES

Fichiers, dossiers et traitements documentaires

8 capacités analysées

services/files · Python, FastAPI, SQLAlchemy asynchrone ; PostgreSQL pour hiérarchie et index, S3/MinIO pour objets, ARQ et Redis/Valkey pour jobs ; Kafka pour faits et antivirus/extraction spécialisés.

Fournir un espace de fichiers personnels ou de projet, navigation, recherche, copies, téléchargement, corbeille et traitements asynchrones. Ce service n'est pas aujourd'hui une autorité d'édition FILESET.

Lecture de code

Pas de qualification runtime nouvelle

Domaines cible : Files, Writer, Publisher, Audit, Maintenance, IAM, Catalog, Semantics, Embeddings.

Périmètre de lecture

- AGENTS.md ; rapport Files existant ; routes files, folders, projects, search, trash, jobs et config_assets.
- ContentStore, parties ciblées des services de fichiers, chunks, assets, extraction, worker et intention de suppression.

Objets et cycle de vie

- File et Folder portent identité, hiérarchie et état logique en PostgreSQL. Les octets sont des objets S3 à clés plates.
- Le SHA-256 actuel décrit le contenu ; il ne fournit ni édition publiée ni déduplication globale prouvée.
- ConfigAsset est séparé des fichiers privés ; les jobs et intentions de suppression ont leurs propres états.

Dépendances du service

- IAM et APISIX contrôlent ressources de chemin et destinations de corps.
- PostgreSQL, S3/MinIO, ARQ avec Redis/Valkey, antivirus et extraction PDF.
- Kafka relaie les faits actuels ; sa substitution fait partie de la cible sans JVM.

Fonctionnalités et compatibilité à préserver

FS-01 Cible non qualifiée

Recevoir un fichier et contrôler sa destination avant stockage

COMPORTEMENT OBSERVÉ

POST upload valide la destination via IAM lorsque la gateway ne peut pas la déduire du chemin. Pour un dossier neuf, le handler peut réessayer le même contrôle pendant la propagation de son tuple, sans substituer un droit de projet. ContentStore borne la taille, détecte le MIME dans les octets, applique la politique et calcule SHA-256 avant l'écriture S3. L'objet utilise une clé UUID/nom ; le hash stocké ne constitue pas une déduplication globale. La ligne initiale peut attendre son scan antivirus.

UTILISATEURS ET RÔLES

Utilisateur ou intégration qui ajoute un fichier.

INTERFACES DANS LE CODE

POST /api/v1/files/upload ; `_authorize_upload_target` ; `ContentStore.ingest_streaming`.

DONNÉES ET ÉTATS

File PostgreSQL, clé de blob, taille, MIME détecté, SHA-256 et état de scan.

DÉPENDANCES ET ÉCHANGES

IAM broker, gateway, stockage S3 et worker antivirus.

COMPOTEMENT CIBLE PROPOSÉ

Files Rust crée une session d'upload et écrit dans un espace privé durable. Le reçu de réception ne vaut pas publication. Writer prépare l'index FILESET ; Publisher approuve l'édition Iceberg après audit, avec origin.* au candidat.

MIGRATION DE CETTE CAPACITÉ

Préserver IDs, noms, limites et erreurs. Auditer les objets existants puis adopter uniquement une version immuable avec hash contrôlé ; reconstruire une édition publiée sans ériger la navigation PostgreSQL en checkpoint.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un MIME falsifié ou un fichier trop grand n'atteint pas le stockage. Un dossier refusé ne devient pas autorisé par son projet. Aucun octet reçu n'est lisible avant son édition approuvée.

Références de l'analyse

[monorepo](#) · [services/files/src/file_management/api/v1/files.py:63](#) Contrôle de la destination et propagation des droits.

[monorepo](#) · [services/files/src/file_management/storage/content_store.py:170](#) Validation complète avant écriture des octets.

FS-02 **Cible non qualifiée****Assembler les uploads par morceaux et nettoyer les abandons****COMPORTEMENT OBSERVÉ**

L'upload par chunks utilise propriétaire, projet, nom et `upload_id` pour isoler le fichier temporaire. Le nom est ramené à son basename ; l'écriture prend flock et revérifie la taille sous verrou. Le service calcule ensuite le checksum du fichier assemblé et poursuit le parcours d'upload. Le worker nettoie les uploads périmés. Ce mécanisme dépend du fichier local et ne démontre pas une reprise depuis un worker arbitraire.

UTILISATEURS ET RÔLES

Clients qui transfèrent des fichiers volumineux.

INTERFACES DANS LE CODE

POST /api/v1/files/upload/chunk ; `ChunkedUploadService.upload_chunk` ; `cleanup_stale_uploads`.

DONNÉES ET ÉTATS

Fichiers temporaires locaux et métadonnées de requête ; objet final et ligne File après assemblage.

DÉPENDANCES ET ÉCHANGES

Disque temporaire du service, stockage S3 et ARQ.

COMPOTEMENT CIBLE PROPOSÉ

Sessions Rust durables avec identifiant de transfert, plages ou parties, taille attendue, checksum et état de finalisation. Les workers reprennent depuis le stockage durable ; la fermeture du transfert produit un candidat FILESET puis une publication WAP.

MIGRATION DE CETTE CAPACITÉ

Garder la route compatible mais router chaque session vers le protocole durable. Terminer ou invalider explicitement les sessions locales ouvertes avant de retirer les anciens pods.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Interrompre une réception puis reprendre sur un autre worker restitue exactement le même fichier. Le rejeu d'une partie ne la double pas. Un nettoyage n'efface pas un upload actif.

Références de l'analyse

[monorepo](#) · [services/files/src/file_management/services/chunked_upload_service.py:85](#) Isolation des chemins et noms.

[monorepo](#) · [services/files/src/file_management/services/chunked_upload_service.py:105](#) Verrou et contrôle de taille.

[monorepo](#) · [services/files/src/file_management/workers/tasks.py:1127](#) Nettoyage des transferts abandonnés.

FS-03 **Cible non qualifiée****Naviguer, déplacer et copier les fichiers et dossiers****COMPOTEMENT OBSERVÉ**

Le service expose contenu personnel, contenu de projet, résolution de chemin, dossiers simples ou par lot, renommage, déplacement et copie. La hiérarchie réside en PostgreSQL ; les clés S3 sont plates. Les destinations fournies dans le corps demandent une autorisation IAM. Les copies de dossiers peuvent produire un job ; son statut est réservé à l'utilisateur qui l'a demandé. Un non-propriétaire voit 404. Si un statut Redis manque après acquittement de l'intention durable, l'API retourne 503 plutôt qu'un pending permanent.

UTILISATEURS ET RÔLES

Contributeurs de projets et propriétaires d'espace personnel.

INTERFACES DANS LE CODE

/api/v1/my-files ; /api/v1/projects/{project_id}/contents et /resolve ; /api/v1/folders ; /api/v1/files/{file_id}/move ou /copy ; GET /api/v1/jobs/{job_id}.

DONNÉES ET ÉTATS

File, Folder, Project, hiérarchie, intention durable de copie et statut temporaire de job.

DÉPENDANCES ET ÉCHANGES

IAM, PostgreSQL, S3, ARQ et statut Redis/Valkey.

COMPOTEMENT CIBLE PROPOSÉ

Files Rust garde des noms logiques et dossiers comme projection de navigation. Déplacer ou copier prépare une nouvelle édition FILESET ou une commande de présentation explicitement séparée. Un job expose un état durable et l'édition résultante.

MIGRATION DE CETTE CAPACITÉ

Importer toute la hiérarchie et les alias projet. Préserver conflits de noms et droits de destination.

Reconstituer les jobs encore ouverts avant bascule et éviter une copie physique inutile lorsque les blobs immuables peuvent être référencés.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Copier vers un dossier interdit échoue même si la source est lisible. Une coupure après création de l'intention reprend sans doubler les entrées. Un autre utilisateur ne peut sonder le statut du job.

Références de l'analyse

[monorepo · services/files/src/file_management/api/v1/folders.py:264](#) Contrat de copie et destination.

[monorepo · services/files/src/file_management/api/v1/jobs.py:33](#) Propriété du job et traitement d'un statut absent.

[monorepo · services/files/src/file_management/api/v1/projects.py:97](#) Résolution du chemin de navigation.

FS-04 **Cible non qualifiée****Télécharger, prévisualiser et produire des archives****COMPOTEMENT OBSERVÉ**

Le téléchargement lit les octets S3 via le service. La garde de scan refuse INFECTED, PENDING et ERROR avec des réponses distinctes ; PENDING signale une attente temporaire. L'aperçu textuel a ses limites de type et de taille, mais ne bloque explicitement que INFECTED dans le code lu. Les dossiers et sélections se téléchargent en ZIP. Le pipeline ZIP borne les fichiers temporaires en vol, contrôle les dérives de taille et nettoie les ressources à l'arrêt du consommateur. Aucun presign n'est établi par les routes examinées.

UTILISATEURS ET RÔLES

Lecteurs autorisés et clients de téléchargement.

INTERFACES DANS LE CODE

GET /api/v1/files/{file_id}/download et /preview ; GET /api/v1/folders/{folder_id}/download ; POST /api/v1/projects/{project_id}/download.

DONNÉES ET ÉTATS

Blob, état de scan, réponse stream et fichiers temporaires ZIP.

DÉPENDANCES ET ÉCHANGES

IAM/gateway, S3 et pipeline ZIP.

COMPOTEMENT CIBLE PROPOSÉ

Un lecteur Rust résout une version de blob dans une édition approuvée puis applique IAM sur tous les chemins. Un EXPORT durable est un effet avec reçu et reprise. Un téléchargement interactif reste un stream médié avec contrôle des limites et révocation.

MIGRATION DE CETTE CAPACITÉ

Aligner les gardes preview, ZIP et download avant ouverture des corpus. Conserver CSV/texte et ZIP compatibles là où utilisés, puis ajouter l'identifiant d'édition dans le contrat de lecture.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un fichier en attente ou en erreur de scan n'est accessible ni par preview ni par ZIP. Annuler un stream libère les fichiers temporaires. Une lecture épinglée ne change pas de blob pendant le transfert.

Références de l'analyse

[monorepo · services/files/src/file_management/services/file_service.py:287](#) États bloquant le téléchargement.

[monorepo · services/files/src/file_management/services/file_service.py:1091](#) Garde spécifique et limites de preview.

[monorepo · services/files/src/file_management/services/zip_pipeline.py:9](#) Borne de ressources du pipeline ZIP.

FS-05 **Cible non qualifiée****Scanner, compresser et extraire du texte dans les workers****COMPORTEMENT OBSERVÉ**

ARQ exécute `scan_file_for_virus`, `compress_file`, `extract_file_text` et copie des objets d'un dossier. Les retries du scan sont bornés et les erreurs terminales sont persistées ; une tâche périodique reprend des scans PENDING restés sans résultat. L'extraction traite textes et PDF avec une limite de taille de fichier et de longueur de texte. La compression écrit une nouvelle clé puis fait pointer la ligne vers celle-ci avant de retirer l'ancien objet. Ces opérations enrichissent l'état de fichier actuel, sans créer une édition gouvernée.

UTILISATEURS ET RÔLES

Utilisateurs en attente de disponibilité ; exploitation des traitements.

INTERFACES DANS LE CODE

`WorkerSettings.functions` ; tâches `scan_file_for_virus`, `compress_file`, `extract_file_text` ; POST `/api/v1/projects/{project_id}/extract-text` selon le routeur projet.

DONNÉES ET ÉTATS

États antivirus, extraction textuelle, clé de blob compressé, jobs ARQ et intentions durables.

DÉPENDANCES ET ÉCHANGES

ARQ, antivirus, extraction PDF, stockage et base métier.

COMPORTEMENT CIBLE PROPOSÉ

Workers fongibles exécutant des runs bornés, avec résultat d'audit lié à une version immuable. Les transformations qui changent les octets créent un candidat et passent WAP ; l'audit post-candidat est lié durablement sans réécriture du summary par fast-forward.

MIGRATION DE CETTE CAPACITÉ

Conserver les résultats de scan avec leur hash/version. Identifier les transformations de compression déjà réalisées, réauditer les objets sans preuve suffisante et déplacer les tâches vers la file sans JVM retenue.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une panne d'antivirus finit en état explicite et ne publie pas le fichier. Une compression interrompue ne laisse aucune édition pointant vers des octets manquants. Un rejeu ne change pas une preuve d'audit déjà liée.

Références de l'analyse

[monorepo · services/files/src/file_management/workers/worker.py:83](#) Liste des tâches, bornes et reprise.

[monorepo · services/files/src/file_management/services/text_extraction_service.py:12](#) Limites de l'extraction.

[monorepo · services/files/src/file_management/workers/tasks.py:423](#) Parcours de compression asynchrone.

FS-06 **Cible non qualifiée****Rechercher noms et contenu sans élargir le périmètre****COMPORTEMENT OBSERVÉ**

La recherche distingue le chemin personnel du chemin de projet. `/my-files/search` fixe le propriétaire à l'appelant et ne reçoit pas de `projectId` permettant d'élargir la portée. `/projects/{id}/search` utilise le projet résolu et aucun filtre propriétaire, sous le contrôle `can_view` de la gateway. Le contrat accepte recherche

fulltext ou nom, type fichier/dossier, filtre dossier et pagination de 1 à 100 résultats par type. Fichiers et dossiers sont interrogés séquentiellement sur la session SQL partagée.

UTILISATEURS ET RÔLES

Utilisateurs recherchant leurs documents ou ceux d'un projet.

INTERFACES DANS LE CODE

GET /api/v1/my-files/search ; GET /api/v1/projects/{project_id}/search.

DONNÉES ET ÉTATS

Index PostgreSQL et texte extrait ; filtres sur projet, propriétaire et dossier.

DÉPENDANCES ET ÉCHANGES

PostgreSQL, extraction et gateway IAM.

COMPORTEMENT CIBLE PROPOSÉ

Files et Catalog Rust indexent seulement les documents appartenant à des éditions approuvées. L'index est reconstructible ; snippets, compteurs et résultats appliquent la même politique que le téléchargement. Semantics/Embeddings reste une extension distincte de la recherche fulltext actuelle.

MIGRATION DE CETTE CAPACITÉ

Préserver chemins et filtres, puis reconstruire l'index depuis les éditions migrées. Ne pas reprendre le texte extrait d'un fichier supprimé ou d'un scan non accepté sans audit.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une recherche personnelle ne retourne pas les fichiers d'un autre utilisateur. Un projet interdit ne révèle ni texte ni compteur. Reconstruire l'index conserve les résultats autorisés de la même édition.

Références de l'analyse

[monorepo](#) · [services/files/src/file_management/api/v1/search.py:132](#) Portée personnelle fixée au sujet.

[monorepo](#) · [services/files/src/file_management/api/v1/search.py:165](#) Portée projet et gardes gateway.

FS-07 Cible non qualifiée

Restaurer la corbeille et différer la suppression physique

COMPORTEMENT OBSERVÉ

Les routes distinguent suppression logique, corbeille de projet, restauration de fichier ou dossier, suppression permanente et vidage. La destination de restauration est contrôlée. Pour supprimer définitivement un fichier, le service verrouille le namespace et la ressource, écrit le fait inverse d'autorisation, enregistre une intention durable de suppression S3 puis supprime la ligne. Les octets ne sont retirés qu'après commit, pour éviter une ligne conservée qui pointe vers un objet effacé.

UTILISATEURS ET RÔLES

Propriétaires, gestionnaires du projet et politique de conservation.

INTERFACES DANS LE CODE

/api/v1/trash/files/{file_id}/restore ; /api/v1/trash/folders/{folder_id}/restore ; DELETE /api/v1/trash/files/{file_id} ; POST /api/v1/projects/{project_id}/trash/empty.

DONNÉES ET ÉTATS

Tombstones logiques, parents de restauration, audit et intention durable de suppression des clés.

DÉPENDANCES ET ÉCHANGES

Outbox, IAM, stockage et futurs Maintenance/Audit.

COMPORTEMENT CIBLE PROPOSÉ

Maintenance Rust distingue retrait du service, conservation et purge physique. Les racines de GC viennent des éditions conservées, tags, runs et obligations de restauration. Les projections et refcounts restent reconstructibles ; un plan audité vérifie à nouveau les références avant effacement.

MIGRATION DE CETTE CAPACITÉ

Importer les éléments de corbeille et leurs délais, relier blobs aux éditions conservées et traiter les intentions anciennes. Inclure textes extraits et dérivés dans le plan de purge.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un rollback de transaction ne supprime aucun blob encore référencé. Une restauration respecte les droits du nouveau dossier. Une purge refuse un blob encore utilisé par une édition retenue, même si l'index de navigation est en retard.

Références de l'analyse

[monorepo](#) · [services/files/src/file_management/services/file_service.py:1000](#) Intention de suppression après commit et inverse authz.

[monorepo](#) · [services/files/src/file_management/api/v1/trash.py:62](#) Restauration et destination.

[monorepo](#) · [services/files/src/file_management/api/v1/trash.py:139](#) Vidage de la corbeille.

FS-08 **Cible non qualifiée****Servir les assets publics de branding dans un stockage distinct****COMPORTEMENT OBSERVÉ**

Les assets de configuration vivent dans une table dédiée, séparée des fichiers privés. Les routes d'administration requièrent une identité et la gateway impose le rôle global ; seule `/public/config/assets/{key}` sert sans authentification. Le service limite clés, MIME réel et taille, calcule un ETag et gère le cache conditionnel. Une mise à jour prend un verrou par clé, écrit un nouvel objet, commit la ligne puis nettoie l'ancien. Ces uploads d'administrateur n'ont pas de scan antivirus.

UTILISATEURS ET RÔLES

Administrateurs de branding et visiteurs de la page de connexion.

INTERFACES DANS LE CODE

GET `/api/v1/config/assets` ; PUT/DELETE `/api/v1/config/assets/{key}` ; GET `/api/v1/config/assets/{key}/info` ; GET `/api/v1/public/config/assets/{key}`.

DONNÉES ET ÉTATS

ConfigAsset, version de clé S3, hash/ETag et auteur de mise à jour.

DÉPENDANCES ET ÉCHANGES

Gateway IAM, S3 et caches HTTP.

COMPORTEMENT CIBLE PROPOSÉ

Files Rust conserve ce stockage public explicitement séparé, avec types et tailles admis. Ce parcours de configuration ne doit pas rendre public un FILESET métier ni contourner les politiques d'édition.

MIGRATION DE CETTE CAPACITÉ

Importer les assets et leur hash, conserver URLs et cache HTTP, puis changer l'implémentation derrière la même frontière gateway.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un utilisateur ordinaire ne peut modifier un asset. Un MIME simulé est refusé avant S3. Deux uploads ou un upload concurrent à un delete ne laissent pas la ligne pointer sur un objet effacé.

Références de l'analyse

[monorepo](#) · [services/files/src/file_management/api/v1/config_assets.py:145](#) Route publique et réponse HTTP.

[monorepo](#) · [services/files/src/file_management/services/config_asset_service.py:120](#) Verrou, clé neuve et ordre commit/nettoyage.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- Sessions d'upload, droits, dossiers, jobs, conservation et reçus d'effets restent des commandes/projections PostgreSQL.
- Une édition FILESET est un index Iceberg publié autoritaire, avec identité et version immuable de chaque blob.

Plan de données proposé

- Réception durable privée, calcul de hash, scan et extraction dans des runs bornés.
- Lecture médiée de blobs appartenant à l'édition autorisée ; pas de droit général au bucket ni aux fichiers staging.

Découpage du core Rust

- Sessions et navigation Files, ContentStore Rust, lecteur d'édition, outbox et adaptateurs d'objets.
- Writer et Publisher communs pour FILESET ; Maintenance pour GC et purge, indépendants du retard de navigation.

Identité, autorisation et isolation

- Uniformiser les gardes antivirus et publication sur download, preview, ZIP, extraction et indexation.
- Préserver le contrôle de destination sans remplacer un refus de dossier par un droit plus large. Les assets publics restent séparés.

Exploitation et reprise

- Surveiller scans PENDING, erreurs terminales, backlog ARQ/outbox, espace temporaire et intentions de suppression.

- Le protocole chunks actuel n'est pas qualifié pour reprise sur un autre pod. La restauration cible combine index Iceberg, blobs et commandes PG, sans considérer un index de recherche comme autorité.

Séquence de migration

- Inventorier IDs, dossiers, blobs, corbeille et jobs ouverts.
- Contrôler hash, immutabilité et audit avant adoption dans des éditions FILESET.
- Basculer les sessions et lecteurs puis les traitements vers Rust, un corpus à la fois avec un seul Publisher.
- Qualifier panne après publication avant ack, lecture de staging, reprise de chunks et GC sous retard de projection.

Écarts et décisions encore ouverts

- La différence de garde antivirus entre preview et download est observée statiquement ; son impact en déploiement n'a pas été testé.
- L'adoption sans copie exige maîtrise de la version physique et de la conservation du blob.
- Aucun scénario de charge, reprise, scan ou restauration n'a été exécuté dans ce lot.

Références de l'analyse

[monorepo](#) · [services/files/src/file_management/main.py:629](#) Assemblage des API Files.

[monorepo](#) · [services/files/src/file_management/workers/worker.py:80](#) Tâches et maintenance du worker.

SERVICE ACTUEL · DATA-MODEL

Catalogue, contrats et cycle de vie des datasets

21 capacités analysées

services/data-model · Kotlin 2.4.10 et Spring Boot 4.1.1 sur JVM ; modules domaine et adaptateurs Spring ; PostgreSQL, JPA/JDBC, Flyway ; Kafka et Protobuf ; clients HTTP IAM et Datasphere.

Référentiel métier utilisé par le catalogue, les projets, les producteurs de données et les responsables qualité. Il réserve les identités de datasets, conserve les métadonnées de branches et l'historique des rafraîchissements, gère les liens vers les projets et diffuse les faits nécessaires aux autres services. Il ne lit pas les lignes des tables.

Lecture de code

Pas de qualification runtime nouvelle

Domaines cible : Catalog, Datasets, IAM, Flows, Connectors, Writer, Publisher, Audit, Maintenance, Notifications, Egress, Migration.

Périmètre de lecture

- Les 14 contrôleurs REST, leurs DTO et les familles de commandes/requêtes dataset, label et promotion.
- Agrégats Dataset, DatasetBranch, schéma, refresh, qualité, connecteurs et formats ; repositories de catalogue, labels et suppressions.
- Consommateurs data-collect, data-dev, process-data, data-serve et IAM ; outbox ; runner de migration et tâches planifiées.
- AGENTS.md, registre des contrôles résiduels, propriétés de service, versions Gradle et charts.

Objets et cycle de vie

- Dataset porte l'identité, le nom technique, le projet et les drapeaux reserved/hidden/inProduction/isPublished. Les propriétés affichées principales viennent de la branche métier main. SourceDataset décrit une origine et son éventuelle suppression.
- DatasetBranch est une branche métier persistée dans PostgreSQL, distincte d'une ref physique Iceberg. Elle porte propriétaire, état, qualité, sensibilité, annotations et schéma. DatasetColumn utilise aujourd'hui un UUID métier et un en-tête ; il ne représente pas explicitement un field ID Iceberg.
- DatasetRefreshConfig décrit déclenchement, append/override, schéma MERGE/REPLACE/PERMISSIVE_FIXED/STRICT_FIXED, schedules et retry. DatasetRefreshHistory décrit une exécution ou une opération de schéma avec ses entrées, opérations et logs.
- DatasetProjectPermission porte demande/accord/refus et validation. DatasetMembership porte un abonnement tel que DATASET_QUALITY, distinct d'un droit de lecture.
- Label et SchemaLabel sont deux vocabulaires distincts. Rule/RuleHistory/Compute/ComputedRule sont des projections de qualité reçues de data-serve.
- LabelPolicy est tenue par le service Datasphere optionnel. PendingPromotion, DatasetPromoted et DatasetPromotionHistory suivent des demandes d'effets distants.
- DatasetDeletion conserve l'identité physique et bloque la réutilisation du nom jusqu'au drop. Les tables de miroirs IAM et correspondances servent la migration et la présentation.

Dépendances du service

- data-collect publie les créations/évolutions de source et les états de collecte en Protobuf sur Kafka ; data-model enregistre configuration et refresh.
- data-dev publie versions, branches et sorties de flows ; process-data publie événements de traitement, horaires et résultats. data-model publie dataset, lien projet, schedule et empty vers les consommateurs.
- data-serve publie les règles et leurs résultats ; il peut aussi émettre les corrections de types sur le topic de migration de schéma.
- IAM HTTP décide consultation par lot, édition de projet, gestion de labels et transfert de propriétaire. IAM consomme les faits authz et écrit OpenFGA.
- notification-center reçoit abonnements et notifications, puis alimente la projection locale des memberships.
- Datasphere HTTP optionnel possède les labels de sécurité, les décisions de clearance et l'effet encrypt. PostgreSQL est le stockage métier actuel ; aucun moteur SQL de données n'est embarqué ici.

Fonctionnalités et compatibilité à préserver

DM-01 **Cible non qualifiée**

Réserver une identité et un nom de table, seul ou en lot

COMPORTEMENT OBSERVÉ

La création alloue un UUID et une branche métier main vide en état CREATED dans PostgreSQL. Ce parcours réserve des métadonnées et ne crée pas de snapshot ni de ref physique Iceberg. Le dataset est reserved, hors production et caché lorsqu'il appartient à un projet. Sans projet, sa visibilité catalogue est déjà publiée. Le DTO exige un nom affiché non blanc et un nom technique composé de deux parties imprimables séparées par un point. Le handler normalise la partie table, refuse les collisions dans le projet ou le catalogue et vérifie que le propriétaire est actif. Le lot accepte de 1 à 100 éléments, impose un seul projet et détecte aussi les collisions entre éléments du même lot. Il peut abonner l'auteur aux échecs de collecte. Une attente bornée laisse IAM propager le droit du propriétaire après création ; elle ne transforme jamais une création réussie en refus.

UTILISATEURS ET RÔLES

Auteur de flow ou de collecte, propriétaire, interface catalogue et SDK.

INTERFACES DANS LE CODE

POST /datasets ; POST /datasets/bulk ; GET et POST /datasets/check ; GET /projects/{projectId}/table-name.

DONNÉES ET ÉTATS

Dataset, branche métier main, réservation et namespace PostgreSQL ; miroir utilisateurs/projets ; faits de création.

DÉPENDANCES ET ÉCHANGES

IAM pour le droit sur le projet ; outbox authz ; notifications ; collecte ultérieure qui consomme la réservation.

COMPORTEMENT CIBLE PROPOSÉ

Datasets Rust conserve UUID et alias, avec une commande idempotente de réservation et une contrainte d'unicité transactionnelle. La réservation ne crée aucune édition publiée. Catalog expose les noms et états, IAM garde la décision sur le projet et le propriétaire.

MIGRATION DE CETTE CAPACITÉ

Importer UUID, namespaces, alias et réservations encore utilisées. Conserver la limite de lot et les codes d'erreur via une façade. Remplacer les sujets Keycloak et IDs projet hérités par des IDs IAM stables avec table de correspondance.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un lot qui contient deux noms se normalisant de la même manière est refusé avant toute création. Après une réponse perdue, la même clé de commande retrouve la même réservation. Le propriétaire peut ensuite lire l'objet sans devoir le recréer.

Références de l'analyse

[monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/command/CreateDatasetCommandHandler.kt:51](#) État initial, branche, vérifications et création en lot.

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/controller/DatasetCreateController.kt:124](#) Borne de lot, canonisation et attente de propagation.

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/controller/dto/DatasetCreateDTO.kt:22](#) Contraintes du contrat public et propriétaire par défaut.

DM-02 **Cible non qualifiée****Modifier les métadonnées et transférer la propriété****COMPORTEMENT OBSERVÉ**

Le détail assemble propriétaire, origine, description, état, médaillon qualité, sensibilité, branche, planning, labels, volumes, configuration et abonnements de l'utilisateur. PUT permet nom, description, ownerId, qualité, labels et certains réglages de refresh. Un dataset encore produit par un dataflow vivant ne peut pas être modifié par ce parcours. Le transfert effectif du propriétaire demande `can_manage_shares` à IAM avant de vérifier le nouvel utilisateur. Seul le changement du propriétaire de la branche métier main déplace le droit global et retire les anciennes notifications qualité. La sensibilité et le mode de schéma ne sont pas modifiables par ce DTO.

UTILISATEURS ET RÔLES

Propriétaire de dataset, gestionnaire de catalogue, consommateur du détail.

INTERFACES DANS LE CODE

GET `/datasets/{datasetId}` ; PUT `/datasets/{datasetId}` avec `branchName` ; `DatasetDetailQueryHandler` et `UpdateDatasetFromFrontCommand`.

DONNÉES ET ÉTATS

Dataset global et `DatasetBranch` avec propriétaire, dates, description, qualité BRONZE/SILVER/GOLD et sensibilité INTERNAL/RESTRICTED/SENSIBLE.

DÉPENDANCES ET ÉCHANGES

IAM pour transfert ; miroirs utilisateurs ; événements dataset, planning et `owner_changed` ; notifications.

COMPORTEMENT CIBLE PROPOSÉ

Catalog Rust porte les métadonnées descriptives et leur historique de commandes. IAM porte propriété et classification. Datasets lie toute métadonnée technique à une édition ou à un draft explicite. La visibilité, la disponibilité des données et le résultat du dernier run restent des champs distincts.

MIGRATION DE CETTE CAPACITÉ

Reprendre les métadonnées par branche et les alias d'identités. Documenter le fait qu'un null dans le DTO actuel conserve souvent la valeur plutôt que l'effacer. Introduire une version de commande pour éviter les modifications concurrentes perdues.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un changement de description conserve l'édition lue. Un transfert refusé ne révèle pas si le nouvel utilisateur existe et ne modifie aucune ligne. Un transfert autorisé de main retire le droit de l'ancien propriétaire et l'abonnement associé une seule fois.

Références de l'analyse

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/controller/dto/DatasetDetailDTO.kt:26](#) Champs du détail réellement exposés.

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/controller/dto/DatasetUpdateDTO.kt:16](#) Champs modifiables et exclusion du mode de schéma.

[monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/command/UpdateDatasetCommandHandler.kt:522](#) Préconditions, autorisation du transfert et événements.

DM-03 **Cible non qualifiée****Rechercher le catalogue et distinguer découverte et accès aux données****COMPOTEMENT OBSERVÉ**

Trois recherches couvrent le catalogue publié, les datasets d'un projet et le catalogue enrichi de l'état de lien à un projet. Elles filtrent le texte, sensibilité, qualité, état, type et ID d'origine, déclenchement, mode de refresh et labels ; le projet ajoute la diffusion. Les cartes du catalogue restent découvrables même si le lecteur ne peut pas consulter les lignes, afin de permettre une demande d'accès. Le repository impose published, in_production, non reserved et non hidden pour le catalogue. Le tri est une liste de champs admis avec UUID de départage. REPLACE est traduit en OVERRIDE et l'ID d'origine accepte sa forme nue ou préfixée. Une taille non positive signifie actuellement une lecture sans borne pratique.

UTILISATEURS ET RÔLES

Lecteur du catalogue et membre de projet qui cherche ou demande un dataset.

INTERFACES DANS LE CODE

GET /catalog/datasets ; GET /catalog/projects/{projectId}/datasets ; GET /catalog/dataset-with-project-links.

DONNÉES ET ÉTATS

Vue dataset_catalog et projections PostgreSQL des propriétaires, origines, labels et permissions de projet.

DÉPENDANCES ET ÉCHANGES

Gateway IAM pour la route ; correspondance des IDs projet ; aucune lecture de table physique dans ce parcours.

COMPOTEMENT CIBLE PROPOSÉ

Catalog Rust conserve deux décisions séparées, découvrir une carte et lire une édition. Les index restent reconstructibles à partir des commandes métier et des publications approuvées. Les recherches ont des limites explicites et des curseurs stables, sans faire fuiter les schémas protégés.

MIGRATION DE CETTE CAPACITÉ

Rejouer les filtres, normalisations et tris existants sur un corpus comparé. Annoncer le remplacement de `size<=0` par une limite ou une tâche d'extraction. Ne pas masquer les cartes restreintes si le parcours de demande d'accès les exige.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un utilisateur qui découvre une carte restreinte peut demander son accès mais reçoit un refus sur son schéma et ses lignes. Tous les filtres combinés et leurs totaux correspondent au corpus migré ; les réservations et datasets cachés restent exclus.

Références de l'analyse

`monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/controller/DatasetCatalogController.kt:33` Trois contrats de recherche et paramètres.

`monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/query/SearchCatalogDatasetsQueryHandler.kt:30` La découverte publiée ne vaut pas droit de lecture.

`monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/repository/DatasetCatalogRepositoryImpl.kt:36` Prédicats de visibilité et requêtes.

`monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/repository/DatasetCatalogRepositoryImpl.kt:320` Filtres, alias, pagination et tri.

DM-04 **Cible non qualifiée****Mesurer les usages du catalogue****COMPOTEMENT OBSERVÉ**

Les KPI de datasets retournent un total publié non caché et les plus utilisés, mesurés par le nombre de liens accordés avec des projets publics. Ils ne mesurent pas les requêtes SQL ou les consultations. Le KPI de sources compte les datasets groupés par origine en excluant LOCAL et DATAFLOW. Les prédicats du total ne sont pas identiques à ceux de la recherche du catalogue, qui exclut aussi réservations et hors production.

UTILISATEURS ET RÔLES

Administrateur et tableau de bord du catalogue.

INTERFACES DANS LE CODE

GET /catalog/kpi/datasets ; GET /catalog/kpi/datasources, paramètre `maxResult`.

DONNÉES ET ÉTATS

Agrégations PostgreSQL de dataset, dataset_branch, dataset_permission et project.

DÉPENDANCES ET ÉCHANGES

Gateway sur les routes KPI ; projections des projets publics et sources.

COMPOTEMENT CIBLE PROPOSÉ

Catalog Rust préserve les indicateurs de liens en les nommant précisément. Un indicateur distinct de consommation peut venir des reçus Query et des EXPOSURE déclarées, avec son périmètre et sa durée de conservation.

MIGRATION DE CETTE CAPACITÉ

Reprendre les définitions existantes avant de changer les compteurs. Décider si le total doit suivre les mêmes règles que la recherche. Ne pas transformer un nombre de liens en métrique d'usage réel.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Deux liens accordés et une demande en attente produisent deux usages de partage. Un lien à un projet privé n'entre pas dans le classement public. Un téléchargement n'augmente pas ce KPI historique.

Références de l'analyse

[monorepo · services/data-model/app/data-model-](#)

[app/src/main/kotlin/com/cleyrop/data/model/app/dataset/controller/DatasetCatalogKpiController.kt:23](#) Endpoints KPI.

[monorepo · services/data-model/app/data-model-](#)

[app/src/main/kotlin/com/cleyrop/data/model/app/dataset/repository/DatasetCatalogRepositoryImpl.kt:234](#) Calcul exact des compteurs et classement.

DM-05 **Cible non qualifiée****Lire schémas et métadonnées par lot, et servir le SDK historique****COMPOTEMENT OBSERVÉ**

Le service expose le schéma d'un dataset ou de plusieurs datasets, éventuellement par projet et branche. Sans IDs explicites ni branche, il ne garde que la production. Les résultats de schéma passent un lot de décisions IAM fondées sur dataset, branche et liens accordés ; une réponse mal alignée échoue. POST /datasets/list peut déléguer le contrôle du corps à la gateway lorsque le drapeau de déploiement est activé, sinon interroge IAM. Le SDK dispose d'une liste dédiée /lib/datasets qui renvoie nom affiché, UUID, nom technique et table exprimée en UUID@main. Un ancien alias sélectionné par User-Agent pylib subsiste.

UTILISATEURS ET RÔLES

Interface SQL, éditeur de flow, bibliothèque Python et clients de catalogue.

INTERFACES DANS LE CODE

GET /datasets/{datasetId}/schema ; GET /datasets/schema ; POST /datasets/list ; GET /lib/datasets ; alias GET /datasets avec User-Agent pylib.

DONNÉES ET ÉTATS

Colonnes de branche, IDs de labels de colonnes, description et permission calculée ; projection en lecture.

DÉPENDANCES ET ÉCHANGES

IAM batch consultation ; repository de liens accordés ; IDs projet hérités traduits.

COMPOTEMENT CIBLE PROPOSÉ

Datasets Rust fournit un SchemaRef versionné associé à une EditionRef ; Catalog conserve les annotations. La façade SDK garde temporairement les anciennes formes d'identifiant. Chaque collection de résultats filtre les objets autorisés avant de calculer pagination et totaux.

MIGRATION DE CETTE CAPACITÉ

Mapper UUID@main vers une édition publiée résolue au début du run. Migrer les clients vers la route dédiée, puis retirer l'alias User-Agent seulement après inventaire. Tester séparément les deux modes de contrôle du corps.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une requête mélangeant deux datasets autorisés et un interdit ne révèle aucun schéma du troisième. Une décision IAM incomplète ne retourne pas un résultat partiel. Un ancien client peut toujours résoudre les tables de son projet pendant la période de compatibilité.

Références de l'analyse

[monorepo · services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/query/DatasetSchemaQueryHandler.kt:72](#) Sélection des branches et filtrage central.

[monorepo](#) · [services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/query/DatasetListQueryHandler.kt:26](#) Bascule entre contrôle du corps gateway et IAM.

[monorepo](#) · [services/data-model/app/data-model-](#)

[app/src/main/kotlin/com/cleyrop/data/model/app/dataset/controller/DatasetForLibController.kt:28](#) Contrat SDK et route de compatibilité.

DM-06 Cible non qualifiée

Synchroniser les branches issues des dataflows

COMPORTEMENT OBSERVÉ

Les événements de data-dev portent les sorties des scripts et les actions SAVE, MERGE et PUBLISH. Ils mettent à jour ou créent les branches métier en PostgreSQL, reprennent propriétaire, qualité, sensibilité et labels, et retirent les sorties devenues orphelines selon la branche et l'état du flow. Une nouvelle branche duplique les colonnes de la branche métier main avec de nouveaux UUID de colonne et sans annotations. inProduction est monotone. Le mode de schéma du flow n'est adopté que lors de la création de la configuration. La suppression d'un flow peut conserver explicitement certains datasets en marquant leur source supprimée.

UTILISATEURS ET RÔLES

Auteur de dataflow, ordonnanceur et consommateurs des événements de versions.

INTERFACES DANS LE CODE

GET /datasets/{datasetId}/branches ; DataflowDecisionCommandHandler ; événements data-dev-graph-execution et data-dev-dataflow.

DONNÉES ET ÉTATS

DatasetBranch métier, SourceDataset, configuration, branches retirées et sorties conservées.

DÉPENDANCES ET ÉCHANGES

data-dev fournit le graphe et ses sorties ; process-data confirme les données et schémas.

COMPORTEMENT CIBLE PROPOSÉ

Datasets et Flows Rust partagent des identités stables de sorties, drafts, environnements et éditions. Une branche de travail référence explicitement sa base ; un merge prépare des candidats sous WAP et ne vaut pas publication. Les field IDs Iceberg remplacent la duplication aveugle des UUID de colonnes.

MIGRATION DE CETTE CAPACITÉ

Importer le mapping branche métier vers ref physique et relever les divergences. Conserver les règles de sorties gardées ou supprimées. Couper l'ancien consommateur pour chaque cohorte avant que Rust devienne le seul propriétaire des transitions.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un merge qui retire une sortie de la branche ne supprime pas une édition encore référencée en production. Une nouvelle branche garde sa base vérifiable. Un événement répété ne crée ni deuxième branche ni nouvelle publication.

Références de l'analyse

[monorepo](#) · [services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/command/DataflowDecisionCommandHandler.kt:174](#) Actions réellement acceptées et nettoyage des sorties.

[monorepo](#) · [services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/aggregate/Dataset.kt:104](#) Création et mise à jour des branches et configuration.

[monorepo](#) · [services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/aggregate/Schema.kt:52](#) Duplication de colonnes et perte volontaire des annotations.

DM-07 **Cible non qualifiée**

Éditer la structure et les annotations de colonnes

COMPORTEMENT OBSERVÉ

PUT schema distingue les annotations seules des renommages, casts et suppressions. Il exige une commande non vide, des IDs de colonnes présents une seule fois et des labels existants. Les noms renommés suivent une grammaire normalisée et restent uniques après toutes les opérations. Les casts suivent une matrice codée ; une branche secondaire ne peut modifier une colonne héritée de la branche métier main. Il faut garder au moins deux colonnes et ne pas supprimer la dernière. Une opération structurelle est refusée si le dernier refresh n'est pas final, sauf s'il est jugé périmé après quinze minutes. Le catalogue écrit immédiatement le schéma souhaité et un historique CREATED ; une annotation seule crée un historique SUCCESS.

UTILISATEURS ET RÔLES

Gestionnaire du dataset ou auteur autorisé à modifier son schéma.

INTERFACES DANS LE CODE

PUT /datasets/{datasetId}/schema ; UpdateSchemaCommand et UpdateColumnCommand.

DONNÉES ET ÉTATS

DatasetColumn, labels, schemaBefore/schemaAfter et historique UPDATE_COLUMN ; événements de traitement ultérieurs.

DÉPENDANCES ET ÉCHANGES

Process-data/data-dev exécutent la modification physique ; data-serve peut réconcilier les types physiques.

COMPORTEMENT CIBLE PROPOSÉ

Datasets Rust sépare ProposedSchema et schéma de l'édition publiée. Writer réalise la transformation sur candidat, Audit contrôle les types et Publisher publie après CAS. Le schéma visible aux lecteurs ne change pas avant publication. Les annotations restent liées aux field IDs.

MIGRATION DE CETTE CAPACITÉ

Importer schémas déclarés et physiques côte à côte puis traiter les écarts. Requalifier la matrice actuelle, notamment les conversions vers flottant, sans la déclarer sans perte par son commentaire. Remplacer le délai de quinze minutes par un lease et une décision de reprise explicites.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un cast qui échoue laisse l'ancien schéma et l'ancienne édition visibles. Un champ renommé conserve son field ID et ses annotations. Deux commandes concurrentes sur la même base aboutissent à un conflit explicite ou à un seul résultat publié.

Références de l'analyse

[monorepo](#) · [services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/command/UpdateSchemaCommandHandler.kt:132](#) Validations des opérations de colonne.

[monorepo](#) · [services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/command/UpdateSchemaCommandHandler.kt:298](#) Écriture immédiate et historique avant exécution physique.

[monorepo](#) · [services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/aggregate/Schema.kt:9](#) Matrice des casts réellement codée.

DM-08 **Cible non qualifiée**

Conserver les contrats de format et de préparation des entrées

COMPORTEMENT OBSERVÉ

Les configurations de refresh gardent CSV, tableur, JSON, XML et Parquet. Les formats portent encodage, compression NONE/GZIP, MIME, suffixes et présence d'en-tête. CSV ajoute séparateur, commentaire, lignes et colonnes ignorées, trim et format temporel ; tableur ajoute feuille ; JSON et XML ajoutent leur chemin d'extraction. Les préparations comprennent transposition, nom de fichier, URL et date ou date-heure d'import. Les filtres de fichiers portent égalité, exclusion ou sous-chaîne du nom. Ce service conserve et expose ces choix ; il n'en exécute pas les parseurs.

UTILISATEURS ET RÔLES

Utilisateur de collecte et lecteur de la configuration historique.

INTERFACES DANS LE CODE

DatasetConfigDTO ; configurations dans GET /datasets/{datasetId} et dans le détail de refresh ; événements de collecte.

DONNÉES ET ÉTATS

ConfigConnector, FileFormat, Preparation et filtres rattachés à la source/historique.

DÉPENDANCES ET ÉCHANGES

data-collect et process-data produisent les événements et réalisent les lectures/préparations.

COMPORTEMENT CIBLE PROPOSÉ

Connectors et Datasets Rust versionnent un FormatContract utilisable par des workers bornés. Le parsing et les normalisations reçoivent une version de contrat, des limites d'octets et un rapport d'erreurs. Les fichiers originaux peuvent former un FILESET publié distinct de la TABLE interprétée.

MIGRATION DE CETTE CAPACITÉ

Traduire chaque option avec un corpus de fichiers réels anonymisés. Préserver la configuration appliquée au run, pas seulement le réglage courant. Ne pas reprendre comme garantie la constante XLSX non utilisée ou les commentaires sur dates sans qualification.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Le même fichier, contrat et moteur versionné donnent le même schéma et le même compte de lignes. L'encodage invalide, la feuille absente et le dépassement de taille produisent un échec borné sans publication. Les colonnes ajoutées restent traçables.

Références de l'analyse

`monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/aggregate/Dataset.kt:463` Formats source et filtres déclarés.

`monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/aggregate/fileFormat/CsvFileFormat.kt:5` Options CSV.

`monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/aggregate/fileFormat/XlsxFileFormat.kt:5` Options tableur.

`monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/aggregate/fileFormat/JsonFileFormat.kt:5` Chemin JSON.

`monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/aggregate/fileFormat/XmlFileFormat.kt:5` Chemin XML.

`monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/aggregate/Preparation.kt:3` Préparations représentées.

DM-09 **Cible non qualifiée**

Restituer les paramètres des sources et des connecteurs

COMPORTEMENT OBSERVÉ

Le domaine distingue connexions et configurations de collecte. Il représente fichiers locaux, SFTP, dépôt, S3, API, PostgreSQL, MySQL, MSSQL, Oracle, BigQuery, Google Sheets, Jira, Cleyrop, Snowflake, Salesforce, MS Dynamics, SAP S/4HANA, Business Central et SharePoint. Les sources SQL portent table, type et requête personnalisée. Les APIs portent pagination par page, offset, curseur, lien JSON ou Link HTTP. Les quatre connecteurs métier typés gardent entity/select/order/filter/limit et version API, avec service ou company selon le cas. Les DTO ne les réduisent pas à une API générique et les credentials restent dans la couche datasource de data-collect.

UTILISATEURS ET RÔLES

Utilisateur qui inspecte une collecte, support et futur migrateur de Sources.

INTERFACES DANS LE CODE

DatasetConfigDTO.toDTO ; Connector/ConfigConnector ; détail dataset et historique.

DONNÉES ET ÉTATS

Configuration typée dans PostgreSQL, associée au refresh ; origine source et éventuel chemin de fichier.

DÉPENDANCES ET ÉCHANGES

data-collect autoritaire pour les connexions ; protocole Protobuf et mappings DAO/DTO.

COMPORTEMENT CIBLE PROPOSÉ

Connectors Rust possède les références de secrets et les capacités des drivers. Catalog affiche une projection expurgée du contrat exécuté. Le type de source reste indépendant du kind de dataset. Une pagination source n'est jamais utilisée comme checkpoint publié par simple copie.

MIGRATION DE CETTE CAPACITÉ

Reprendre les discriminants et configurations sans exporter de secrets. Vérifier chaque round-trip DAO/Protobuf/DTO. Porter les curseurs durables vers origin.* du candidat et ne les acquitter qu'après publication.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une configuration SQL personnalisée ou OData conserve tous ses champs après migration. Le détail historique ne révèle aucune valeur secrète. Une reprise de pagination ne saute aucune page après une publication refusée.

Références de l'analyse

[monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/aggregate/connector/Connector.kt:9](#) Familles de connecteurs actuelles.

[monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/aggregate/configconnector/PaginationConfig.kt:13](#) Six modes de pagination.

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/controller/dto/config/DatasetConfigDTO.kt:38](#) Correspondance exhaustive des DTO.

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/controller/dto/config/DatasetConfigDTO.kt:114](#) Conservation des champs typés et séparation des credentials.

DM-10 **Cible non qualifiée**

Suivre les collectes, traitements et historiques de refresh

COMPORTEMENT OBSERVÉ

Le service agrège les événements de collecte et de traitement dans un historique par dataset et branche. Il conserve numéro d'exécution, auteur, déclenchement, append/override, état, opérations ingestion/process, dates, volumes, logs et entrées fichiers ou tables avec erreurs. Il calcule les durées et associe le compute qualité au refresh. Les routes de détail et logs vérifient que le refresh appartient au dataset. Les statuts terminaux et les opérations de schéma suivent des chemins séparés pour ne pas écraser un résultat final par un événement running tardif.

UTILISATEURS ET RÔLES

Auteur, exploitant, support et utilisateur du détail dataset.

INTERFACES DANS LE CODE

GET /datasets/{datasetId}/refresh ; GET /datasets/{datasetId}/refresh/{refreshId} ; GET /datasets/{datasetId}/refresh/{refreshId}/logs/{logType}.

DONNÉES ET ÉTATS

DatasetRefreshHistory, DataInputFileRefreshRecord, DataInputTableRefreshRecord, Ingestion/Process/UpdateSchema et logs PostgreSQL.

DÉPENDANCES ET ÉCHANGES

Consommateurs data-collect/process-data ; données qualité reçues de data-serve ; notifications d'échec.

COMPORTEMENT CIBLE PROPOSÉ

Flows Rust conserve les runs et leurs tentatives ; Datasets expose leur relation avec candidats et éditions. Audit garde les preuves immuables. Le checkpoint publié provient de l'édition Iceberg autoritaire, même si une projection de run a pris du retard.

MIGRATION DE CETTE CAPACITÉ

Conserver les anciens IDs de refresh en références de migration et importer logs avec accès contrôlé. Rejouer les événements hors ordre pour vérifier les transitions. Mapper volumes et durées sans inventer de snapshot pour les échecs historiques.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un événement running tardif ne rouvre pas un run terminal. Le détail d'un refresh d'un autre dataset retourne un introuvable. Après perte de la projection, le dernier checkpoint se reconstruit de l'édition publiée sans réémission d'effet.

Références de l'analyse

[monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/aggregate/DatasetRefresh.kt:19](#) Modèle des historiques et entrées.

[monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/aggregate/RefreshHistoryRecord.kt:38](#) Protection des états terminaux de process.

[monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/query/DatasetRefreshHistoryDetailQueryHandler.kt:46](#) Lien parent/enfant, durées et qualité.

[monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/query/OperatorLogsQueryHandler.kt:62](#) Restriction des logs au couple dataset/refresh.

DM-11 **Cible non qualifiée**

Régler le déclenchement et les reprises planifiées

COMPORTEMENT OBSERVÉ

Le modèle conserve déclenchements manuel, planifié, récurrent ou événements dataset/dataflow/fichier, avec schedules, fuseau et RetryPolicy. La mise à jour depuis le catalogue ne prend en charge que MANUAL ou RECURRENT et ne modifie le planning que pour une source vivante autre que DATAFLOW et LOCAL. Elle émet DatasetScheduleUpdatedEvent ; ce service n'exécute pas lui-même les collectes planifiées. La configuration du schéma reste séparée de cette modification.

UTILISATEURS ET RÔLES

Propriétaire qui règle le refresh et ordonnanceur consommateur.

INTERFACES DANS LE CODE

PUT /datasets/{datasetId} avec refreshConfig ; DatasetScheduleUpdatedEvent ; SynchronizedRefreshCommandHandler.

DONNÉES ET ÉTATS

DatasetRefreshConfig, Scheduler et RetryPolicy ; événements de planning.

DÉPENDANCES ET ÉCHANGES

process-data/data-collect pour déclenchement et retours ; zone horaire du contrat.

COMPORTEMENT CIBLE PROPOSÉ

Flows Rust devient l'unique propriétaire du schedule, de ses occurrences, reprises et arrêts. Catalog conserve une projection lisible. Chaque occurrence lance un run borné avec ID stable, deadline, budget de retry et checkpoint publié de départ.

MIGRATION DE CETTE CAPACITÉ

Importer cron, timezone et retry en préservant l'heure locale et les passages heure été/hiver. Pour chaque cohorte, désactiver le propriétaire ancien avant d'activer Rust. Décider explicitement le traitement des occurrences manquées.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une occurrence à la transition de fuseau suit une règle documentée sans double publication. Un redémarrage d'ordonnanceur retrouve le même ID d'occurrence. Modifier les labels seuls ne change pas le prochain déclenchement.

Références de l'analyse

[monorepo · services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/aggregate/DatasetRefreshConfig.kt:10](#) Schedules et paramètres de reprise.

[monorepo · services/data-model/app/data-model-](#)

[app/src/main/kotlin/com/cleyrop/data/model/app/dataset/controller/dto/DatasetDTO.kt:183](#) Déclenchements exposés et mises à jour admises.

[monorepo · services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/command/UpdateDatasetCommandHandler.kt:582](#) Sources réellement modifiables et événement de planning.

DM-12 **Cible non qualifiée**

Rendre visible un dataset de projet

COMPORTEMENT OBSERVÉ

publish refuse un dataset caché puis met seulement isPublished à true, sauvegarde le dataset et émet les faits de visibilité. Il n'effectue aucun commit Iceberg, audit de candidat ou déplacement de ref. unhide est une action distincte. Les données peuvent donc avoir été produites avant cette publication de catalogue. Les permissions et affichages actuels s'appuient sur cette distinction projet privé, projet publié et catalogue global.

UTILISATEURS ET RÔLES

Gestionnaire autorisé à éditer le dataset et lecteurs du catalogue.

INTERFACES DANS LE CODE

POST /datasets/{datasetId}/publish ; POST /datasets/{datasetId}/unhide.

DONNÉES ET ÉTATS

Booléens hidden, inProduction, isPublished et faits authz ; dernier refresh associé au message.

DÉPENDANCES ET ÉCHANGES

IAM/gateway pour can_edit ; événement dataset et dataset.published.

COMPORTEMENT CIBLE PROPOSÉ

Catalog Rust nomme cette opération diffusion. Publisher Rust possède séparément la publication d'une édition approuvée. Une édition reçoit origin.* au summary du candidat, une preuve d'audit durable et une référence approuvée ; la diffusion ne remplace aucun de ces contrats.

MIGRATION DE CETTE CAPACITÉ

Conserver une façade /publish qui préserve d'abord la diffusion historique. Créer une commande séparée de publication de données et migrer les usages un par un. Ne jamais transformer rétroactivement tous les snapshots physiques en publications approuvées.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Publier la diffusion d'un dataset ne modifie ni snapshot ni checkpoint. Un candidat refusé par Audit ne devient pas lisible parce que isPublished est vrai. L'historique utilisateur ne contient que les éditions approuvées.

Références de l'analyse

[monorepo · services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/command/PublishProjectDatasetCommandHandler.kt:58](#) La publication actuelle ne fait que modifier le référentiel et émettre des événements.

[monorepo · services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/aggregate/Dataset.kt:302](#) Implémentation des drapeaux publish et unhide.

[monorepo · services/data-model/app/data-model-](#)

[app/src/main/kotlin/com/cleyrop/data/model/app/dataset/controller/DatasetCreateController.kt:252](#) Actions REST distinctes.

DM-13 **Cible non qualifiée**

Demander, accorder et retirer un accès de projet

COMPORTEMENT OBSERVÉ

Un dataset de projet non publié ne peut pas être lié ailleurs. Pour INTERNAL, la commande vérifie la politique de labels puis accorde immédiatement le lien. Pour RESTRICTED ou SENSIBLE, elle crée une demande avec auteur et commentaire, notifie et abonne le demandeur à la réponse. Accepter enregistre validateur, date et justification, revalide les labels et émet le fait d'accès. Refuser un lien déjà accordé révoque l'alias de projet, sauf le projet propriétaire. La route de retrait exige une justification non blanche, vérifie l'appartenance permission/dataset et n'accepte que l'état ACCEPTED. La liste /access ne montre que cet état, alors que les liens INTERNAL automatiques emploient un autre état accordé.

UTILISATEURS ET RÔLES

Demandeur membre de projet, responsable du dataset, administrateur de partage.

INTERFACES DANS LE CODE

POST /datasets/{datasetId}/projects/{projectId} ; GET /datasets/permissions ; POST /datasets/permissions/{permissionId}/{permission} ; GET /datasets/{datasetId}/permissions/projects ; GET /datasets/{datasetId}/access ; POST /datasets/{datasetId}/access/{permissionId}/withdraw.

DONNÉES ET ÉTATS

DatasetProjectPermission, CREATED/REQUESTED/ACCEPTED/REFUSED/LINKED et justifications ; tuple IAM seulement pour droits accordés.

DÉPENDANCES ET ÉCHANGES

Policy Datasphere, utilisateurs/projets, notification-center, faits authz vers IAM et liens vers data-serve.

COMPORTEMENT CIBLE PROPOSÉ

Catalog et IAM Rust conservent demande, approbation et révocation comme commandes auditées. L'accès porte une portée claire, dataset ou édition, et une politique versionnée. La révocation doit invalider les prochaines pages et téléchargements selon le contrat de lecture.

MIGRATION DE CETTE CAPACITÉ

Importer les états et rattachements sans accorder les demandes en attente. Revoir l'écart entre lien automatique et liste/retrait des accès pour offrir un retrait cohérent. Préserver le projet propriétaire comme relation distincte.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une demande en attente ne permet jamais une lecture. Une acceptation répétée ne crée pas deux liens. Le retrait d'un partage bloque la prochaine page et laisse le projet propriétaire autorisé. La permission d'un autre dataset reste introuvable.

Références de l'analyse

[monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/command/CreateDatasetLinkToProjectCommandHandler.kt:140](#)
Différence entre partage immédiat et demande.

[monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/command/AcceptDatasetLinkToProjectCommandHandler.kt:72](#)
Acceptation et révocation des liens accordés.

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/controller/DatasetAccessController.kt:35](#) Filtre de liste et conditions de retrait.

Administrer les labels de datasets et de colonnes

COMPORTEMENT OBSERVÉ

Les labels de dataset portent nom et couleur ; la création refuse un nom existant sans tenir compte de la casse. Leur suppression retire le label des branches avant de supprimer la définition et diffuse un événement. Les labels de colonnes sont une entité séparée, avec nom/couleur non blancs, utilisée par UpdateSchema. Le code des handlers create et update de SchemaLabel lève AlreadyExists lorsque getByName retourne null. Le repository renvoie bien null pour un nom absent. Cette condition paraît inversée ; sa conséquence est à reproduire, elle ne doit pas être décrite comme une unicité correcte. Les contrôleurs demandent actuellement à IAM le droit de gérer les labels.

UTILISATEURS ET RÔLES

Administrateur des vocabulaires et éditeur de métadonnées.

INTERFACES DANS LE CODE

GET/POST /labels ; PUT/DELETE /labels/{labelId} ; GET/POST /schema/labels ; PUT/DELETE /schema/labels/{schemaLabelId}.

DONNÉES ET ÉTATS

Label, SchemaLabel, liens aux branches et colonnes ; événements de labels pour data-serve.

DÉPENDANCES ET ÉCHANGES

IAM label management ; repositories et outbox des labels dataset.

COMPORTEMENT CIBLE PROPOSÉ

Catalog Rust garde deux usages explicites, tags de dataset et annotations de champs, avec IDs stables, unicité normalisée et suppression contrôlée. Ces tags descriptifs restent distincts des politiques de sécurité et des classifications IAM.

MIGRATION DE CETTE CAPACITÉ

Importer les IDs et références ; définir la normalisation du nom. Corriger le comportement SchemaLabel avec un ticket vérifiable avant de figer la compatibilité. Conserver l'historique des annotations sur éditions anciennes.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Créer un nouveau label de colonne réussit ; un doublon est refusé selon la règle documentée. Supprimer un label courant ne modifie pas les preuves historiques d'une édition. Deux créations concurrentes ne produisent pas deux définitions équivalentes.

Références de l'analyse

[monorepo · services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/label/command/CreateLabelCommandHandler.kt:24](#) Unicité du label dataset.

[monorepo · services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/label/command/DeleteLabelCommandHandler.kt:25](#) Retrait des références de branches.

[monorepo · services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/label/command/CreateSchemaLabelCommandHandler.kt:25](#) Condition d'existence observée.

[monorepo · services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/label/command/UpdateSchemaLabelCommandHandler.kt:26](#) Même condition dans update.

[monorepo · services/data-model/app/data-model-](#)

[app/src/main/kotlin/com/cleyrop/data/model/app/label/repository/SchemaLabelRepositoryImpl.kt:27](#) Sémantique effective de getByName.

DM-15 **Cible non qualifiée**

Présenter la qualité et les abonnements de suivi

COMPORTEMENT OBSERVÉ

data-model reçoit les définitions et résultats qualité de data-serve, puis expose le dernier compute par dataset et les résultats associés aux refreshs. Le tableau global sélectionne ALL, OWNER, PROJECTS, CATALOG ou MEMBERSHIP, peut combiner catalogue et projets, et filtre qualité, sensibilité et état. Il autorise les candidats par IAM avant de calculer les stats et ne nomme que les projets accessibles. Il distingue dataset avec règles, succès, erreurs et niveaux CRITICAL/MAJOR/MINOR. Les memberships DATASET_QUALITY viennent aussi des événements de notification ; ils représentent une souscription de suivi.

UTILISATEURS ET RÔLES

Responsable qualité, propriétaire et utilisateur abonné aux alertes.

INTERFACES DANS LE CODE

GET /projects/{projectId}/datasets/monitoring ; POST /quality/monitoring ; DataServeConsumer ; NotificationCenterEventConsumer.

DONNÉES ET ÉTATS

Projections Rule, RuleHistory, Compute, ComputedRule et vue de monitoring ; DatasetMembership.

DÉPENDANCES ET ÉCHANGES

data-serve produit règles/calculs ; notification-center produit abonnements ; IAM filtre consultation et projets.

COMPORTEMENT CIBLE PROPOSÉ

Audit Rust possède les exécutions de règles liées aux candidats et aux éditions ; Catalog en affiche une projection. Notifications possède les préférences de suivi. Une mesure peut bloquer Publisher selon une politique explicite, ce que ce tableau actuel ne garantit pas.

MIGRATION DE CETTE CAPACITÉ

Reprendre définitions, versions et résultats en conservant leur provenance d'exécution. Mapper les abonnements sans les convertir en autorisations. Comparer les agrégats pour chaque catégorie et documenter le statut des résultats historiques sans snapshot.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un dataset interdit ne contribue ni aux items ni aux compteurs personnels. Une règle échouée avant publication laisse l'édition antérieure lisible. Le dernier compute affiché indique la version de règle et l'édition effectivement contrôlée.

Références de l'analyse

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/controller/DatasetQualityController.kt:40](#) Les deux API de monitoring.

[monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/query/DatasetQualityMonitoringQueryHandler.kt:125](#) Sélection, autorisation et statistiques.

[monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/query/DatasetProjectRulesQueryHandler.kt:86](#) Projection de qualité par projet.

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/event/consumer/DataServeConsumer.kt:23](#) Réception des règles et computes.

DM-16 **Cible non qualifiée**

Vérifier les politiques de labels et lire les fichiers SPIF

COMPORTEMENT OBSERVÉ

Une politique est un triplet `policy/classification/category` lié au dataset. Sa modification appelle Datasphere, puis inscrit une promotion en attente. Le contrôle de partage envoie les emails des membres du projet à `/access/check` et refuse les sujets DENY. Une réponse absente du client actuel devient une liste vide, donc aucune interdiction ; la couverture complète des sujets n'est pas vérifiée ici. Si Datasphere est désactivé, l'implémentation `no-op` permet les contrôles et ne fournit pas de politique. Le proxy SPIF exige une URI HTTP(S) absolue et un préfixe autorisé à frontière de chemin avant le GET sortant.

UTILISATEURS ET RÔLES

Administrateur de politique, gestionnaire de partage et lecteur du label de sécurité.

INTERFACES DANS LE CODE

PUT/GET `/datasets/{datasetId}/label/policy` ; GET `/datasets/{datasetId}/projects/{projectId}/label/policy/check` ; PUT `/spif-proxy/files`.

DONNÉES ET ÉTATS

Policy autoritaire externe ; PendingPromotion local ; décisions PERMIT/DENY.

DÉPENDANCES ET ÉCHANGES

Datasphere HTTP `/labels`, `/access/check` et fichier SPIF externe autorisé ; miroir des membres de projet.

COMPORTEMENT CIBLE PROPOSÉ

IAM Rust décide sur une politique versionnée et une liste complète de sujets. Catalog garde les références et l'état. Egress centralise le fetch SPIF avec contrôle de destination et de taille ; indisponibilité ou décision incomplète bloque l'opération dépendante.

MIGRATION DE CETTE CAPACITÉ

Inventorier les tenants qui utilisent Datasphere et ceux qui utilisent le `no-op`. Porter le contrat de classification sans inventer de chiffrement. Remplacer l'interprétation permissive des réponses vides par un refus explicite qualifié avec les clients.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un service de politique indisponible ou qui omet un membre bloque le partage. Une URL dont l'hôte imite un préfixe admis est refusée avant l'appel réseau. Une politique modifiée conserve sa version et l'identité de son auteur.

Références de l'analyse

[monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/promotion/command/LabelPolicyCommandHandler.kt:53](#) Écriture externe puis promotion en attente.

[monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/promotion/query/CheckLabelPolicyQueryHandler.kt:36](#) Décision sur tous les emails du miroir.

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/promotion/service/LabelPolicyServiceRestImpl.kt:151](#) Décodage des décisions, valeur vide et appels.

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/promotion/service/LabelPolicyServiceRestImpl.kt:217](#) Garde URL SPIF et `no-op` conditionnel.

DM-17 **Cible non qualifiée**

Promouvoir sous politique et suivre les demandes de chiffrement

COMPORTEMENT OBSERVÉ

L'action encrypt vérifie qu'une politique existe, marque le dataset comme promoted puis demande à Datasphere /labels/encrypt sur le nom technique et le dataset. Elle prend le dernier refresh réussi et conserve statut/log de promotion lié à ce refresh. Les datasets marqués sont retraités lors de refreshs ultérieurs. Une tâche périodique traite les changements de label lorsque la politique distante correspond à celle en attente. Sans historique réussi, le handler journalise l'absence mais garde le flag promoted. Ce code ne prouve ni algorithme de chiffrement ni atomicité physique.

UTILISATEURS ET RÔLES

Administrateur de promotion et exploitant qui suit son résultat.

INTERFACES DANS LE CODE

POST/GET /datasets/{datasetId}/label/encrypt ; GET /datasets/{datasetId}/history/{datasetRefreshHistoryId}/encrypt ; AsyncPromoteServiceImpl.

DONNÉES ET ÉTATS

DatasetPromoted, PendingPromotion et DatasetPromotionHistory avec SUCCESS/FAILURE.

DÉPENDANCES ET ÉCHANGES

Datasphere /labels/encrypt ; historique de refresh et tâche périodique locale.

COMPORTEMENT CIBLE PROPOSÉ

Egress et Audit Rust représentent cette opération comme un effet idempotent consommant une EditionRef publiée, avec reçu, politique et preuve. Si un chiffrement change les données ou artefacts de référence, il produit un nouveau candidat sous Writer/Publisher plutôt qu'un alias de publication opaque.

MIGRATION DE CETTE CAPACITÉ

Vérifier le contrat effectif du service distant, les données chiffrées, les lecteurs et les reprises. Importer les historiques comme reçus hérités non qualifiés. Remplacer le simple booléen promoted par un état d'effet demandé/réussi/échoué.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une demande sans édition publiée est refusée explicitement. Après réponse HTTP perdue, le retry retrouve le même reçu et ne chiffre pas deux fois. L'interface n'annonce un succès que lorsque le reçu de l'édition et de la politique attendues existe.

Références de l'analyse

monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/promotion/command/LabelPolicyPromoteCommandHandler.kt:68 Effet externe et historique.

monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/promotion/command/LabelPolicyPromoteCommandHandler.kt:98 Flag avant appel et absence possible de refresh.

monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/promotion/command/LabelPolicyPromoteCommandHandler.kt:118 Traitement des relabels.

monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/promotion/service/AsyncPromoteServiceImpl.kt:12 Tâche périodique.

DM-18 **Cible non qualifiée**

Vider, supprimer et libérer les noms sans réexposer des données

COMPORTEMENT OBSERVÉ

Empty vérifie dataset et branche puis émet un événement ; il ne vide pas lui-même la table. Delete refuse une sortie de dataflow dont la source est encore vivante. La suppression habituelle conserve un tombstone DatasetDeletion, le nom technique et les tuples IAM jusqu'au succès du job DATASET_DELETE déclenché côté data-dev. Un échec ou timeout relance l'événement avec nombre d'essais borné ; ABANDONED conserve le nom pour éviter qu'une nouvelle identité lise les anciennes lignes. Les suppressions de projet utilisent toutefois un chemin sans attente de drop. Une tâche libère aussi les réservations sans type trop anciennes.

UTILISATEURS ET RÔLES

Gestionnaire, suppression de projet ou flow, opérateur de reprise.

INTERFACES DANS LE CODE

DELETE /datasets/{datasetId} ; DELETE /datasets/{datasetId}/reserved ; POST /datasets/{datasetId}/empty ; DatasetDropSucceededCommand/FailedCommand.

DONNÉES ET ÉTATS

DatasetDeletion PENDING/FAILED/ABANDONED, branches, nom réservé et faits IAM différés.

DÉPENDANCES ET ÉCHANGES

data-dev/process-data pour drop physique ; événements Kafka et tâches de retry/rétention.

COMPORTEMENT CIBLE PROPOSÉ

Datasets Rust garde le tombstone métier. Maintenance exécute un plan audité pour vider, expirer ou supprimer après vérification de toutes les racines de rétention. Writer/Publisher publie une édition vide sous WAP si nécessaire ; aucun worker n'a le droit REST générique de drop.

MIGRATION DE CETTE CAPACITÉ

Importer toutes les suppressions pendantes et réservations avant de réouvrir les noms. Reprendre le cas projet supprimé avec une autorité Maintenance dédiée. Séparer effacement visible, fin d'accès et GC des octets.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un drop échoué ne libère jamais le nom. Une publication conservée, un run actif ou une release multi-sorties empêche le GC. Une confirmation répétée de drop ne réémet pas les mêmes effets. Empty produit une édition vide approuvée avant de modifier les lectures.

Références de l'analyse

[monorepo · services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/command/EmptyDatasetCommandHandler.kt:29](#) Empty est une demande asynchrone.

[monorepo · services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/command/DeleteDatasetCommandHandler.kt:130](#) Tombstone et conservation des tuples.

[monorepo · services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/command/DeleteDatasetCommandHandler.kt:211](#) Exception de suppression de projet.

[monorepo · services/data-model/app/data-model-](#)

[app/src/main/kotlin/com/cleyrop/data/model/app/dataset/service/DatasetDeletionRetryService.kt:56](#) Retry borné et abandon sûr.

[monorepo · services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/command/DatasetDropCommandHandler.kt:28](#) Finalisation idempotente de la suppression.

DM-19 **Cible non qualifiée**

Fournir une liaison de table avant la projection de serving

COMPORTEMENT OBSERVÉ

Le protocole interne retourne datasetId, branche métier, techName, projet, propriétaire, reserved et hasSchema. Il lit la réservation autoritaire avant qu'une première ingestion existe dans data-serve. Si le dataset a disparu, il peut répondre à partir du tombstone en ajoutant deleting. La branche doit exister dans le dataset ou le tombstone. Le contrôleur exige le JWT du compte de service data-serve, en plus du contrôle gateway, et refuse un simple utilisateur du même client OAuth.

UTILISATEURS ET RÔLES

data-serve et protocole de capacité pour jobs de lecture/écriture/suppression.

INTERFACES DANS LE CODE

GET /internal/execution-datasets/{datasetId}?branch=... ; ExecutionDatasetBindingDTO.

DONNÉES ET ÉTATS

Réservation active ou DatasetDeletion autoritaire ; aucune lecture des octets.

DÉPENDANCES ET ÉCHANGES

IAM/gateway et identité de service ; futur consommateur de capacité.

COMPORTEMENT CIBLE PROPOSÉ

Datasets Rust fournit un Binding immuable signé ou référencé qui lie dataset UUID, table UUID Iceberg, namespace, branche, schéma, spec, action, run et époque. Writer vérifie ce contrat et médie staging ; Publisher médie publication. Les CAS et contrôles d'ascendance ne remplacent pas un fencing natif Hemera à construire.

MIGRATION DE CETTE CAPACITÉ

Conserver le chemin compatible pour les exécutions anciennes, mais ne pas réutiliser techName seul comme autorité. Capturer UUID de table et références lors du basculement. Définir un Binding de suppression indépendant du projet déjà effacé.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un worker muni d'un binding expiré ou d'une époque précédente ne peut modifier ni supprimer la table. Renommer un alias ne change pas la table liée. Un compte humain du client data-serve est refusé par le protocole interne.

Références de l'analyse

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/controller/ExecutionDatasetBindingController.kt:16](#) Payload du protocole.

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/controller/ExecutionDatasetBindingController.kt:34](#) Vérification du compte de service et branche.

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/controller/ExecutionDatasetBindingController.kt:59](#) Résolution à partir du tombstone.

DM-20 **Cible non qualifiée**

Réconcilier les identités et publier les faits d'autorisation

COMPORTEMENT OBSERVÉ

Des miroirs utilisateurs/projets/membres sont alimentés par les anciens événements et, sous drapeaux, par IAM. Les IDs externes des projets et sujets utilisateur sont traduits aux frontières. Les événements IAM couvrent création, mise à jour, suppression et offboarding avec remplacement de propriétaires ; les membres arrivés avant leurs parents passent par une file persistée et un drainage. Les faits dataset créé, publié, reclassifié, propriétaire ou projet changé et supprimé partent vers IAM via outbox. IAM reste l'écrivain OpenFGA. Les drapeaux de migration et de traitement destructif ne constituent pas une preuve de bascule dans un environnement donné.

UTILISATEURS ET RÔLES

IAM, administrateur de migration, responsables des datasets réattribués.

INTERFACES DANS LE CODE

IamEventConsumer ; ProjectIdResolver ; UserIdResolver ; DatasetAuthzEventHandler ; topics IAM et data-model.authz.events.

DONNÉES ET ÉTATS

Miroirs user/project/member, tables de correspondance, pending members, high-water marks et outbox.

DÉPENDANCES ET ÉCHANGES

IAM et anciennes sources benutzer/data-dev ; OpenFGA écrit uniquement par IAM ; notification-center pour transfert.

COMPORTEMENT CIBLE PROPOSÉ

IAM Rust possède les identités canoniques ; Catalog consomme des événements ordonnés et dédoublés pour sa présentation. Les changements de propriétaire et de classification ont une révision et un reçu. Un retard de projection ne permet pas d'autoriser avec une identité périmée.

MIGRATION DE CETTE CAPACITÉ

Importer les correspondances au lieu de supposer IAM UUID égal au subject. Drainer les pending members et prouver la parité avant coupure. Rejouer les faits sur un groupe isolé, puis désactiver les consommateurs hérités cohorte par cohorte.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un offboarding réattribue toutes les branches et n'accorde rien au compte supprimé. Rejouer le journal ne double ni membres ni droits. Un membre reçu avant son projet devient visible après création sans intervention manuelle ni perte d'événement.

Références de l'analyse

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/event/consumer/IamEventConsumer.kt:47](#) Activation et consommation des événements IAM.

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/event/consumer/iam/IamUserEventHandler.kt:42](#) Création, suppression et offboarding.

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/service/ProjectIdResolver.kt:95](#) Traductions de clés aux frontières.

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/event/consumer/iam/IamPendingMemberDrain.kt:90](#) Drainage des membres différés.

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/common/outbox/OutboxPublisher.kt:13](#) Contrat de persistance des messages.

DM-21 **Cible non qualifiée**

Ajouter éditions, lignage et collections sans confondre les autorités

COMPORTEMENT OBSERVÉ

Le modèle observé conserve une origine source, les branches, les sorties de flows et des historiques de refresh. Aucun contrôleur dédié aux collections, aucun graphe de lignage par édition et aucune entité Edition avec table UUID, snapshot, schema/spec et preuve d'audit n'ont été trouvés dans ce service. Les types actuels décrivent des connecteurs ou DATAFLOW ; ils ne sont pas les huit kinds de datasets de la cible. Cette fiche décrit donc un besoin cible, pas une capacité livrée.

UTILISATEURS ET RÔLES

Équipe Catalog/Datasets, utilisateurs qui organisent et tracent les données.

INTERFACES DANS LE CODE

Existant : Dataset, SourceDataset, DatasetRefreshHistory et DataflowDecision. API cible de collections/éditions/lignage à spécifier, aucun chemin inventé.

DONNÉES ET ÉTATS

Actuel : PostgreSQL métier et provenance partielle. Cible : commandes/drafts PG et éditions Iceberg autoritaires, index Catalog reconstruisible.

DÉPENDANCES ET ÉCHANGES

Flows fournit entrées et sorties ; Writer crée les candidats ; Audit produit la preuve ; Publisher approuve ; Query épingle les éditions.

COMPORTEMENT CIBLE PROPOSÉ

Datasets Rust définit TABLE, INCREMENTAL, VIEW, FILESET, MODEL, EMBEDDINGS, FEATURESET et STREAM. Chaque kind publie une édition Iceberg avec origin.* dès le candidat et références d'artefacts immuables. Catalog propose des collections de références, sans recopier données ni droits. Le lignage relie éditions d'entrée, contrat, run et édition de sortie. Une release multi-sorties est elle-même une édition avec checkpoint commun et participants, y compris inchangés.

MIGRATION DE CETTE CAPACITÉ

Importer la provenance ancienne avec un niveau de complétude déclaré. Constituer l'historique approuvé sans exposer tous les snapshots techniques. Ne pas déduire de checkpoint depuis un historique de commandes PG. Introduire collections et nouveaux kinds progressivement derrière des contrats versionnés.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Perdre l'index Catalog permet de reconstruire les éditions et le checkpoint depuis Iceberg. Une collection ne confère aucun droit supplémentaire. Une release visible ne contient pas de mélange de versions ; ses lecteurs, ack et GC utilisent le même manifeste. STREAM passe aussi par WAP après journal durable.

Références de l'analyse

[monorepo · services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/aggregate/Dataset.kt:66](#) Le dataset actuel ne contient pas de référence d'édition.

[monorepo · services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/aggregate/DatasetRefresh.kt:28](#) Le refresh historique n'est pas une autorité de snapshot.

[monorepo · services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/command/DataflowDecisionCommandHandler.kt:73](#) Sorties de flows présentes, sans lignage complet par édition.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- Conserver les IDs métier, alias, descriptions, collections, demandes de partage, définitions de règles, commandes et drafts dans PostgreSQL avec révisions et idempotency keys. IAM reste seul arbitre des droits.
- Définir EditionRef et DatasetContract avec kind, table UUID, snapshot/ref approuvée, schema/spec, inputs, run, policy et provenance. Résoudre une édition publiée est une opération distincte de la recherche du catalogue.
- Indexer les publications approuvées et les manifests de releases multi-sorties. Une projection peut être en retard ; elle ne peut déplacer le checkpoint publié ni autoriser le GC.
- Transformer la promotion/chiffrement en commande d'effet munie d'un reçu. Conserver les erreurs et reprises sans convertir une demande en succès.

Plan de données proposé

- Writer Rust de confiance valide les bindings et commit le staging dans Lakekeeper. Les workers fournissent fichiers préparés et métadonnées bornées ; ils ne reçoivent pas table.modify générique qui autorise aussi commit et drop.
- Publisher Rust vérifie candidat, droits, preuve post-candidat durable, UUID/refs/schema/spec et ascendance avant publication. Le fencing Hemera, la confidentialité des branches et les races de droits sont des travaux explicites, pas des garanties du CAS seul.
- Écrire origin.system/cursor/state/provenance dans le summary du candidat. Lier ensuite la preuve immuable à l'identifiant préalloué sans prétendre que le fast-forward réécrit ce summary.
- Soumettre compaction, schéma structurel et édition vide à WAP. Maintenance expire les snapshots et objets via plan audité, racines de rétention et relecture après délai de sécurité.

Découpage du core Rust

- Proposition catalog-core : identités de dataset, aliases, métadonnées, recherche, collections et sérialisation des DTO compatibles.
- Proposition dataset-contracts : huit kinds, schémas, field IDs, formats, Binding, EditionRef, PreparedWrite, PublicationReceipt et ReleaseManifest.
- Proposition catalog-projector : projections depuis événements métier et éditions approuvées ; déduplication, version par agrégat, resynchronisation et métriques de retard.
- Proposition dataset-lifecycle : réservation, partage, tombstones et demandes de maintenance avec transactions communes pour état et outbox.
- Proposition migration-adapters : lecteurs PG/Protobuf hérités, mapping IAM, contrôle de parité et adoption des tables existantes sans publication fictive.

Identité, autorisation et isolation

- Les décisions de route sont centralisées dans APISIX/IAM/OpenFGA. Le code conserve des contrôles portant sur des cibles cachées dans le corps, les lots, le transfert d'owner et la liaison permission/dataset. Ne pas déduire de l'AGENTS qu'aucun appel IAM n'existe dans le pod.

- La découverte de cartes publiées est volontairement plus large que l'accès au détail, au schéma et aux octets. Porter cette distinction dans Catalog Rust et les outils MCP.
- La confiance X-Userinfo dépend d'un chemin réseau contrôlé. Le protocole execution-datasets impose aussi une identité de service précise. Les références de secrets de connecteurs ne doivent pas entrer dans origin.* ni les preuves publiques.
- Le périmètre de tenant visible ici repose sur le déploiement et IAM ; les agrégats examinés n'ont pas un tenantId universel. Un service Rust mutualisé doit ajouter une clé de tenant aux commandes, index, caches, objets et décisions.
- Points à corriger/qualifier : réponse de clearance vide interprétée sans DENY ; vérification inversée de SchemaLabel ; contrôle des données potentiellement sensibles dans logs et configurations historiques.

Exploitation et reprise

- Le chart prévoit Deployment, Service, HPA optionnel, PVC optionnel, NetworkPolicy APISIX et PodMonitor. L'activation du PVC force un seul replica et une stratégie de recréation selon le contrat des values. Ce rapport ne prouve aucun déploiement actif.
- PostgreSQL et Kafka sont nécessaires aux commandes et projections. Les migrations Flyway acceptent l'out-of-order ; les migrations de code ont leurs propres lignes de suivi. Le runner data-model actuel lève une exception en cas d'échec, contrairement à la mention exit 0 encore présente dans AGENTS.md.
- Le relay outbox publie au moins une fois, conserve erreurs/essais et purge les messages envoyés selon rétention. La transaction de l'outbox ne prouve l'atomicité état/événement que si l'appelant possède une transaction englobante ; contrôler chaque parcours lors du portage.
- Tâches distinctes : libération des réservations, retry des suppressions, relabel/promotion, drainage des membres IAM et rétention outbox. Toute réplication exige un claim ou verrou distribué pour les tâches mutantes.
- Sauvegarder les commandes métier, demandes de partage, mappings d'identités, tombstones et reçus, ainsi que les preuves et éditions Iceberg. Tester restauration et rejeu sans seconde publication ni second effet.
- Observabilité à préserver : santé/readiness, métriques Prometheus, parité des miroirs IAM, retard outbox/consommateurs, suppressions abandonnées, promotions échouées et divergence schéma physique/déclaré.

Séquence de migration

- Établir un inventaire par dataset : UUID, propriétaire IAM et sujet hérité, projet, nom technique, branches, table UUID Iceberg, schéma déclaré/physique, labels, droits, historique, tombstones et origine.
- Importer les métadonnées dans Catalog Rust et construire une projection parallèle en lecture. Comparer catalogue, schémas, détails, recherches, permissions de partage, compteurs et états sans écrire dans les dépôts sources.
- Formaliser les contrats de formats et de schéma ainsi que les scénarios des capacités DM-01 à DM-20. Traiter les conditions inversées/ambiguës par tickets, pas par copie silencieuse des comportements.
- Adopter les tables existantes et leur état retenu comme base migrée avec provenance déclarée. Ne pas fabriquer d'audit antérieur. Passer les nouvelles écritures par Writer/Audit/Publisher avec un seul propriétaire de publication.
- Basculer commandes et schedules par cohorte ; drainer l'outbox et les consommateurs existants. Garder les façades REST/SDK tant que les clients n'ont pas migré.

- Répéter restauration de Catalog depuis commandes et éditions, révocation pendant lecture, conflit après audit et drop échoué. Retirer Kotlin, Spring et dépendances Kafka/Keycloak Java une fois leur remplacement qualifié et le rollback de cohorte vérifié.

Écarts et décisions encore ouverts

- Aucun service ni test n'a été exécuté. Les comportements décrits sont des observations statiques à la révision donnée par le brief, et les acceptances sont des scénarios à implémenter ou exécuter.
- Quelle API publique doit porter diffusion, publication d'édition, promotion de sécurité et chiffrement pour que quatre opérations différentes ne partagent plus le même vocabulaire ?
- Comment adopter les datasets historiques dont schéma déclaré, ref physique et dernier refresh ne concordent pas ? L'outil de correction de data-serve n'est pas un historique d'éditions.
- Les limites artificielles de suppression de colonnes et la matrice de casts doivent-elles rester dans une façade historique ou évoluer dans le contrat Rust ?
- Comment retirer un lien INTERNAL accordé automatiquement via les routes actuelles dont /access/withdraw ne traite que ACCEPTED ?
- La politique Datasphere doit-elle migrer dans IAM Rust ou rester un adaptateur externe sans JVM ? Son résultat doit couvrir tous les sujets avant de permettre un partage.
- Quelles données de refresh/configuration/logs ont une rétention métier et quelles projections peut-on reconstruire ? Aucun checkpoint publié ne doit devenir dépendant de leur sauvegarde.
- Les collections, le lignage par édition et les kinds autres que la table doivent encore recevoir des API et des règles d'autorisation détaillées. Ils ne sont pas présents dans les contrôleurs examinés.

Références de l'analyse

[monorepo](#) · [services/data-model/AGENTS.md:1](#) Périmètre et consignes du composant.

[monorepo](#) · [services/data-model/app/gradle/libs.versions.toml:14](#) Versions Kotlin et Spring Boot déclarées.

[monorepo](#) · [services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/codemigration/runner/CleyropMigrationRunner.kt:37](#) Le code actuel fait échouer le démarrage sur migration ratée.

[monorepo](#) · [services/data-model/chart/values.yaml:60](#) Isolation réseau et identité gateway.

[monorepo](#) · [services/data-model/chart/values.yaml:224](#) Contrat de persistance et headers.

[monorepo](#) · [services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/common/outbox/OutboxRelay.kt:25](#) Publication au moins une fois et rétention.

SERVICE ACTUEL · DATA-DEV

Développement et versionnement des flows

23 capacités analysées

services/data-dev · Kotlin 2.4.10, Spring Boot 4.1.1 et JVM 21 observés dans Gradle. Domaines CQRS, JPA/PostgreSQL, Flyway, Kafka/Protobuf, JGraphT, génération Python/SQL/PySpark, S3 et REST Gitea/IAM/Jupyter.

Service des développeurs et administrateurs de projet. Il conserve les graphes, branches et versions de définition, produit les scripts exécutables, prépare les accès aux datasets et relie projets, Git, notebooks, modules et notifications. Il conserve aussi des fonctions de projets désormais en transition vers IAM.

Lecture de code

Pas de qualification runtime nouvelle

Domaines cible : Flows, IAM, Datasets, Connectors, Writer, Publisher, Query, Runtimes, Applications, Notifications, Audit, Migration, Maintenance, Frontends.

Périmètre de lecture

- Contrôleurs project, definition, branch, version, internalmodule, dataset, favorite et schedule.
- Handlers des créations et mises à jour de projets, offboarding, graphes, sauvegarde, publication, preview, branches, MR et résolution.
- ScriptGeneratorServiceImpl, calcul des schémas, conversion Protobuf des versions et adaptateur DataflowForLibServiceImpl.
- Agrégats Version, DataflowDefinition, DataflowBranch, variables, modules et miroirs datasets.
- Adaptateur Gitea, intégration notebooks, visibilité IAM, chiffrement SSH, outbox, DLT, run modes et manifestes Gradle/Helm.

Objets et cycle de vie

- Project, ProjectMember, User, Group et Subscription sont des objets legacy ou miroirs IAM. Leur correspondance canonique doit précéder l'import des ressources du projet.
- Application représente Gitea/Jupyter et leur provisioning. Les effets distants et leurs erreurs ont un cycle distinct de l'enregistrement projet.
- DataflowDefinition conserve identité, projet, propriétaire et type PYTHON/PYSPARK/SQL. DataflowBranch conserve filiation, propriétaire, statut DRAFT/PRODUCTION, cluster, suppression et dernières versions.
- GraphVersion contient le graphe et ses scripts. VERSION_SAVE, VERSION_MERGE, VERSION_PUBLISH, VERSION_MODULE et VERSION_ENV_VAR décrivent des changements de définition ; CollectVersion, DeleteVersion, EmptyDatasetVersion et ColumnOperationVersion décrivent des opérations techniques.
- Script conserve dépendances, transformations/étapes, sorties, type et manifeste dataset/branche/mode. L'artefact de code S3 est une seconde ressource à sauvegarder avec sa référence.
- InternalModule possède draft et publication séparés. EnvVar possède portée globale/locale et un masquage de réponse pour secrets, sans chiffrement applicatif visible dans son DAO.
- MergeRequest conserve auteur, reviewer, branches, historique de statut et choix de suppression de source. MergeRequestResolution conserve le hash de script cible de la décision.
- DatasetBranch, Execution, Cluster et ExecutionSchedule sont des projections d'autres services. Leur stockage local ne leur donne pas l'autorité sur les données publiées.

- OutboxEvent et DeadLetterEvent portent les commandes/événements non délivrés, les tentatives et les reçus. Ils font partie du périmètre de sauvegarde et de bascule.

Dépendances du service

- IAM fournit canonicalisation des projets, décisions de visibilité et d'autorisations spécialisées, comptes de service et événements d'identité. IAM est le writer des tuples, data-dev émet des faits.
- APISIX autorise les routes humaines puis le service authentifie. Les contrôles de cohérence de payload, de propriété d'objet et certains rôles métier résiduels restent dans les handlers lus.
- Process-data reçoit VersionEvent et les triggers ; il renvoie exécutions, schedules et clusters par Kafka/Protobuf.
- Data-model fournit datasets, branches, schémas, labels et liens projet/dataset. Ses demandes de vidage/suppression/changement de colonnes peuvent provoquer des jobs.
- Data-collect émet les collectes CREATED/UPDATED ; data-dev compile leurs transformations et les rend exécutables.
- Gitea est appelé par REST pour organisation, dépôt, compte et collaboration. Jupyter passe par ApplicationServiceJupyter et événements d'applications.
- Notification-center reçoit les abonnements et notifications de MR/flows ainsi que les changements de favoris.
- PostgreSQL/JPA/Flyway porte graphes et état de commande. S3 conserve les archives et scripts. Kafka transporte les événements de contrôle ; ce transport JVM doit aussi quitter la cible finale.

Fonctionnalités et compatibilité à préserver

DD-01 **Cible non qualifiée**

Projets, espaces personnels et compatibilité avec IAM

COMPORTEMENT OBSERVÉ

Les routes historiques listent les projets avec recherche, tri, pagination, filtre onlyMine et inclusion des espaces personnels. La visibilité du demandeur passe par une décision IAM avant restitution. Les réponses exposent la correspondance avec les identifiants IAM et des en-têtes de dépréciation. La création valide manager et slug, crée les applications initiales et émet les événements projet. Un mode de passage par IAM écrit d'abord le projet central puis le miroir local. PATCH /projects/{id} répond 410. La suppression est jouable si un tombstone prouve une suppression antérieure. Un espace SINGLE exige une règle particulière d'administration pour sa suppression et sa création est conditionnée au profil utilisateur.

UTILISATEURS ET RÔLES

Développeur, responsable de projet, administrateur d'organisation et IAM.

INTERFACES DANS LE CODE

GET/POST /projects ; GET/DELETE/PATCH /projects/{id} ; GET /projects/exists ; POST /projects/list ; GET /projects/personal-space ; GET /users/{id}/projects ; GET /kpi/projects.

DONNÉES ET ÉTATS

Project et ProjectMember en PostgreSQL, visibilité PUBLIC ou SINGLE, nom historique et identifiant IAM canonique. Initialisation des applications et enregistrement de compte de service peuvent prolonger le cycle après création.

DÉPENDANCES ET ÉCHANGES

IAM par le pont d'écriture et décisions de visibilité ; événements vers consommateurs de projets ; applications Gitea/Jupyter ; enregistrement IAM du compte de service.

COMPORTEMENT CIBLE PROPOSÉ

IAM Rust possède les projets, adhésions, identifiants et espaces personnels. Flows ne garde que la référence canonique nécessaire aux graphes. Applications prend en charge les effets de provisionnement, avec reçus persistants.

MIGRATION DE CETTE CAPACITÉ

Importer les correspondances legacy/IAM, inventorier les appelants des routes dépréciées, puis fournir une façade temporaire. Déplacer les effets applicatifs dans une procédure de provisionnement reprise sur incident avant de supprimer les tables miroir devenues inutiles.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Créer un projet avec membres, interrompre le provisionnement Gitea puis reprendre. IAM garde un seul projet, les applications finissent par converger et la façade restitue le même identifiant. Un utilisateur sans droit de lecture ne modifie ni les lignes ni le total visible d'une page.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-](#)

[app/src/main/kotlin/com/cleyrop/data/dev/app/project/controller/ProjectController.kt:60](#) Routes, canonicalisation, dépréciation et arrêt du PATCH.

[monorepo · services/data-dev/app/data-dev-](#)

[domain/src/main/kotlin/com/cleyrop/data/dev/domain/project/command/handler/CreateProjectWithMembersCommandHandler.kt:48](#) Pont IAM avant création locale et membres.

[monorepo · services/data-dev/app/data-dev-](#)

[domain/src/main/kotlin/com/cleyrop/data/dev/domain/project/command/handler/DeleteProjectCommandHandler.kt:33](#) Idempotence de suppression et invariant SINGLE.

[monorepo · services/data-dev/app/data-dev-](#)

[domain/src/main/kotlin/com/cleyrop/data/dev/domain/project/service/ProjectVisibilityService.kt:16](#) Filtrage par décision centrale.

DD-02 **Cible non qualifiée**

Miroirs d'identité, membres et départ d'un utilisateur**COMPORTEMENT OBSERVÉ**

Les événements utilisateurs, groupes, abonnements et membres alimentent des miroirs locaux et les effets applicatifs. Lors d'une désactivation, les flows du membre et les projets dont il était manager peuvent être réaffectés. Lors d'une suppression, l'espace personnel peut être supprimé, les adhésions ordinaires retirées et les propriétaires remplacés. L'absence de remplaçant ne crée pas de nouveau propriétaire implicite. Le nettoyage Git vérifie l'identité du compte par email avant verrouillage. Une erreur Git laisse le miroir révoqué mais le compte distant inchangé et incrémente un compteur. Les rôles historiques servent encore à certaines règles métier de provisioning et d'éligibilité, même si la décision d'accès des routes est centrale.

UTILISATEURS ET RÔLES

IAM, administrateurs et consommateurs des événements de lifecycle.

INTERFACES DANS LE CODE

`OnUserDeactivatedCommandHandler` ; `OnUserDeletedCommandHandler` ;

`ReapUserFootprintCommandHandler` ; événements de membres et d'identité. `GET /projects/{id}/members` ;

`GET /users/{id}/check-owner`.

DONNÉES ET ÉTATS

User, groupes, abonnements et ProjectMember en miroir ; propriétaires de projets/flows ; états des applications ; compte Git distant verrouillable.

DÉPENDANCES ET ÉCHANGES

Flux IAM ou legacy, événements de transfert de propriétaire, Gitea et applications du projet.

COMPORTEMENT CIBLE PROPOSÉ

IAM Rust émet des événements versionnés et ordonnés par identité. Flows et Applications appliquent des changements idempotents. L'offboarding devient une opération suivie jusqu'aux reçus des effets distants.

MIGRATION DE CETTE CAPACITÉ

Reprendre chaque règle de transfert, conserver les références historiques d'auteur et ne pas assimiler désactivation à effacement. Introduire une file de réconciliation du verrouillage Git, car le chemin lu signale l'échec sans réparateur automatique.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Désactiver le propriétaire pendant une indisponibilité Gitea. Les nouveaux accès et runs sont refusés, l'historique reste attribué, le transfert décidé par IAM est appliqué une seule fois et un effet de verrouillage non terminé reste visible jusqu'à son acquittement.

Références de l'analyse

`monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/project/command/handler/OnUserDeactivatedCommandHandler.kt:35` Ordre de réaffectation des flows puis managers.

`monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/project/command/handler/OnUserDeletedCommandHandler.kt:34` Espace personnel, adhésions et transfert.

`monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/project/command/handler/ReapUserFootprintCommandHandler.kt:76` Vérification du compte Git et échec observé sans réconciliation.

DD-03 **Cible non qualifiée****Dépôts Gitea, comptes Git et clés SSH****COMPORTEMENT OBSERVÉ**

L'initialisation d'une application Gitea crée un état WAITING et émet une commande asynchrone. L'adaptateur sait assurer l'organisation, créer/supprimer un dépôt, migrer un dépôt, gérer les collaborateurs, créer/retrouver des comptes et verrouiller leur connexion. La résolution d'un login existant tient compte du login conservé et de l'email. Le retrait d'un collaborateur peut refaire la résolution si le serveur ne connaît plus le login stocké. Les viewers et business analysts sont exclus de certains effets de collaboration. Les clés SSH locales portent un convertisseur de chiffrement et une procédure de rechiffrement existe. Ces dépôts de projet ne sont pas les branches de graphes métier, conservées en base.

UTILISATEURS ET RÔLES

Membres techniques du projet et opérateurs de migration Git.

INTERFACES DANS LE CODE

InitGitCommandHandler ; CreateGitRepositoryCommandHandler ; EditGitMemberCommandHandler ; GitCrudService ; GiteaCrudServiceImpl. Pas de préfixe public Git inventé.

DONNÉES ET ÉTATS

Application GITEA avec statut de provisioning, login Git dans le miroir utilisateur, dépôt/collaborateurs distants et SSHKey chiffrée en PostgreSQL.

DÉPENDANCES ET ÉCHANGES

REST Gitea, événements d'applications, utilisateurs/membres et chiffrement partagé avec l'infrastructure existante.

COMPORTEMENT CIBLE PROPOSÉ

Applications Rust encapsule Gitea, tiers Go compatible avec la cible zéro JVM. IAM possède l'identité. Flows référence un commit de code ou un artefact immuable quand un run dépend d'un dépôt.

MIGRATION DE CETTE CAPACITÉ

Séparer l'adaptateur Git du compilateur de graphes. Migrer les clés par une procédure contrôlée sans valeurs dans les exports. Conserver les identifiants de comptes et reçus de provisionnement pour éviter les créations concurrentes.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Rejouer deux fois la création d'application puis renommer un compte distant. Un seul dépôt et un seul compte sont liés au projet ; la suppression d'adhésion retire le bon collaborateur. Une identité Git non prouvée produit un échec explicite et aucun verrouillage d'un tiers.

Références de l'analyse

`monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/project/command/handler/InitGitCommandHandler.kt:31` Application WAITING et événement de création.

`monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/project/command/handler/EditGitMemberCommandHandler.kt:32` Éligibilité, provisioning et retrait avec nouvelle résolution.

`monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/project/service/gitea/GiteaCrudServiceImpl.kt:45` Adaptateur REST organisation, dépôts et comptes.

`monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/project/repository/user/SSHKeyDAO.kt:19` Convertisseur de chiffrement de la clé.

DD-04 Cible non qualifiée

Provisionnement des notebooks et contexte de projet

COMPORTEMENT OBSERVÉ

NotebookUseCase initialise une application JUPYTER, soit immédiatement soit par événement. Avant de provisionner, il vérifie que l'application et le projet existent encore pour éviter de recréer un projet supprimé pendant la file d'attente. Il initialise ensuite manager et membres. Les erreurs métier de reprise donnent RETRY, les autres ERROR. La route de lien calcule l'URL par utilisateur et slug du projet. La route d'informations destinée au notebook vérifie la permission centrale de création, la visibilité du projet et retourne le contexte utilisateur/projet/Git. Le paramètre applicationName de cette dernière route n'est pas le mécanisme de sélection du projet dans le handler lu.

UTILISATEURS ET RÔLES

Utilisateur ouvrant un notebook, IAM et service d'applications.

INTERFACES DANS LE CODE

GET /projects/{projectId}/notebooks/link ; GET /users/me/application/{applicationName}/{projectSlug} ; NotebookUseCase.createNotebook.

DONNÉES ET ÉTATS

Application JUPYTER et statuts WAITING, RETRY ou ERROR ; associations aux membres ; contexte référencé depuis PostgreSQL.

DÉPENDANCES ET ÉCHANGES

ApplicationServiceJupyter, événements d'applications, décisions IAM et dépôt Gitea du projet.

COMPORTEMENT CIBLE PROPOSÉ

Applications et Runtimes Rust gèrent le lancement et les quotas ; l'interface notebook peut rester Python/TypeScript. Chaque accès aux données passe par les contrats Query/Writer associés au projet et au run, sans session Spark finale.

MIGRATION DE CETTE CAPACITÉ

Préserver les liens, volumes et attribution des notebooks. Remplacer le contexte de credentials ambiants par une délégation courte et bornée. Retirer les chemins Spark Connect seulement après qualification du SDK utilisateur de remplacement.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Supprimer un projet entre l'émission et le traitement du provisioning. Aucun notebook ne réapparaît. Ouvrir un notebook autorisé fournit le bon contexte canonique et les accès au stockage expirent après révocation du run.

Références de l'analyse

`monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/project/usecase/NotebookUseCase.kt:101` Provisionnement vérifié puis RETRY/ERROR.

`monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/project/query/handler/GetApplicationInfoQueryHandler.kt:24` Visibilité du projet et contexte Git.

`monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/project/controller/UserController.kt:77` Décision IAM de création de notebook.

`monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/project/controller/ProjectNotebookController.kt:33` Route du lien personnalisé.

DD-05 Cible non qualifiée

Découverte des flows, versions, KPI et favoris

COMPORTEMENT OBSERVÉ

Les contrôleurs offrent listes de flows par projet et cluster, détail avec branche/version, métadonnées, sorties, état avant publication, historique des versions et dernier script exécuté d'une transformation. Les versions ont des actions visibles ou cachées, notamment collecte, suppression, preview et retouche de colonnes. Les KPI de projets distinguent SINGLE et PUBLIC. Le favori est une association propre à l'utilisateur, idempotente à l'ajout, et émet un événement de création/suppression. L'ajout exige la visibilité de la cible si le point d'entrée n'a pas déjà imposé l'autorisation de gateway.

UTILISATEURS ET RÔLES

Développeur, frontends et administrateur.

INTERFACES DANS LE CODE

GET /dataflows ; GET /dataflows/list ; GET /dataflows/clusters/{clusterId} ; GET /dataflows/{id} ; GET /dataflows/{id}/metadata ; GET /dataflows/{id}/outputs ; GET /dataflows/{id}/branches/{branchName}/versions ; GET /kpi/dataflows ; POST /users/favorite.

DONNÉES ET ÉTATS

Définitions, branches, versions, miroir d'exécutions et favoris en PostgreSQL. Les actions techniques ont un indicateur unshown ; elles ne doivent pas être perdues à cause du filtrage d'interface.

DÉPENDANCES ET ÉCHANGES

Miroirs process-data pour l'exécution ; Notifications pour les favoris et abonnements ; IAM/gateway pour la visibilité.

COMPORTEMENT CIBLE PROPOSÉ

Flows Rust expose les contrats de recherche et l'historique des définitions. Audit conserve les opérations techniques. Frontends garde les vues distinctes entre versions de définition, runs et éditions de données approuvées.

MIGRATION DE CETTE CAPACITÉ

Construire une matrice de champs, ordres, pagination et filtrage pour chaque route. Importer l'historique caché avec son type d'origine. Migrer les favoris sans les convertir en droits persistants.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Comparer les pages et filtres sur un jeu comportant plusieurs branches, versions cachées et projets non visibles. Les totaux restent cohérents. Un favori ajouté deux fois ne crée qu'une relation et ne redonne pas accès après révocation.

Références de l'analyse

`monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/controller/DataflowGetterController.kt:77` Parcours de lecture et sorties.

`monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/version/aggregate/Version.kt:19` Actions de version et visibilité.

`monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/favorite/command/UserCreateFavoriteCommandHandler.kt:37` Idempotence et décision de visibilité des favoris.

`monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/version/controller/VersionController.kt:26` Historique par branche.

DD-06 **Cible non qualifiée****Sauvegarde de graphes et concurrence d'édition****COMPORTEMENT OBSERVÉ**

POST /dataflows crée ou sauvegarde un flow. À la création, owner doit désigner l'appelant. À la sauvegarde, le handler charge le flow et refuse de changer son projet ou son propriétaire par ce parcours. Le type et le cluster doivent être compatibles. Le graphe est validé avant génération. Un baseVersionId optionnel provoque un verrou de sauvegarde puis une comparaison avec la dernière version. Un décalage lève un conflit. Le diff distingue changement fonctionnel, métadonnées et absence de changement. Sans baseVersionId, certains changements de métadonnées actualisent la version existante ; avec cette précondition une nouvelle version est générée. Un changement fonctionnel de main déjà en production doit passer par proposition.

UTILISATEURS ET RÔLES

Auteur et collaborateurs du flow.

INTERFACES DANS LE CODE

POST /dataflows ; DataflowCreateCommandHandler ; DataflowSaveConflictAdvice.

DONNÉES ET ÉTATS

DataflowDefinition, DataflowBranch et GraphVersion. Le graphe contient entrées, transformations, sorties, arêtes et positions. Le graphe sauvegardé et ses scripts sont liés à une version, mais le chemin historique peut actualiser une version pour un changement de métadonnées.

DÉPENDANCES ET ÉCHANGES

Cluster miroir, variables, compilateur de scripts, S3, événements de versions et faits d'autorisation.

COMPORTEMENT CIBLE PROPOSÉ

Flows Rust sépare draft mutable et révision exécutable immuable. Chaque sauvegarde porte une révision attendue. Le manifeste compilé fixe code, schémas, dépendances, paramètres non secrets et références d'entrées.

MIGRATION DE CETTE CAPACITÉ

Importer les graphes et actions de version sans réécrire les anciennes preuves. Rendre la précondition obligatoire dans le nouveau contrat. Conserver un adaptateur de sauvegarde legacy pendant la migration de l'éditeur TypeScript.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Deux collaborateurs sauvegardent le même draft à partir de la même révision. Une seule écriture gagne et l'autre reçoit un conflit avec la révision courante. Le manifeste d'un run déjà lancé reste identique après déplacement de nœuds ou changement de cluster.

Références de l'analyse

`monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/controller/DataflowDevelopController.kt:107` Transaction, auteur authentifié et canonicalisation du projet.

`monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/definition/command/DataflowCreateCommandHandler.kt:153` Refus des changements de propriétaire/projet.

`monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/definition/command/DataflowCreateCommandHandler.kt:456` Verrou, précondition, diff et règles de production.

DD-07 **Cible non qualifiée****Règles du graphe et compilation des transformations****COMPORTEMENT OBSERVÉ**

GraphValidator refuse les imports wildcard. PYSPARK et SQL exigent exactement une sortie par transformation ; une sortie doit avoir une transformation productrice. Python accepte plusieurs sorties. Le compilateur calcule les dépendances par les entrées, sorties et transformations amont, parcourt les transformations prêtes et propage leurs schémas. Il rend des templates différents pour Python, PySpark et SQL. SQL reste aujourd'hui une exécution PySpark dans le contrat Protobuf. Chaque transformation reçoit un ScriptId, un type, des dépendances, des sorties et éventuellement un cluster. Les archives contiennent main.py et les modules publiés du projet, puis sont écrites en S3 sous la version et le script.

UTILISATEURS ET RÔLES

Développeur de flow, éditeur et exécutateur.

INTERFACES DANS LE CODE

ScriptGeneratorService.generateGraphVersion ; GraphValidator.valid ; GraphExecutionEventProducer ; GET /dataflows/{id}/transformations/{tid} pour le contrat pylib.

DONNÉES ET ÉTATS

GraphVersion avec scripts ; artefacts S3 par version/script ; dépendances de scripts ; mappings des transformations et étapes.

DÉPENDANCES ET ÉCHANGES

JGraphT, templates Python/SQL, modules internes, schémas des datasets, S3 et process-data via VersionEvent.

COMPORTEMENT CIBLE PROPOSÉ

Flows Rust compile un graphe typé indépendant du moteur. Les opérateurs tabulaires produisent un plan SQL/Arrow exécuté par Query/Runtimes Rust. Python utilisateur reste possible dans un processus isolé. Aucune dépendance finale à PySpark, Spark Connect ou runtime JVM.

MIGRATION DE CETTE CAPACITÉ

Classer chaque flow par langage et fonctions utilisées. Transpiler les opérateurs déclaratifs et traiter les appels Spark spécifiques comme incompatibilités explicites. Conserver les artefacts historiques ; produire de nouveaux artefacts avec empreinte et version du compilateur.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Pour un graphe Python à deux sorties et un graphe SQL à jointure, vérifier ordre de dépendances, schémas, résultats et erreurs équivalentes. Une fonction exclusivement Spark est signalée avant activation du runtime Rust, avec localisation de la transformation.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/definition/aggregate/graph/GraphValidator.kt:13](#) Contraintes wildcard et nombre de sorties.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/service/ScriptGeneratorServiceImpl.kt:229](#) Calcul des dépendances et Script par transformation.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/service/ScriptGeneratorServiceImpl.kt:420](#) Parcours et propagation de schémas.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/service/ScriptGeneratorServiceImpl.kt:776](#) Archive et modules publiés.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/event/producer/GraphExecutionEventProducer.kt:107](#) SQL encodé comme PYSPARK.

DD-08 **Cible non qualifiée**

Transformations low-code et schémas intermédiaires**COMPORTEMENT OBSERVÉ**

Le vocabulaire low-code comprend filtrage/renommage/standardisation de colonnes, group by, jointures gauche/droite/interne/externe, colonne conditionnelle, suppression de doublons, filtres de lignes, concaténation, calcul mathématique, cast, union, calcul et recherche textuelle, top N et preview. Un alias historique FILTER_COLUMN reste accepté. Les étapes sont ordonnées, valident leur payload et calculent leurs schémas avant génération. Les opérations à deux entrées doivent nommer des sources disponibles. Les endpoints generate et validate opèrent sur un graphe et une étape cible ; si un flow existant est nommé, son projet doit correspondre au projet du payload.

UTILISATEURS ET RÔLES

Utilisateur de l'éditeur visuel et concepteur de modèles de transformation.

INTERFACES DANS LE CODE

POST /dataflows/generate ; POST /dataflows/validate ; StepProcessorServiceFactoryImpl et DataflowGraphProcessorServiceImpl.

DONNÉES ET ÉTATS

Étapes typées avec identifiant, ordre et payload ; OwnerSchemasCompute avec schéma rattaché au nœud/étape ; pas d'édition Iceberg créée par la seule validation.

DÉPENDANCES ET ÉCHANGES

Schémas des branches datasets, factory des processors et templates PySpark.

COMPORTEMENT CIBLE PROPOSÉ

Flows et Query Rust portent un registre versionné d'opérateurs. Chaque opérateur définit types, nulls, dates, ordre, validation et lowering vers le moteur sans JVM. L'interface TypeScript consomme le même contrat de schéma.

MIGRATION DE CETTE CAPACITÉ

Établir une fiche de parité par opérateur, y compris null, timezone, doublons, casts invalides, ordre des colonnes et stabilité top N. Garder les identifiants d'étapes et alias de payload pendant l'import.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Exécuter une campagne par opérateur sur types mixtes et valeurs limites. generate et validate du même draft annoncent le schéma réellement obtenu à l'exécution. Une jointure désignant une entrée absente échoue avant création d'un job.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/definition/aggregate/graph/step/LowCodeTypeEnum.kt:5](#) Inventaire exhaustif des opérateurs et alias.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/service/ScriptGeneratorService.kt:101](#) Validation des entrées et calcul des schémas avant génération.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/service/factory/DataflowGraphProcessorService.kt:116](#) Parcours récursif jusqu'à l'étape cible.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/controller/DataflowSchemaCompute.kt:70](#) Cohérence projet/flow du payload.

DD-09 **Cible non qualifiée****Manifestes d'accès et séparation des branches de données****COMPORTEMENT OBSERVÉ**

Chaque script généré porte une liste d'entrées dataset/branche et de sorties dataset/branche/mode. APPEND et OVERRIDE deviennent APPEND et REPLACE. Les versions techniques de suppression utilisent DELETE. Le manifeste est sérialisé dans VersionEvent et conservé côté exécution. DataflowForLibService calcule les entrées/sorties de la transformation et choisit aujourd'hui main pour une branche en production ou une branche Nessie dérivée de l'identifiant de branche pour un draft. Sur une sortie en production, le schemaMode est relu depuis le dataset main. Les opérations techniques peuvent porter executionProjectId pour ne pas traiter leur référence synthétique de dataset comme un vrai projet.

UTILISATEURS ET RÔLES

Compilateur, service d'exécution, bibliothèque Python et services d'autorisation du stockage.

INTERFACES DANS LE CODE

ExecutionStorageManifest ; DatasetInputAccess ; DatasetOutputAccess ; VersionEvent.storageManifest ; DataflowForLibServiceImpl.getTransformationFromBranch.

DONNÉES ET ÉTATS

Manifeste par script et metadata execution_project_id pour versions techniques. Les branches métier, les branches de datasets et les références de catalogue doivent rester distinguées.

DÉPENDANCES ET ÉCHANGES

Miroir de schémas data-model, process-data, S3 et catalogue actuel Nessie dans le runtime.

COMPORTEMENT CIBLE PROPOSÉ

Flows Rust produit un manifeste immuable qui fixe les éditions d'entrée autorisées et les huit kinds cible. Writer de confiance autorise le staging exact ; Publisher décide la publication Iceberg. Le worker reçoit uniquement la portée nécessaire, sans table.modify générique qui permettrait aussi drop/commit.

MIGRATION DE CETTE CAPACITÉ

Reprendre les manifestes déjà présents et mettre à part les versions anciennes nulles. Un backfill s'appuie sur la version historique prouvée, jamais sur le draft courant. Adapter les références Nessie vers UUID/ref/schema/spec Iceberg et garder l'identité d'origine dans origin.* du candidat.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un script qui tente une écriture vers un dataset hors manifeste est refusé. Une ancienne version sans preuve de ses entrées ne reçoit pas d'accès large. Un candidat conserve les références d'édition de ses entrées après une publication concurrente de main.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-](#)

[app/src/main/kotlin/com/cleyrop/data/dev/app/definition/service/ScriptGeneratorServiceImpl.kt:309](#) Construction du manifeste par transformation.

[monorepo · services/data-dev/app/data-dev-](#)

[app/src/main/kotlin/com/cleyrop/data/dev/app/definition/service/DataflowForLibServiceImpl.kt:55](#) Branches Nessie et schemaMode production.

[monorepo · services/data-dev/app/data-dev-](#)

[app/src/main/kotlin/com/cleyrop/data/dev/app/definition/event/producer/GraphExecutionEventProducer.kt:144](#) Manifeste dans le contrat Protobuf.

[monorepo · services/data-dev/app/data-dev-](#)

[domain/src/main/kotlin/com/cleyrop/data/dev/domain/definition/command/DataflowCreateOrUpdateCollectDatasetCommandHandler.kt:45](#) Projet réel des versions techniques.

DD-10 **Cible non qualifiée****Variables globales, locales et valeurs secrètes****COMPORTEMENT OBSERVÉ**

Le flow porte des variables globales et chaque branche des variables locales. Les globales ne changent que depuis main. Leur modification déclenche une génération VERSION_ENV_VAR sur les autres branches. Un secret reçu sans nouvelle valeur conserve la valeur stockée par son identifiant, alors qu'une valeur absente sans remplacement possible est refusée. Les doublons de clés globales sont vérifiés. Le DTO masque un secret par une chaîne vide. Le DAO stocke toutefois la valeur dans une colonne String sans convertisseur de chiffrement visible, et le générateur transforme les variables en code d'environnement inclus dans le script. Le masquage d'API ne prouve donc pas un chiffrement ni une absence du secret dans les artefacts.

UTILISATEURS ET RÔLES

Auteur du flow et opérateur des secrets.

INTERFACES DANS LE CODE

EnvVarDTO dans POST /dataflows ; EnvVarCommand.toEnvVar ;
DataflowCreateCommandHandler.buildEnvVars.

DONNÉES ET ÉTATS

EnvVar avec portée, clé, description, isSecret et valeur en PostgreSQL ; versions régénérées et scripts S3.

DÉPENDANCES ET ÉCHANGES

Générateur de scripts et bus de versionnement automatique.

COMPORTEMENT CIBLE PROPOSÉ

Flows Rust conserve les noms et références de secrets ; Runtimes résout une version de secret au lancement via IAM. Le manifeste et les artefacts publiés contiennent la référence et une preuve de version, jamais la valeur. Les logs et erreurs n'exposent pas cette valeur.

MIGRATION DE CETTE CAPACITÉ

Inventorier les variables et artefacts qui peuvent incorporer des secrets sans exporter leur contenu. Migrer vers un magasin chiffré et faire tourner les secrets concernés selon une procédure contrôlée. Fixer la règle de priorité entre portée globale et locale dans le contrat.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une sauvegarde d'un secret masqué préserve sa référence. Un changement global crée de nouvelles révisions pour les branches concernées. Une recherche sur les artefacts et événements du nouveau runtime ne retrouve aucune valeur secrète injectée au lancement.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/definition/command/DataflowCreateCommandHandler.kt:74](#) Conservation d'une valeur secrète omise.

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/definition/command/DataflowCreateCommandHandler.kt:251](#) Portées, doublons et régénération des autres branches.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/controller/dto/EnvVarDTO.kt:39](#) Masquage en restitution.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/repository/dao/EnvVarDAO.kt:18](#) Valeur String sans convertisseur visible.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/service/ScriptGeneratorServiceImpl.kt:431](#) Variables rendues dans les scripts.

DD-11 **Cible non qualifiée****Branches de développement et restauration contrôlée****COMPORTEMENT OBSERVÉ**

Une branche naît d'une branche source existante, après validation du flow, du nom et de l'unicité active. Le nom doit satisfaire à la fois les contraintes Git historiques et Spark ; main est réservé au parcours prévu. La création d'une branche précédemment supprimée peut réutiliser son identifiant puis la restaurer explicitement. Le simple upsert de graphe refuse au contraire de ressusciter une branche supprimée. La branche reçoit un

propriétaire, un graphe copié puis régénéré, des versions et une configuration de cluster. La suppression clôt les MR source ou cible non finales et émet les faits d'autorisation correspondants. Le handler contient encore un contrôle local de propriétaire de branche ou de flow.

UTILISATEURS ET RÔLES

Développeur et propriétaire du flow.

INTERFACES DANS LE CODE

GET /dataflows/{id}/branches ; GET /dataflows/{id}/branches/exist ; POST /dataflows/{id}/branches ; DELETE /dataflows/{id}/branches/{branchName} ; PUT /dataflows/{id}/branches/{branchName}/cluster.

DONNÉES ET ÉTATS

DataflowBranch avec sourceld, statut, owner, deleted, lastMergedAt, cluster et versions. Noms de branche main et personnalisés.

DÉPENDANCES ET ÉCHANGES

Générateur, variables, versions, MR, cluster miroir et IAM par faits de branches.

COMPORTEMENT CIBLE PROPOSÉ

Flows Rust conserve la filiation et les droits de branches. La branche de définition devient indépendante des contraintes de nom de Spark et des refs Iceberg de staging, ces dernières étant produites par Writer/Publisher.

MIGRATION DE CETTE CAPACITÉ

Conserver les IDs, noms et tombstones. Retirer les contraintes de nom propres à Spark seulement avec un mapping explicite. Clarifier dans IAM les règles de suppression encore contrôlées localement pour éviter un changement de droit durant le transfert.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Créer, supprimer puis recréer une branche par la route dédiée préserve la filiation attendue. Un upsert ordinaire du même nom reste refusé. Les MR touchées se ferment ou se retargetent selon le contrat, sans référence orpheline.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/branch/command/DataflowBranchCreateCommandHandler.kt:43](#) Validation, copie, restauration et nouvelle version.

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/branch/command/DataflowBranchDeleteCommandHandler.kt:36](#) Contrôle local résiduel et clôture des MR.

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/branch/aggregate/DataflowBranch.kt:134](#) Contraintes combinées Git/Spark.

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/definition/command/DataflowCreateCommandHandler.kt:226](#) Refus de résurrection par upsert.

DD-12 Cible non qualifiée

Propositions de fusion, diff et résolution des conflits

COMPORTEMENT OBSERVÉ

Une MR référence source, cible, auteur, reviewer, titre, description et suppression éventuelle de source. La création vérifie que le demandeur possède la source, que le reviewer existe et appartient au projet, puis calcule le statut de conflit. Elle émet des abonnements, notifications et faits d'autorisation. Les API listent les

MR, les comptent, retournent leurs changements et le contenu d'un nœud. La résolution applique un choix de nœud au graphe source et conserve le hash du script cible contre lequel la décision a été prise. Une MR finale n'est plus résoluble. L'appartenance de la MR au flow de l'URI est vérifiée. Les diffs distinguent entrées, sorties, arêtes, code et étapes low-code.

UTILISATEURS ET RÔLES

Auteur, reviewer et collaborateurs du projet.

INTERFACES DANS LE CODE

POST/GET /dataflows/{id}/merge_requests ; GET /dataflows/{id}/merge_requests/{mrId} ; GET /dataflows/{id}/merge_requests/{mrId}/content/{nodeId} ; POST /dataflows/{id}/merge_requests/{mrId}/resolve ; GET /dataflows/{id}/merge_requests/countOpen.

DONNÉES ET ÉTATS

DataflowMergeRequestDefinition, historique de statuts et MergeRequestResolution avec auteur, date et hash du script cible ; nouvelles versions de la source après résolution.

DÉPENDANCES ET ÉCHANGES

GraphDiffCalculator, ConflictResolutionApplier, versions et modules compilés, Notifications et IAM.

COMPORTEMENT CIBLE PROPOSÉ

Flows Rust reprend le diff sémantique et associe chaque revue aux révisions source/cible exactes. Audit conserve les décisions de résolution. Une revue de définition reste distincte de l'audit d'un candidat de données.

MIGRATION DE CETTE CAPACITÉ

Importer les MR ouvertes et leurs preuves de résolution. Conserver les identifiants des nœuds et les hashes déjà produits. Lors du basculement, recalculer les conflits contre les révisions importées et invalider les revues dont la cible a changé.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Résoudre un conflit, modifier ensuite la cible puis tenter la fusion. La résolution obsolète est détectée et une nouvelle revue est demandée. Une MR d'un autre flow donne la même absence que si elle n'existait pas dans le flow demandé.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/branch/command/DataflowMergeRequestCreateCommandHandler.kt:71](#) Validations de création, droits métier et notifications.

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/branch/command/ResolveConflictsCommandHandler.kt:54](#) Application, version et hash de cible.

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/branch/aggregate/DataflowMergeRequestDefinition.kt:55](#) Appartenance au flow.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/branch/controller/MergeRequestController.kt:60](#) Routes de revue, diff et résolution.

DD-13 Cible non qualifiée

Approbation, fusion et maintenance des MR dépendantes

COMPORTEMENT OBSERVÉ

L'approbation charge le flow et la MR, vérifie leur relation, puis construit une version `VERSION_MERGE` sur la branche cible à partir du graphe fusionné et des variables de la cible. Un refus doit porter un commentaire. Après fusion, la source peut être supprimée ou conserver `lastMergedAt`. Si elle disparaît, les MR qui portaient de cette source sont clôturées ; celles qui la visaient sont retargetées vers la nouvelle cible, sauf si cela ferait de la branche sa propre source. Les triggers devenus sans objet peuvent être retirés. Le retour indique aussi si les clusters source et cible sont identiques. Un recalcul des statuts des autres MR est tenté après la fusion et son échec est journalisé.

UTILISATEURS ET RÔLES

Reviewer, auteur et responsable de production.

INTERFACES DANS LE CODE

POST `/dataflows/{id}/merge_requests/{mrid}/approval` ; `DataflowMergeRequestApproveCommandHandler`.

DONNÉES ET ÉTATS

Statut final `MERGED` ou `DELETED`, nouvelle version de cible, source supprimée ou `lastMergedAt` mis à jour, MR restantes modifiées.

DÉPENDANCES ET ÉCHANGES

`BranchService` pour fusion, variables et scripts, triggers, versions et événement exécutable.

COMPORTEMENT CIBLE PROPOSÉ

Flows Rust réalise la fusion par comparaison atomique des révisions approuvées. Les retargets et suppressions font partie d'une commande durable. Audit enregistre qui a approuvé quel diff et la révision résultante.

MIGRATION DE CETTE CAPACITÉ

Porter les règles de retarget des MR en chaîne. Faire des échecs de recalcul une tâche persistante observable. Préserver le comportement des variables de cible et clarifier le changement de profil runtime entre source et cible.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Fusionner A dans main avec suppression de A alors qu'une MR B vise A. La MR B vise ensuite main ; une MR main vers A est clôturée. Une panne après création de la révision cible permet de reprendre les retargets sans deuxième fusion.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-](#)

[domain/src/main/kotlin/com/cleyrop/data/dev/domain/branch/command/DataflowMergeRequestApproveCommandHandler.kt:122](#)

Version cible et variables utilisées.

[monorepo · services/data-dev/app/data-dev-](#)

[domain/src/main/kotlin/com/cleyrop/data/dev/domain/branch/command/DataflowMergeRequestApproveCommandHandler.kt:176](#)

Clôture et retarget des MR dépendantes.

[monorepo · services/data-dev/app/data-dev-](#)

[domain/src/main/kotlin/com/cleyrop/data/dev/domain/branch/command/DataflowMergeRequestApproveCommandHandler.kt:292](#)

Ordre de fusion, statut et recalcul.

Mise en production d'une définition de flow

COMPORTEMENT OBSERVÉ

La publication cible aujourd'hui main. Elle refuse un flow absent ou supprimé, lit la dernière exécution pour le résumé, passe la branche au statut PRODUCTION et génère VERSION_PUBLISH. Le script et les sorties sont ensuite émis vers process-data. Le résumé renseigne la dernière exécution et si elle a démarré après la dernière modification. Dans le handler lu, ces indicateurs ne constituent pas une exigence de succès avant publication. L'opération publie donc une définition exécutable, pas une édition de données ayant franchi un audit WAP. La route lazy fournit l'état préalable sans exécuter cette transition.

UTILISATEURS ET RÔLES

Propriétaire habilité à publier le flow.

INTERFACES DANS LE CODE

POST /dataflows/{id}/publish ; GET /dataflows/{id}/publish/lazy ; DataflowPublishCommandHandler.

DONNÉES ET ÉTATS

DataflowStatus DRAFT/PRODUCTION sur la branche ; GraphVersion VERSION_PUBLISH ; information d'exécution distincte.

DÉPENDANCES ET ÉCHANGES

Versions, variables, miroir des exécutions et producteur GraphExecutionEvent.

COMPORTEMENT CIBLE PROPOSÉ

Flows Rust publie une révision de définition. Publisher Rust publie séparément les résultats de chaque run après audit. Les huit kinds, y compris STREAM, suivent WAP ; la réussite du job ne détermine pas à elle seule l'autorité des données.

MIGRATION DE CETTE CAPACITÉ

Renommer les concepts d'interface et de contrat pour garder publication du flow et publication des résultats distinctes. Importer le statut de production existant comme activation de définition, sans le convertir en preuve d'approbation d'un snapshot.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Activer une définition puis exécuter un run dont l'audit échoue. La définition demeure activée, le candidat reste non publié et les lecteurs restent épinglés sur la dernière édition approuvée. L'historique distingue les deux décisions.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/definition/command/DataflowPublishCommandHandler.kt:112](#) Résumé lazy et publication effective.

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/definition/command/DataflowPublishCommandHandler.kt:131](#) Passage PRODUCTION et VERSION_PUBLISH.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/controller/DataflowDevelopController.kt:186](#) Route publique de mise en production.

Preview d'une étape low-code et preuve de portée

COMPORTEMENT OBSERVÉ

Le preview charge le flow et la branche, utilise le graphe fourni ou le dernier graphe sauvegardé, exige une transformation low-code et une étape existante. Les datasets amont doivent avoir un état SUCCESS dans les branches attendues. Le graphe est tronqué à l'étape demandée avec un nouvel identifiant, puis le compilateur produit une version VERSION_PREVIEW et l'émet. Un endpoint interne destiné à Data Serve retourne les entrées d'un preview seulement si la version persistée est de ce type et si tous ses scripts ont un manifeste. Il vérifie en plus le client du bearer. Le chemin de génération lu ne sauvegarde pas directement cette version ; ce raccord de persistance est une question ouverte vérifiée dans le code.

UTILISATEURS ET RÔLES

Utilisateur de l'éditeur, runtime de preview et Data Serve.

INTERFACES DANS LE CODE

POST /dataflows/{id}/preview ; GET /internal/dataflow-versions/{versionId}/preview-scope.

DONNÉES ET ÉTATS

Graphe tronqué, identifiant de preview, scripts et artefacts temporaires ; PreviewScope lié aux manifestes persistés attendus.

DÉPENDANCES ET ÉCHANGES

Schémas/états des datasets, générateur, process-data et contrôle Data Serve.

COMPORTEMENT CIBLE PROPOSÉ

Flows Rust crée un RunSpec de preview immuable et persistant. Runtimes applique bornes de lignes, temps et mémoire ; Writer ne publie aucune édition de production. Query n'ouvre le résultat qu'après vérification de sa portée et de sa durée de rétention.

MIGRATION DE CETTE CAPACITÉ

Corriger ou établir le raccord entre émission et stockage du preview avant reprise fonctionnelle. Faire du résultat un artefact associé au run et non une permission fondée sur un UUID fourni par le client.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Créer un preview d'une étape intermédiaire, perdre le navigateur puis reprendre par son ID. Les mêmes entrées épinglées sont restituées, le scope reste disponible au consommateur autorisé et un bearer humain direct reçoit un refus.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/definition/command/DataflowPreviewCommandHandler.kt:113](#) Construction et émission du preview.

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/version/query/PreviewScopeQueryHandler.kt:38](#) Exigence de version persistée et manifeste complet.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/version/controller/PreviewScopeController.kt:52](#) Client interne autorisé et 404.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/event/producer/GraphExecutionEventProducer.kt:48](#) Émission outbox sans persistance de version dans le producteur.

Modules Python réutilisables et republication des flows

COMPORTEMENT OBSERVÉ

Un module interne appartient à un projet et n'accepte que PYTHON ou PYSPARK. Son nom suit des limites, une liste de mots interdits et les contraintes de nom de fichier importable. La sauvegarde distingue script/description/snippet publiés et drafts ; les imports wildcard sont refusés. La publication inscrit publicationId, publishedBy et lastPublished et copie le draft comme script publié. La suppression et la publication déclenchent une régénération asynchrone de toutes les branches du projet. Les archives de run chargent les modules publiés par type. isUsableInRun repose sur publicationId afin qu'une attribution historique manquante n'efface pas l'usage d'un module publié.

UTILISATEURS ET RÔLES

Auteur de module, publisher du projet et utilisateurs des flows.

INTERFACES DANS LE CODE

GET/POST /projects/{projectId}/internal-modules ; GET /projects/{projectId}/internal-modules/{moduleId} ; POST/DELETE /projects/{projectId}/internal-modules/{id} ; POST /projects/{projectId}/internal-modules/{id}/publish ; GET /projects/{projectId}/internal-modules/exists.

DONNÉES ET ÉTATS

InternalModule avec champs draft/publiés et identité de publication ; versions VERSION_MODULE régénérées pour les branches du projet.

DÉPENDANCES ET ÉCHANGES

Registre de modules, compilateur d'archives, événement automatique par branche, IAM par faits de module.

COMPORTEMENT CIBLE PROPOSÉ

Flows et Runtimes Rust portent un registre d'artefacts de modules immuables avec digest, dépendances et provenance. Python utilisateur reste supporté ; les modules PySpark doivent migrer vers les API de données sans JVM.

MIGRATION DE CETTE CAPACITÉ

Importer chaque version publiée séparément du draft. Capturer les dépendances exactes d'un run au lieu de recharger une liste mutable de modules. Remplacer la régénération globale par des dépendances déclarées tout en gardant une réconciliation des branches affectées.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Publier un module pendant qu'un ancien run est en cours. Ce run conserve l'ancien digest ; un nouveau run utilise le nouveau. Un module draft ou un nom de fichier invalide n'entre pas dans l'archive exécutable.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/internalmodule/command/InternalModuleCommandHandler.kt:86](#) Validation, sauvegarde de draft et publication.

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/internalmodule/aggregate/InternalModule.kt:98](#) Champs de versions et règle isUsableInRun.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/internalmodule/service/AsynchPublisherImpl.kt:20](#) Régénération des branches du projet.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/service/ScriptGeneratorServiceImpl.kt:800](#) Modules publiés inclus dans les artefacts.

DD-17 **Cible non qualifiée****Métadonnées, transfert de propriétaire et faits d'autorisation****COMPORTEMENT OBSERVÉ**

La mise à jour des métadonnées change nom et description et peut modifier le trigger. Elle refuse un owner différent du propriétaire courant. Le transfert dispose d'une route distincte. Le nouveau propriétaire doit exister, être éligible selon le rôle métier historique et appartenir au projet. Une mise à jour conditionnelle compare l'ancien owner ; zéro ligne modifiée produit une raison explicite, dont conflit concurrent. Le controller ouvre une transaction couvrant le changement et ses événements. Le writer de transfert associe l'événement consommé par process-data au fait d'autorisation consommé par IAM. Une réaffectation après départ de membre emprunte également ce mécanisme spécialisé.

UTILISATEURS ET RÔLES

Propriétaire du flow, propriétaire de projet autorisé et workflow IAM d'offboarding.

INTERFACES DANS LE CODE

POST /dataflows/{id}/metadata ; PATCH /dataflows/{dataflowId}/owner/{newOwnerId} ;
ChangeDataflowOwnerCommandHandler.

DONNÉES ET ÉTATS

ownerId courant, version d'autorité attendue au transfert, métadonnées et faits outbox.

DÉPENDANCES ET ÉCHANGES

IAM/APISIX pour la permission de route ; miroirs users/members pour règles métier ; process-data et IAM pour propagation.

COMPORTEMENT CIBLE PROPOSÉ

IAM Rust décide les droits et éligibilités. Flows Rust conserve la propriété métier et enregistre transfert plus événements dans une même transaction. L'identité capturée d'un run existant ne change pas lors d'un transfert ultérieur.

MIGRATION DE CETTE CAPACITÉ

Conserver la séparation entre auteur de révision, propriétaire courant et principal d'exécution. Supprimer les contrôles de rôle hérités seulement après définition d'une règle centrale équivalente.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un collaborateur sauvegarde un graphe sans devenir propriétaire. Deux transferts concurrents ne produisent qu'un gagnant et une seule chaîne d'autorité cohérente. Un ancien run garde l'identité responsable de son lancement.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/definition/command/DataflowUpdateMetadataCommandHandler.kt:130](#)
Refus de transfert via métadonnées.

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/definition/command/ChangeDataflowOwnerCommandHandler.kt:51](#) Éligibilité et compare-and-set de propriétaire.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/controller/DataflowDevelopController.kt:229](#) Transaction de transfert avec rollback.

DD-18 **Cible non qualifiée**

Déclencheurs fonctionnels et miroir de planification

COMPORTEMENT OBSERVÉ

Les métadonnées d'un flow peuvent référencer un dataset, un autre flow ou un chemin de fichiers du projet comme trigger. Le handler vérifie l'existence du dataset ou du flow et refuse les flows supprimés ; il émet création, modification ou suppression du trigger selon la différence avec la valeur existante. La liste des triggers possibles parcourt toutes les pages des flows du projet, exclut le flow courant et expose aujourd'hui un choix de chemin racine. Les événements de schedules provenant de process-data alimentent le miroir de développement. L'exécution des cron n'appartient pas à ce service.

UTILISATEURS ET RÔLES

Développeur configurant une dépendance événementielle.

INTERFACES DANS LE CODE

GET /dataflows/{id}/triggers ; POST /dataflows/{id}/metadata ; TriggerAtEventDatasetUpdateCommand, TriggerAtEventDataflowUpdateCommand et TriggerAtEventProjectFileUpdateCommand.

DONNÉES ET ÉTATS

TriggerAtEvent unique associé au flow, dataset/flow déclencheur ou projet/chemin ; miroir ExecutionSchedule.

DÉPENDANCES ET ÉCHANGES

Événements de triggers vers process-data, miroirs de datasets/flows et retour des événements de schedule.

COMPORTEMENT CIBLE PROPOSÉ

Flows Rust possède la définition des dépendances et Scheduler dans Runtimes produit des commandes idempotentes. Une dépendance de données se déclenche sur une publication approuvée, avec son édition et sa clé de déduplication.

MIGRATION DE CETTE CAPACITÉ

Inventorier cycles et changements de triggers, fixer la règle d'événement attendu pour les anciennes dépendances et importer les chemins. Pour STREAM, exiger un journal durable avant gouvernance et découper des runs bornés par epoch/range.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Rejouer une notification de publication et un événement de fichier. Chaque occurrence légitime crée au plus un run ; une occurrence dupliquée n'en crée pas. Un candidat refusé par audit ne déclenche aucun flow dépendant de données publiées.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-](#)

[domain/src/main/kotlin/com/cleyrop/data/dev/domain/definition/command/DataflowUpdateMetadataCommandHandler.kt:155](#)

Validation des ressources déclencheuses.

[monorepo · services/data-dev/app/data-dev-](#)

[domain/src/main/kotlin/com/cleyrop/data/dev/domain/definition/command/DataflowUpdateMetadataCommandHandler.kt:202](#) Diff et événements de trigger.

[monorepo · services/data-dev/app/data-dev-](#)

[domain/src/main/kotlin/com/cleyrop/data/dev/domain/schedule/query/ListAvailableTriggersQueryHandler.kt:32](#) Découverte paginée et chemin racine.

[monorepo · services/data-dev/app/data-dev-](#)

[app/src/main/kotlin/com/cleyrop/data/dev/app/definition/event/DataProcessKafkaConsumer.kt:79](#) Consommation des schedules process-data.

Demandes de création de datasets et compilation des collectes

COMPORTEMENT OBSERVÉ

POST /datasets et /datasets/bulk préparent des demandes de collecte après décision IAM sur les cibles du payload. Le batch doit être non vide et viser un seul projet. Les fréquences PLANNED et les triggers événementiels sont refusés dans ce parcours. La validation des triggers porte sur tout le batch, mais les événements sont ensuite émis dataset par dataset et le controller documente l'absence d'atomicité globale. À réception d'une collecte CREATED ou UPDATED, le service attend brièvement le miroir du dataset, vérifie qu'il existe des entrées valides puis produit une CollectVersion. Il convertit CSV, XLSX, JSON, XML et Parquet ainsi que schéma cible, préparations et modes APPEND/OVERRIDE.

UTILISATEURS ET RÔLES

Utilisateur de collecte, data-collect et IAM.

INTERFACES DANS LE CODE

POST /datasets ; POST /datasets/bulk ; DatasetCollectEvent ; DataflowCreateOrUpdateCollectDatasetCommandHandler.

DONNÉES ET ÉTATS

Commande de dataset créée par événement ; miroir DatasetBranch ; CollectVersion cachée avec executionProjectId réel, script et manifeste de sortie.

DÉPENDANCES ET ÉCHANGES

IAM dataset creation authorization, data-collect, data-model, stockage des fichiers importés, process-data et templates de collecte.

COMPORTEMENT CIBLE PROPOSÉ

Datasets et Connectors Rust prennent les commandes de collecte. Flows compose les transformations ; Writer écrit les candidats. Le résultat et le checkpoint source publié sont autoritaires dans l'édition Iceberg ; PostgreSQL garde la commande et les reçus.

MIGRATION DE CETTE CAPACITÉ

Sortir la génération de scripts Spark du parcours de collecte. Remplacer le batch partiel sans reçu par une commande avec résultat par item et clé d'idempotence. Reprendre chaque préparation et règle de schéma avant changement de moteur.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Interrompre un batch au troisième dataset puis le rejouer. Chaque item déjà accepté garde le même résultat, les autres reprennent et le client voit le détail. Une collecte sans entrée valide ne crée pas de publication vide implicite.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/dataset/controller/DatasetController.kt:31](#) Routes, décision IAM et limite de batch documentée.

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/dataset/command/CreateDatasetWithDatasourceCommandHandler.kt:86](#) Validation globale puis émissions individuelles.

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/definition/command/DataflowCreateOrUpdateCollectDatasetCommandHandler.kt:28](#) Attente du miroir, entrées valides et CollectVersion.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/event/DataCollectKafkaConsumer.kt:71](#) Formats, types de source et préparations.

DD-20 **Cible non qualifiée**

Opérations techniques sur datasets et schémas

COMPOTEMENT OBSERVÉ

Les événements de data-model peuvent générer des versions techniques de suppression de table/branches, vidage et retouche de colonnes. La suppression supprime le miroir local puis produit une version DELETE par dataset avec branches et projet réel ; si un tombstone sans branches arrive, elle retire le miroir sans relancer un script. Le vidage exige un miroir correspondant à la branche. Une mise à jour compare les anciens/nouveaux schémas, conserve le schemaMode utile et peut générer suppression, renommage ou cast de colonnes. Une garde refuse une suppression massive qui ferait passer un schéma existant d'au moins deux colonnes à moins de deux. Ces opérations sont des exécutions, pas de simples modifications de métadonnées.

UTILISATEURS ET RÔLES

Data-model, administrateur de données et runtime technique.

INTERFACES DANS LE CODE

DeleteDatasetCommandHandler ; EmptyDatasetCommandHandler ; UpdateDatasetCommandHandler ; VersioningAction VERSION_DELETE, VERSION_EMPTY_DATASET, VERSION_UPDATE_COLUMN.

DONNÉES ET ÉTATS

Miroirs DatasetBranch et versions techniques cachées ; scripts et manifestes DELETE/REPLACE selon opération ; identifiant de projet réel séparé des références synthétiques.

DÉPENDANCES ET ÉCHANGES

Événements data-model, générateur de templates, S3 et process-data.

COMPOTEMENT CIBLE PROPOSÉ

Datasets Rust conserve les commandes métier ; Writer/Publisher applique WAP pour mutations de contenu et schéma. Maintenance traite expiration et GC avec plan audité et gardes de rétention. Une demande de suppression logique ne donne pas au worker le droit de drop de catalogue.

MIGRATION DE CETTE CAPACITÉ

Classier chaque action historique en suppression logique, nouvelle édition vide, évolution de schéma ou destruction physique. Importer les traces et remplacer les versions synthétiques par des RunSpec techniques explicitement typés.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un vidage validé publie une nouvelle édition vide et l'ancienne reste lisible selon rétention. Un cast non compatible ou une suppression de colonnes non approuvée laisse la dernière édition publiée intacte. Le GC refuse un fichier encore référencé par un lecteur épinglé.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/dataset/command/DeleteDatasetCommand.kt:29](#) Tombstone, nettoyage miroir et versions de suppression.

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/dataset/command/EmptyDatasetCommandHandler.kt:22](#) Vidage par branche et projet réel.

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/dataset/command/UpdateDatasetCommandHandler.kt:67](#) Diff de colonnes et garde de schéma.

DD-21 **Cible non qualifiée**

Retour d'exécution dans les graphes et historique de scripts

COMPORTEMENT OBSERVÉ

Data-dev consomme les événements de process-data pour conserver états d'exécution, schedules et clusters utilisés par l'éditeur. Le domaine expose les statuts CREATED, PENDING, RUNNING, FAILURE, SUCCESS, SKIPPED, WARNING et CANCELED. Les graphes peuvent mettre à jour les états des nœuds/sorties depuis ces événements. Les lectures de dernière exécution servent au résumé de publication et au dernier script exécuté d'une transformation. Ce miroir de développement n'est pas l'autorité d'exécution ni la preuve qu'un dataset a été publié après audit.

UTILISATEURS ET RÔLES

Éditeur, utilisateur consultant un flow et process-data.

INTERFACES DANS LE CODE

DataProcessKafkaConsumer ; UpdateGraphExecutionStateCommandHandler ; GET /dataflows/{id}/transformations/{transformationId}/lastExecutedScript ; GET /dataflows/{id}/publish/lazy.

DONNÉES ET ÉTATS

Execution miroir, états de graphe et références aux versions ; clusters/schedules miroir.

DÉPENDANCES ET ÉCHANGES

Protobuf d'exécutions process-data, stockage des versions et graphes.

COMPORTEMENT CIBLE PROPOSÉ

Flows Rust expose une projection reconstruisible depuis le journal de runs et les éditions approuvées. Frontends présente séparément état du calcul, état d'audit et état de publication.

MIGRATION DE CETTE CAPACITÉ

Conserver les identifiants de transformation, script, job et version dans les événements de remplacement. Importer les états sans les interpréter comme des preuves WAP. Prévoir la réparation de projection par replay.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Rejouer les événements de run dans un ordre retardé. Un état terminal ne redevient pas RUNNING et le graphe pointe vers le bon run. Un job réussi avec publication refusée apparaît comme calcul réussi et données non publiées.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-](#)

[app/src/main/kotlin/com/cleyrop/data/dev/app/definition/event/DataProcessKafkaConsumer.kt:59](#) Consommateurs d'exécution, schedule et cluster.

[monorepo · services/data-dev/app/data-dev-](#)

[domain/src/main/kotlin/com/cleyrop/data/dev/domain/definition/aggregate/DataflowDefinition.kt:132](#) Vocabulaire des statuts.

[monorepo · services/data-dev/app/data-dev-](#)

[domain/src/main/kotlin/com/cleyrop/data/dev/domain/branch/command/UpdateGraphExecutionStateCommandHandler.kt:89](#)

Projection des états dans le graphe.

[monorepo · services/data-dev/app/data-dev-](#)

[app/src/main/kotlin/com/cleyrop/data/dev/app/definition/controller/DataflowGetterController.kt:181](#) Lecture du dernier script exécuté.

Événements fiables, faits IAM et traitement des messages en échec

COMPORTEMENT OBSERVÉ

Les producteurs de versions et de faits d'autorisation utilisent une outbox PostgreSQL. Le relay réclame des lignes par verrou SKIP LOCKED, attend l'acquittement Kafka puis marque sentAt. Pour les messages avec clé, sa requête bloque les successeurs tant qu'un prédécesseur du même topic/clé n'est pas envoyé. Un envoi réussi suivi d'une panne avant commit peut être livré de nouveau. Les messages en échec sont parqués avec statut, cause et informations d'origine. Data-dev dispose d'un drainer, d'un probe de dépendances, de retrait après tombstone, d'expiration et d'un endpoint JMX de census/replay. Les mutations administratives ne sont pas exposées comme endpoints web génériques dans la configuration lue.

UTILISATEURS ET RÔLES

Consommateurs de domaine, IAM et opérateurs de reprise.

INTERFACES DANS LE CODE

OutboxPublisher/OutboxRelay ; DeadLetterEventConsumer/DeadLetterEventDrainer ; JMX dlt-replay ; AuthzFactOutboxPublisher.

DONNÉES ET ÉTATS

outbox_event ordonné par relay_sequence ; dead_letter_event avec reçu de traitement et statut ; faits de ressources pour IAM.

DÉPENDANCES ET ÉCHANGES

Kafka/Protobuf et JSON de certains événements, PostgreSQL, IAM comme writer des tuples.

COMPORTEMENT CIBLE PROPOSÉ

Audit et Flows Rust conservent outbox/inbox et schémas d'événements versionnés. Le transport final n'a aucune JVM ; les clés de déduplication et d'ordre sont contractuelles. Les reçus de publication ou d'effets restent durables sans dépendre d'un fan-out UI.

MIGRATION DE CETTE CAPACITÉ

Importer outbox non envoyée et messages parqués sans modifier leurs payloads. Construire des adaptateurs de double lecture temporaire avec un seul writer par ressource. Qualifier les reprises et l'ordre par agrégat avant remplacement du transport Kafka.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Faire tomber le relay après acquittement distant et avant sentAt. Le replay est déduplicqué par le consommateur et ne double ni version ni autorité. Un message dépendant d'un projet absent reste parqué puis reprend quand le projet apparaît.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/common/outbox/OutboxEventJPA.kt:11](#) Ordre par clé et verrou SKIP LOCKED.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/common/outbox/OutboxRelay.kt:25](#) Envoi, acquittement, erreurs et rétention.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/common/deadletter/DeadLetterEventDrainer.kt:143](#) Drainer et sweep des messages parqués.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/common/deadletter/DeadLetterReplayEndpoint.kt:37](#) Administration JMX de census et replay.

Migrations de code, réconciliation et exploitation

COMPORTEMENT OBSERVÉ

Le service combine migrations Flyway et migrations de code suivies par une table cleyrop_migration. Il possède des modes de démarrage dédiés migrate et reconcile, dont un parcours de réparation des applications et de leurs membres. Un reconciler de comptes de service reprend les projets qui ne sont pas encore provisionnés dans IAM. Les déploiements incluent des sondes, un port de management, un espace temporaire pour les archives, un PVC optionnel et des règles réseau de gateway. Les métriques couvrent notamment outbox, DLT et échecs de verrouillage Git. L'état opérationnel dépend donc de PostgreSQL, des artefacts S3 et d'effets distants, pas du seul pod API.

UTILISATEURS ET RÔLES

Opérateur de plateforme et automatisation de migration.

INTERFACES DANS LE CODE

MigrateMode ; ReconcileMode ; UpgradeReconcile ; CleyropMigrationRunner ; ServiceAccountReconcileJob ; health/prometheus et endpoints JMX configurés.

DONNÉES ET ÉTATS

Journal cleyrop_migration, marqueurs de provisioning du projet, miroir d'applications, outbox/DLT, archives en S3.

DÉPENDANCES ET ÉCHANGES

Flyway, PostgreSQL, IAM, Git, applications, Kafka et S3.

COMPORTEMENT CIBLE PROPOSÉ

Migration Rust orchestre des étapes idempotentes avec préconditions, journal et validation. Audit conserve les preuves de conversion. Maintenance suit rétention et sauvegarde des commandes, graphes et artefacts.

MIGRATION DE CETTE CAPACITÉ

Exporter un inventaire vérifiable des tables et artefacts, prendre un point de reprise cohérent puis importer par cohortes. Le retrait de JVM inclut Flyway embarqué et les migrations Kotlin restantes. Garder les outils legacy isolés uniquement pendant la transition.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Interrompre l'import entre deux lots puis reprendre. Les IDs et graphes importés restent identiques ; aucun effet Git/IAM n'est doublé. Une restauration de sauvegarde retrouve les artefacts référencés et les commandes non terminées.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/runmode/UpgradeReconcile.kt:55](#)
Actions de backfill et applications.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/runmode/ReconcileMode.kt:51](#)
Mode de réconciliation dédié.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/project/service/iam/ServiceAccountReconcileJob.kt:96](#) Réconciliation du provisioning IAM.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/codemigration/runner/CleyropMigrationRunner.kt:30](#) Migrations de code au démarrage.

[monorepo · services/data-dev/chart/templates/deployment.yaml:68](#) Sondes et montage des volumes.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- Flows Rust possède drafts, révisions exécutables, branches, graphes, MR, résolution, activation et manifestes. La révision inclut digest du code/modules et version du compilateur.
- IAM Rust possède projets, membres et droits. Les contrats conservent une vérification de cohérence entre l'objet nommé par une route, le projet du payload et les objets réellement persistés.
- Définir un RunSpec qui nomme principal, projet réel, version de flow, éditions d'entrée, profils de ressources, sorties autorisées, secret refs et deadline. Les huit kinds sont TABLE, INCREMENTAL, VIEW, FILESET, MODEL, EMBEDDINGS, FEATURESET et STREAM.
- Writer médie le staging et Publisher médie la publication. Flows n'accorde pas au worker un accès REST générique table.modify ; le droit inclut commit/drop et dépasserait le besoin du job.
- La définition publiée et la donnée publiée ont des cycles distincts. L'historique de publications approuvées ne se reconstruit pas en listant tous les snapshots.
- Les événements de dépendance et les checkpoints publiés référencent l'édition Iceberg autoritaire. PostgreSQL reste autoritaire pour drafts, commandes, identités et reçus d'effets.
- Applications traite provisionnement Git/notebooks comme effets idempotents, avec état de réparation. Notifications traite le fan-out UI séparément des preuves durables.

Plan de données proposé

- Compiler le low-code vers un plan typé sans Spark. Chaque opérateur doit avoir une sémantique prouvée sur schéma, nulls, types, dates, ordre et erreurs.
- Exécuter SQL et transformations tabulaires dans le moteur Rust retenu après campagnes. Python/SQL utilisateur reste un format d'usage, dans des processus bornés et sans API Spark conservée par défaut.
- Stager des fichiers immuables, produire un candidat avec origin.* au summary puis faire auditer ce candidat. La preuve post-candidat est liée durablement, sans réécriture du summary par fast-forward.
- Pour un flow à plusieurs sorties, produire une release qui est elle-même une édition Iceberg avec checkpoint source/epoch/range/participants et lecteurs épinglés.
- Un preview est un résultat de run temporaire avec portée et rétention, jamais une exception aux règles d'accès. Les résultats STREAM viennent d'un journal durable et franchissent WAP.
- EXPORT est un effet avec reçu et EXPOSURE un consommateur déclaré. Ni l'un ni l'autre n'ajoute un neuvième kind de sortie.

Découpage du core Rust

- Proposition flows-model : contrats serde pour graphe, étapes, branches, révisions, modules, manifestes et types d'erreurs, avec import du JSON historique.
- Proposition flows-compiler : tri du graphe, validation des opérateurs, calcul des schémas, dépendances d'artefacts et génération de plans SQL/Arrow ou invocation Python.
- Proposition flows-revisions : sauvegarde à révision attendue, diff sémantique, MR, règles de résolution et activation de définition.
- Proposition flow-run-contract : RunSpec immuable, liaisons vers Writer/Publisher, preuves origin.*, entrées épinglées et sorties déclarées.

- Proposition applications-adapters : Gitea, notebooks, comptes de service et procédures d'offboarding sous commande durable.
- Proposition event-runtime : outbox/inbox, déduplication, ordre par agrégat, DLT et réconciliation indépendants du transport final.
- Réutilisation d'un moteur Rust existant à qualifier, sans annoncer qu'Icegres ou Eidos couvrent les opérateurs de ce service. Leur capacité concrète est hors de cette lecture.

Identité, autorisation et isolation

- Garder séparés appelant, auteur de version, propriétaire et principal de run. Les transferts ne réécrivent pas les preuves historiques.
- Conserver le refus des payloads croisant projet et ressource. La canonicalisation legacy/IAM est obligatoire avant comparaison.
- La directive générale d'autorisation gateway ne suffit pas à décrire le code : suppression de branche, création de MR, éligibilité du propriétaire et création de dataset gardent des contrôles locaux. Ils doivent entrer dans la matrice de migration IAM.
- Les secrets EnvVar sont masqués dans le DTO mais visibles au DAO et incorporés aux scripts générés. La migration doit retirer ces valeurs des artefacts et adopter des références de secrets versionnées.
- Les clés SSH ont un convertisseur de chiffrement ; leur migration et rotation doivent préserver la compatibilité avec la clé partagée existante sans export de clé dans le dossier.
- La portée preview n'est fournie qu'aux clients internes configurés et exige un manifeste complet. Les travaux de migration doivent réparer tout écart entre génération et persistance de cette preuve.
- Un worker cible ne peut ni publier ni drop une table. Les vérifications UUID/ref/schema/spec/ascendance appartiennent au Publisher et ne sont pas présentées comme un fencing Hemera natif.

Exploitation et reprise

- Deux modules Gradle séparent domaine et adaptateurs Spring. Les versions lues sont celles du dépôt ; aucune version déployée n'a été interrogée.
- La sauvegarde/restauration doit couvrir PostgreSQL et artefacts S3, ainsi que le rapprochement avec les dépôts Git, clés chiffrées et applications externes.
- Les sondes et le management Prometheus sont définis dans le chart et les propriétés. L'existence d'un PodMonitor dépend des conditions de chart et de la CRD de monitoring.
- Le chart permet un stockage persistant optionnel ; sa documentation indique que cette option impose un seul replica et une stratégie Recreate. L'espace temporaire sert notamment aux ZIP générés.
- L'outbox réclame les événements par verrous et conserve l'ordre par clé, mais les consommateurs doivent supporter la relivraison. Les métriques DLT et de provisioning sont nécessaires au pilotage de bascule.
- Les migrations Kotlin/Flyway et les modes migrate/reconcile constituent une dette JVM opérationnelle à remplacer. Les migrations de code doivent être qualifiées en reprise partielle.
- Un échec de verrouillage Git peut laisser un compte distant ouvert malgré le miroir révoqué. Le dossier propose un suivi durable de cet effet et une réconciliation avant fermeture d'offboarding.
- Aucun test Gradle ou Helm n'a été exécuté pendant cet audit en lecture seule.

Séquence de migration

- 1. Inventorier IDs legacy/IAM, graphes, branches actives/supprimées, versions, MR, modules, variables, artefacts S3 et événements non terminés.
- 2. Construire un importeur sans secrets en clair qui conserve IDs, dates, auteurs et liens d'artefacts. Réconcilier les versions dépourvues de manifeste avec leur preuve historique.
- 3. Formaliser les contrats API et de payload observés, dont 410 projet, erreurs de sauvegarde concurrente, batch partiel, branches restaurables et MR dépendantes.
- 4. Porter le modèle et le compilateur de low-code vers Rust, puis qualifier chaque opérateur et chaque format. Classer les programmes PySpark qui exigent une réécriture utilisateur.
- 5. Introduire RunSpec et le protocole Writer/Audit/Publisher WAP. Séparer activation d'une définition et publication de résultats, avec sorties multiples et checkpoints publiés dans Iceberg.
- 6. Migrer secrets/modules en artefacts immuables, puis déplacer Git/notebooks et leurs effets vers Applications. Finir le transfert des projets et membres vers IAM.
- 7. Basculer par projet avec un seul writer de drafts, compare de projections et replay d'outbox. Maintenir des lecteurs temporaires de l'historique sans autoriser les anciens runtimes à republier.
- 8. Retirer générateur Spark, clients JVM, Spring/Flyway Kotlin et transport Kafka JVM seulement après les campagnes de résultats, révocation, reprise et restauration.

Écarts et décisions encore ouverts

- Preview : le handler lit VersionRepository mais ne sauvegarde pas la version produite ; le producteur ne fait qu'émettre. PreviewScopeQueryHandler exige pourtant findById. Vérifier l'existence d'un autre chemin de persistance et qualifier ce raccord avant d'en faire un contrat cible.
- Publication : le code lit la dernière exécution pour le résumé sans exiger son succès dans le handler de publication. La politique d'activation d'un flow doit être décidée séparément de WAP.
- Variables secrètes : le périmètre historique d'artefacts qui incorporent des valeurs n'a pas été inspecté, conformément à la consigne. Une campagne sécurisée d'inventaire est requise.
- Concurrence : baseVersionId reste optionnel et certains chemins actualisent une GraphVersion. Il faut décider la durée de compatibilité de ces écritures historiques avant immutabilité stricte.
- Autorisation : le code conserve des règles de propriétaire et d'éligibilité dans des handlers malgré la note générale gateway. Réconcilier code, routes IAM et contrat métier avant suppression de ces règles.
- Suppression de dataset : distinguer les événements sans branches, qui ne génèrent pas de script, et les suppressions de contenu. Aucun tombstone importé ne doit être converti en destruction physique automatique.
- La compatibilité des fonctions Python/PySpark privées, modules utilisateurs et images installées dépend de données client non lues. Aucun taux de compatibilité n'est annoncé.
- Le couplage observé à Nessie et aux noms Spark de branches doit être traduit ; un renommage de ref seul ne fournit pas un protocole de publication autorisé et audité.

Références de l'analyse

[monorepo](#) · [services/data-dev/app/gradle/libs.versions.toml:23](#) Versions Kotlin et Spring dans le dépôt.

[monorepo](#) · [services/data-dev/app/data-dev-app/build.gradle.kts:24](#) Toolchain JVM 21.

[monorepo](#) · [services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/version/aggregate/Version.kt:19](#) Familles de versions.

[monorepo](#) · [services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/service/ScriptGeneratorServiceImpl.kt:339](#) Compilation d'une version exécutable.

[monorepo](#) · [services/data-dev/chart/templates/deployment.yaml:50](#) Processus non root, sondes et volumes.

[monorepo](#) · [services/data-dev/chart/values.yaml:239](#) Contraintes du mode persistence.

SERVICE ACTUEL · PROCESS-DATA

Exécution des flows, planification et ressources de calcul

22 capacités analysées

services/process-data · Kotlin 2.4.10, Spring Boot 4.1.1 et JVM 21 observés dans Gradle. PostgreSQL/JPA/Flyway, Kafka/Protobuf avec outbox/inbox, Quartz JDBC clusterisé pour cron, boucles Spring de réconciliation, S3, REST app-management et HTTP vers les processus de session Python/PySpark.

Service qui reçoit les versions exécutables produites par data-dev, crée et suit les runs et jobs, orchestre les sessions Python/PySpark, gère les déclenchements, clusters et images, expose historique et temps réel, et relaie les états aux autres services.

Lecture de code

Pas de qualification runtime nouvelle

Domaines cible : Runtimes, Flows, IAM, Writer, Publisher, Query, Applications, Notifications, Audit, Migration, Maintenance, Egress.

Périmètre de lecture

- Controllers execution, schedule, session, image et admin, DTO de soumission et filtres.
- Agrégats DataflowVersion, Script, ExecutionStorageManifest, DataflowExecution, jobs, files persistantes, Cluster, Session et schedules.
- Handlers de run manuel/automatique/partiel, soumission, suivi, logs, fin de job, annulation et callback Spark.
- Adaptateur app-management et appels directs de sessions, cycle de clusters/images, bibliothèque PyPI, startup et réconciliation.
- Consumers DataDev/ProcessData/IAM, outbox/inbox/DLT, maintenance PostgreSQL, RunnerAuthorizationClient, manifestes Gradle et chart.

Objets et cycle de vie

- DataflowVersion : copie exécutable reçue de data-dev, avec scripts, dépendances, manifestes, propriétaire, branche et projet réel d'exécution ; ce n'est pas une édition de données publiée.
- DataflowExecution : run avec statut, déclencheur, principal, cluster, échéance de capacité, dates et jobs. L'exécution partielle conserve des résultats antérieurs marqués comme réutilisés.
- DataflowJobExecution et JobDetails : script, étapes, sorties, logs et flag historique isSparkCommitted. Les statuts finaux ne décrivent pas à eux seuls un reçu de publication.
- DataflowSubmission, JobSubmission, JobSubmitted, JobCompleted et ExecutionCancelled : intentions et étapes de progression durables utilisées pour reprendre après incident.
- Cluster et Session : capacité persistante Python/PySpark, ressources, images, état observé, pause et suppression différée.
- Image, ImageUpdate et Library : base, environnement personnalisé, demandes de packages, builds et historique.
- DataflowSchedule : expression/fuseau ou déclenchement événementiel ; Quartz stocke jobs, triggers et heartbeat.
- Miroirs User/Project/ProjectMember, correspondances IAM, outbox, inbox, quarantaine, chunks de logs et suivi de migration.

Dépendances du service

- Data-dev produit VersionEvent, changements de propriétaire, suppressions et triggers ; process-data renvoie états de run, schedules et clusters.
- App-management reçoit les appels REST de session, serveur Spark Connect, submission, cancellation, inventaire et build d'image ; les sessions exposent aussi leur état/job en HTTP.
- S3 héberge les archives de scripts ; le run obtient une URL signée vers la version/script. Le contenu effectif et les versions déployées ne sont pas inspectés.
- IAM autorise les opérations de runner et fournit les miroirs ; gateway protège les routes externes ; Notification Center relaie les mises à jour de clusters.
- PostgreSQL conserve contrôle et historique ; Kafka transporte Protobuf et événements ; Quartz ajoute la planification clusterisée ; PyPI/index interne fournit les bibliothèques.

Fonctionnalités et compatibilité à préserver

PD-01 **Cible non qualifiée**

Réception des versions et intégrité du manifeste

COMPORTEMENT OBSERVÉ

Le consommateur VersionEvent réclame une clé d'inbox dans sa transaction puis construit une version locale. VERSION_SAVE conserve la version sans run. MERGE, PUBLISH, COLLECT, PREVIEW, DELETE, EMPTY_DATASET et UPDATE_COLUMN peuvent lancer automatiquement le traitement si le démarrage des clusters est actif. Les actions inconnues sont rejetées pour retry/DLT. Une version dupliquée ne ressuscite pas une version supprimée. Le handler peut enrichir uniquement un manifeste absent ; il vérifie le même ensemble de scripts et refuse un manifeste non nul divergent. Il préserve les autres métadonnées de la version existante. La portée execution_project_id vient du producteur et une valeur invalide ne devient pas un runner global.

UTILISATEURS ET RÔLES

Data-dev, service de migration et exécuter.

INTERFACES DANS LE CODE

Kafka VersionEvent ; DataDevKafkaEventConsumer.consumeVersionEvent ; DataflowVersionCreateCommandHandler ; VersionEventProtoConverter.

DONNÉES ET ÉTATS

DataflowVersion immatriculée par VersionId, branches et projet, scripts avec dépendances et manifestes ; inbox de déduplication ; marqueur deletedAt.

DÉPENDANCES ET ÉCHANGES

Data-dev par Protobuf/Kafka, PostgreSQL et contrôleur de runs.

COMPORTEMENT CIBLE PROPOSÉ

Flows/Runtimes Rust acceptent un RunSpec immuable et versionné. Un même ID ne peut désigner des artefacts ou droits différents. Les versions anciennes sans manifeste sont isolées jusqu'à preuve ou conversion explicite.

MIGRATION DE CETTE CAPACITÉ

Importer versions, scripts, manifestes et inbox de manière cohérente. Conserver les références d'origine mais remplacer les IDs synthétiques d'opérations techniques par un type de run explicite. Ne jamais reconstruire des droits depuis le flow courant.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Rejouer la même version avec un manifeste égal, absent puis divergent. Les deux premiers cas conservent les droits existants ; le troisième est refusé. Rejouer une version supprimée ne la remet ni en file ni en service.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/common/event/consumer/DataDevKafkaEventConsumer.kt:45](#) Inbox, stockage et déclenchement conditionnel.

[monorepo · services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/DataflowVersionCreateCommandHandler.kt:44](#)

Immutabilité et enrichissement limité du manifeste.

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/common/event/consumer/converter/VersionEventProtoConverter.kt:142](#) Portée projet et matrice d'actions automatiques.

PD-02 **Cible non qualifiée**

Lancement manuel d'un flow complet**COMPORTEMENT OBSERVÉ**

Le lancement manuel attend que la version demandée soit visible, refuse une version supprimée et une version sans script, puis consulte la dernière exécution de sa branche. Si elle est CREATED, PENDING ou RUNNING, il refuse un nouveau run concurrent selon la règle métier locale. Le cluster vient de l'override de requête, de la version ou du précédent run, avec repli vers un cluster par défaut du type attendu. Le service crée un DataflowExecution, calcule son nom numérique par comptage des exécutions du projet, persiste ses jobs éventuels puis émet ExecutionCreatedEvent. Le demandeur authentifié devient le principal MANUAL_CALLER distinct du propriétaire de version.

UTILISATEURS ET RÔLES

Utilisateur autorisé à exécuter le flow.

INTERFACES DANS LE CODE

POST /dataflows/{id}/versions/{version}/run, paramètre cluster optionnel ;
DataflowVersionRunManualCommand.

DONNÉES ET ÉTATS

Execution CREATED puis événement vers le traitement de file ; principal capturé, cluster, version, timestamps et deadline.

DÉPENDANCES ET ÉCHANGES

Versions répliquées de data-dev, clusters, miroirs utilisateurs/projets, outbox d'exécution et notifications.

COMPORTEMENT CIBLE PROPOSÉ

Runtimes Rust accepte une commande idempotente par projet et version, acquiert une réservation atomique de concurrence et fige RunSpec. Le run est borné en temps, octets et ressources, indépendamment du pod worker.

MIGRATION DE CETTE CAPACITÉ

Conserver la politique de concurrence par branche ou la faire évoluer explicitement. Remplacer le contrôle lecture-puis-écriture par une contrainte/lease transactionnelle. Corréler le flow de l'URI avec la version persistée dans le nouveau contrat.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Envoyer deux requêtes concurrentes de même clé de commande sur deux replicas. Elles retournent un seul run. Deux clés différentes visant une branche limitée à un run produisent un gagnant et un conflit déterministe.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/execution/controller/DataflowPipelineController.kt:206](#) Route et identité du demandeur.

[monorepo · services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/DataflowVersionRunCommandHandler.kt:198](#)

Conditions de lancement et choix du cluster.

[monorepo · services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/DataflowVersionRunCommandHandler.kt:101](#) Création, persistance et événements du run.

[monorepo · services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/session/aggregate/Cluster.kt:196](#) Sélection d'un cluster ou d'un défaut par type.

PD-03 Cible non qualifiée**Exécution partielle d'une transformation et réutilisation d'amont****COMPOTEMENT OBSERVÉ**

La route de transformation résout le script correspondant à l'identifiant métier de transformation. Elle refuse un dernier run de branche encore actif, choisit un cluster puis calcule les dépendances à relancer. Un amont dont le job précédent est SUCCESS ou WARNING n'est pas ajouté à la fermeture de dépendances. Le nouveau run peut copier des jobs du précédent avec de nouveaux IDs et `isFromAnotherExecution=true`, tout en relançant les scripts sélectionnés. Les dates de démarrage, la session et la deadline sont renouvelées. Le résultat est marqué `isPartialRun` ; le statut global ignore certains jobs copiés dans son calcul d'échec.

UTILISATEURS ET RÔLES

Développeur qui corrige ou inspecte une transformation.

INTERFACES DANS LE CODE

POST `/dataflows/{id}/versions/{version}/transformations/{transformation}/run` ; `DataflowVersion.getDepends` ; `DataflowExecution.singleScriptExecution`.

DONNÉES ET ÉTATS

Nouveau run partiel, jobs réutilisés marqués, nouveaux jobs pour le sous-graphe nécessaire et nouvelle attribution du lancement.

DÉPENDANCES ET ÉCHANGES

Historique des jobs de branche, graphe de dépendances de la version et runtime de session.

COMPOTEMENT CIBLE PROPOSÉ

Runtimes Rust réutilise uniquement des résultats liés à des éditions d'entrée et artefacts immuables prouvés. La sélection d'un sous-graphe enregistre les dépendances réutilisées, leur provenance et la politique d'invalidation.

MIGRATION DE CETTE CAPACITÉ

Ne pas convertir `isFromAnotherExecution` en preuve suffisante de réutilisation. Importer cette information comme provenance historique, puis exiger digests, éditions et paramètres pour les nouveaux runs partiels.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Relancer une transformation après modification d'une entrée amont. Le runtime invalide l'ancien résultat et recalcule le sous-graphe requis. Si aucun élément du manifeste n'a changé, il réutilise l'édition exacte et l'explicite dans le reçu du run.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/DataflowVersionRunCommandHandler.kt:163](#) Sous-graphe, dernier run et principal du run partiel.

[monorepo · services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/aggregate/DataflowVersion.kt:68](#) Amonts SUCCESS/WARNING réutilisés.

[monorepo · services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/aggregate/DataflowExecution.kt:83](#) Copie de jobs et renouvellement du run.

[monorepo · services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/aggregate/DataflowExecution.kt:137](#) Calcul de résultat partiel.

PD-04 **Cible non qualifiée**

Identité d'exécution, projet réel et durée d'autorisation**COMPOTEMENT OBSERVÉ**

L'exécution conserve ExecutionPrincipal avec MANUAL_CALLER, SCHEDULED_OWNER ou INTERNAL_OWNER. Les runs planifiés capturent le propriétaire de la version utilisée ; les manuels capturent le demandeur. La deadline d'autorisation est fixée à la création et le repository conserve celle de la ligne verrouillée lors des sauvegardes. Submit refuse de lancer un job après cette borne. Pour les jobs de flow/preview, le projet vient de la version ; pour collecte/suppression/vidage/colonnes, il vient de executionProjectId, distinct des anciennes références synthétiques. Le service n'envoie pas directement le manifeste dans SubmitJobRequestDTO ; app-management reçoit les IDs du job et doit résoudre les droits selon son propre contrat.

UTILISATEURS ET RÔLES

Utilisateur, scheduler, run technique, IAM et app-management.

INTERFACES DANS LE CODE

ExecutionPrincipal ; DataflowExecution.capabilityDeadline ; SubmitCommandHandler ; SubmitJobRequestDTO.

DONNÉES ET ÉTATS

Principal et deadline persistés dans dataflow_execution ; version et type de job ; projet d'exécution réel optionnel pour l'historique.

DÉPENDANCES ET ÉCHANGES

IAM et broker de credentials via app-management ; miroirs de projet ; manifeste de scripts conservé en base.

COMPOTEMENT CIBLE PROPOSÉ

IAM/Runtimes Rust délivrent une capacité liée à tenant, projet, run, tentative, manifestes et échéance. La révocation coupe les nouveaux accès. Les workers sont fongibles et ne possèdent ni credentials de catalogue génériques ni état autoritaire.

MIGRATION DE CETTE CAPACITÉ

Importer le principal seulement quand il est prouvé. Les exécutions anciennes sans principal ne doivent pas recevoir de fallback global. Fixer le protocole de résolution app-management et la borne de renouvellement avant remplacement.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Changer le propriétaire après lancement d'un run puis réessayer une soumission. L'identité d'origine reste inchangée. Après l'échéance, aucune nouvelle délégation n'est émise et la tentative se termine avec un motif contrôlable.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/aggregate/DataflowExecution.kt:20](#) Types de principal et deadline.

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/execution/repository/DataflowExecutionRepositoryImpl.kt:76](#) Préservation de la deadline sur ligne verrouillée.

[monorepo · services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/SubmitCommandHandler.kt:99](#) Borne de soumission et choix du projet réel.

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/session/service/appmanagement/dto/RequestDTO.kt:67](#) Contrat de soumission sans valeur de credential.

PD-05 **Cible non qualifiée**

Files persistantes et progression du graphe de jobs

COMPOTEMENT OBSERVÉ

ExecutionCreatedEvent fait passer une exécution CREATED à PENDING et crée DataflowSubmission. La file est ordonnée par cluster et expose une position. Le démarreur charge les clusters et jobs déjà soumis, cherche une session libre et crée des JobSubmission pour les racines du graphe. Une session est choisie parmi les IDLE sans job en attente/soumis ; les jobs d'un même flow peuvent rester rattachés à sa session. Les enfants sont déclenchés après succès et seulement si tous leurs amonts sont SUCCESS. Le traitement parcourt successivement nettoyage, sessions, annulation, démarrage, soumission, suivi et fin de job. Cette boucle est un @Scheduled du pod ; elle n'est pas le job Quartz utilisé pour les cron.

UTILISATEURS ET RÔLES

Ordonnanceur interne et opérateur suivant les files.

INTERFACES DANS LE CODE

DataflowExecutionRunCommandHandler ; StartExecutionCommandHandler ;
Orchestrator.manageOrchestratorLifecycle.

DONNÉES ET ÉTATS

DataflowSubmission, JobSubmission, JobSubmitted, JobCompleted et ExecutionCancelled en PostgreSQL ;
run CREATED/PENDING/RUNNING.

DÉPENDANCES ET ÉCHANGES

PostgreSQL, ClusterService et événements de succès/session.

COMPOTEMENT CIBLE PROPOSÉ

Runtimes Rust possède un ordonnanceur durable avec claims atomiques, leases bornés, tentative monotone et déduplication. Les files ne donnent pas de droits au stockage. Le worker remplaçable ne modifie jamais le checkpoint publié.

MIGRATION DE CETTE CAPACITÉ

Conserver les identifiants de commande/job lors de la transition. Remplacer les séquences d'appels au repository par des transactions définies. Qualifier la fenêtre entre création distante du job et enregistrement local avant montée en replicas.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Perdre un pod entre réservation d'une session et création distante du job. Un autre contrôleur reprend après expiration de lease ; un seul effet de soumission est accepté et la file ne reste pas bloquée.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/DataflowExecutionRunCommandHandler.kt:82](#) Passage PENDING et file persistante.

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/StartExecutionCommandHandler.kt:84](#) Déqueue, session et racines.

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/session/aggregate/Cluster.kt:156](#) Sélection de session libre.

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/execution/service/Orchestrator.kt:45](#) Boucle périodique du pod.

PD-06 **Cible non qualifiée****Soumission aux sessions Python et Spark Connect****COMPOTEMENT OBSERVÉ**

Submit examine JobSubmission et évite un nouveau job si la session possède déjà un JobSubmitted. Il exige une session IDLE, charge l'exécution, refuse un run final ou hors deadline et soumet via ClusterService. L'adaptateur crée une URL S3 présignée pour l'artefact version/script puis envoie job, cluster, session et projet à app-management. Une création distante valide devient job RUNNING, JobSubmitted et session BUSY. Une session absente, invalide ou une réponse nulle déclenche FAILURE. Le statut et l'annulation du job passent ensuite directement par HTTP vers le processus de session. Les appels distants et écritures PostgreSQL ne forment pas une transaction atomique dans les handlers lus.

UTILISATEURS ET RÔLES

Ordonnanceur, app-management et processus de calcul.

INTERFACES DANS LE CODE

POST app-management /private/sessions/jobs/submit ; HTTP de session submit/{statementId} ; ClusterConnectServerServiceAppManagementImpl.submitJob/getJobState/cancel.

DONNÉES ET ÉTATS

JobSubmission vers JobSubmitted avec StatementId distant ; session IDLE/BUSY ; artefact S3 référencé par URL courte.

DÉPENDANCES ET ÉCHANGES

App-management authentifié par client de service, S3, réseau de sessions Python/PySpark.

COMPOTEMENT CIBLE PROPOSÉ

Runtimes Rust soumet un JobSpec à un adaptateur Kubernetes/processus non JVM. L'ID de tentative est idempotent côté exécuter ; la réponse distante peut être retrouvée après perte réseau. L'artefact est vérifié par digest avant exécution.

MIGRATION DE CETTE CAPACITÉ

Préserver les contrats de job et les reçus pendant le remplacement de l'adaptateur. Introduire une opération query-by-attempt pour résoudre une soumission ambiguë. Remplacer les sessions Spark Connect par workers bornés et pools Python adaptés.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Faire créer un job distant puis couper la réponse. La reprise retrouve le même job par son ID de tentative, vérifie son digest et ne le crée pas une deuxième fois. Une URL d'artefact expirée donne une erreur reprise sans fallback de credentials.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/SubmitCommandHandler.kt:56](#) Garde de session et transitions locales.

[monorepo · services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/SubmitCommandHandler.kt:128](#) Traitement des résultats de soumission.

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/session/service/appmanagement/ClusterConnectServerServiceAppManagementImpl.kt:173](#) Présignature et soumission distante.

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/session/service/appmanagement/ClusterConnectServerServiceAppManagementImpl.kt:198](#) Suivi et annulation HTTP directs.

PD-07 **Cible non qualifiée**

États des jobs, étapes et sorties**COMPOTEMENT OBSERVÉ**

Le run et les jobs utilisent CREATED, PENDING, RUNNING, FAILURE, SUCCESS, SKIPPED, WARNING et CANCELED. Les états finaux d'un job ne sont pas remplacés par un nouvel état. Les détails associent transformation, étape, sortie, logs et un outputStatus distinct. Le suivi de session convertit available en succès, error en échec et un état inconnu en échec. Un job échoué marque les descendants non réussis SKIPPED. Une réussite déclenche les enfants admissibles. Le run devient terminal quand tous les scripts possèdent un job final ; WARNING peut contribuer à l'échec global, avec une règle propre aux runs partiels. Les messages Python renseignent aussi les étapes sans déterminer seuls le résultat global.

UTILISATEURS ET RÔLES

Exécuteur, bibliothèque Python, frontends et consommateurs de datasets.

INTERFACES DANS LE CODE

ExecutionUpdateStatusCommandHandler ; JobCompletedCommandHandler ; ScriptExecutionEvent ; ExecuteFinalPipelineEvent.

DONNÉES ET ÉTATS

DataflowExecution, DataflowJobExecution, JobDetail et ExecutionState de sortie. Historiques terminal et de provenance des jobs repris.

DÉPENDANCES ET ÉCHANGES

Runtime de session pour résultat, Kafka pour messages de scripts et événements de fin, projection data-dev/data-model.

COMPOTEMENT CIBLE PROPOSÉ

Runtimes Rust sépare état du calcul, état du candidat, décision d'audit et publication. Une sortie n'est disponible au lecteur que par une édition approuvée. Le vocabulaire legacy peut être exposé par une projection de compatibilité.

MIGRATION DE CETTE CAPACITÉ

Écrire une table de transition et une matrice de conversion des états inconnus, WARNING, SKIPPED et reprises partielles. Éviter de recopier SUCCESS dans un statut de publication. Corréler les événements au run/tentative, pas seulement à version/script.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Livrer successivement RUNNING, SUCCESS puis un RUNNING retardé pour une même tentative. Le job reste terminal. Un run avec un enfant sauté et un candidat refusé expose séparément ces faits et ne publie aucune sortie partielle non approuvée.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/aggregate/DataflowExecution.kt:137](#) Calcul de fin et état global.

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/aggregate/DataflowExecution.kt:488](#) Monotonie des statuts de jobs.

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/ExecutionUpdateStatusCommandHandler.kt:187](#) Succès, échec, descendants et messages d'étape.

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/JobCompletedCommandHandler.kt:50](#) Interprétation des résultats de session.

PD-08 **Cible non qualifiée****Logs bornés, historique des transformations et conservation finale****COMPOTEMENT OBSERVÉ**

Les messages de scripts écrivent des chunks associés à run/job/étape dans `dataflow_job_execution_log`. La normalisation gère les retours de ligne réels et échappés ; le traitement borne les lignes et peut filtrer les lignes marquées administrateur. Le poll de session extrait stdout, message et traceback. À la fin du job, le handler reconstruit les logs finaux, les sauvegarde dans les détails puis peut retirer les chunks déjà compactés. Les routes permettent détail du run, logs d'une transformation et choix `showAll/showAllLogs`. Des logs finaux tardifs peuvent encore être ajoutés alors que des transitions non finales tardives sont ignorées.

UTILISATEURS ET RÔLES

Développeur, support et opérateur.

INTERFACES DANS LE CODE

`GET /executions/{id}?showAllLogs=...` ; `GET /dataflows/{dataflowDefinitionId}/transformations/{transformationId}/logs` ; `DataflowJobExecutionLogRepository` ; `LogUtils`.

DONNÉES ET ÉTATS

Chunks PostgreSQL puis logs finaux dans les détails du job ; IDs `run/job/step` et `createdAt`.

DÉPENDANCES ET ÉCHANGES

Messages Python, résultats HTTP du runtime, repository des jobs et fonctions de filtrage.

COMPOTEMENT CIBLE PROPOSÉ

Audit/Runtimes Rust conserve des logs structurés bornés avec tenant, run, tentative, étape et offsets. Les gros volumes vont en objet immuable ; PostgreSQL garde index et reçus. Le filtrage de secrets précède diffusion et stockage des vues utilisateur.

MIGRATION DE CETTE CAPACITÉ

Préserver formats et choix de filtre requis par l'interface, documenter explicitement la troncature et migrer les historiques accessibles. Ne pas assimiler l'agrégation actuelle de lignes à une piste d'audit de publication.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Produire un log volumineux, un message final retardé et une valeur secrète de test. Le log reste consultable par pages, signale sa borne, conserve le final une seule fois et ne diffuse pas le secret.

Références de l'analyse

`monorepo · services/process-data/app/data-exec-`

`app/src/main/kotlin/com/cleyrop/data/exec/app/execution/repository/dao/DataflowJobExecutionLogDAO.kt:9` Stockage des chunks par run/job/étape.

`monorepo · services/process-data/app/data-exec-`

`domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/ExecutionUpdateStatusCommandHandler.kt:170` Finalisation puis nettoyage des chunks.

`monorepo · services/process-data/app/data-exec-`

`domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/ExecutionUpdateStatusCommandHandler.kt:217` Messages retardés et borne des logs.

`monorepo · services/process-data/app/data-exec-`

`domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/util/LogUtils.kt:5` Normalisation, filtre et borne de lignes.

`monorepo · services/process-data/app/data-exec-`

`domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/SubmittedCommandHandler.kt:63` Extraction des sorties de session.

PD-09 **Cible non qualifiée****Annulation et signal de commit Spark****COMPOTEMENT OBSERVÉ**

DELETE /executions/{id} est idempotent pour une exécution déjà finale. Pour un job unique actuellement marqué Spark committed, l'annulation est refusée. Pour plusieurs jobs avec un job committé et des travaux en attente, le service annule mais remonte une exception de commit partiel. Le marqueur est posé par un callback authentifié qui charge la dernière exécution de la version, vérifie le projet persistant et le job actif. L'annulation inscrit ExecutionCancelled, passe le run en CANCELED et émet l'événement. Une boucle retire ensuite les files et tente l'annulation distante. Ce booléen ne constitue pas une preuve d'audit ni un historique de publications approuvées.

UTILISATEURS ET RÔLES

Utilisateur qui annule, runtime appelant le callback et IAM.

INTERFACES DANS LE CODE

DELETE /executions/{id} ; POST /spark-hook/versions/{versionId}/scripts/{scriptId}/commit avec contrat pylib ; DataflowExecutionCancelCommandHandler ; SparkCommitCommandHandler.

DONNÉES ET ÉTATS

Flag isSparkCommitted sur job, run CANCELED, commande ExecutionCancelled et files supprimées ; résultat partiel possible.

DÉPENDANCES ET ÉCHANGES

IAM runner authorize, PostgreSQL, runtime HTTP et événements d'exécution.

COMPORTEMENT CIBLE PROPOSÉ

Runtimes Rust définit l'annulation par phase. Avant publication, elle arrête la tentative et abandonne le candidat. Après publication, elle n'efface pas l'édition approuvée. Publisher conserve le reçu exact ; une compensation devient une nouvelle commande audité.

MIGRATION DE CETTE CAPACITÉ

Importer isSparkCommitted comme signal historique limité. Remplacer la résolution dernière-exécution-par-version par run/tentative explicites. Garder un reçu d'annulation distante jusqu'à confirmation au lieu de déduire le succès de la suppression de file.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Annuler avant audit, entre audit et publication, puis après publication. Les deux premiers cas ne changent pas l'édition visible ; le dernier conserve l'édition et retourne son reçu. Un callback d'une vieille tentative ne peut marquer un nouveau run.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/DataflowExecutionCancelCommandHandler.kt:115](#)
Idempotence et règles committed/partial committed.

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/SparkCommitCommandHandler.kt:30](#) Autorisation, recherche et flag commit.

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/CancelExecutionCommandHandler.kt:43](#) Retrait des files et annulation distante.

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/execution/controller/SparkHookController.kt:23](#) Callback transactionnel du runner.

PD-10 **Cible non qualifiée****Planification cron, fuseaux et récupération des horaires****COMPORTEMENT OBSERVÉ**

Le service conserve les schedules et exige une version en production non supprimée pour les modifier. La commande temporelle accepte notamment MANUAL et RECURRENT ; elle refuse PLANNED et les types événementiels dans ce parcours. Les schedules temporels existants sont diffés tandis que les schedules événementiels sont préservés. Quartz conserve jobs et triggers dans PostgreSQL, demande recovery et interdit une exécution concurrente d'un même job. L'adaptateur normalise certaines expressions, applique le fuseau et peut relancer les schedules dont une prochaine occurrence existe au démarrage. Le contrôle initial de cron utilise directement CronExpression.isValidExpression, distinct du chemin de normalisation. Un listener journalise les misfires ; il ne décrit pas à lui seul une politique métier exhaustive de rattrapage.

UTILISATEURS ET RÔLES

Utilisateur planifiant un flow et opérateur du scheduler.

INTERFACES DANS LE CODE

POST/GET /dataflows/{id}/schedules ; DataflowScheduleTimeCommand ; DataflowSchedulerServiceQuartzImpl ; DataflowJob.

DONNÉES ET ÉTATS

DataflowSchedule avec fréquence, expression, timezone et date ; tables Quartz JDBC avec état de cluster.

DÉPENDANCES ET ÉCHANGES

Version de production, PostgreSQL/Quartz, contrôleur de runs et événements de schedules vers data-dev.

COMPORTEMENT CIBLE PROPOSÉ

Runtimes Rust possède un scheduler durable. Chaque occurrence est identifiée par schedule/version/instant logique, avec politique explicite de DST, retard, chevauchement, rattrapage et annulation. Le déclenchement produit une commande idempotente.

MIGRATION DE CETTE CAPACITÉ

Exporter les expressions et fuseaux puis comparer une séquence d'occurrences avant bascule. Décider le support du type PLANNED historique et des formats acceptés. Éviter la double activation Quartz/Rust pour un même schedule.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Comparer les occurrences sur changement d'heure, arrêt prolongé et redémarrage de deux contrôleurs. Les occurrences exécutées, ignorées ou rattrapées ont chacune un reçu et aucune occurrence n'est créée deux fois.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/schedule/command/DataflowScheduleCommandHandler.kt:104](#) Diff des schedules et types refusés.

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/schedule/service/quartz/DataflowSchedulerServiceQuartzImpl.kt:87](#) Création Quartz avec recovery.

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/schedule/service/quartz/DataflowSchedulerServiceQuartzImpl.kt:175](#) Validation, normalisation, fuseau et reprise.

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/schedule/service/quartz/DataflowSchedulerServiceQuartzImpl.kt:217](#) Job non concurrent transactionnel.

[monorepo · services/process-data/app/data-exec-app/src/main/resources/application.properties:292](#) Quartz JDBC clusterisé.

PD-11 Cible non qualifiée**Déclenchements par fin de flow, dataset ou fichier****COMPORTEMENT OBSERVÉ**

Le consumer d'exécution traite FINISHED en recherchant les schedules dépendants. Seules les versions de branche par défaut fournissent des candidats. Selon le type de script, les candidats identifient le flow et éventuellement un dataset ; les opérations techniques hors collecte sont exclues. Pour chaque schedule, le handler cherche la dernière version en production et déclenche si elle n'est pas supprimée, si un cluster est disponible et si sa dernière exécution n'est pas active. Un endpoint projet/fichier cherche les schedules par projet et chemin. Le déclenchement planifié temporel exige dans le chemin lu une précédente exécution de production. Le handler de fin lu ne filtre pas lui-même le succès de l'exécution source ; la sémantique FINISHED doit être prise en compte.

UTILISATEURS ET RÔLES

Producteur d'événement de fichier, runtime amont et scheduler.

INTERFACES DANS LE CODE

POST /dataflows/events/projects/{id} ; EventExecutionFinishedCommand ;
DataflowVersionRunProjectIdCommand ; TriggerAtEventEvent.

DONNÉES ET ÉTATS

DataflowSchedule avec eventTriggerId/path ; nouveau run avec triggerId, triggerDisplayName et principal SCHEDULED_OWNER.

DÉPENDANCES ET ÉCHANGES

Événements process-data et data-dev, données de projet, versions de production et clusters.

COMPORTEMENT CIBLE PROPOSÉ

Flows/Runtimes Rust distinguent événement technique de fin, publication approuvée et événement source. Les dépendances de données se basent sur une édition Iceberg approuvée. Un flux éphémère est d'abord écrit dans un journal durable ; son epoch/range déclenche des runs bornés sous WAP.

MIGRATION DE CETTE CAPACITÉ

Inventorier quels flows dépendent actuellement d'une fin même échouée et proposer une règle explicite. Donner un ID durable aux événements de fichiers et une clé de déduplication par occurrence. Ne pas perdre les triggers consommés pendant une période de désactivation.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un calcul amont qui échoue n'active pas une dépendance déclarée publication réussie. Rejouer la même édition ou le même fichier ne crée pas un doublon. Une nouvelle occurrence légitime du même chemin crée un nouveau run.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/DataflowVersionRunCommandHandler.kt:231](#)
Précondition de dernière exécution pour cron.

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/DataflowVersionRunCommandHandler.kt:288](#)
Déclenchement sur fin et projet/fichier.

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/schedule/aggregate/ScheduleCandidate.kt:14](#) Candidats selon type de script.

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/schedule/controller/ScheduleController.kt:60](#) Route de fichier projet.

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/common/event/consumer/ProcessDataKafkaEventConsumer.kt:38](#) Traitement de FINISHED.

PD-12 Cible non qualifiée**Profils de clusters, capacité et commandes de cycle de vie****COMPORTEMENT OBSERVÉ**

Les API créent, vérifient, mettent à jour, listent, arrêtent, reprennent et suppriment des clusters. Deux familles de configuration existent : Python avec CPU/RAM/image, et PySpark avec ressources du driver et des exécuteurs. La validation vérifie notamment nom disponible, image compatible, taille mémoire minimale et au moins une session. Une mise à jour ne peut changer de famille ; un changement de configuration remplace les sessions, tandis qu'un changement de nombre augmente ou réduit leur effectif. La réduction préfère les sessions supprimables puis marque les autres à supprimer. Les clusters par défaut sont protégés de certaines

commandes de pause et suppression. La pause mémorise le nombre souhaité de sessions pour la reprise. L'état du cluster est dérivé des états des sessions. Le parcours `/projects/{projectId}/execution-clusters` restitue les profils disponibles sans appliquer de filtre projet dans le handler de liste lu.

UTILISATEURS ET RÔLES

Administrateur des ressources, utilisateur choisissant un profil et moteur de planification.

INTERFACES DANS LE CODE

POST `/clusters/check` ; GET/POST `/clusters` ; GET/PATCH/DELETE `/clusters/{clusterId}` ; POST `/clusters/{clusterId}/stop` et `/start` ; GET `/clusters/{clusterId}/sessions` ; GET `/projects/{projectId}/execution-clusters` ; GET `/projects/{projectId}/execution-clusters/{clusterId}/libraries`.

DONNÉES ET ÉTATS

Cluster avec owner, configuration, default, paused, toDelete, nombre avant pause et image de serveur ; collection de Session avec état. La suppression physique du cluster attend la disparition de ses sessions.

DÉPENDANCES ET ÉCHANGES

Catalogue d'images et bibliothèques, app-management pour les ressources, événements de cluster vers data-dev et historique d'exécutions.

COMPORTEMENT CIBLE PROPOSÉ

Runtimes Rust remplace les clusters persistants par des profils de capacité et des pools de workers fongibles. Les quotas organisation/projet, admission, priorité et limites par run sont explicites. Python utilisateur reçoit un profil isolé ; aucun profil final ne dépend de Spark.

MIGRATION DE CETTE CAPACITÉ

Conserver des identifiants et noms de profil pour les flows importés, traduire les ressources utiles et signaler les options Spark sans équivalent. Distinguer arrêter l'admission, drainer les tentatives et détruire la capacité. Décider si les profils sont globaux ou limités au projet.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Réduire un pool pendant deux runs : les runs admis terminent ou sont annulés selon la politique et aucune session occupée n'est perdue implicitement. Un projet à quota épuisé reste en attente avec raison visible, puis démarre après libération. Le changement de moteur exige requalification du manifeste.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/session/controller/ClusterController.kt:80](#) Toutes les commandes de cluster et lectures de sessions.

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/session/aggregate/Cluster.kt:40](#) Remplacement, réduction, pause et reprise des sessions.

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/session/aggregate/Cluster.kt:208](#) Configurations et validations de ressources.

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/session/command/UpdateClusterCommandHandler.kt:36](#) Famille du cluster immuable.

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/session/controller/ExecutionClusterController.kt:27](#) Route de sélection de profil dans un projet.

Réconciliation des sessions et des serveurs Spark Connect

COMPOTEMENT OBSERVÉ

L'orchestrateur compare les sessions persistées, les files d'exécution et les états remontés par app-management. Pour PySpark, un serveur Spark Connect est assuré tant que le cluster conserve des sessions. Une nouvelle image provoque le remplacement du serveur. La réconciliation récupère un inventaire des sessions et serveurs, actualise les images effectivement déployées et oublie un serveur absent pour le recréer. Les sessions mortes ou revenues à STARTING/NOT_STARTED font échouer leurs jobs, retirent leurs soumissions et peuvent conduire à une annulation distante. Le log de ce chemin emploie un message out-of-memory même si la cause exacte n'est pas établie ici. L'adaptateur suit aussi localement le dernier retour des sessions et applique des délais distincts de démarrage et de silence. Ces caches de temps et l'état désiré PostgreSQL n'ont pas le même périmètre de reprise.

UTILISATEURS ET RÔLES

Orchestrateur process-data et opérateur d'exploitation.

INTERFACES DANS LE CODE

SessionCheckCommandHandler ; SessionGroupingResultBuilder ; REST app-management
/private/sessions/reconcile, /private/sessions/start, /private/sessions/delete, /private/sessions/server/start et /private/sessions/server/delete.

DONNÉES ET ÉTATS

Session avec sessionKey, startedAt, updatedAt, deployedImage et flags de suppression ; Cluster.serverImage ; files des jobs ; caches locaux de dernier retour dans l'adaptateur.

DÉPENDANCES ET ÉCHANGES

App-management et runtime Python/PySpark ; catalogue d'images ; journal d'exécutions et événements de sessions.

COMPOTEMENT CIBLE PROPOSÉ

Runtimes Rust réconcilie tentative, bail et worker observé. Un worker disparu devient une tentative perdue, avec cause structurée et nouvelle tentative décidée par politique. L'ancien worker ne peut plus publier : Writer/Publisher vérifient l'autorité de tentative avant toute transition approuvée.

MIGRATION DE CETTE CAPACITÉ

Importer les diagnostics et règles utiles de nettoyage ; supprimer entièrement Spark Connect à la fin de migration. Remplacer les caches de silence significatifs par des heartbeats et échéances persistés. Prévoir un inventaire Kubernetes encapsulé et une réconciliation sans dépendre d'un nom d'image mutable.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Perdre un pod, un serveur et une réponse d'inventaire à des moments différents. La disparition est distinguée d'une indisponibilité de l'API. Une reprise ne crée pas deux publications ; les octets d'une tentative expirée restent candidats non approuvés et sont traités par rétention.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/SessionCheckCommandHandler.kt:107](#) Cycle de serveur Spark Connect selon sessions et image.

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/SessionCheckCommandHandler.kt:234](#) Échec des jobs attachés aux sessions mortes.

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/SessionCheckCommandHandler.kt:286](#) Réconciliation de l'inventaire et serveurs absents.

[monorepo](#) · [services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/session/service/appmanagement/ClusterConnectServerServiceAppManagementImpl.kt:85](#) Polling, délais et inventaire distant.

PD-14 Cible non qualifiée

Images d'exécution, builds et historique des mises à jour

COMPOTEMENT OBSERVÉ

Une image personnalisée est créée à partir d'une image de base avec des bibliothèques validées. L'appel de build est délégué à app-management ; l'état et l'historique de mise à jour sont conservés localement. Une image de base ne suit pas le parcours de modification/suppression des images personnalisées. Une mise à jour refuse une construction encore RUNNING et refuse une liste inchangée. Le rebuild repart du dernier besoin exprimé, y compris après un échec, et peut retargeter le registre interne configuré. Les callbacks de build ne modifient l'image que si le dernier update est RUNNING et correspond exactement au nom/tag attendu. Au démarrage, les images de base sont synchronisées ; les changements de tags peuvent redémarrer les sessions. Les images personnalisées construites avant une base actualisée peuvent être reconstruites si aucun build ne tourne déjà. La suppression d'image déclenche aussi la suppression des clusters concernés.

UTILISATEURS ET RÔLES

Administrateur et auteur d'un environnement Python personnalisé.

INTERFACES DANS LE CODE

POST/GET /images ; PATCH/DELETE/GET /images/{imageld} ; POST /images/{imageld}/builds ; GET /images/{imageld}/updates ; ImageServiceAppManagement ; UpdateImageStatusCommandHandler.

DONNÉES ET ÉTATS

Image, ImageUpdate et historique incrémental de bibliothèques ; RUNNING puis SUCCESS/FAILURE ; nom de base, nom d'image construite et dates de build.

DÉPENDANCES ET ÉCHANGES

App-management /private/build/image, registre de conteneurs, callbacks Kafka de build, images de base configurées et clusters.

COMPOTEMENT CIBLE PROPOSÉ

Runtimes/Applications Rust conservent une spécification d'environnement immuable et un digest de résultat, avec provenance, SBOM, politique de sécurité et reçu de build. Les builders tiers peuvent être Go/C++ ou outils sans JVM encapsulés. Le run référence le digest qualifié.

MIGRATION DE CETTE CAPACITÉ

Exporter historique, base et besoins de packages, puis construire des environnements sans Spark/JVM. Ne pas assimiler nom/tag à identité immuable. Traiter les échecs et callbacks retardés comme des tentatives de build distinctes. Conserver les images legacy uniquement pendant la transition contrôlée.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un callback ancien reçu pendant une reconstruction n'écrase pas le nouveau résultat. Une base changée crée un nouveau digest et nécessite la requalification attendue ; les runs déjà admis conservent leur digest d'origine. La suppression d'une image encore retenue par l'audit est refusée ou différée.

Références de l'analyse

[monorepo](#) · [services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/image/controller/ImageController.kt:58](#) Contrats de création, mise à jour, rebuild et lecture.

[monorepo](#) · [services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/image/command/UpdateImageCommandHandler.kt:42](#) Restrictions et diff des besoins.

[monorepo](#) · [services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/image/command/RebuildImageCommandHandler.kt:44](#) Reconstruction après échec et reprise des besoins.

[monorepo](#) · [services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/image/command/UpdateImageStatusCommandHandler.kt:130](#) Protection contre callback obsolète.

[monorepo](#) · [services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/image/command/RebuildOutdatedCustomImagesCommandHandler.kt:49](#) Reconstruction des images dépendantes devenues anciennes.

PD-15 **Cible non qualifiée**

Validation et consultation des bibliothèques Python

COMPOTEMENT OBSERVÉ

La validation compare les bibliothèques demandées avec celles de l'image de base. Une bibliothèque de base imposée ne peut être redéfinie avec une version contradictoire. Les bibliothèques personnalisées sont résolues puis enregistrées avec leur contrainte demandée. Les erreurs sont agrégées pour retourner plusieurs problèmes à la fois ; une demande sans bibliothèque personnalisée est refusée dans ce service de vérification. L'adaptateur consulte les métadonnées PyPI ou un index particulier pour la bibliothèque interne. Il normalise les noms/extras et vérifie les versions et extras quand les métadonnées permettent de conclure. Une absence de métadonnées extras n'équivaut pas à une validation complète. Les lectures distinguent bibliothèques appliquées à l'image et besoins du dernier update.

UTILISATEURS ET RÔLES

Auteur d'environnement et administrateur du registre.

INTERFACES DANS LE CODE

GET /images/{imageld}/libraries?lastUpdate=... ; GET /projects/{projectId}/execution-clusters/{clusterId}/libraries ; LibraryService.checkLibraries ; LibraryServicePyPiImpl.

DONNÉES ET ÉTATS

Library et relations base/custom/update avec nom, contrainte et version résolue ; historique des demandes d'environnement.

DÉPENDANCES ET ÉCHANGES

Index Python public/interne, image de base, service de build et réseau sortant du service.

COMPOTEMENT CIBLE PROPOSÉ

Runtimes Rust définit le lock d'environnement, tandis qu'un builder Python encapsulé résout les dépendances. Le lock fixe versions transitives, hashes, provenance et plateforme. Egress gère l'accès aux index ; Audit conserve le besoin demandé et le résultat exact.

MIGRATION DE CETTE CAPACITÉ

Rejouer les demandes historiques contre un index contrôlé, détecter les packages disparus, natifs ou dépendants de Java, puis qualifier les locks. Préserver les contraintes initiales comme information sans les utiliser seules pour reproduire un run.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une contrainte inchangée produit le même environnement quand le lock existe, même si l'index publie une nouvelle version. Un package lié à la JVM ou un extra inconnu est bloqué avant activation selon la règle de qualification ; une indisponibilité d'index reste une erreur distincte d'un package inexistant.

Références de l'analyse

[monorepo](#) · [services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/image/service/LibraryService.kt:20](#) Contrôle des packages de base, custom et agrégation des erreurs.

[monorepo](#) · [services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/image/service/LibraryServicePyPiImpl.kt:28](#) Résolution des métadonnées et index.

[monorepo](#) · [services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/image/service/LibraryServicePyPiImpl.kt:89](#) Normalisation et traitement des extras.

[monorepo](#) · [services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/image/controller/ImageController.kt:134](#) Lecture appliquée ou dernier update.

PD-16 **Cible non qualifiée****Historique des exécutions, filtres, facettes et consultation****COMPOTEMENT OBSERVÉ**

POST /executions exige le projet, convertit les identifiants IAM/legacy et retourne une recherche paginée d'exécutions visibles. Les critères entre dimensions sont combinés par AND et les valeurs d'une dimension par OR. Les facettes ignorent leur propre sélection pour présenter les alternatives. La recherche peut viser noms d'exécution/flow, branches, clusters, utilisateurs, statuts et bornes temporelles. Le repository filtre les versions techniques selon toShow et utilise la date de création pour les bornes. Les tris autorisés sont contrôlés dans le controller. Les autres routes servent l'historique d'une branche, le détail d'un run, les logs d'une transformation, les exécutions d'un cluster et les exécutions RUNNING. Le paramètre showAllLogs influe sur la restitution des logs, pas sur leur autorisation.

UTILISATEURS ET RÔLES

Développeur, opérateur et frontends.

INTERFACES DANS LE CODE

POST /executions ; GET /executions/{executionId} ; GET /dataflows/{dataflowId}/branches/{branchName}/executions ; GET /dataflows/{dataflowDefinitionId}/transformations/{transformationId}/logs ; GET /clusters/{clusterId}/executions ; GET /clusters/executions/running.

DONNÉES ET ÉTATS

DataflowExecution, JobExecution, versions toShow, utilisateurs/projets miroirs et logs stockés ; DTO détail et réponses paginées/facettes.

DÉPENDANCES ET ÉCHANGES

PostgreSQL, canonicalisation IAM, agrégats d'exécution et authorization gateway.

COMPOTEMENT CIBLE PROPOSÉ

Runtimes/Audit Rust exposent une vue indexée des runs, tentatives et transitions, liée aux éditions approuvées sans confondre statut technique et publication. Les totaux, facettes et logs appliquent la même portée d'autorisation que les lignes.

MIGRATION DE CETTE CAPACITÉ

Reproduire un corpus de requêtes UI et comparer tri, pagination, filtres vides, fenêtres temporelles et versions cachées. Ajouter les IDs de tentative et de publication de manière additive avant de retirer les DTO Spark/session.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Comparer historique et facettes sur des runs de plusieurs projets, branches, statuts et utilisateurs. Les données invisibles ne changent ni les options ni les compteurs. Un run techniquement terminé sans publication reste distinguable d'une édition approuvée.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/execution/controller/DataflowPipelineController.kt:90](#) Recherche, facettes, tri et canonicalisation.

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/execution/controller/DataflowPipelineController.kt:147](#) Historique, détail et logs.

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/execution/repository/DataflowExecutionFilteredQuery.kt:35](#) Prédicats, dates, recherche et toShow.

[monorepo · services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/query/ExecutionFilter.kt:6](#) Dimensions du filtre.

PD-17 **Cible non qualifiée****Suivi temps réel SSE et notifications de clusters****COMPOTEMENT OBSERVÉ**

Les routes exposent un abonnement SSE par utilisateur, version de flow et éventuellement exécution, ainsi que des routes pour un cluster ou toutes les exécutions en cours. Les emitters sont conservés dans une map du processus et réutilisés pour une même clé. Ils expirent et sont supprimés sur erreur, fin ou timeout. Le chemin d'émission lu envoie directement aux clés de flow compatibles ; les données de cluster et de toutes les exécutions sont relayées vers Notification Center par Kafka. Les clés SSE de cluster peuvent être enregistrées, mais ce chemin n'émet pas directement vers elles. L'état SQL et les endpoints de détail restent nécessaires pour rétablir l'affichage après reconnexion ; aucun replay durable par Last-Event-ID n'a été constaté dans ce service.

UTILISATEURS ET RÔLES

Frontend de développement et console d'exploitation.

INTERFACES DANS LE CODE

GET /dataflows/{dataflowId}/versions/{versionId}/sse ; GET /clusters/{clusterId}/executions/sse ; GET /clusters/executions/running/sse ; SseEmitterService ; NotificationCenterRelayKafkaImpl.

DONNÉES ET ÉTATS

Emitters locaux indexés par SseKey ; DTO d'état construit au fil des événements ; relay Kafka vers rooms de cluster.

DÉPENDANCES ET ÉCHANGES

Runtimes d'exécution, serveur HTTP/SSE, Notification Center et Kafka.

COMPOTEMENT CIBLE PROPOSÉ

Notifications Rust fournit un flux de mises à jour autorisé avec curseur durable et reprise. Les messages signalent un changement identifié ; les détails sont relus avec contrôle de portée. Les clients gèrent doublons, réordonnancement et perte de connexion.

MIGRATION DE CETTE CAPACITÉ

Confirmer les routes effectivement utilisées par les interfaces et décider le retrait des anciens abonnements cluster SSE. Introduire une séquence de changement par run et une resynchronisation explicite. Retirer Kafka JVM lorsque le nouveau journal/relay sans JVM est qualifié.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Déconnecter le client pendant plusieurs transitions puis reconnecter sur un autre pod. Il récupère toutes les transitions utiles ou un signal explicite de resynchronisation, sans accéder à un run dont le droit a été retiré. Deux onglets d'un même utilisateur ne se neutralisent pas.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/execution/controller/DataflowPipelineController.kt:248](#) Abonnement par flow/version et exécution optionnelle.

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/session/controller/ClusterController.kt:185](#) Routes SSE de clusters et exécutions actives.

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/execution/service/SseEmitterService.kt:39](#) Map locale et cycle de vie des emitters.

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/execution/service/SseEmitterService.kt:94](#) Émission flow directe et relais cluster vers Notification Center.

PD-18 **Cible non qualifiée****Autorisation des runners et configuration lakehouse****COMPORTEMENT OBSERVÉ**

Deux opérations techniques sont contrôlées par RunnerAuthorizationClient : la lecture de configuration et le signal de commit. Le client exige une identité BEARER_JWT et un clientId ; il appelle IAM avec les credentials entrants, l'utilisateur, le client, l'opération et éventuellement le projet. Une réponse négative donne un refus ; un IAM indisponible donne une erreur de service. Le client n'a pas de cache ou d'autorisation de secours dans le code lu. /lakehouse/conf retourne les noms de catalogue et namespace après autorisation et exige le User-Agent pylib. Le hook commit reprend le projet du run trouvé puis vérifie l'opération dédiée. Le User-Agent est un critère de route, pas une preuve d'identité autonome.

UTILISATEURS ET RÔLES

Runner Python/PySpark et IAM.

INTERFACES DANS LE CODE

GET /lakehouse/conf ; POST /spark-hook/versions/{versionId}/scripts/{scriptId}/commit ; RunnerAuthorizationClient.requireConfiguration ; RunnerAuthorizationClient.requireCommit ; IAM /internal/runners/authorize.

DONNÉES ET ÉTATS

Décision centrale à la demande ; contexte d'identité entrante ; signal isSparkCommitted sur le job. Aucune preuve de publication Iceberg n'est créée par la réponse de configuration.

DÉPENDANCES ET ÉCHANGES

IAM runner authorization, bearer entrant et paramètres de catalogue configurés.

COMPOTEMENT CIBLE PROPOSÉ

IAM/Runtimes Rust délivrent des capacités de run limitées au projet, à la tentative, aux entrées, sorties et à l'échéance. Query contrôle la lecture ; Writer médie le staging ; Publisher seul approuve la publication. Le worker n'obtient jamais table.modify générique qui permet aussi commit/drop.

MIGRATION DE CETTE CAPACITÉ

Remplacer le hook Spark par un protocole explicite de proposition de candidat et de publication. Propager runId/tentativeId et portée réelle jusque dans les appels de données. Conserver une façade de configuration temporaire avec les mêmes refus explicites.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une identité de service sans délégation de run, un mauvais projet, un client non prévu ou une capacité expirée ne peuvent ni écrire ni publier. Une panne IAM échoue sans fallback permissif. Le worker ne peut pas utiliser son accès staging pour supprimer la table.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/configuration/RunnerAuthorizationClient.kt:26](#) Opérations et appel central, refus et indisponibilité.

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/configuration/RunnerAuthorizationClient.kt:53](#) Exigence bearer JWT et clientId.

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/execution/controller/ConfigurationForLibController.kt:19](#) Configuration limitée après autorisation.

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/SparkCommitCommandHandler.kt:30](#) Projet persisté utilisé pour la décision commit.

PD-19 **Cible non qualifiée****Miroirs IAM, identifiants canoniques et réconciliation des droits****COMPOTEMENT OBSERVÉ**

Le service peut consommer l'identité legacy ou IAM selon une propriété exclusive. Les événements IAM créent/actualisent les utilisateurs et projets, lient leurs identifiants et alimentent les adhésions. Une suppression utilisateur désactive le miroir. Une suppression projet laisse le miroir en place car versions et exécutions le référencent. Le transfert de propriétaire arrive via les événements de data-dev, pas directement via l'offboarding IAM. Une adhésion dont les correspondances sont indisponibles est refusée et suit retry/DLT. Les API résolvent les projets IAM vers les IDs historiques pour interroger les tables, puis réexposent l'ID canonique si connu. Une route d'administration lance une réconciliation asynchrone des faits d'autorisation.

UTILISATEURS ET RÔLES

IAM, gateway et administrateur de migration.

INTERFACES DANS LE CODE

IamKafkaEventConsumer ; ProjectIdResolver ; POST /admin/reconcile-tuples.

DONNÉES ET ÉTATS

User/Project/ProjectMember miroirs, tables de correspondance IAM/legacy et faits d'autorisation émis.

DÉPENDANCES ET ÉCHANGES

IAM, événements data-dev de propriétaire, PostgreSQL et Kafka.

COMPOTEMENT CIBLE PROPOSÉ

IAM Rust reste l'autorité ; Runtimes garde uniquement les références canoniques et l'attribution historique. Les contrôles de run utilisent la décision courante et une capacité bornée.

MIGRATION DE CETTE CAPACITÉ

Importer les correspondances, réparer les événements refusés puis basculer une seule source d'identité. Préserver les auteurs historiques quand une identité est supprimée.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un membre reçu avant son projet est rejoué après réconciliation sans création de faux projet. Une suppression utilisateur interdit les nouveaux runs et conserve les historiques attribués.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/common/event/consumer/IamKafkaEventConsumer.kt:56](#) Miroirs, suppression et membres.

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/common/projectid/ProjectIdResolver.kt:58](#) Conversion des identifiants entrants et sortants.

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/admin/controller/AdminReconcileTuplesController.kt:49](#) Réconciliation administrative asynchrone.

PD-20 **Cible non qualifiée**

Outbox, déduplication des événements et quarantaine**COMPOTEMENT OBSERVÉ**

Les producteurs utilisent une outbox transactionnelle. Le relay réclame les lignes non envoyées avec SKIP LOCKED, attend l'acquittement Kafka puis marque sentAt ; un échec conserve la ligne, incrémente attempts et stocke l'erreur. Le tri par date ne constitue pas la barrière par clé présente dans data-dev. Des consumers emploient ProcessEventInbox dans la transaction : la clé est le messageId ou une empreinte de payload, unique par type d'événement. La couverture n'est pas universelle à tous les listeners. Les dead letters sont conservées avec provenance, payload et erreur ; un échec de stockage est relancé pour empêcher l'acquittement. La rétention supprime seulement les dead letters résolues et les lignes outbox envoyées selon des délais distincts.

UTILISATEURS ET RÔLES

Consumers interservices et exploitation.

INTERFACES DANS LE CODE

OutboxRelay.drain ; ProcessEventInbox.claim ; DeadLetterEventConsumer.consume ; DeadLetterRetention.cleanupResolved.

DONNÉES ET ÉTATS

Outbox SQL, inbox SQL, dead_letter_event et offsets Kafka ; livraison au moins une fois et déduplication locale pour les handlers équipés.

DÉPENDANCES ET ÉCHANGES

PostgreSQL, Kafka, événements data-dev/process-data/app-management/IAM et métriques.

COMPOTEMENT CIBLE PROPOSÉ

Runtimes/Audit Rust utilisent un journal durable sans JVM, une inbox et des séquences par agrégat. Les effets externes ont leur reçu idempotent. L'acquittement d'une progression de données suit la publication approuvée.

MIGRATION DE CETTE CAPACITÉ

Cartographier chaque listener et sa clé d'idempotence, importer les éléments non acquittés, puis arrêter l'ancien producteur avant de transférer une partition logique. Définir explicitement l'ordre par run.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Couper après acquittement transport mais avant commit SQL du relay : le doublon est accepté sans double effet. Rejouer une dead letter après réparation conserve son lien avec l'événement initial et son résultat.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/common/outbox/OutboxRelay.kt:25](#) Envoi acquitté, tentatives et rétention.

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/common/outbox/OutboxEventJPA.kt:18](#) Réclamation ordonnée avec SKIP LOCKED.

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/common/event/consumer/ProcessEventInbox.kt:14](#) Déduplication dans la transaction.

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/common/deadletter/DeadLetterEventConsumer.kt:37](#) Quarantaine persistante et refus d'ack si échec.

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/common/deadletter/DeadLetterRetention.kt:22](#) Nettoyage limité aux éléments résolus.

PD-21 **Cible non qualifiée****Démarrage, reprise des files et coexistence des réplicas****COMPOTEMENT OBSERVÉ**

Au démarrage de production, le service cherche un autre pair vivant dans les tables Quartz. Si un pair existe, il évite les opérations de cold start qui annuleraient les runs et réinitialiseraient clusters/schedules. Sinon, il annule les runs orphelins tout en conservant ceux dont une file durable prouve une progression reprenable, initialise les clusters si activés et relance les schedules selon les flags. La disponibilité app-management est attendue avec un nombre borné de tentatives. Les images et bibliothèques sont synchronisées même en présence d'un pair. La détection de pair et les protections SQL observées ne qualifient pas à elles seules toute la boucle d'orchestration pour plusieurs réplicas.

UTILISATEURS ET RÔLES

Opérateur du déploiement et orchestrateur.

INTERFACES DANS LE CODE

`RunAfterStartup.resetSessions` ; `StartCluster.startClusters` ; `ClusterPeerCheck.hasOtherLivePeer` ; `DataflowExecutionCancelRunningCommand`.

DONNÉES ET ÉTATS

Files persistées, exécutions actives, état Quartz/heartbeat et images ; état local de la boucle d'orchestration.

DÉPENDANCES ET ÉCHANGES

PostgreSQL, Quartz, app-management, Kafka et readiness des runtimes.

COMPOTEMENT CIBLE PROPOSÉ

Runtimes Rust utilise des baux persistants et une réconciliation idempotente par run/tentative. Un rolling update ne change pas la propriété d'une publication ni l'identité de sa tentative. Les workers ne possèdent aucun état métier durable indispensable.

MIGRATION DE CETTE CAPACITÉ

Qualifier crash et rolling update pour chaque étape de file avant de migrer les runs actifs. Importer les intentions restantes ou laisser finir les runs legacy ; empêcher l'activation simultanée de deux orchestrateurs sur un même run.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Redémarrer tous les contrôleurs puis effectuer un rolling update avec des jobs en attente, soumis et terminés mais non finalisés. Chaque run retrouve une issue expliquée et au plus une publication approuvée.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/RunAfterStartup.kt:94](#) Différence cold start/rolling update et flags.

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/RunAfterStartup.kt:54](#) Attente bornée d'app-management.

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/common/cluster/ClusterPeerCheck.kt:24](#) Détection de pair Quartz.

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/DataflowExecutionCancelCommandHandler.kt:179](#) Conservation des runs possédant une progression durable.

PD-22 **Cible non qualifiée****Maintenance SQL, migrations et exploitation du service****COMPOTEMENT OBSERVÉ**

Le service comporte les migrations Flyway et un runner de migrations de code. Ce runner marque le résultat dans la table de suivi ; le chemin d'échec demande une sortie Spring avec code 0, ce qui nécessite une vérification explicite côté déploiement. La maintenance planifie VACUUM ANALYZE des tables d'exécution/jobs et collecte des statistiques de taille PostgreSQL. Le chart configure utilisateur non-root, probes et ressources ; le stockage persistant optionnel impose des contraintes de réplica et stratégie de mise à jour documentées. Les caches, outbox, quarantaine, tables Quartz et artefacts S3 ne se sauvegardent pas par un seul mécanisme.

UTILISATEURS ET RÔLES

Exploitation, équipe de migration et administrateur PostgreSQL.

INTERFACES DANS LE CODE

DatabaseMaintenanceService ; CleyropMigrationRunner ; Flyway ; probes et configuration Helm.

DONNÉES ET ÉTATS

Schémas SQL, table de migrations, statistiques, logs et volumes optionnels.

DÉPENDANCES ET ÉCHANGES

PostgreSQL, Kubernetes/Helm, S3, métriques et plateforme de sauvegarde.

COMPOTEMENT CIBLE PROPOSÉ

Migration/Maintenance Rust gèrent les opérations suivies, idempotentes et à échec explicite. Les migrations suivent Expand/Migrate/Contract. Audit établit les plans de rétention des candidats et preuves ; compaction de données sous WAP et GC séparé par garde de rétention.

MIGRATION DE CETTE CAPACITÉ

Conserver une sauvegarde restaurable des commandes, historiques et reçus avant conversion. Remplacer le runner de migration JVM et ses codes de sortie ambigus. Documenter la restauration coordonnée avec les éditions Iceberg et artefacts immuables.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une migration échouée bloque l'activation et expose son erreur. Restaurer la base de contrôle avec le catalogue plus récent reconstruit les projections sans faire reculer une édition approuvée ni republier un effet déjà reçu.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/maintenance/DatabaseMaintenanceService.kt:39](#) Maintenance SQL et collecte de statistiques.

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/codemigration/runner/CleyropMigrationRunner.kt:30](#) Suivi et sortie sur échec de migration.

[monorepo · services/process-data/chart/templates/deployment.yaml:50](#) Exécution non-root et probes du chart.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- API Rust de commandes de run, admission, tentative, annulation, planification et progression, avec idempotence, révision attendue et journal durable.
- Flows fournit une révision immuable et un manifeste typé. Runtimes épingle entrées approuvées, environnement, paramètres, portée IAM, budget et échéance avant admission.
- PostgreSQL reste l'autorité des commandes, identités et reçus d'effets ; les checkpoints publiés se lisent dans les éditions Iceberg approuvées.
- Publisher coordonne les sorties et crée une release multi-sorties elle-même édition Iceberg, comprenant source/epoch/range/participants ; les lecteurs épinglent cette release.
- Le statut du run distingue calcul, staging, audit, publication et effets. EXPORT est un effet ; EXPOSURE est un consommateur déclaré.

Plan de données proposé

- Workers fongibles pour runs bornés, moteurs Rust/Arrow/SQL et Python isolé. Suppression finale des sessions Spark, Spark Connect, Quartz JVM, Kafka JVM et de toute autre dépendance Java.
- Le worker soumet les octets au Writer de confiance. Il ne détient pas table.modify générique ; Publisher contrôle la publication après audit de fichiers immuables.
- Tous les kinds TABLE, INCREMENTAL, VIEW, FILESET, MODEL, EMBEDDINGS, FEATURESET et STREAM utilisent WAP. Un flux éphémère est journalisé durablement avant gouvernance et traité par intervalles bornés.
- Le candidat porte origin.* ; la preuve post-candidat est liée durablement sans réécriture du summary par fast-forward. UUID/refs/schema/spec/ascendance ne remplacent pas un protocole de fencing Hemera.
- Publish-then-ack lie checkpoint publié et édition approuvée. Compaction suit le même circuit WAP ; expiration/GC utilisent un plan audité et les gardes de rétention.

Découpage du core Rust

- runtimes-api : DTO compatibles, admission, autorisations et résolution des identités canoniques.

- run-controller : machine d'états, files persistées, baux de tentative, réconciliation, annulation et receipts.
- scheduler : occurrences durables, cron/fuseaux, événements, déduplication et politique de rattrapage.
- environment-registry : locks Python, digests de conteneur, provenance et builders encapsulés.
- worker-gateway : lancement, état, logs structurés et capacités bornées ; aucune API de publication générique vers le worker.
- audit-projection : historique des runs/tentatives, lien aux publications approuvées, métriques et flux de notification reprenables.

Identité, autorisation et isolation

- Conserver la distinction initiateur manuel, owner interne/planifié et projet réel des opérations fabriquées. Ne pas dériver des droits du seul owner courant ou du dataflowId synthétique.
- Les dates de capacité persistées doivent survivre à la reprise. Une nouvelle tentative ne prolonge pas implicitement l'autorisation du run.
- Le code utilisateur, packages, logs, URL signées et fichiers sont des surfaces tenant ; limiter octets, durée, réseau, secrets et scopes, avec redaction dans les diagnostics.
- La suppression de projet/utilisateur conserve l'attribution historique sans conserver un droit d'accès. Les projections et caches ne sont pas une source d'autorisation autonome.

Exploitation et reprise

- La lecture établit les mécanismes présents, pas leur qualification en multi-réplicas. Quartz clusterisé ne suffit pas à prouver la sûreté de la boucle Orchestrator de chaque pod.
- Surveiller âge des files, durée d'admission, tentatives perdues, dead letters, relay attempts, silences runtime, dérive d'image, retard de schedule et différences entre fin technique et publication.
- Sauvegarder PostgreSQL, artefacts et catalogue avec une procédure de restauration qui réconcilie les éditions déjà publiées ; ne pas traiter la base de contrôle comme checkpoint data autoritaire.
- Qualifier redémarrage complet, rolling update, perte réseau après submit, arrêt pendant commit et callback retardé. Aucun de ces parcours n'a été exécuté dans cet audit.

Séquence de migration

- Geler le contrat API/événements et un corpus représentatif : Python, SQL, PySpark, partial, preview, collect, delete, empty et update-column.
- Importer IDs canoniques, versions, manifestes, profils, schedules et historique ; isoler les versions anciennes sans manifeste complet.
- Construire scheduler et controller Rust en observation passive puis comparer leurs décisions sans lancer de travail concurrent.
- Qualifier l'exécution sans JVM par opérateur, package et type de flow ; rendre les incompatibilités explicites avant activation.
- Déployer Writer/Publisher, audit immuable, checkpoints d'édition et receipts ; qualifier les scénarios de panne et de multi-sorties.

- Basculer par projet/profil après drainage ou reprise explicite des runs legacy ; attribuer chaque occurrence et run à un seul controller.
- Réconcilier queues/outbox/DLT puis retirer process-data JVM, Spark Connect, sessions Spark, Quartz et transport Kafka JVM. Conserver les lectures d'historique nécessaires.

Écarts et décisions encore ouverts

- Le contrat exact de succès attendu par les triggers FINISHED doit être décidé ; le handler lu ne filtre pas le succès source.
- Le parcours schedule temporel requiert une exécution antérieure de production : confirmer la création initiale attendue côté produit.
- Le signal isSparkCommitted pointe le dernier run d'une version et un script ; préciser les garanties de corrélation actuelles avant de le remplacer.
- Le submit distant peut réussir avec réponse perdue. La clé d'idempotence réellement garantie par app-management doit être qualifiée.
- La sûreté multi-répliques de la boucle de files et les limites des verrous observés exigent des tests de panne ; aucune garantie globale exactly-once n'est déduite.
- Les anciennes routes SSE cluster enregistrent des clés qui ne sont pas ciblées par le chemin d'émission direct lu ; vérifier les clients encore concernés.
- Quelles fonctions PySpark, packages natifs et ressources personnalisées n'ont pas d'équivalent sans JVM ? L'inventaire des définitions clientes n'a pas été consulté.
- Les paramètres réellement déployés, volumes, budgets et versions runtime restent inconnus ; le dossier décrit le code du commit de référence.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/aggregate/DataflowExecution.kt:55](#) État, identité et lifecycle d'un run.

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/aggregate/DataflowVersion.kt:35](#) Version et manifestes de scripts.

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/execution/service/Orchestrator.kt:45](#) Boucle opérationnelle.

[monorepo · services/process-data/app/gradle/libs.versions.toml:16](#) Versions Kotlin et Spring déclarées.

[monorepo · services/process-data/app/data-exec-app/build.gradle.kts:25](#) Cible JVM du module applicatif.

SERVICE ACTUEL · DATA-SERVE

Consultation, SQL, exposition et qualité des données

18 capacités analysées

services/data-serve · Kotlin 2.4.10, Spring Boot 4.1.1 et JVM ; PostgreSQL/JPA/Flyway ; Kafka/Protobuf ; Trino JDBC et JSqlParser ; MinIO/S3 pour les previews ; OAuth et client IAM.

Service de lecture pour l'interface, les clients OAuth et le MCP dataset. Il traduit les métadonnées métier en lectures Trino sur Iceberg, expose aperçus, lignes filtrées, SQL et téléchargements, et possède les définitions et calculs des règles qualité. Son nom historique apitoken ne signifie plus une gestion locale de clés API.

Lecture de code

Pas de qualification runtime nouvelle

Domaines cible : Query, Catalog, Datasets, Audit, Egress, IAM, Flows, Applications, MCP, Maintenance, Migration.

Périmètre de lecture

- ServeController, ExpositionApiController et DataQualityController, leurs DTO et handlers domaine.
- QueryAnalyser, SQLQueryHandler, SqlFragmentGuard, TrinoJDBC, repositories de lecture et LosslessNdjson.
- Règles, historiques, comparateurs, templates, computes et événements de qualité.
- Authentification, résolution MCP, client IAM, projections Kafka, migration de schéma, outbox/dead letters, configuration Trino et charts.

Objets et cycle de vie

- Dataset et DatasetBranch sont des projections de data-model, avec types de colonnes, labels, état, sensibilité, qualité et owner. Elles ne constituent pas actuellement un registre d'éditions Iceberg.
- Query, Statement et ExecutionContext portent le SQL et les datasets détectés. DataFrame, Table, TabularSchema et Pagination décrivent la réponse ; aucune session durable de pagination n'est présente dans les contrats lus.
- DatasetContent possède un Stream de lignes et le nom/MIME de fichier. Les frames NDJSON conservent schéma, valeurs positionnelles, compteur et digest, sans constituer une publication de dataset.
- Rule est l'identité d'une règle avec suppression logique ; RuleHistory versionne le script et les seuils. Compute est lié à une exécution de dataset et ComputedRule porte une valeur typée et son résultat.
- PreviewScope porte le projet et les datasets/branches d'entrée d'une version. Le fichier preview reste dans le stockage objet et reçoit les droits de ce scope.
- Les tables utilisateur, labels et liens de projets sont des miroirs. Outbox et dead letters gardent les événements et leur état de traitement.

Dépendances du service

- data-model alimente datasets, branches, schémas, labels et liens sur Kafka/Protobuf. data-serve publie règles et computes en retour.
- process-data publie les événements de fin d'exécution qui déclenchent les contrôles qualité sur main.

- Trino JDBC lit les lignes et schémas Iceberg, résout les refs, exécute les SQL utilisateurs et règles, et fournit les types des exports. Retirer seulement Spring ne retire donc pas la JVM du parcours.
- MinIO/S3 fournit les fichiers preview. data-dev expose leur scope de projet et d'entrées via HTTP interne.
- IAM HTTP arbitre CONSULT/DOWNLOAD/PROJECT et valide l'acteur MCP ; Keycloak fournit actuellement les JWT et tokens techniques. La bibliothèque partagée conserve des résolutions d'identité en cache.
- PostgreSQL conserve projections, règles, histories, computes, outbox, dead letters et état de migration. Notification-center reçoit les abonnements qualité.

Fonctionnalités et compatibilité à préserver

DS-01 Cible non qualifiée

Construire la projection de serving depuis les faits du catalogue

COMPORTEMENT OBSERVÉ

Le consommateur data-model crée ou remplace la projection locale du dataset et de ses branches. UPDATED est un upsert utilisable même sans CREATE antérieur ; il exige un snapshot after dont l'ID correspond à l'agrégat. DELETE supprime une projection existante et tolère son absence. Les liens de projet ne sont conservés que pour les états ACCEPTED ou OWNED ; demandes et refus suppriment un ancien lien pendant le rejeu. Les labels sont répliqués séparément. Les utilisateurs ont un miroir local alimenté par un parcours IAM et un listener historique encore présent sans condition sur LegacyUserCreatedConsumer.

UTILISATEURS ET RÔLES

Consommateurs Kafka et toutes les lectures de data-serve.

INTERFACES DANS LE CODE

DataModelEventKafkaConsumer.consumeCreateDataset ; consumeUserCreated pour les liens ; DatasetLabelEventKafkaConsumer ; LegacyUserCreatedConsumer.

DONNÉES ET ÉTATS

Dataset, DatasetBranch, schémas, liens projet, labels et user_table_serve PostgreSQL ; aucune EditionRef dans l'agrégat actuel.

DÉPENDANCES ET ÉCHANGES

data-model-dataset, data-model-project-dataset-link, data-model-label et événements d'identité ; Kafka au moins une fois.

COMPORTEMENT CIBLE PROPOSÉ

Catalog Rust publie une projection versionnée et Query conserve seulement les index nécessaires à la résolution. Les lignes restent lues depuis des éditions approuvées. Chaque fait comporte ID, agrégat et révision ; un événement ancien ne peut ressusciter une projection supprimée.

MIGRATION DE CETTE CAPACITÉ

Rejouer le catalogue dans une projection parallèle puis comparer les UUID, branches, labels et liens accordés. Conserver la compatibilité Protobuf pendant le basculement. Vérifier la coexistence des deux sources utilisateur au lieu de supposer que le drapeau IAM arrête le listener historique.

CRITÈRE D'ACCEPTATION À EXÉCUTER

UPDATED reçu avant CREATE établit la projection. Un refus rejoué retire un ancien lien autorisé. Une suppression puis un événement ancien ne rend pas le dataset visible. La perte de la projection se répare sans nouvelle publication de données.

Références de l'analyse

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/event/DataModelEventKafkaConsumer.kt:32](#) Upsert et contrôles du snapshot reçu.

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/event/DataModelEventKafkaConsumer.kt:85](#) Seuls les liens accordés restent dans la projection.

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[app/src/main/kotlin/com/cleyrop/data/serve/app/user/event/LegacyUserCreatedConsumer.kt:18](#) Listener historique non conditionné dans ce fichier.

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataset/aggregate/Dataset.kt:128](#) Agrégat de serving sans référence d'édition.

DS-02 **Cible non qualifiée**

Authentifier les lecteurs OAuth et préserver l'acteur des appels MCP

COMPORTEMENT OBSERVÉ

Les routes natives `/api/v1` et `/internal/mcp` utilisent une chaîne Bearer obligatoire. Le reste peut accepter X-Userinfo uniquement sous configuration gateway, tout en conservant le resource server Bearer. L'interceptor résout l'utilisateur, exige qu'il soit actif et ignore les anciens en-têtes de clés API. Le MCP exige le client `cleyrop-mcp-dataset` et demande à IAM de résoudre acteur et contexte à partir du Bearer de transport ; l'acteur retourné doit correspondre à celui demandé. Le compte de transport n'accorde pas lui-même la lecture des données.

UTILISATEURS ET RÔLES

Utilisateur web, client OAuth, outil MCP opérant pour un utilisateur.

INTERFACES DANS LE CODE

ExpositionApiController ; TokenHandlerInterceptor ; McpReadIdentityResolver ; POST IAM `/api/v1/internal/mcp/read-identity`.

DONNÉES ET ÉTATS

Identité de requête et miroir utilisateur ; cache de résolution IAM et cache du token de service, pas clé API locale.

DÉPENDANCES ET ÉCHANGES

Fournisseur OIDC actuellement Keycloak, gateway APISIX, IAM et `cleyrop-spring-auth`.

COMPORTEMENT CIBLE PROPOSÉ

IAM/Query Rust valident le jeton du transport et le sujet/acteur/context distincts. Ils appliquent les audiences et destinations prévues sans dépendre de Keycloak. MCP reçoit une capacité de lecture limitée aux éditions et opérations autorisées, jamais une clé globale.

MIGRATION DE CETTE CAPACITÉ

Conserver les familles de routes et les enveloppes d'erreur. Porter les mappings subject vers IAM ID, puis supprimer le couplage JVM et Keycloak. Tester les chemins directs MCP et gateway séparément, y compris les protections réseau.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un en-tête historique `token/apikey` n'authentifie pas un appel. Le bon compte MCP avec le mauvais acteur ou contexte est refusé. Une identité inactive ne peut ni lister ni lire, même si une résolution précédente est en cache.

Références de l'analyse

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/configuration/WebSecurityConfiguration.kt:43](#) Chaîne Bearer native et chaîne gateway distinctes.

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/configuration/interceptor/TokenHandlerInterceptor.kt:26](#) Résolution et activité utilisateur.

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/authz/McpReadIdentityResolver.kt:28](#) Validation du transport et de l'acteur auprès d'IAM.

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/configuration/CurrentUserResolver.kt:25](#) Cache de résolution d'identité.

DS-03 **Cible non qualifiée****Déléguer consultation et téléchargement à IAM****COMPORTEMENT OBSERVÉ**

Le service construit des faits sur dataset, projet propriétaire, projets liés, diffusion et sensibilité de la branche, puis interroge le broker IAM. Les opérations CONSULT et DOWNLOAD sont distinctes, ainsi que l'accès PROJECT. Les liens sont découpés par groupes de douze et les demandes par cent contrôles ; les réponses d'un dataset sont combinées seulement après obtention complète. Une erreur, réponse vide, booléen mal typé ou vecteur incomplet produit une indisponibilité, sans politique de secours locale. Une erreur 401 du broker renouvelle une fois le token de service. /permissions ne retourne actuellement que canDownload.

UTILISATEURS ET RÔLES

Tout lecteur de lignes ou de fichiers et interface qui prépare le téléchargement.

INTERFACES DANS LE CODE

GET /datasets/{datasetId}/permissions ; DatasetAuthorizationService ; POST IAM /api/v1/internal/data-serve/authorize.

DONNÉES ET ÉTATS

Projection de faits du dataset et liens accordés ; décisions par appel ; cache du token technique.

DÉPENDANCES ET ÉCHANGES

IAM/OpenFGA, miroir user pour retrouver le subject, gateway pour le contrôle de route.

COMPORTEMENT CIBLE PROPOSÉ

Query Rust applique une autorisation centralisée à l'édition, au schéma et à l'opération, avec policy revision et identité complètes. Les pages et les effets d'export réévaluent la révocation selon le contrat ; aucun cache de résultat ne contourne IAM.

MIGRATION DE CETTE CAPACITÉ

Porter les limites de lots et l'échec fermé. Stabiliser les identités sans dépendre d'une projection utilisateur périmée. Étendre le contrat à tenant, édition et droits de colonnes/lignes seulement avec une politique spécifiée et qualifiée.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un dataset avec plus de douze projets liés reste lisible si un groupe l'autorise et tous les contrôles répondent. Si un autre groupe échoue, aucun octet ne sort. Le bouton téléchargement et le download réel utilisent exactement la même décision.

Références de l'analyse

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[domain/src/main/kotlin/com/cleyrop/data/serve/domain/authz/DatasetAuthorizationService.kt:45](#) Lots, combinaison des droits et échec fermé.

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[app/src/main/kotlin/com/cleyrop/data/serve/app/authz/IamDatasetAuthorizationClient.kt:32](#) Réponse complète, retry 401 et décodage strict.

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataset/query/DatasetPermissionsQueryHandler.kt:31](#) canDownload vient du même arbitre que le téléchargement.

DS-04 **Cible non qualifiée**

Lister les datasets réellement consultables

COMPORTEMENT OBSERVÉ

L'API d'exposition liste les datasets accessibles, avec filtres qualité, sensibilité, état, propriétaires, description, nom affiché et nom du propriétaire ; ownerId prime sur ownerDisplayName. includeColumns ajoute les colonnes et descriptions. Le handler parcourt les candidats par lots de cent, interroge IAM sur chaque branche main puis conserve tous les autorisés en mémoire avant la pagination demandée. Les totaux décrivent donc les datasets consultables. Le coût croît avec l'ensemble des candidats et des décisions IAM, même pour une petite première page.

UTILISATEURS ET RÔLES

Client d'intégration, MCP dataset et applications de lecture.

INTERFACES DANS LE CODE

GET /api/v1/datasets ; GET /internal/mcp/v1/datasets.

DONNÉES ET ÉTATS

Projection locale des datasets et utilisateurs ; collection de candidats puis liste autorisée.

DÉPENDANCES ET ÉCHANGES

IAM batch ; PostgreSQL de serving ; labels/schémas de data-model.

COMPORTEMENT CIBLE PROPOSÉ

Catalog/Query Rust conserve une liste filtrée par droits et une pagination bornée. Le plan de recherche doit éviter de charger tous les objets pour chaque page, sans produire de totaux qui révèlent des objets interdits. Les droits sont revérifiés à l'ouverture de lecture.

MIGRATION DE CETTE CAPACITÉ

Comparer champs, ordre, includeColumns, pagination externe à base un et types d'erreur. Mesurer le coût sur le volume réel du catalogue avant de choisir index d'autorisation, curseur ou total approximatif explicitement annoncé.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une liste de cent candidats dont trente sont autorisés annonce trente, sans IDs des autres. La priorité ownerId est conservée. Une révocation entre liste et lecture bloque la lecture et la charge mémoire reste bornée sur un gros catalogue.

Références de l'analyse

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[app/src/main/kotlin/com/cleyrop/data/serve/app/apitoken/controller/ExpositionApiController.kt:182](#) Paramètres et alias MCP de la liste.

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[domain/src/main/kotlin/com/cleyrop/data/serve/domain/apitoken/query/ListDatasetApiTokenQueryHandler.kt:43](#) Résolution des propriétaires puis filtrage de tous les candidats.

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[domain/src/main/kotlin/com/cleyrop/data/serve/domain/apitoken/query/ListDatasetApiTokenQueryHandler.kt:65](#) Coût de parcours complet avant pagination.

DS-05 Cible non qualifiée

Lire une table par UUID ou nom avec projection et filtres bornés

COMPORTEMENT OBSERVÉ

Les lectures simples et structurées résolvent un UUID ou un techName, prennent main, vérifient CONSULT, refusent CREATED/FAILURE et valident `page>=1` et `size` dans la borne configurée. `select` est une liste de colonnes de ce dataset ; `where` est reconstruit depuis une grammaire admise de comparaisons, AND/OR/NOT, NULL, BETWEEN, IN et LIKE/ILIKE. Fonctions, casts, sous-requêtes, commentaires et séparateurs de statements sont refusés. Les fragments ont une limite de longueur, profondeur et nombre de valeurs. `orderBy` accepte colonnes retournées ou positions, direction et NULLS FIRST/LAST ; un départage par positions complète le tri.

UTILISATEURS ET RÔLES

Applications, intégrations et lecteurs MCP sans éditeur SQL complet.

INTERFACES DANS LE CODE

GET `/api/v1/datasets/{dataset}` ; GET `/api/v1/datasets/structured/{dataset}` ; mêmes chemins sous `/internal/mcp/v1`.

DONNÉES ET ÉTATS

Réponse JSON brute ou structurée avec schéma et pagination ; lecture de snapshot Trino résolu pour main.

DÉPENDANCES ET ÉCHANGES

IAM CONSULT, schéma répliqué, SqlFragmentGuard et Trino.

COMPORTEMENT CIBLE PROPOSÉ

Query Rust expose le même sous-ensemble via un AST typé sans concaténation de fragments clients. Il lie noms et field IDs à une édition autorisée et versionne la sérialisation des nombres, dates, binaires et valeurs complexes. Une lecture sans schéma fiable doit obtenir le schéma autoritaire de l'édition.

MIGRATION DE CETTE CAPACITÉ

Réutiliser un corpus de sélections et filtres valides/invalides, y compris colonnes accentuées et noms réservés. Préserver les formats de réponse historiques dans la façade. Remplacer le fallback actuel d'identifiants simples sur schéma vide par une résolution autoritaire.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une injection dans `select/where/orderBy` n'atteint pas le moteur. Une projection de deux colonnes ne permet pas de trier par une troisième cachée. Une page avec valeurs nulles, binaire, date et struct garde les types prévus par le contrat.

Références de l'analyse

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[domain/src/main/kotlin/com/cleyrop/data/serve/domain/apitoken/query/ReadDatasetApiTokenQueryHandler.kt:99](#) Résolution, droits, états et pagination.

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[app/src/main/kotlin/com/cleyrop/data/serve/app/common/SqlFragmentGuard.kt:41](#) Grammaire admise et reconstruction.

[monorepo](#) · [services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/common/SqlFragmentGuard.kt:124](#) Bornes des fragments.

[monorepo](#) · [services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/repository/TableRepositoryTrinoImpl.kt:138](#) Lecture de page et tri stable.

DS-06 **Cible non qualifiée**

Retourner les métadonnées métier du dataset exposé

COMPORTEMENT OBSERVÉ

L'endpoint de métadonnées résout UUID ou nom technique, exige inProduction et une branche main, puis CONSULT. Il retourne type d'origine, date de création, description, qualité, sensibilité, dernier refresh, nom du propriétaire, état, fréquence, append/override, mode de schéma, diffusion et noms des labels. Les métadonnées sont issues de la projection PostgreSQL et ne portent ni snapshot ID ni version de contrat ni preuve d'approbation.

UTILISATEURS ET RÔLES

Intégrateur, application et MCP souhaitant décrire les données avant lecture.

INTERFACES DANS LE CODE

GET /api/v1/datasets/{dataset}/metadata ; GET /internal/mcp/v1/datasets/{dataset}/metadata.

DONNÉES ET ÉTATS

DatasetMetadataView construit depuis dataset/main, utilisateur et labels.

DÉPENDANCES ET ÉCHANGES

Projections data-model/utilisateurs ; IAM CONSULT.

COMPORTEMENT CIBLE PROPOSÉ

Catalog Rust renvoie les métadonnées métier avec une EditionRef et la révision de politique. Le propriétaire et les tags actuels sont distingués des métadonnées de l'édition historique. Le kind est explicite et ne reprend pas le type de connecteur.

MIGRATION DE CETTE CAPACITÉ

Maintenir les noms de champs et valeurs enum dans la façade. Ajouter des champs d'édition optionnels puis une version d'API qui les impose. Mesurer et rendre visible la fraîcheur de la projection.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un dataset hors production reste introuvable via ce parcours. Une EditionRef obtenue avec les métadonnées permet une lecture répétable ; une nouvelle collecte ne change pas les données de cette référence. Les droits sont vérifiés même si les métadonnées sont en cache.

Références de l'analyse

[monorepo](#) · [services/data-serve/app/data-serve-domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataset/query/GetDatasetMetadataQueryHandler.kt:28](#) Champs réellement exposés.

[monorepo](#) · [services/data-serve/app/data-serve-domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataset/query/GetDatasetMetadataQueryHandler.kt:61](#) Production, branche, droits, propriétaire et labels.

[monorepo](#) · [services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/apitoken/controller/ExpositionApiController.kt:398](#) Route et alias MCP.

DS-07 Cible non qualifiée

Afficher un aperçu de dataset

COMPORTEMENT OBSERVÉ

Le preview d'un dataset résout la branche demandée ou main. Une branche CREATED ou FAILURE retourne aucun aperçu. Après autorisation CONSULT, le repository lit par défaut cinquante lignes via Trino et assemble les valeurs avec le schéma répliqué et la date de refresh. WARNING reste lisible. Les erreurs de branche manquante et dataset manquant sont distinctes dans le domaine. La route rend 404 lorsqu'aucun DataFrame n'est produit.

UTILISATEURS ET RÔLES

Utilisateur du catalogue et de l'éditeur qui inspecte un échantillon.

INTERFACES DANS LE CODE

GET /datasets/{datasetId}/preview?branchName=... ; PreviewDatasetQueryDataset.

DONNÉES ET ÉTATS

DataFrame contenant Table, TabularSchema et refreshedDate ; projection de branche.

DÉPENDANCES ET ÉCHANGES

IAM CONSULT ; Trino/Iceberg ; data-model pour schéma et état.

COMPORTEMENT CIBLE PROPOSÉ

Query Rust fournit un échantillon borné d'une édition explicitement autorisée, avec schema ID, type précis et budget d'octets. Le résultat cite l'édition plutôt que la seule date d'exécution. Les erreurs d'une tentative récente ne doivent pas masquer une édition approuvée antérieure sans règle métier explicite.

MIGRATION DE CETTE CAPACITÉ

Comparer valeurs et typage sur les cinquante premières lignes des corpus de référence. Décider et versionner l'ordre de l'aperçu, qui n'est pas un ordre métier garanti actuellement. Revoir l'effet de FAILURE lorsque des données approuvées existent déjà.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un aperçu d'une édition répétée reste cohérent après refresh. L'utilisateur sans CONSULT ne reçoit ni lignes ni types protégés. Une valeur très volumineuse ne dépasse pas le budget du preview.

Références de l'analyse

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/controller/ServeController.kt:122](#) Contrat HTTP et branche optionnelle.

[monorepo · services/data-serve/app/data-serve-domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataset/query/PreviewDatasetQueryHandler.kt:109](#) État, droits et assemblage.

[monorepo · services/data-serve/app/data-serve-domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataset/repository/TableRepository.kt:21](#) Limite par défaut de cinquante lignes.

DS-08 Cible non qualifiée

Consulter les résultats intermédiaires des transformations low-code

COMPORTEMENT OBSERVÉ

POST /version/preview prend version et transformation. data-serve demande à data-dev le projet et toutes les entrées de la version, vérifie l'identité de la version retournée et la forme des branches, puis exige accès au projet et CONSULT sur toutes les entrées. Une version sans entrée reste régie par le projet. Il lit ensuite un fichier preview.json dans le stockage MinIO/S3, indexé par version et transformation. Un scope absent ou une entrée introuvable produit aucun preview ; une autorité scope indisponible produit une indisponibilité explicite.

UTILISATEURS ET RÔLES

Auteur ou lecteur de flow et interface low-code.

INTERFACES DANS LE CODE

POST /version/preview ; GET interne /api/v1/internal/dataflow-versions/{id}/preview-scope appelé vers data-dev.

DONNÉES ET ÉTATS

PreviewScope projet/inputs et objet S3 intermédiaire ; pas d'édition publiée de dataset.

DÉPENDANCES ET ÉCHANGES

data-dev HTTP avec identité de service ; IAM ; MinIO/S3 ; artefacts process-data.

COMPORTEMENT CIBLE PROPOSÉ

Flows et Query Rust lient chaque preview à run, transformation, révision de plan et éditions d'entrée. Les artefacts sont privés, immuables, bornés et munis d'une rétention. Leur contrôle de lecture utilise sujet, acteur, projet et toutes les entrées, même après retrait d'un partage.

MIGRATION DE CETTE CAPACITÉ

Importer le mapping des versions vivantes et conserver la lecture des anciens artefacts jusqu'à leur expiration. Remplacer les chemins conventionnels par un manifeste de preview dont le producteur est authentifié. Éviter qu'un simple UUID d'objet fournisse le droit de lecture.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un utilisateur autorisé sur le projet mais interdit sur une entrée ne lit pas le preview. Une version inconnue et une entrée supprimée n'exposent pas d'informations supplémentaires. Un preview généré sans entrée reste limité aux droits du projet.

Références de l'analyse

[monorepo · services/data-serve/app/data-serve-domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataset/query/PreviewDatasetQueryHandler.kt:66](#) Vérification du scope et des entrées avant lecture.

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/repository/DataDevPreviewScopeClient.kt:41](#) Autorité distante, validation stricte et erreurs.

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/repository/TableRepositoryTrinoImpl.kt:241](#) Lecture du preview stocké en objet.

Exécuter du SQL de lecture avec analyse des relations

COMPORTEMENT OBSERVÉ

L'éditeur SQL accepte un projet et une branche facultative ; l'exposition propose SQL global ou par projet, toujours sur main. Query découpe les statements sur les points-virgules et n'en accepte qu'un. JSqlParser extrait opérations et relations ; seules SELECT et EXPLAIN passent, les syntaxes non parsables échouent. Les fonctions de table sont refusées sauf UNNEST. Le service vérifie l'accès au projet puis à chaque dataset et, en contexte projet, son appartenance ou lien. L'éditeur permet une requête sans dataset après contrôle du projet ; exposition/MCP la refusent. Il n'y a pas de CRUD SQL d'écriture dans ces contrôleurs.

UTILISATEURS ET RÔLES

Analyste SQL, application OAuth et MCP dataset.

INTERFACES DANS LE CODE

POST /projects/{projectID}/datasets/sql/query ; POST /api/v1/projects/{projectID}/datasets/sql/query ; POST /api/v1/datasets/sql/query ; POST /internal/mcp/v1/datasets/sql/query.

DONNÉES ET ÉTATS

ExecutionContext transitoire, statement, opérations, noms de datasets et inputIds dans la réponse.

DÉPENDANCES ET ÉCHANGES

IAM CONSULT/PROJECT, liens de projet, JSqlParser et Trino JDBC.

COMPORTEMENT CIBLE PROPOSÉ

Query Rust analyse, lie puis exécute un AST sur des EditionRef approuvées avec Icegres ou un moteur sans JVM qualifié. Les capacités dialectales sont explicites. L'analyse de sécurité doit porter sur tout le plan, sous-requêtes, fonctions et EXPLAIN compris, avec aucune résolution physique hors liste autorisée.

MIGRATION DE CETTE CAPACITÉ

Créer une matrice de compatibilité SELECT, CTE, joins, agrégations, types, fonctions, EXPLAIN et erreurs. Rejouer un corpus expurgé et comparer résultats. Remplacer le split textuel et les réécritures regex par un parseur et binder communs.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une jointure exige le droit sur chaque entrée et un contexte projet cohérent. TABLE d'accès indirect est refusé avant moteur. Un point-virgule dans un littéral suit le dialecte documenté. Une requête sans dataset conserve la différence éditeur/exposition tant que la façade la promet.

Références de l'analyse

[monorepo · services/data-serve/app/data-serve-domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataset/aggregate/Query.kt:6](#) Découpage actuel des statements.

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/service/QueryAnalyserImplJSQL.kt:20](#) Extraction des relations et refus des fonctions de table.

[monorepo · services/data-serve/app/data-serve-domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataset/service/QueryAnalyser.kt:113](#) Droits sur projet et chaque dataset.

[monorepo · services/data-serve/app/data-serve-domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataset/query/SQLQueryHandler.kt:148](#) Distinction éditeur et exposition.

Épingler les snapshots et paginer les résultats SQL

COMPORTEMENT OBSERVÉ

TrinoJDBC lit `snapshot_id` depuis la table de refs pour la branche, puis utilise `FOR VERSION AS OF`. La lecture de dataset résout le snapshot pour chaque appel. Le SQL arbitraire résout chaque entrée séparément et réécrit le statement par regex ; un snapshot manquant laisse actuellement le nom sans clause. Le SQL paginé interroge d'abord le schéma, enveloppe la requête avec `COUNT OVER`, tri et `offset/limit`, puis conserve total et `hasMore`. Si une page dataset est vide au-delà du début, le comptage de secours résout de nouveau la ref. Aucune session de pagination exposée ne fixe la même édition sur plusieurs pages ou plusieurs tables.

UTILISATEURS ET RÔLES

Clients paginés, analystes et équipes qui exigent une lecture cohérente.

INTERFACES DANS LE CODE

`SQLQueryPaged` ; `TrinoJDBC.queryForListWithTotal/queryForRowSetPaged` ; réponses Pagination.

DONNÉES ET ÉTATS

Snapshot IDs transitoires ; tri dérivé des colonnes sélectionnées ; aucun cursor d'édition persistant dans le contrat actuel.

DÉPENDANCES ET ÉCHANGES

Trino et refs Iceberg ; schéma de résultat JDBC ; politique de pages du service.

COMPORTEMENT CIBLE PROPOSÉ

Query Rust résout un `ReadSet` avant exécution. Ce `ReadSet` contient table UUID, édition/snapshot, schema/spec et policy revision pour toutes les entrées. Les pages reprennent un curseur signé lié à ce `ReadSet` et au sujet ; une release multi-sorties fournit un ensemble atomique de participants. Une ref absente ne retombe jamais sur l'état courant.

MIGRATION DE CETTE CAPACITÉ

Conserver la pagination numérique dans la façade mais lui associer un handle stable. Réécrire le plan au niveau AST/scan, sans regex. Ajouter des scénarios de branches absentes, noms quotés, ref déplacée, lignes identiques et publication concurrente.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Publier entre page 1 et page 2 ne produit ni saut ni mélange d'éditions. Une requête multi-sorties lit tous les membres de la même release. Une ref inconnue échoue avant scan. Le total d'une page vide est calculé sur le même snapshot.

Références de l'analyse

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/repository/jpa/TrinoJDBC.kt:69](#) Résolution de ref par branche.

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/repository/jpa/TrinoJDBC.kt:122](#) Page avec total et recomptage.

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/repository/jpa/TrinoJDBC.kt:254](#) Plan de pagination SQL.

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/repository/jpa/TrinoJDBC.kt:473](#) Réécriture regex et absence de clause si snapshot introuvable.

Télécharger un dataset ou une requête en CSV

COMPORTEMENT OBSERVÉ

Le download dataset prend branche, quoted, separator et format. Pour CSV, il exige DOWNLOAD, refuse CREATED/FAILURE, écrit une ligne d'en-tête puis les lignes en streaming. Le download SQL réutilise l'analyse de requête avec l'opération DOWNLOAD pour toutes les relations et refuse une requête sans dataset. Le résultat porte un nom de fichier et Content-Disposition. Le corps ferme le Stream après transfert. Le pool Trino de download est séparé du pool de lecture pour qu'un client lent n'accapare pas les connexions interactives ; sa saturation aboutit à un timeout borné et une erreur 503.

UTILISATEURS ET RÔLES

Utilisateur qui extrait des données et intégrateur consommant un dump.

INTERFACES DANS LE CODE

GET /datasets/{datasetId}/download ; POST /datasets/download ; POST /api/v1/datasets/download.

DONNÉES ET ÉTATS

Stream HTTP/JDBC, filename et paramètres CSV ; aucun reçu d'export durable ni destination externe persistée.

DÉPENDANCES ET ÉCHANGES

IAM DOWNLOAD ; Trino download pool ; MVC StreamingResponseBody.

COMPORTEMENT CIBLE PROPOSÉ

Query/Egress Rust produit un flux borné à partir d'une EditionRef approuvée, avec limites de durée, volume, concurrence et annulation. Un EXPORT vers un tiers est un effet distinct avec destination déclarée, clé d'idempotence et reçu. Le résultat peut être préparé en objet privé puis téléchargé sous droit réévalué.

MIGRATION DE CETTE CAPACITÉ

Conserver séparateur, quoting, en-têtes, charset, nom de fichier et comportements null/types dans la façade. Qualifier l'échappement CSV sur textes contenant séparateur et retour ligne. Décider le comportement d'une révocation pendant un transfert long.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un utilisateur qui peut consulter mais ne peut télécharger ne reçoit aucun octet. Une jointure nécessite DOWNLOAD sur chaque source. Annuler un transfert libère le scan et les ressources. Plusieurs clients lents n'empêchent pas les previews de répondre.

Références de l'analyse

[monorepo · services/data-serve/app/data-serve-](#)

[domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataset/query/DownloadDatasetQueryHandler.kt:62](#) Droit de téléchargement, états et formats.

[monorepo · services/data-serve/app/data-serve-](#)

[domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataset/query/DownloadDatasetQueryHandler.kt:89](#) SQL export avec droit DOWNLOAD.

[monorepo · services/data-serve/app/data-serve-](#)

[app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/repository/TableRepositoryTrinoImpl.kt:195](#) Production CSV en flux.

[monorepo · services/data-serve/app/data-serve-](#)

[app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/controller/CsvStreamingResponse.kt:6](#) Fermeture du flux et écriture HTTP.

[monorepo · services/data-serve/app/data-serve-](#)

[app/src/main/kotlin/com/cleyrop/data/serve/app/configuration/TrinoConfiguration.kt:24](#) Isolation des connexions de téléchargement.

DS-12 **Cible non qualifiée**

Exporter les types sans perte via le protocole NDJSON

COMPORTEMENT OBSERVÉ

Le format NDJSON du download dataset n'est pas une simple liste d'objets métier. Il commence par un frame schema portant noms et signatures de types, continue avec des frames row positionnels et termine par rowCount et SHA-256. Les décimaux et flottants sont encodés en texte, le binaire en base64, les dates/temps en forme textuelle. Les ROW restent positionnels pour ne pas perdre les noms de champs dupliqués ; les maps deviennent des paires key/value. L'imbrication est bornée. Le digest couvre les octets des frames row encodés, sans les retours ligne ajoutés par le contrôleur, ni les frames schema/end.

UTILISATEURS ET RÔLES

Client de copie, migration de dataset et intégration qui doit préserver les types.

INTERFACES DANS LE CODE

GET /datasets/{datasetId}/download?format=NDJSON ; DetailedRowStream.toLosslessNdjson.

DONNÉES ET ÉTATS

Frames schema/row/end ; compteur et digest calculés pendant le flux ; schema et lignes épinglés au même snapshot dans ce parcours.

DÉPENDANCES ET ÉCHANGES

Trino JDBC pour signatures et valeurs typées ; IAM DOWNLOAD ; lecteur de frames compatible.

COMPORTEMENT CIBLE PROPOSÉ

Query/Egress Rust conserve ce protocole versionné ou fournit Arrow IPC avec un adaptateur compatible. Le digest, les bornes et la représentation des types font partie du contrat. Un consommateur rejette un flux tronqué sans end et ne publie pas une copie partielle.

MIGRATION DE CETTE CAPACITÉ

Documenter le checksum exact avant portage. Comparer valeurs complexes, NaN/infinis, grands entiers/décimaux, timezone, maps à clés non textuelles et ROW avec noms dupliqués. Prévoir une négociation de version de format.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un aller-retour préserve null, précision décimale, timezone, binaire, ordre des champs et clés de map. Couper le transfert avant end empêche le commit du destinataire. Le digest recalculé selon la définition correspond exactement au frame final.

Références de l'analyse

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/repository/LosslessNdjson.kt:23](#) Frames, compteur et digest.

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/repository/LosslessNdjson.kt:57](#) Encodage des types et limite de profondeur.

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/repository/jpa/TrinoJDBC.kt:190](#) Schéma détaillé et lignes sur le même snapshot.

[monorepo · services/data-serve/app/data-serve-domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataset/query/DownloadDatasetQueryHandler.kt:81](#) Choix NDJSON dans le contrat.

DS-13 **Cible non qualifiée**

Créer, versionner et supprimer les règles qualité

COMPORTEMENT OBSERVÉ

Une Rule identifie le dataset, l'auteur et sa suppression logique ; chaque création ou modification écrit une RuleHistory avec nom, description, script et seuils. Le nom actif est unique sans distinction de casse. Le service vérifie comparateurs, champs non blancs, existence du dataset, syntaxe SQL et exécution de la requête avant sauvegarde. Les comparateurs varient par type boolean, numeric, instant ou string et sont ordonnés ; la première condition satisfaite détermine le niveau d'alerte. Les modifications et lectures portant ruleId passent aussi datasetId afin qu'une règle d'un autre dataset soit introuvable. isNotify crée une souscription d'échec qualité.

UTILISATEURS ET RÔLES

Éditeur de règles et responsable qualité.

INTERFACES DANS LE CODE

POST /datasets/{datasetId}/rules ; PUT/DELETE /datasets/{datasetId}/rules/{ruleId} ; GET règles, règle, check du nom et history.

DONNÉES ET ÉTATS

Rule, RuleHistory, suppression logique, auteur, script et niveaux CRITICAL/MAJOR/MINOR.

DÉPENDANCES ET ÉCHANGES

Gateway IAM can_edit_rules/can_view_rules ; Trino pour validation ; outbox vers data-model ; notification-center.

COMPORTEMENT CIBLE PROPOSÉ

Audit Rust conserve une définition versionnée de règle et un contrat de résultat. Les seuils numériques utilisent un type exact adapté, sans conversion systématique vers Double. La règle est compilée et autorisée sur toutes ses entrées, puis exécutée sur un candidat ou une édition précise.

MIGRATION DE CETTE CAPACITÉ

Importer toutes les versions et tombstones. Garder l'historique des valeurs et comparateurs, y compris les limitations de conversion du type instant et numeric. Distinguer validation syntaxique, test utilisateur et audit bloquant de publication.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Modifier une règle crée une version nouvelle sans altérer les preuves d'une édition ancienne. Un ruleId d'un autre dataset est introuvable sur toutes les routes. Un seuil numérique supérieur à 2^{53} garde sa valeur exacte. Deux créations concurrentes du même nom ne réussissent pas ensemble.

Références de l'analyse

[monorepo · services/data-serve/app/data-serve-domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataquality/command/SaveRuleCommandHandler.kt:86](#) Validations, unicité et exécution avant sauvegarde.

[monorepo · services/data-serve/app/data-serve-domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataquality/command/SaveRuleCommandHandler.kt:172](#) Versions créées/modifiées et notifications.

[monorepo · services/data-serve/app/data-serve-domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataquality/aggregate/RuleType.kt:59](#) Ordre et arité des comparateurs.

[monorepo · services/data-serve/app/data-serve-domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataquality/aggregate/RuleType.kt:189](#) Conversion numérique actuelle en Double.

[monorepo](#) · [services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/dataquality/controller/DataQualityController.kt:64](#) Binding ruleId/datasetId.

DS-14 Cible non qualifiée

Fournir les modèles et l'essai interactif des règles qualité

COMPOTEMENT OBSERVÉ

Quinze templates couvrent volumes, duplicats de lignes, unicité de colonnes, combinaisons distinctes, distribution, intervalles, email, appartenance à une liste, minimum, maximum, médiane, moyenne et non-null. Ils retournent du SQL avec le nom technique ou un placeholder. Le check exécute le script sur main et exige une seule colonne, une seule ligne et une valeur non nulle d'un type admis. Le contrôle de statement exige SELECT et la présence du dataset ciblé, mais ne vérifie pas que l'ensemble des relations se réduit à ce dataset et ne demande pas CONSULT sur d'autres relations. Certains templates ne correspondent pas à leur libellé, par exemple maximum inférieur utilise $\geq$ et médiane égale utilise $\geq$.

UTILISATEURS ET RÔLES

Utilisateur qui prépare une règle avant enregistrement.

INTERFACES DANS LE CODE

GET /datasets/{datasetId}/rules/templates/{templateType} ; POST /datasets/{datasetId}/rules/check.

DONNÉES ET ÉTATS

Résultat transitoire type/valeur ou erreur métier ; aucun compute persistant pour ce check.

DÉPENDANCES ET ÉCHANGES

JSqlParser, schéma/techName local et Trino ; gateway autorise le dataset du chemin.

COMPOTEMENT CIBLE PROPOSÉ

Audit/Query Rust fournit un catalogue de templates versionnés, paramètres typés et sémantique testable. Toutes les relations du script doivent être liées et autorisées ; une règle monodataset refuse les autres, une règle multi-entrées les déclare explicitement. Les essais sont bornés et distincts des preuves de publication.

MIGRATION DE CETTE CAPACITÉ

Traiter l'écart d'autorisation des relations supplémentaires avant de reproduire le parcours. Versionner les corrections des templates pour ne pas changer en silence les règles enregistrées. Rejouer les cas vide, null et limites numériques/temporelles.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un script qui joint une table non autorisée est refusé avant Trino. Chaque template donne le résultat annoncé sur des jeux connus, y compris table vide. Un résultat de plusieurs lignes ou colonnes retourne l'erreur qualité attendue sans être enregistré comme preuve.

Références de l'analyse

[monorepo](#) · [services/data-serve/app/data-serve-domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataquality/aggregate/Template.kt:7](#) Les quinze modèles disponibles.

[monorepo](#) · [services/data-serve/app/data-serve-domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataquality/aggregate/Template.kt:101](#) Opérateurs codés de maximum et médiane.

[monorepo](#) · [services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/dataquality/service/QualityRuleServiceImpl.kt:22](#) Le contrôle vérifie la présence de la cible, pas l'exclusivité des relations.

[monorepo](#) · [services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/dataquality/service/QualityRuleServiceImpl.kt:49](#) Forme du résultat exigée.

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataquality/query/RuleCheckQueryHandler.kt:31](#) Exécution interactive.

DS-15 **Cible non qualifiée**

Calculer la qualité après les traitements et diffuser les résultats

COMPORTEMENT OBSERVÉ

Les événements process-data déclenchent CreateCompute. Le handler ignore les exécutions échouées et les branches autres que main. Il crée un Compute lié au dataset et à l'exécution, charge la dernière version de chaque règle active, valide et exécute son SQL, puis produit ComputedRule avec résultat ou erreur et publie un événement pour data-model. L'évaluation se fait sur la ref main résolue au moment de chaque requête, pas sur un snapshot fixé par l'événement de fin de traitement. Le handler présenté ne montre pas de déduplication de commande par executionId avant allocation d'un nouvel ID de compute.

UTILISATEURS ET RÔLES

Consommateur d'exécution, responsable qualité et notifications de résultat.

INTERFACES DANS LE CODE

ProcessDataEventConsumer ; CreateComputeCommandHandler ; ComputedRuleCreatedEvent.

DONNÉES ET ÉTATS

Compute, dernière RuleHistory choisie, ComputedRule et statut SUCCESS/FAIL/ERROR/PENDING ; publication outbox.

DÉPENDANCES ET ÉCHANGES

process-data Kafka, Trino, repositories de règles et topic data-serve-computed-rule vers data-model.

COMPORTEMENT CIBLE PROPOSÉ

Audit Rust évalue toutes les règles retenues pour un candidat précis avec un ensemble de versions figé. La preuve post-candidat est immuable et liée durablement à la provenance préallouée. Publisher consomme cette preuve avant d'approuver ; une alerte métier peut être non bloquante selon politique explicite. Le run d'audit est borné et idempotent.

MIGRATION DE CETTE CAPACITÉ

Importer les résultats historiques comme observations sans prétendre qu'ils ont bloqué la publication. Introduire clé candidat+ruleset+audit-policy et binding de snapshot, puis déplacer le déclenchement avant Publisher. Supprimer la dépendance au simple événement de succès de main.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un second traitement publié pendant l'audit ne change pas les données auditées. Rejouer le même message retrouve le même compute et la même preuve. Un veto garde l'ancien snapshot publié et interdit l'ack source. Tous les participants d'une release multi-sorties sont couverts.

Références de l'analyse

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataquality/command/CreateComputeCommandHandler.kt:122](#) Conditions de déclenchement et versions de règles.

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataquality/command/CreateComputeCommandHandler.kt:58](#) Exécution sur main et construction des résultats.

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[app/src/main/kotlin/com/cleyrop/data/serve/app/dataquality/event/consumer/ProcessDataEventConsumer.kt:19](#) Source événementielle du calcul.

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[app/src/main/kotlin/com/cleyrop/data/serve/app/dataquality/event/producer/ComputedRuleEventHandlerKafkaImpl.kt:1](#)

Production des résultats de règles.

DS-16 **Cible non qualifiée**

Réparer la projection des types depuis le schéma physique

COMPORTEMENT OBSERVÉ

Une migration parcourt les datasets et branches, lit leur schéma physique par Trino, puis compare les types par nom de colonne sans distinction de casse. Elle conserve l'ordre et la casse déclarés et ignore les colonnes absentes physiquement. Si la lecture de branche échoue, elle essaie main. Elle émet une correction JSON vers data-model seulement lorsqu'un type diffère. Les exceptions d'une branche sont journalisées et le parcours continue. Cette correction passe encore par un producteur Kafka direct, exception au mécanisme outbox.

UTILISATEURS ET RÔLES

Opérateur de migration et mainteneur de la cohérence du catalogue.

INTERFACES DANS LE CODE

CorrectSchemaCommandHandler ; DatasetSchemaMigrationObject ; topic migration-topic-dataset-branch-schema. Aucune route REST publique de correction trouvée.

DONNÉES ET ÉTATS

Projection de types corrigée par événement ; schéma physique lu via snapshot/ref.

DÉPENDANCES ET ÉCHANGES

Trino/Iceberg, data-model consommateur de migration, runner de code migrations.

COMPORTEMENT CIBLE PROPOSÉ

Migration/Datasets Rust compare field IDs, schéma d'édition et annotations sans remplacer une branche inconnue par main. Le plan de réparation est audité, rejouable et sépare correction d'index de changement réel de données. Il ne publie pas automatiquement un nouveau snapshot.

MIGRATION DE CETTE CAPACITÉ

Utiliser la lecture physique pour l'inventaire initial, mais enregistrer UUID, snapshot et raison de l'écart. Remplacer le fallback implicite par un statut à résoudre. Préserver les anciens événements le temps de drainer leurs corrections.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une relance sans divergence n'émet aucune correction. Une branche inexistante est signalée sans lui attribuer le schéma de main. Un renommage de colonne conserve son field ID et n'est pas confondu avec une nouvelle colonne.

Références de l'analyse

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataset/command/CorrectSchemaCommandHandler.kt:33](#) Comparaison par noms et conservation de l'ordre.

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataset/command/CorrectSchemaCommandHandler.kt:116](#) Fallback main et correction conditionnelle.

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/event/CorrectSchemaEventHandlerKafkaImpl.kt:1](#) Producteur historique direct de correction.

DS-17 Cible non qualifiée**Borner les ressources et restaurer les projections après panne****COMPOTEMENT OBSERVÉ**

Les lectures interactives et dumps ont des pools Trino séparés. Les dumps utilisent `StreamingResponseBody` et une durée maximale MVC configurée. Le service utilise une outbox durable, un relay avec retries et purge des envois, des dead letters et un drainage qui clôt certaines courses où l'entité a disparu tout en signalant les erreurs permanentes. Le runner de migrations de code consigne les résultats mais appelle encore une sortie Spring de code zéro en cas d'échec et contient une réserve explicite sur l'absence de lock. Aucun cache de résultats SQL n'a été trouvé dans les implémentations lues ; seuls les caches d'identité/token et les projections sont observés.

UTILISATEURS ET RÔLES

Exploitant, SRE et clients soumis aux limites de concurrence.

INTERFACES DANS LE CODE

TrinoConfiguration ; OutboxRelay ; KafkaConfiguration ; DeadLetterEventDrainer ; CleyropMigrationRunner ; endpoints santé/metrics selon configuration.

DONNÉES ET ÉTATS

Pools et flux transitoires ; outbox_event, dead_letter_event et cleyrop_migration durables ; caches locaux reconstructibles.

DÉPENDANCES ET ÉCHANGES

PostgreSQL, Kafka, Trino, S3 et IAM ; chart Deployment/HPA/PVC/NetworkPolicy.

COMPOTEMENT CIBLE PROPOSÉ

Query Rust possède admission, budget CPU/mémoire/octets, annulation et échéances par requête/run. Les workers sont fongibles. Tout cache de données est une optimisation indexée par tenant, édition, schéma, requête normalisée et contexte de sécurité, avec réévaluation des droits. Le journal durable sans JVM et les reçus remplacent la dépendance finale à Kafka Java.

MIGRATION DE CETTE CAPACITÉ

Établir des budgets d'exploitation mesurés et conserver l'isolement des téléchargements. Drainer outbox/dead letters avant coupure et qualifier les duplicates. Ajouter un lock et un statut d'échec observable aux migrations Rust. Tester restauration de PG, index et objets séparément.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une panne broker n'acquiesce pas une mutation non journalisée. Une migration échouée empêche readiness et fournit une erreur identifiable. Supprimer un cache ne provoque ni nouvelle publication ni nouvel export. Un utilisateur révoqué ne récupère pas un résultat d'un cache partagé.

Références de l'analyse

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/configuration/TrinoConfiguration.kt:17](#) Pools et timeouts distincts.

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/common/outbox/OutboxPublisher.kt:9](#) Transactions et garanties au moins une fois.

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/common/deadletter/DeadLetterEventDrainer.kt:25](#) Cycle des dead letters et backlog.

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/codemigration/runner/CleyropMigrationRunner.kt:29](#) Comportement d'échec et réserve sur lock.

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[app/src/main/kotlin/com/cleyrop/data/serve/app/authz/KeycloakServiceTokenProvider.kt:24](#) Cache de token local observé.

DS-18 **Cible non qualifiée**

Distinguer API de lecture, exposition déclarée et export durable

COMPORTEMENT OBSERVÉ

ExpositionApiController est aujourd'hui une famille d'API OAuth de listing, lecture, métadonnées, SQL et téléchargement. Aucun objet EXPOSURE déclarant consommateur, responsable, politique de refresh ou édition cible, ni ressource EXPORT avec destination, état et reçu, n'a été trouvé dans les contrôleurs et agrégats examinés. Il n'y a pas non plus de gestion locale de tokens API dans ces contrôleurs. Les téléchargements restent des requêtes HTTP transitoires. Cette fiche spécifie le besoin cible et évite d'attribuer au nom du contrôleur une capacité absente.

UTILISATEURS ET RÔLES

Application, dashboard, intégration et responsable d'un consommateur externe.

INTERFACES DANS LE CODE

Existant : routes ExpositionApiController. API cible EXPOSURE/ExportEffect à définir sans inventer de chemin compatible.

DONNÉES ET ÉTATS

Cible : consommateur déclaré en PostgreSQL, EditionRef source, demande d'effet, clé d'idempotence et reçu ; pas de neuvième kind EXPORT.

DÉPENDANCES ET ÉCHANGES

Catalog, Query, Applications, Egress, IAM et Audit ; destination externe validée.

COMPORTEMENT CIBLE PROPOSÉ

Applications/Catalog Rust décrivent EXPOSURE comme consommateur déclaré d'éditions. Egress traite EXPORT comme effet consommant une édition publiée avec destination, contrat de format, autorisation, preuve de livraison et politique de reprise. MODEL/EMBEDDINGS peuvent avoir un alias de serving sans déplacer l'autorité de leur édition Iceberg.

MIGRATION DE CETTE CAPACITÉ

Inventorier clients OAuth/MCP, dashboards et téléchargements programmés, puis les rattacher à des consommateurs déclarés. Garder l'API native pour les clients existants. Migrer les effets vers des jobs bornés qui ne relisent pas implicitement main après retry.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une demande d'export sur un candidat non publié est refusée. Une livraison dont la réponse a été perdue retrouve son reçu ou déclenche une réconciliation, sans doublon silencieux. Une exposition épinglée reste sur son édition jusqu'à une mise à jour autorisée.

Références de l'analyse

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[app/src/main/kotlin/com/cleyrop/data/serve/app/apitoken/controller/ExpositionApiController.kt:69](#) Famille de routes de lecture, sans entité de consommateur.

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataset/query/DownloadDatasetQueryHandler.kt:44](#) Téléchargement transitoire, sans reçu d'effet.

monorepo · services/data-serve/app/data-serve-

domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataset/aggregate/Dataset.kt:128 Modèle de serving observé.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- Conserver les façades REST nécessaires, les définitions et versions de règles, les déclarations EXPOSURE, commandes d'export et reçus d'effet. Leur stockage PG ne remplace pas les éditions publiées ni leur checkpoint.
- Créer ReadSet et QuerySession avec sujet/acteur, tenant, édition et schema de chaque entrée, politique, paramètres, budget et échéance. La pagination reprend ce contrat et ne résout pas une nouvelle version de main.
- Exprimer les capacités SQL du moteur sans JVM et refuser les plans non pris en charge avant exécution. Une interface pgwire/Flight SQL n'est pas une preuve de compatibilité avec tous les clients Trino ou Postgres.
- Versionner les règles qualité et publier un manifeste de preuve attaché au candidat. Historique utilisateur des publications et historique technique des snapshots restent distincts.

Plan de données proposé

- Icegres ou un autre moteur Rust qualifié exécute les scans/plans sur ReadSet ; Arrow sert de contrat interne typé. La sélection du moteur n'est pas qualifiée par cette lecture du service Kotlin.
- Séparer budgets et admission des lectures interactives, checks qualité, previews, exports et scans de migration. Annulation et deadline doivent libérer objets, streams et slots.
- Les caches sont restructurables et identifiés par édition/plan/schéma/tenant ; les droits sont réévalués. Un index de serving, d'embeddings ou un alias d'application ne devient pas autorité de publication.
- Egress produit CSV, NDJSON ou formats nouveaux depuis une édition publiée et livre via des jobs bornés. Les accusés de livraison et reprises sont des reçus d'effets, séparés de l'ack de source.
- Audit exécute sur candidat immuable et lié par provenance durable. Les workers ne reçoivent ni credential global de table ni droit de commit/drop ; Writer et Publisher restent les seuls médiateurs d'écriture et publication.

Découpage du core Rust

- Proposition query-api : routes compatibles, enveloppes d'erreurs, parsing des paramètres, authentification IAM et propagation du contexte MCP.
- Proposition query-planner : SQL AST, binding dataset/field IDs, résolution du ReadSet, validation des fonctions et règles d'accès sur l'ensemble du plan.
- Proposition query-engine-adapter : interface vers Icegres/moteur sans JVM, Arrow, annulation, budgets, restitution des types et métriques.
- Proposition query-pages : curseurs signés liés au ReadSet et au sujet, total cohérent, tri stable et réautorisation.
- Proposition egress-codecs : CSV compatible, NDJSON schema/row/end, checksum et formats typés futurs avec versions explicites.

- Proposition audit-rules : templates, règles versionnées, comparateurs exacts, exécution sur candidat et preuves immuables ; migration des scripts SQL historiques.
- Proposition serving-projector : consommation dédoublée des événements de catalogue, contrôle de révisions et reconstruction des miroirs.

Identité, autorisation et isolation

- La décision du gateway ne suffit pas pour les datasets nommés dans un SQL ou un scope preview. Le code délègue ces décisions à IAM et échoue fermé sur réponse invalide ; conserver ce comportement.
- Les routes de règles vérifient le lien ruleId/datasetId. En revanche, QualityRuleServiceImpl vérifie seulement que le dataset ciblé est présent dans les relations SQL. Qualifier puis corriger le traitement des relations supplémentaires avant migration.
- Les identités gateway, OAuth natif et MCP sont trois transports à préserver. Les headers de compte technique ne doivent pas remplacer le sujet utilisateur. Les chemins directs au pod nécessitent des règles réseau compatibles avec leur identité.
- La ref physique résolue pour une requête n'est pas encore une preuve de publication approuvée. L'API future doit empêcher lecture des candidats, snapshots techniques et fichiers d'audit non autorisés.
- Les logs observés contiennent des statements SQL ou fragments rejetés ; un programme de portage doit définir expurgation, rétention et accès car ils peuvent contenir données personnelles ou literals sensibles.
- Le service n'expose pas de tenantId universel dans ses agrégats. Un déploiement Rust mutualisé doit rendre le tenant explicite dans les décisions, pools, caches, sessions et destinations d'export.

Exploitation et reprise

- Chart Helm de Deployment avec HPA et PVC optionnels, probes de santé et NetworkPolicy configurable. La politique réseau n'est pas une preuve qu'un environnement l'a activée ; elle doit couvrir gateway, MCP direct, IAM, PostgreSQL, Kafka, Trino et S3 selon le déploiement.
- Le pool de téléchargement est indépendant du pool interactif. Le délai MVC borne les dumps ; la saturation du pool de download ne doit pas consommer tous les slots de lecture. Aucun chiffre de charge ni SLO n'est qualifié ici.
- Les événements sortants passent par outbox sauf la correction de schéma historique. La durabilité de l'insert outbox ne suffit pas à prouver une transaction commune avec toutes les écritures du handler ; vérifier les frontières lors du portage.
- KafkaConfiguration traite retry, dead letters et repli local ; DeadLetterEventDrainer distingue les erreurs permanentes des courses d'entité disparue et expose des métriques de backlog. Préserver un traitement opérable au remplacement du journal.
- Flyway et code migrations se lancent au démarrage. Le runner data-serve présente encore un chemin de sortie zéro sur échec et un TODO de verrouillage. La cible doit empêcher deux instances de lancer la même migration mutable.
- Sauvegarder règles et histories, commandes/reçus d'effets, preuves et mappings d'identité. Reconstruire projections et caches depuis les sources autoritaires sans rejouer les téléchargements ou exporter une deuxième fois.

Séquence de migration

- Inventorier les clients des API natives, éditeur, previews, SDK et MCP ; documenter leurs formats, SQL, limites, erreurs et modes d'identité. Associer chaque dataset à une table UUID et à une base publiée retenue.
- Construire une façade Rust en lecture sur les mêmes projections et ReadSets autorisés. Comparer métadonnées, filtres, previews, paginations et exports sur des fixtures expurgées.
- Qualifier le moteur sans JVM pour SQL, types, fonctions, timezone, nulls, decimal, binaire et structs. Vérifier EXPLAIN, annulation, dépassements de mémoire et garanties d'isolation. Aucun retrait de Trino avant ces critères.
- Introduire éditions épinglées et sessions de pagination, avec tests de publication et révocation concurrentes. Corriger l'analyse des règles et les écarts de templates avant d'en faire des gates WAP.
- Migrer Rule/RuleHistory et calculs vers Audit ; figer versions et candidats. Basculer de l'événement post-traitement vers preuve avant publication, avec idempotence par candidat/ruleset.
- Déplacer les téléchargements lourds et effets externes vers Egress, reçus et runs bornés. Drainer outbox/dead letters et l'ancien calcul qualité pour éviter deux producteurs.
- Retirer l'ancien service, Trino, JDBC, JSqlParser, bibliothèques Spring et dépendances Keycloak/Kafka Java après bascule de toutes les cohortes et exercice de restauration.

Écarts et décisions encore ouverts

- Lecture statique uniquement. Aucun benchmark, requête Trino/Icegres, test de contrat ni restauration n'a été exécuté pour ce rapport.
- Quel dialecte SQL est contractuel pour les clients existants, et quelles fonctions ou types doivent être portés, traduits ou refusés explicitement par le moteur Rust ?
- Quelle garantie fournir aux requêtes multi-tables hors release commune ? La résolution indépendante de refs actuelle ne prouve aucune visibilité atomique de groupe.
- Quel protocole d'édition et de curseur ajouter à la pagination numérique pour conserver une lecture stable sans rompre les intégrations ? Que faire d'une révocation au milieu d'un stream déjà ouvert ?
- Le contrôle SQL de qualité doit-il devenir strictement monodataset ou supporter des règles multi-entrées explicitement autorisées ? Le code actuel ne vérifie pas toutes les relations de ce parcours.
- Comment versionner la correction des templates maximum/médiane et l'adoption de nombres exacts sans modifier les preuves et règles historiques en place ?
- Quels computes ont été réellement évalués sur l'état correspondant à leur executionId ? Le handler lit main au moment de chaque règle et ne porte pas de snapshot explicite.
- Quelle rétention et quels droits appliquer aux SQL, erreurs moteur et previews objets ? Les sources peuvent contenir des données sensibles même sans credentials.
- Le cache de données n'est pas une capacité constatée du service actuel. Sa conception future doit spécifier clés, invalidation, révocation et coûts avant toute promesse de performance.
- EXPOSURE et EXPORT nécessitent de nouveaux contrats. L'API nommée ExpositionApiController ne possède pas aujourd'hui ce cycle de vie de consommateur ou d'effet.

Références de l'analyse

[monorepo](#) · [services/data-serve/AGENTS.md:1](#) Périmètre et contraintes du composant.

[monorepo](#) · [services/data-serve/app/gradle/libs.versions.toml:20](#) Versions Kotlin/Spring déclarées.

[monorepo](#) · [services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/controller/ServeController.kt:55](#) Aperçu, droits, SQL et téléchargements UI.

[monorepo](#) · [services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/apitoken/controller/ExpositionApiController.kt:182](#) Exposition native et MCP.

[monorepo](#) · [services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/dataquality/controller/DataQualityController.kt:83](#) Contrats qualité publics.

[monorepo](#) · [services/data-serve/chart/values.yaml:147](#) Bascule du miroir d'identité dans le chart.

[monorepo](#) · [services/data-serve/chart/values.yaml:353](#) Isolation réseau configurable.

[monorepo](#) · [services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/codemigration/runner/CleyropMigrationRunner.kt:29](#) Cycle de migration observé.

SERVICE ACTUEL · APP-MANAGEMENT

Applications, images et espaces de développement

10 capacités analysées

services/app-management · Python, FastAPI, SQLAlchemy/SQLAlchemy, PostgreSQL/Alembic, FastStream/Kafka, Kubernetes, Helm, Kaniko et Trivy.

Provisionne les applications Kubernetes, leurs images, instances, dépôts et devspaces ; fournit aussi à process-data un adaptateur de sessions Python/PySpark.

Lecture de code

Pas de qualification runtime nouvelle

Domaines cible : Applications, Runtimes, IAM, Egress, Audit, Migration, Maintenance.

Périmètre de lecture

- Routers applications, instances, lifecycle, images, sessions privées, dépôts internes/externes et devspaces.
- Services create_application, deploy_instance, cancel_deployment, create_devspace, image_build et internal_repository_file.
- Adaptateur SessionRuntimeServiceImpl et instructions de chiffrement, secrets SSH, Kafka et migrations.

Objets et cycle de vie

- Application/Namespace : identité, projet, propriétaire et configuration désirée.
- Instance et DeploymentExecution : configuration et historique de déploiement, principale, annulation et dates.
- Registry/Image/Build : source Git, commit, image, credentials chiffrés et scans.
- Repository : dépôt interne/externe, projet, visibilité et liens applicatifs.
- Devspace/SSHKey : espace personnel, instance et clés.
- Outbox d'autorisation, miroirs utilisateurs et tombstones projets.

Dépendances du service

- IAM/APISIX pour identité, droits et faits de relation.
- PostgreSQL pour entités, chiffrement et historique ; Kafka/FastStream pour demandes et résultats.
- Kubernetes/Helm pour les workloads, Kaniko/Trivy et registre OCI pour les images.
- Gitea ou autres fournisseurs Git ; process-data pour sessions de calcul.

Fonctionnalités et compatibilité à préserver

AM-01 **Cible non qualifiée**

Catalogue d'applications, propriété et rattachement au projet

COMPORTEMENT OBSERVÉ

Les routes créent, listent, modifient et suppriment les applications, vérifient leur existence et transfèrent leur propriétaire. La création lie une application à son namespace et éventuellement à un projet. Elle vérifie la cohérence du projet dans les arguments, refuse un projet supprimé et sérialise les opérations avec des verrous par propriétaire, projet et application. Les liens vers des applications agents sont autorisés avant création ; une cible absente ou refusée échoue sans laisser un lien. Les faits d'autorisation sont déposés dans l'outbox pour les applications concernées.

UTILISATEURS ET RÔLES

Développeur, responsable de projet et administrateur.

INTERFACES DANS LE CODE

Application CRUD router : /apps ; CreateApplicationService.create_application ; update_application_owner.

DONNÉES ET ÉTATS

ApplicationEntity, NamespaceEntity, lien application-agent, propriétaire, project_id et tombstone de projet.

DÉPENDANCES ET ÉCHANGES

IAM/gateway, outbox SQL et Kubernetes par les déploiements ultérieurs.

COMPORTEMENT CIBLE PROPOSÉ

Applications Rust possède application, rattachement, propriétaire et configuration désirée. IAM reste l'autorité d'accès ; la création et ses faits utilisent une transaction et une clé idempotente.

MIGRATION DE CETTE CAPACITÉ

Importer IDs et rattachements ; préserver les verrous contre création concurrente/offboarding et la validation de chaque cible. Garder une façade HTTP compatible pendant la bascule.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Créer une application pendant la suppression de son projet : elle est refusée ou appartient explicitement au résultat ordonné de l'opération, sans namespace orphelin ni droits tardifs.

Références de l'analyse

[monorepo · services/app-management/src/app_manager/application/router/application_crud_router.py:158](#) Contrat de création et autres commandes du router.

[monorepo · services/app-management/src/app_manager/domain/service/application/create_application_service.py:51](#) Validation, verrous, namespace et outbox.

AM-02 Cible non qualifiée

Instances, configuration déployable et instance principale

COMPORTEMENT OBSERVÉ

Une application possède plusieurs instances dont le détail, la liste, la configuration et les déploiements sont consultables. Le router fournit création, mise à jour, suppression et changement de statut principal. Les dépendances de route contrôlent la parenté application/instance ou application/image afin qu'un identifiant valide d'une autre application ne soit pas accepté sous un parent arbitraire. Les ressources et historiques de sévérité ont des lectures dédiées.

UTILISATEURS ET RÔLES

Développeur applicatif et opérateur.

INTERFACES DANS LE CODE

Instance router : create_instance, update_instance, update_instance_main_status, list_instance_deployments ; enforce_path_parentage.

DONNÉES ET ÉTATS

InstanceEntity, application parente, image, configuration, historique de deployment et sélection principale.

DÉPENDANCES ET ÉCHANGES

Applications, images, autorisation de gateway et ressources Kubernetes.

COMPORTEMENT CIBLE PROPOSÉ

Applications Rust gère une révision immuable de configuration par déploiement, une référence de digest d'image et une sélection principale explicite. Le namespace n'est pas une preuve de droit.

MIGRATION DE CETTE CAPACITÉ

Importer les instances et leur parenté, traduire les secrets en références et conserver les IDs d'historique. Qualifier les contraintes de principale et la concurrence avant activation.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Changer deux fois simultanément l'instance principale conserve un résultat cohérent. Une instance d'une autre application ne peut être lue ou déployée par substitution de parent.

Références de l'analyse

[monorepo · services/app-management/src/app_manager/application/router/instance_router.py:84](#) Création et cycle de configuration.

[monorepo · services/app-management/src/app_manager/application/router/instance_router.py:221](#) Sélection de l'instance principale.

[monorepo · services/app-management/src/app_manager/application/router/parent_scope.py:100](#) Validation de la parenté du chemin.

AM-03 **Cible non qualifiée****Déployer, arrêter, reprendre et annuler une instance****COMPORTEMENT OBSERVÉ**

Le lancement valide l'instance et son état puis contrôle que l'utilisateur peut employer le compte de service configuré. Il crée une exécution de déploiement, passe l'instance à DEPLOYING, synchronise le statut application et commit avant d'émettre la demande. Une précédente demande encore pending peut être marquée CANCELED avant la nouvelle. L'annulation charge le déploiement et exige son état annulable ; elle conserve la date et le résultat en base. Les routes offrent également start, stop et logs. L'émission après commit doit être distinguée d'une transaction unique avec le transport.

UTILISATEURS ET RÔLES

Propriétaire autorisé et exploitation.

INTERFACES DANS LE CODE

InstanceLifecycleRouter.deploy_instance, stop_instance, start_instance, cancel_deployment, get_instance_logs.

DONNÉES ET ÉTATS

DeploymentExecution incrémental, états d'instance/application et dates d'annulation.

DÉPENDANCES ET ÉCHANGES

Consumer Kafka, Helm/Kubernetes, compte de service IAM, watchers de déploiement.

COMPORTEMENT CIBLE PROPOSÉ

Applications/Runtimes Rust utilisent des commandes persistées, reçoivent d'effets Helm/Kubernetes et réconciliation de l'état désiré. Une annulation a une phase précise ; un timeout ne vaut pas preuve d'arrêt.

MIGRATION DE CETTE CAPACITÉ

Introduire un reçu durable couvrant commit puis publication d'événement. Reprendre les déploiements en attente ou les drainer avant de changer le propriétaire de l'orchestration.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Couper le service après commit et avant émission : le déploiement reste reprenable et s'exécute une seule fois logiquement. Annuler une demande déjà exécutée retourne son état réel.

Références de l'analyse

[monorepo · services/app-management/src/app_manager/domain/service/instance/deploy_instance_service.py:58](#) Validation, compte de service et lancement.

[monorepo · services/app-management/src/app_manager/domain/service/instance/deploy_instance_service.py:102](#) Commit avant événement et états.

[monorepo · services/app-management/src/app_manager/domain/service/instance/cancel_deployment_service.py:35](#) Annulabilité et persistance.

[monorepo · services/app-management/src/app_manager/application/router/instance_lifecycle_router.py:65](#) Lecture des logs et lifecycle public.

AM-04 Cible non qualifiée**Construire, reconstruire et inspecter une image****COMPORTEMENT OBSERVÉ**

La construction à partir d'un dépôt vérifie l'application et son lien vers ce dépôt, résout la référence Git en commit et crée un build. Un dépôt non lié répond comme absent. Le service réutilise les entités registre/image/credentials, commit le build puis publie ses événements. Le rebuild retrouve le build précédent et sa source. Le router expose détail, historique, vulnérabilités et logs ; pour un build actif, les logs sont streamés depuis le pod, sinon lus dans les logs conservés. Le service ferme la transaction de lecture avant le flux long.

UTILISATEURS ET RÔLES

Auteur d'application et administrateur des environnements.

INTERFACES DANS LE CODE

Image router : create_image, rebuild_image, list_image_builds, get_image_vulnerabilities, get_build_logs ; ImageBuildService.

DONNÉES ET ÉTATS

BuildEntity, commit source, image/registre, historique et résultats de scan.

DÉPENDANCES ET ÉCHANGES

Git, Kaniko, Trivy, registre OCI, Kubernetes et événements Kafka.

COMPORTEMENT CIBLE PROPOSÉ

Applications/Runtimes Rust gèrent la spécification du build et son reçu ; les exécutants sans JVM restent encapsulés. Le déploiement référence un digest qualifié avec provenance, SBOM et audit.

MIGRATION DE CETTE CAPACITÉ

Conserver commits, builds et diagnostics ; épingler le digest avant déploiement et qualifier l'idempotence de la demande. Reprendre les packages Python personnalisés via le registre d'environnements Runtimes.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un rebuild produit un nouveau reçu sans écraser l'historique. Une réponse perdue ne crée pas deux effets non corrélés. Les logs ne gardent pas une transaction SQL ouverte pendant la durée du build.

Références de l'analyse

[monorepo · services/app-management/src/app_manager/domain/service/image/image_build_service.py:97](#) Source Git liée et résolue.

[monorepo · services/app-management/src/app_manager/domain/service/image/image_build_service.py:141](#) Persistance et publication du build.

[monorepo · services/app-management/src/app_manager/domain/service/image/image_build_service.py:322](#) Logs streamés ou stockés, transaction relâchée.

[monorepo · services/app-management/src/app_manager/application/router/image_router.py:194](#) Lecture du résultat de scan.

AM-05 **Cible non qualifiée****Dépôts externes, dépôts internes et édition de sources****COMPORTEMENT OBSERVÉ**

Les API distinguent dépôts externes liés aux applications et dépôts internes. Elles vérifient l'accès distant, listent les sources et permettent de lier/déliier un dépôt. Le dépôt interne ajoute création, changement de projet, commits, branches, tags, arborescence, lecture/écriture/suppression et upload de fichier. Le service de fichiers résout le dépôt, associe l'auteur et délègue les opérations au client Git ; un renommage avec mise à jour passe par une opération groupée de fichiers. La source d'un build doit être un dépôt lié à l'application.

UTILISATEURS ET RÔLES

Développeur dans la Factory et administrateur Git.

INTERFACES DANS LE CODE

Routers `/repositories`, `/repositories/internal` et `/apps/{app_id}/repositories` ; `InternalRepositoryFileService`.

DONNÉES ET ÉTATS

`RepositoryEntity`, liens application-dépôt, visibilité, projet, branches/tags/commits Git et auteurs.

DÉPENDANCES ET ÉCHANGES

Gitea interne et autres fournisseurs Git, SSH, utilisateurs et IAM.

COMPORTEMENT CIBLE PROPOSÉ

Applications Rust porte la gestion des dépôts et effets Git, Flows épingle les commits qu'il exécute. Gitea Go peut rester encapsulé. L'édition de source ne publie aucune édition de données.

MIGRATION DE CETTE CAPACITÉ

Préserver IDs, URLs non secrètes, commits et auteurs. Séparer l'accès de lecture à la source de la permission de build/déploiement. Conserver les préconditions Git de mise à jour.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une édition basée sur un fichier devenu obsolète retourne un conflit compréhensible. Changer le dépôt lié ne modifie pas le commit d'un build déjà admis.

Références de l'analyse

[monorepo · services/app-management/src/repository/application/router/internal_repository_router.py:209](#) Branches, tags et historique.

[monorepo · services/app-management/src/repository/application/router/repository_application_router.py:59](#) Rattachement à l'application.

[monorepo](#) · [services/app-management/src/repository/domain/service/internal_repository_file_service.py:154](#) Édition groupée et attribution.

[monorepo](#) · [services/app-management/src/repository/application/router/internal_repository_file_router.py:194](#) Contrats fichiers et arborescence.

AM-06 Cible non qualifiée

Devspace personnel, accès SSH et provisioning

COMPORTEMENT OBSERVÉ

La création d'un devspace vérifie l'utilisateur et l'absence d'espace déjà associé. Elle autorise les liens vers applications agents, réutilise registre/image, persiste application, instance et clé SSH puis assure l'accès au dépôt interne avant de lancer le déploiement. Les opérations sont donc réparties entre transaction SQL et effets distants. Les services distincts couvrent lecture, liste, mise à jour, suppression, authorized keys et logs de déploiement. Les clés privées sont chiffrées au repos selon le service crypto existant.

UTILISATEURS ET RÔLES

Développeur et administrateur des espaces.

INTERFACES DANS LE CODE

DevspaceRouter ; CreateDevspaceService.create_devspace ; UpdateDevspaceAuthorizedKeysService ; DevspaceSSHKeyService.

DONNÉES ET ÉTATS

Application de type DEVSPACE, instance, utilisateur, clé et déploiement.

DÉPENDANCES ET ÉCHANGES

Git interne, Kubernetes, Helm, image devspace, IAM et agent de tunnel.

COMPORTEMENT CIBLE PROPOSÉ

Applications/Runtimes Rust orchestre un espace isolé, quotas et reçus de provisioning. Les accès SSH/tunnel ont identité, durée et révocation propres ; le SDK données utilise les frontières Query/Writer.

MIGRATION DE CETTE CAPACITÉ

Importer clés par une procédure sécurisée sans export clair, préserver volumes et dépôts, puis recréer les espaces depuis des images sans JVM. Rendre chaque effet réconciliable.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Interrompre après persistance mais avant accès Git : la reprise complète le même espace sans doublon. Révoquer le développeur coupe les nouvelles sessions et retire les clés prévues.

Références de l'analyse

[monorepo](#) · [services/app-management/src/devspace/domain/service/create_devspace_service.py:96](#) Parcours SQL, clés, Git puis déploiement.

[monorepo](#) · [services/app-management/src/devspace/domain/service/create_devspace_service.py:207](#) Unicité et utilisateur.

[monorepo](#) · [services/app-management/src/devspace/application/router/devspace_router.py:51](#) Surface devspaces.

AM-07 Cible non qualifiée

Sessions de calcul Python et Spark pour process-data

COMPOTEMENT OBSERVÉ

Les routes privées start/delete/reconcile et jobs délèguent au SessionRuntimeServiceImpl. L'adaptateur rend des templates de pod pour Python, client Spark et serveur Spark Connect, applique services/deployments et réconcilie les objets observés. Le submit adresse le harness de la session ; le chemin concerné résout les credentials du projet avant l'appel. L'état et l'annulation utilisent le statement de cette session. Ces fonctions appartiennent au runtime de données bien qu'elles soient hébergées dans app-management.

UTILISATEURS ET RÔLES

Process-data et exploitant des runtimes.

INTERFACES DANS LE CODE

/private/sessions ; SessionRuntimeServiceImpl.start_sessions, start_connect_server, reconcile_clusters, submit_job, get_job_state, cancel_job.

DONNÉES ET ÉTATS

Ressources Kubernetes de sessions et serveurs ; statement du harness ; process-data conserve le run métier.

DÉPENDANCES ET ÉCHANGES

Kubernetes, templates runtime, credentials projet, Python/PySpark et Spark Connect.

COMPOTEMENT CIBLE PROPOSÉ

Runtimes Rust reprend cette frontière avec des workers fongibles et des tentatives bornées. Writer médie le staging ; Publisher contrôle la publication. Aucun serveur Spark ou composant JVM final.

MIGRATION DE CETTE CAPACITÉ

Conserver un adaptateur temporaire process-data puis déplacer admission, capacités et réconciliation dans Runtimes. Qualifier les scripts Python/SQL ; inventorier les incompatibilités PySpark.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Perdre la réponse du submit puis réconcilier ne produit pas deux publications. Une ancienne tentative ne peut écrire avec la capacité d'une nouvelle ; aucune permission REST générique table.modify ne parvient au worker.

Références de l'analyse

[monorepo · services/app-management/src/app_manager/application/router/private_session_router.py:37](#) Routes privées de sessions.

[monorepo · services/app-management/src/app_manager/infra/adapter/session/session_runtime_service_impl.py:213](#) Création des ressources runtime.

[monorepo · services/app-management/src/app_manager/infra/adapter/session/session_runtime_service_impl.py:453](#) Harness et credentials de projet.

[monorepo · services/app-management/src/app_manager/infra/adapter/session/session_runtime_service_impl.py:372](#) Inventaire et nettoyage.

AM-08 **Cible non qualifiée**

Catalogue d'applications agents et liens de communication

COMPOTEMENT OBSERVÉ

Un router dédié liste les applications agents et gère l'octroi/retrait d'accès entre applications. La création d'application valide chaque cible par la décision IAM de lecture car un lien peut ouvrir un accès réseau lors du déploiement. Les cibles inconnues, DEVSPACE ou refusées ne deviennent pas des liens valides. Cette relation

doit être traitée comme un changement de droit et de réseau, pas seulement comme une association d'affichage.

UTILISATEURS ET RÔLES

Auteur d'application, utilisateur métier et IAM.

INTERFACES DANS LE CODE

ApplicationAgentRouter.list_agents, grant_apps_access, revoke_app_access ; authorize_agent_targets.

DONNÉES ET ÉTATS

AppToAppLink, applications cibles et faits d'autorisation.

DÉPENDANCES ET ÉCHANGES

IAM, catalogue applicatif et NetworkPolicy rendu au déploiement.

COMPORTEMENT CIBLE PROPOSÉ

Applications/IAM/Egress Rust gèrent les consommateurs déclarés et relations d'appel. EXPOSURE décrit une consommation autorisée ; elle n'est pas un nouveau kind de publication.

MIGRATION DE CETTE CAPACITÉ

Importer les liens existants avec leur justification et vérifier droits sur chaque cible. Rendre la mise à jour réseau observable et révocable après retrait.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Le retrait d'un lien supprime le droit et converge vers la fermeture réseau. Un appelant ne peut déduire l'existence d'une application privée en essayant de la lier.

Références de l'analyse

[monorepo · services/app-management/src/app_manager/application/router/application_agent_router.py:55](#) Catalogue et opérations de liens.

[monorepo · services/app-management/src/app_manager/application/router/application_agent_router.py:183](#) Révocation explicite.

[monorepo · services/app-management/src/app_manager/domain/service/application/create_application_service.py:7](#) Validation des cibles avant création.

AM-09 Cible non qualifiée

Secrets, clés de build et limite de confiance des identités

COMPORTEMENT OBSERVÉ

Le service crypto chiffre les passwords basic-auth, credentials de registres, variables secrètes et clés SSH privées. Les builds internes et externes ont des secrets SSH distincts ; seule l'URL exacte du dépôt interne peut utiliser la clé du compte Gitea interne. Les clones écrivent leur URL dans les logs, ce qui interdit d'y embarquer des credentials. Le header X-Userinfo n'est fiable que derrière le proxy configuré qui l'écrase ; sinon l'identité vient du bearer. Ces contraintes sont documentées par le composant et doivent être préservées lors du remplacement du langage.

UTILISATEURS ET RÔLES

Application, builder, opérateur de secrets et gateway.

INTERFACES DANS LE CODE

CryptoService ; build_repository_init_container ; contrat TRUST_PROXY_HEADERS ; user keys router.

DONNÉES ET ÉTATS

Secrets chiffrés en SQL et secrets montés dans les jobs ; identité entrante séparée du propriétaire de ressource.

DÉPENDANCES ET ÉCHANGES

Gestion des clés, Kubernetes Secrets, Gitea, IAM/APISIX et registres.

COMPORTEMENT CIBLE PROPOSÉ

Applications/IAM/Egress Rust utilisent des références de secrets, rotation et délégations bornées. Les secrets de build interne restent inaccessibles aux dépôts externes ; pas d'algorithme crypto maison.

MIGRATION DE CETTE CAPACITÉ

Migrer les ciphertexts avec une procédure de rekey contrôlée et auditée. Tester les mêmes routes derrière et sans proxy ; ne jamais importer de secrets dans le dossier.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une URL externe ressemblant à Gitea n'obtient pas sa clé. Un faux X-Userinfo sur un pod qui ne fait pas confiance au proxy ne donne aucun droit. Les logs ne contiennent pas de credential.

Références de l'analyse

[monorepo](#) · [services/app-management/AGENTS.md:72](#) Contrat de chiffrement documenté.

[monorepo](#) · [services/app-management/AGENTS.md:79](#) Séparation des clés et risque du clone URL.

[monorepo](#) · [services/app-management/AGENTS.md:86](#) Limite de confiance du header.

AM-10 **Cible non qualifiée****Exploitation, événements et migrations avant déploiement****COMPORTEMENT OBSERVÉ**

Les consumers de build et déploiement attendent la fin des watchers, ce qui exige un poll interval adapté. Le composant documente que NACK_ON_ERROR est nécessaire pour conserver un message échoué, car le simple auto_commit=False ne donne pas cette garantie FastStream. Les migrations Alembic sont exécutées par un hook avant install/upgrade en mode migrate, pendant que les anciens pods peuvent encore servir. Les routes health distinguent live/ready et regardent notamment la base et IAM. La politique automatique de migrations du monorepo ne couvre pas cet arbre src/migrations.

UTILISATEURS ET RÔLES

Exploitant Kubernetes et équipe de livraison.

INTERFACES DANS LE CODE

Build/deployment consumers ; /health/live ; /health/ready ; hook migration-job ; Alembic.

DONNÉES ET ÉTATS

Offsets, exécutions longues, migrations et statut de disponibilité.

DÉPENDANCES ET ÉCHANGES

PostgreSQL, Kafka, Helm, IAM et Kubernetes.

COMPORTEMENT CIBLE PROPOSÉ

Migration/Applications Rust conservent Expand/Migrate/Contract, transactions d'événement et opérateurs repris après incident. La livraison reste par digest attesté ; tous les hooks/builders sont inclus dans le contrôle zéro JVM.

MIGRATION DE CETTE CAPACITÉ

Ajouter les migrations Rust à l'enforcement, transférer les messages non traités sans double propriétaire et qualifier l'arrêt/reprise des watchers. Conserver les checks de readiness utiles.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un échec de watcher ne perd pas l'événement. Le binaire précédent reste compatible pendant Expand ; un échec de migration bloque l'activation sans casser les pods servant encore.

Références de l'analyse

[monorepo](#) · [services/app-management/AGENTS.md:58](#) Hook et coexistence ancien schéma/binaire.

[monorepo](#) · [services/app-management/AGENTS.md:66](#) Acquittement et watchers.

[monorepo](#) · [services/app-management/src/app_manager/application/router/healthcheck_router.py:161](#) Disponibilité.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- Core Rust pour applications, dépôts, instances, devspaces et opérations suivies.
- Identité IAM, quotas Runtimes, révisions immuables, idempotence et reçus des effets.
- Séparer gestion applicative et sessions de calcul, actuellement réunies dans ce service.

Plan de données proposé

- Exécutants de build/déploiement encapsulés, aucun runtime JVM requis à l'état final.
- Espaces Python utilisateur isolés, volumes et tunnel ; workers de données sans autorité de publication.

Découpage du core Rust

- applications-api et registry de configuration.
- deployment-controller avec commande persistée, watcher et réconciliation.
- git-adapter, environment-build-controller et secret references.
- devspace-controller et tunnel-coordinator séparés du contenu utilisateur.

Identité, autorisation et isolation

- Préserver parenté des ressources, décisions sur les applications liées et usage des comptes de service.
- Séparer clés SSH internes/externes et conserver chiffrement/rotation des secrets.
- Un header proxy ne fait foi que dans la frontière de confiance explicitement configurée.

Exploitation et reprise

- Pré-install/pre-upgrade migrations Expand compatibles ; policy src/migrations à rendre effective.

- Réconcilier les opérations commit SQL puis événement et les watchers interrompus.
- Sauvegarder base, clés, dépôts et volumes avec procédure de restauration des effets déjà réalisés.

Séquence de migration

- Importer identités, parentés et historique sans changer les IDs.
- Ajouter commandes/reçus et comparer en observation passive.
- Qualifier builds et déploiements sans JVM, puis basculer par application.
- Déplacer sessions de calcul vers Runtimes et retirer les adaptateurs Python métier remplacés.

Écarts et décisions encore ouverts

- Idempotence sous réponse perdue de chaque effet Helm, Git et build à qualifier.
- Configurations clients, état réel des registres/volumes et performances non inspectés.
- Les citations AGENTS sur secrets et transport sont des contrats documentés ; cette mission n'a pas réaudité chaque branche du watcher.

Références de l'analyse

[monorepo](#) · [services/app-management/src/app_manager/domain/service/application/create_application_service.py:36](#) Cœur applicatif lu.

[monorepo](#) · [services/app-management/src/app_manager/infra/adapter/session/session_runtime_service_impl.py:132](#) Frontière runtime également présente.

SERVICE ACTUEL · DEVSPACE-AGENT

Agent sortant de tunnel des devspaces

3 capacités analysées

services/devspace-agent · Python asyncio, websockets et Pydantic Settings.

Relie un devspace au coordinateur par WebSocket sortant et transporte les sessions vers son sshd local.

Lecture de code Pas de qualification runtime nouvelle

Domaines cible : Runtimes, Applications, IAM, Audit.

Périmètre de lecture

- Agent.run et canaux contrôle/données.
- Bridge WebSocket/TCP et configuration ; présence de tests, non exécutés.

Objets et cycle de vie

- Configuration agent, sessionId, connexion contrôle et tâches de pont ; état éphémère.

Dépendances du service

- Coordinateur de tunnel authentifié, sshd local, fichier de token projeté.

Fonctionnalités et compatibilité à préserver

DSA-01 Cible non qualifiée

Enregistrement sortant et ouverture de sessions

COMPORTEMENT OBSERVÉ

L'agent ouvre un WebSocket vers agent/connect avec un bearer lu sur disque. Les messages de contrôle JSON de type session_request créent une tâche par sessionId ; les frames binaires ou mal formées sont ignorées. La session ouvre le sshd local puis un WebSocket agent/session/{sessionId}. Le token est relu à chaque connexion, ce qui permet de prendre en compte une rotation du fichier. Le code lu ne déduplique pas les sessionId avant de créer une tâche.

UTILISATEURS ET RÔLES

Devspace et coordinateur de tunnels.

INTERFACES DANS LE CODE

Agent._connect_and_serve ; Agent._serve_control_channel ; Agent._handle_session.

DONNÉES ET ÉTATS

Connexion de contrôle et ensemble de tâches locales ; aucune base persistante.

DÉPENDANCES ET ÉCHANGES

Coordinateur WSS, token projeté et sshd local.

COMPORTEMENT CIBLE PROPOSÉ

Runtimes/Applications Rust peut reprendre l'agent en Rust ou encapsuler le composant sans JVM transitoirement ; session, identité et durée sont explicitement bornées.

MIGRATION DE CETTE CAPACITÉ

Préserver le protocole binaire et les URLs durant migration du coordinateur ; ajouter limites de sessions et déduplication de demande.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Renvoyer deux fois la même demande ne crée pas deux tunnels logiques. Après rotation du token, la reconnexion utilise la nouvelle identité sans redémarrer le pod.

Références de l'analyse

[monorepo](#) · [services/devspace-agent/src/devspace_agent/agent.py:61](#) Enregistrement authentifié.

[monorepo](#) · [services/devspace-agent/src/devspace_agent/agent.py:74](#) Validation des frames et création de tâches.

[monorepo](#) · [services/devspace-agent/src/devspace_agent/agent.py:127](#) Relecture du token.

DSA-02 **Cible non qualifiée**

Transport des octets et fermeture du tunnel**COMPORTEMENT OBSERVÉ**

Le bridge lance deux pompes asyncio : frames WebSocket vers TCP avec drain, et lectures TCP par chunks vers WebSocket. Les frames textuelles sont encodées en octets ; les payloads vides sont ignorés. La fin d'une pompe entraîne l'annulation de l'autre puis la fermeture du writer TCP. Le code ne parse ni commande SSH ni contenu métier. Les connexions WebSocket emploient `max_size=None` dans l'agent ; des plafonds globaux doivent donc être évalués au niveau du tunnel.

UTILISATEURS ET RÔLES

Client SSH et sshd du devspace.

INTERFACES DANS LE CODE

bridge ; ws_to_tcp ; tcp_to_ws.

DONNÉES ET ÉTATS

Buffers et tâches éphémères ; flux chiffré SSH relayé comme octets.

DÉPENDANCES ET ÉCHANGES

Asyncio streams et WebSocket du coordinateur.

COMPORTEMENT CIBLE PROPOSÉ

Runtimes Rust offre un transport à backpressure avec budgets d'octets, durée et connexions, sans inspection métier du flux SSH.

MIGRATION DE CETTE CAPACITÉ

Qualifier EOF, fermeture distante, annulation et débit avant remplacement. Ne pas déduire la sémantique de half-close du seul commentaire : vérifier le comportement effectif.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un transfert binaire long conserve l'intégrité et borne la mémoire quand le lecteur ralentit. Une fermeture ou annulation libère les tâches et sockets sans tuer un autre tunnel.

Références de l'analyse

[monorepo](#) · [services/devspace-agent/src/devspace_agent/bridge.py:25](#) Pompes bidirectionnelles et fermeture.

[monorepo](#) · [services/devspace-agent/src/devspace_agent/agent.py:69](#) Absence de limite de frame dans les connexions lues.

DSA-03 **Cible non qualifiée****Reconnexion, arrêt et paramètres de l'agent****COMPOTEMENT OBSERVÉ**

La boucle reconnecte après erreur avec backoff exponentiel plafonné et jitter ; une connexion réussie réinitialise le délai. L'arrêt positionne un événement et annule les tâches de sessions connues. La configuration porte l'URL du coordinateur, la cible sshd, le chemin du token, les délais de reconnexion et les paramètres de healthcheck. Aucun journal de session durable n'est conservé dans ce composant.

UTILISATEURS ET RÔLES

Opérateur et orchestrateur du devspace.

INTERFACES DANS LE CODE

Agent.run ; Agent.stop ; AgentConfig.

DONNÉES ET ÉTATS

Backoff, événement d'arrêt, ensemble des tâches et configuration du processus.

DÉPENDANCES ET ÉCHANGES

Cycle de vie du pod, coordinateur et fichier de token.

COMPOTEMENT CIBLE PROPOSÉ

Applications/Runtimes Rust observe enregistrement, sessions actives et refus, avec reconnexion bornée et arrêt drainé. Audit journalise les ouvertures/fermetures côté autorité sans contenu SSH.

MIGRATION DE CETTE CAPACITÉ

Conserver la compatibilité des paramètres publics et mesurer les tempêtes de reconnexion ; décider explicitement les limites de durée et d'inactivité.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Redémarrer le coordinateur pour plusieurs devspaces ne crée pas une boucle serrée. L'arrêt ne laisse pas de session active sans propriétaire identifié.

Références de l'analyse

[monorepo](#) · [services/devspace-agent/src/devspace_agent/agent.py:35](#) Backoff et reprise.

[monorepo](#) · [services/devspace-agent/src/devspace_agent/agent.py:56](#) Arrêt et annulation des sessions.

[monorepo](#) · [services/devspace-agent/src/devspace_agent/config.py:4](#) Paramètres du processus.

Architecture et exploitation du service cible**Plan de contrôle Rust proposé**

- Autoriser chaque ouverture et révocation côté coordinateur ; bornes de durée et sessions.

Plan de données proposé

- Transport opaque WebSocket/TCP avec backpressure et ressources bornées.

Découpage du core Rust

- Agent Rust asynchrone et bibliothèque de bridge ; protocole compatible pendant transition.

Identité, autorisation et isolation

- Token relu sans être imprimé ; le tunnel transporte un accès SSH puissant à limiter au bon devspace.
- Les limites de frame/session doivent être explicites ; `max_size=None` est un état du code lu.

Exploitation et reprise

- Backoff avec jitter ; supervision enregistrement et sessions.
- Aucune donnée métier durable dans l'agent ; restauration par reconnexion.

Séquence de migration

- Qualifier protocole/EOF et token rotation.
- Canary d'un agent Rust avec même coordinateur.
- Remplacer les images et retirer le runtime Python de cet agent si la politique le prévoit.

Écarts et décisions encore ouverts

- La qualification de charge, half-close et révocation côté coordinateur n'a pas été exécutée.
- Aucun plafond de sessions actives constaté dans `Agent._spawn_session`.

Références de l'analyse

[monorepo](#) · [services/devspace-agent/src/devspace_agent/agent.py:29](#) Lifecycle agent.

[monorepo](#) · [services/devspace-agent/src/devspace_agent/bridge.py:25](#) Transport.

SERVICE ACTUEL · AI-GEN-PROXY

Proxy de modèles et contrôle des fournisseurs

5 capacités analysées

services/ai-gen-proxy · Python/FastAPI, HTTPX, PostgreSQL/Alembic, Kafka, Fernet et JWT.

Acheminer chat et embeddings vers les fournisseurs configurés, administrer leurs clés et observer l'usage.

Lecture de code

Pas de qualification runtime nouvelle

Domaines cible : AI, Egress, IAM, Audit, Embeddings.

Périmètre de lecture

- Analyses existantes 11-ml-embeddings et 14-front-api reprises comme contexte ; vérifications ciblées des routes, outils et implémentations citées ci-dessous.
- Instructions racine et AGENTS.md du composant lorsqu'il existe ; aucun binaire propriétaire inspecté.
- Lecture statique ciblée pour finalisation ; les branches non exécutées et le déploiement restent non qualifiés.

Objets et cycle de vie

- API keys chiffrées, AIModel, configurations par rôle, métriques et usages.

Dépendances du service

- Fournisseurs HTTP ; IAM/IdP ; PostgreSQL et Kafka.

Fonctionnalités et compatibilité à préserver

AIGP-01 Cible non qualifiée

Administrer les connexions fournisseurs et leurs clés chiffrées

COMPORTEMENT OBSERVÉ

Le plan de contrôle gère création, lecture, liste, mise à jour, suppression et existence des clés fournisseurs. Les secrets sont chiffrés Fernet au repos. La découverte peut réutiliser une clé persistée quand le client fournit un placeholder ou omet le secret. Les routes de gestion valident un JWT et vivent dans une application/port distincts de ceux du proxy.

UTILISATEURS ET RÔLES

Administrateurs de modèles.

INTERFACES DANS LE CODE

/api_keys ; POST /api_keys/discover ; control_app.

DONNÉES ET ÉTATS

API keys, fournisseurs, endpoints et modèles PostgreSQL.

DÉPENDANCES ET ÉCHANGES

IAM, IdP, Fernet, API fournisseur.

COMPORTEMENT CIBLE PROPOSÉ

AI/Egress Rust stocke des références de secrets et autorise chaque endpoint ; séparation réseau contrôle/données conservée.

MIGRATION DE CETTE CAPACITÉ

Migrer les enveloppes via un outil contrôlé puis révoquer les anciens secrets ; conserver les IDs et comparer les droits de gestion.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une clé masquée se modifie sans perte ; le listener de données refuse toutes les routes de gestion ; une destination interdite ne reçoit jamais la clé.

Références de l'analyse

monorepo · services/ai-gen-proxy/app/src/main.py:163 Séparation des applications.

monorepo · services/ai-gen-proxy/app/src/llm_metric/domain/service/discover_models_service.py:45 Réutilisation de connexion existante.

monorepo · services/ai-gen-proxy/AGENTS.md:63 Chiffrement déclaré et contraintes de cache.

AIGP-02 Cible non qualifiée

Acheminer le chat compatible OpenAI, avec streaming et sélection automatique

COMPORTEMENT OBSERVÉ

Le proxy expose le chat par nom technique de clé et une route automatique. Il résout un modèle et sa connexion, déchiffre la clé, transmet la requête et filtre des headers de réponse. Les branches streaming et non-streaming ont leur gestion de délai et métriques. Le streaming demande include_usage et ferme le flux HTTP partagé. Le mode automatique ordonne des modèles par préférence ; son comportement fournisseur doit être qualifié par scénario.

UTILISATEURS ET RÔLES

Applications IA et agents.

INTERFACES DANS LE CODE

POST /llm/{api_key_tech_name}/v1/chat/completions ; POST /llm/v1/chat/completions ; GET /llm/models.

DONNÉES ET ÉTATS

Configuration de modèles ; flux HTTP et métrique de requête.

DÉPENDANCES ET ÉCHANGES

Fournisseurs compatibles, clients HTTP mutualisés, service de métriques.

COMPORTEMENT CIBLE PROPOSÉ

Gateway AI Rust conserve contrat HTTP/SSE, annulation, budgets et choix de modèles autorisés ; Egress borne destinations et tailles.

MIGRATION DE CETTE CAPACITÉ

Rejouer fixtures streaming, erreurs, délais et sélection multi-fournisseur ; basculer par fournisseur.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un client annulé libère le flux ; un timeout retourne une erreur définie ; aucun changement de fournisseur ne viole les politiques de données.

Références de l'analyse

monorepo · services/ai-gen-proxy/app/src/llm_metric/application/router/llm_router.py:152 Sélection automatique.

monorepo · services/ai-gen-proxy/app/src/llm_metric/application/router/llm_router.py:256 Flux et collecte de métriques.

AIGP-03 Cible non qualifiée

Produire les embeddings via les modèles configurés

COMPORTEMENT OBSERVÉ

L'API liste les modèles d'embedding, expose les informations par clé et transmet une requête d'embeddings au fournisseur sélectionné. Le chemin emploie des retries et enregistre l'usage. Le proxy produit des vecteurs ; il ne publie pas une édition de corpus et ne constitue pas un moteur d'entraînement.

UTILISATEURS ET RÔLES

Ingestions documentaires et applications de recherche.

INTERFACES DANS LE CODE

GET /emb/models ; GET /emb/{api_key_tech_name}/info ; POST /emb/{api_key_tech_name}/v1/embeddings.

DONNÉES ET ÉTATS

Modèles et appels ; vecteurs retournés au consommateur.

DÉPENDANCES ET ÉCHANGES

API fournisseur, clé chiffrée, mesures d'usage.

COMPORTEMENT CIBLE PROPOSÉ

AI sert un contrat modèle/dimension/version ; Embeddings contrôle longueur, type numérique et provenance avant WAP.

MIGRATION DE CETTE CAPACITÉ

Qualifier batch, dimensions, ordre et erreurs de chaque fournisseur ; figer le modèle utilisé dans les éditions.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un batch garde son ordre et sa dimension ; un changement de modèle incompatible est refusé avant publication d'EMBEDDINGS.

Références de l'analyse

monorepo · services/ai-gen-proxy/app/src/llm_metric/application/router/embedding_router.py:114 Route de vecteurs.

monorepo · services/ai-gen-proxy/app/src/llm_metric/application/router/embedding_router.py:201 Reprises fournisseur.

AIGP-04 Cible non qualifiée

Choisir et sonder les modèles par défaut

COMPORTEMENT OBSERVÉ

La configuration par défaut comprend des rôles de modèles. Pour INSTRUCT et CODER, le service résout les modèles puis lance des probes concurrents avec délai, enregistre les métriques et refuse de sauvegarder si un probe refuse ou lève une erreur. Après succès il upsert les choix puis émet un événement de changement. Les paramètres de génération, dont reasoning_effort, font partie du modèle de configuration.

UTILISATEURS ET RÔLES

Administrateurs et initialisation plateforme.

INTERFACES DANS LE CODE

GET/POST /api_keys/configuration ; ConfigureDefaultModelsService.configure.

DONNÉES ET ÉTATS

DefaultModelConfiguration, rôles, références et résultats de probes.

DÉPENDANCES ET ÉCHANGES

Provider gateway, métriques et événements de configuration.

COMPORTEMENT CIBLE PROPOSÉ

AI Rust versionne le choix par rôle et son attestation de compatibilité ; une probe ne vaut pas qualification complète du modèle.

MIGRATION DE CETTE CAPACITÉ

Reprendre les choix et réglages ; exécuter les probes et tests de capacités avant promotion.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un modèle introuvable ou une probe refusée laisse la configuration antérieure ; tous les replicas observent le changement accepté.

Références de l'analyse

[monorepo · services/ai-gen-proxy/app/src/llm_metric/domain/service/configure_default_models_service.py:42](#) Probes, refus et sauvegarde.

[monorepo · services/ai-gen-proxy/app/src/llm_metric/domain/service/configure_default_models_service.py:75](#) Requête de sondage bornée.

AIGP-05 **Cible non qualifiée**

Mesurer l'usage, la latence et les erreurs des appels

COMPORTEMENT OBSERVÉ

Le proxy construit une métrique par appel à partir de l'usage fournisseur, du statut, de la durée et du temps au premier token lorsqu'il existe. Il publie les métriques via Kafka et les consomme pour les conserver. Le contrôle expose les modèles de métrique et un résumé de tokens avec filtres. Les écritures en arrière-plan de streaming doivent être distinguées d'une garantie comptable exhaustive.

UTILISATEURS ET RÔLES

Opérateurs et responsables de consommation.

INTERFACES DANS LE CODE

GET /metrics/models ; GET /metrics/tokens ; LLMMetricService.

DONNÉES ET ÉTATS

LLMMetric, usages de tokens, modèle et clé de provenance.

DÉPENDANCES ET ÉCHANGES

Kafka, PostgreSQL et payload d'usage fournisseur.

COMPORTEMENT CIBLE PROPOSÉ

Audit/AI Rust conserve un reçu d'usage corrélé à l'appel et des quotas explicites ; bus sans JVM.

MIGRATION DE CETTE CAPACITÉ

Reprendre métriques et clés ; comparer comptages sur erreurs, annulations et flux tronqués avant bascule.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un même appel rejoué pour transport ne double pas la facturation interne ; un usage absent est inconnu et non assimilé à zéro.

Références de l'analyse

[monorepo](#) · [services/ai-gen-proxy/app/src/llm_metric/application/router/llm_router.py:355](#) Champs de métrique.

[monorepo](#) · [services/ai-gen-proxy/app/src/llm_metric/application/router/llm_metric_router.py:38](#) Résumé des tokens.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- AI/Egress Rust stocke des références de secrets et autorise chaque endpoint ; séparation réseau contrôle/données conservée.
- Gateway AI Rust conserve contrat HTTP/SSE, annulation, budgets et choix de modèles autorisés ; Egress borne destinations et tailles.
- AI sert un contrat modèle/dimension/version ; Embeddings contrôle longueur, type numérique et provenance avant WAP.
- AI Rust versionne le choix par rôle et son attestation de compatibilité ; une probe ne vaut pas qualification complète du modèle.
- Audit/AI Rust conserve un reçu d'usage corrélé à l'appel et des quotas explicites ; bus sans JVM.

Plan de données proposé

- Flux chat et embeddings bornés, sans accès direct au catalogue de publication.

Découpage du core Rust

- ai-gateway, provider-registry, default-models et usage-ledger ; adaptation par fournisseur.

Identité, autorisation et isolation

- Deux listeners ; contrôle exposé par APISIX et JWT obligatoire.
- Secrets chiffrés, aucune valeur secrète incluse dans le dossier.

- Pas de cache local de clés/modèles sans invalidation inter-réplicas selon AGENTS.

Exploitation et reprise

- Clients HTTP partagés fermés par lifespan ; migrations et initialisation au démarrage web.
- Les tests requièrent Docker ; non exécutés.

Séquence de migration

- Migrer les enveloppes via un outil contrôlé puis révoquer les anciens secrets ; conserver les IDs et comparer les droits de gestion.
- Rejouer fixtures streaming, erreurs, délais et sélection multi-fournisseur ; basculer par fournisseur.
- Qualifier batch, dimensions, ordre et erreurs de chaque fournisseur ; figer le modèle utilisé dans les éditions.
- Reprendre les choix et réglages ; exécuter les probes et tests de capacités avant promotion.
- Reprendre métriques et clés ; comparer comptages sur erreurs, annulations et flux tronqués avant bascule.

Écarts et décisions encore ouverts

- Politique d'égress et garanties exactes de retry/fallback à qualifier par fournisseur.
- Le compteur d'usage ne prouve pas à lui seul un quota, un coût contractuel ni une remise exactement une fois.

Références de l'analyse

monorepo · [services/ai-gen-proxy/app/src/main.py:163](#) Séparation des applications.

monorepo · [services/ai-gen-proxy/app/src/llm_metric/application/router/llm_router.py:152](#) Sélection automatique.

monorepo · [services/ai-gen-proxy/app/src/llm_metric/application/router/embedding_router.py:114](#) Route de vecteurs.

monorepo · [services/ai-gen-proxy/app/src/llm_metric/domain/service/configure_default_models_service.py:42](#) Probes, refus et sauvegarde.

monorepo · [services/ai-gen-proxy/app/src/llm_metric/application/router/llm_router.py:355](#) Champs de métrique.

SERVICE ACTUEL · AI-GEN-ASSISTANT

Assistants, contextes et corpus RAG

6 capacités analysées

services/ai-gen-assistant · Python/FastAPI, PostgreSQL/SQLAlchemy/Alembic, Milvus, S3, Kafka et schedulers ; dépendances Python de génération.

Configurer les assistants et leurs contextes, converser avec outils et sources, ingérer les corpus et gérer les artefacts générés.

Lecture de code Pas de qualification runtime nouvelle

Domaines cible : AI, Embeddings, MCP, Flows, Writer, Publisher, Files, IAM, Audit, Runtimes.

Périmètre de lecture

- Analyses existantes 11-ml-embeddings et 14-front-api reprises comme contexte ; vérifications ciblées des routes, outils et implémentations citées ci-dessous.
- Instructions racine et AGENTS.md du composant lorsqu'il existe ; aucun binaire propriétaire inspecté.
- Lecture statique ciblée pour finalisation ; les branches non exécutées et le déploiement restent non qualifiés.

Objets et cycle de vie

- Contextes et versions ; conversations et messages.
- Corpus, sources, documents, métadonnées, chunks et transactions.
- Toolboxes, variables utilisateur, sessions de fichiers, templates et artefacts.

Dépendances du service

- IAM et outbox de droits ; proxy LLM/embeddings ; data-collect ; MCP ; PostgreSQL/S3/Milvus/Kafka.

Fonctionnalités et compatibilité à préserver

AIGA-01 Cible non qualifiée

Configurer un contexte d'assistant, ses ressources et son partage

COMPORTEMENT OBSERVÉ

Le contexte est rattaché à un projet et porte nom, description, instructions, personnalisation, ressources corpus/datasets/fichiers, toolboxes et choix de modèle avec réglages de génération. Les routes créent, listent et modifient ces configurations, leur statut et leur propriétaire. La création résout le miroir utilisateur avant écriture et canonicalise le projet. Des routes distinctes gèrent le partage. Les faits de création, propriété et partage passent par l'outbox IAM lorsque cette émission est activée.

UTILISATEURS ET RÔLES

Auteurs d’assistants, membres de projets et propriétaires.

INTERFACES DANS LE CODE

ContextRouter ; POST /projects/{project_id}/contexts ; ContextShareRouter.

DONNÉES ET ÉTATS

Context, ContextOwner, ConversationResources, références de sessions de fichiers, miroirs utilisateur/projet.

DÉPENDANCES ET ÉCHANGES

IAM, PostgreSQL, modèles du proxy, événements de droits.

COMPORTEMENT CIBLE PROPOSÉ

AI Rust gère les configurations versionnées et les mandats de ressources ; IAM décide partage et propriété. Les ressources référencent des éditions autorisées.

MIGRATION DE CETTE CAPACITÉ

Importer UUID, propriétaires et références, reconstruire les faits IAM puis comparer les accès avant bascule.

CRITÈRE D’ACCEPTATION À EXÉCUTER

Créer un contexte avec trois types de ressource, le partager puis révoquer ; les autres projets ne découvrent aucune ressource interdite.

Références de l’analyse

[monorepo · services/ai-gen-assistant/app/src/context/application/router/context_router.py:103](#) Création et modèle de configuration.

[monorepo · services/ai-gen-assistant/AGENTS.md:49](#) Faits IAM et outbox transactionnelle.

AIGA-02 Cible non qualifiée

Historiser, comparer et publier les versions de contexte

COMPORTEMENT OBSERVÉ

Le service expose liste et détail des versions, diff, duplication, publication d’une version et suppression simple ou multiple. Une lecture peut résoudre la configuration effective d’une version sans muter le contexte courant. La publication citée ici concerne la configuration de l’assistant ; elle ne démontre aucune publication de données Iceberg.

UTILISATEURS ET RÔLES

Auteurs et responsables de contextes.

INTERFACES DANS LE CODE

ContextVersionRouter : list_context_versions, diff_context_version, duplicate_context_version, publish_context_version.

DONNÉES ET ÉTATS

Versions de contexte et configuration effective ; pagination et auteurs résolus.

DÉPENDANCES ET ÉCHANGES

PostgreSQL, service de versions et miroir d’identité.

COMPORTEMENT CIBLE PROPOSÉ

AI conserve une version de plan immuable ; Publisher reste seul propriétaire des publications de données déclenchées par un plan.

MIGRATION DE CETTE CAPACITÉ

Reprendre versions et diffs ; conserver une distinction de contrat et de vocabulaire entre configuration publiée et édition de données approuvée.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Lire une ancienne version sans changer la tête ; publier une configuration ne rend aucun candidat de données visible.

Références de l'analyse

[monorepo · services/ai-gen-assistant/app/src/context/application/router/context_version_router.py:148](#) Comparaison des versions.

[monorepo · services/ai-gen-assistant/app/src/context/application/router/context_router.py:65](#) Résolution explicite sans mutation.

AIGA-03 Cible non qualifiée

Ouvrir et reprendre les conversations avec streaming

COMPORTEMENT OBSERVÉ

Les conversations existent pour un assistant et pour un contexte. Les routes listent et consultent les messages, créent un tour, reprennent un échange, modifient les propriétés ou suppriment la conversation. Les créations et reprises utilisent une réponse en streaming ; l'absence de modèle instruct par défaut a une réponse dédiée. Une API publique de contextes expose aussi ces parcours avec une dépendance de contrôle d'accès au contexte. Les artefacts graphiques générés possèdent leur propre téléchargement.

UTILISATEURS ET RÔLES

Utilisateurs de chat, applications clientes et assistants intégrés.

INTERFACES DANS LE CODE

ConversationRouter ; ContextConversationRouter ; PublicContextRouter /v1/contexts/{context_id}.

DONNÉES ET ÉTATS

Conversations, messages, ressources de conversation et artefacts ; état en PostgreSQL/S3 selon l'objet.

DÉPENDANCES ET ÉCHANGES

AI proxy, IAM, toolboxes/MCP, stockage d'artefacts.

COMPORTEMENT CIBLE PROPOSÉ

AI Rust orchestre des tours bornés, annulation et budgets ; messages et citations portent la version de contexte et les éditions effectivement lues.

MIGRATION DE CETTE CAPACITÉ

Migrer historiques et identifiants ; qualifier la compatibilité des événements streaming et les reprises avant bascule des clients.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une interruption reprend selon un identifiant de tour sans lancer deux effets ; une révocation du contexte bloque lecture et nouveau tour.

Références de l'analyse

[monorepo · services/ai-gen-assistant/app/src/assistant/application/router/conversation_router.py:207](#) Création et streaming.

[monorepo](#) · [services/ai-gen-assistant/app/src/public_api/public_context_router.py:129](#) Dépendance commune d'accès au contexte public.

AIGA-04 Cible non qualifiée

Définir les corpus et exécuter leurs ingestions documentaires

COMPORTEMENT OBSERVÉ

Les corpus sont créés et listés dans un projet, avec recherche d'existence, documents, métadonnées et configuration de refresh. Les transactions corpus/source/document suivent l'ingestion et la suppression. La boucle source utilise une ligne source_transaction_progress créée avec la transaction, revendiquée sous bail et renouvelée ; un autre worker peut reprendre depuis le curseur validé après disparition du propriétaire. La concurrence des workers est configurée dans le scheduler. Les documents possèdent aussi métadonnées et états de transaction consultables.

UTILISATEURS ET RÔLES

Gestionnaires de connaissances, auteurs de corpus et opérateurs.

INTERFACES DANS LE CODE

CorpusRouter ; DocumentRouter ; CorpusTransactionRouter ; SourceTransactionLoopScheduler.

DONNÉES ET ÉTATS

Corpus, Source, Document, transactions et curseurs de progression PostgreSQL.

DÉPENDANCES ET ÉCHANGES

Data-collect pour les sources, stockage objet, extraction de documents, modèle d'embedding.

COMPORTEMENT CIBLE PROPOSÉ

Flows/AI/Embeddings utilisent des runs bornés. Writer prépare un FILESET et des EMBEDDINGS immuables ; Publisher publie sous WAP avant acquittement de la plage source.

MIGRATION DE CETTE CAPACITÉ

Importer les documents et leurs provenances comme éditions auditées ; reprendre les travaux en attente avec correspondance explicite entre ancien curseur et plage source.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Tuer un worker pendant une tranche ; reprendre sans perdre les documents et ne faire avancer le checkpoint publié qu'après publication approuvée.

Références de l'analyse

[monorepo](#) · [services/ai-gen-assistant/app/src/corpus/application/router/corpus_router.py:111](#) Surface de corpus et contrôle projet.

[monorepo](#) · [services/ai-gen-assistant/app/src/corpus/application/scheduler/source_transaction_loop_scheduler.py:57](#) Concurrence, bail et reprise de source.

AIGA-05 Cible non qualifiée

Indexer les chunks et rechercher les corpus autorisés

COMPORTEMENT OBSERVÉ

La recherche publique liste les corpus accessibles puis recherche des documents. Le stockage de chunks repose sur Milvus. Le remplacement écrit les nouveaux chunks puis retire les anciens ; le commentaire et le code indiquent l'absence de transaction Milvus, avec une période possible de duplication. En perte de propriété, le nettoyage cible les identifiants écrits par ce worker. La dimension des vecteurs est actuellement sondée par une requête au modèle.

UTILISATEURS ET RÔLES

Utilisateurs RAG, assistants et MCP corpus.

INTERFACES DANS LE CODE

PublicCorpusRouter list_corpora/search_corpora ; ChunkAdapter.replace_document_chunks.

DONNÉES ET ÉTATS

Collections Milvus, documents, IDs de chunks et watermark de génération.

DÉPENDANCES ET ÉCHANGES

Milvus, proxy embeddings, corpus en PostgreSQL, autorisation IAM.

COMPORTEMENT CIBLE PROPOSÉ

EMBEDDINGS Iceberg conserve vecteurs, hash du contenu, édition source, découpage, modèle et dimension. L'index Milvus ou son remplaçant devient une projection reconstruisible par édition approuvée.

MIGRATION DE CETTE CAPACITÉ

Écrire une génération candidate, auditer dimensions et droits, publier puis construire l'index privé avant bascule de lecture ; ne pas assimiler l'alias d'index à la vérité publiée.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Changer de modèle/dimension, perdre l'index et le reconstruire ; aucune recherche ne mélange deux éditions ni ne restitue un document révoqué.

Références de l'analyse

[monorepo · services/ai-gen-assistant/app/src/corpus/infra/adapter/storage/chunk_adapter.py:308](#) Ordre écriture/nettoyage et absence de transaction.

[monorepo · services/ai-gen-assistant/app/src/corpus/infra/adapter/storage/chunk_adapter.py:362](#) Sondage de dimension.

[monorepo · services/ai-gen-assistant/app/src/public_api/public_corpus_router.py:61](#) Recherche publique.

AIGA-06 **Cible non qualifiée**

Outils des assistants et gérer fichiers, modèles de documents et exports

COMPORTEMENT OBSERVÉ

Les toolboxes exposent leurs outils et variables utilisateur, ainsi que leur instance. Le service gère des sessions S3 pour upload/suppression de fichiers. Il reçoit des modèles de documents, les liste, les prévisualise, les modifie et les archive. Les documents générés proposent consultation, formats exportables et téléchargement ; les graphiques ont un export dédié. Des outils de lecture de fichiers, génération de documents/PPTX, graphiques et exécution Python sont présents ; cette lecture n'établit pas à elle seule leur isolation runtime.

UTILISATEURS ET RÔLES

Auteurs d'assistants, utilisateurs de pièces jointes et administrateurs d'outils.

INTERFACES DANS LE CODE

ToolboxRouter ; S3FileSessionRouter ; DocumentTemplateRouter ; GeneratedDocumentRouter ;
GeneratedChartRouter.

DONNÉES ET ÉTATS

Toolbox/Tool, variables par utilisateur, sessions de fichiers, modèles, documents et graphiques générés.

DÉPENDANCES ET ÉCHANGES

MCP, S3, moteur de génération et runtime Python.

COMPORTEMENT CIBLE PROPOSÉ

AI/Runtimes exécutent les outils sous mandat et budgets ; Files contrôle les pièces jointes. Un EXPORT est un effet à reçu, un artefact durable peut être un FILESET publié via Writer/Publisher.

MIGRATION DE CETTE CAPACITÉ

Migrer métadonnées et objets avec hashes ; qualifier isolation Python, droits d'outils, formats et suppression des artefacts avant transfert.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un outil n'accède qu'aux fichiers du mandat ; le même export rejoué retrouve son reçu ; une suppression retire objet, index et liens conservés selon politique.

Références de l'analyse

[monorepo · services/ai-gen-assistant/app/src/assistant/application/router/toolbox_router.py:211](#) Variables de toolbox utilisateur.

[monorepo · services/ai-gen-assistant/app/src/assistant/application/router/document_template_router.py:47](#) Cycle de vie des templates.

[monorepo · services/ai-gen-assistant/app/src/assistant/application/router/generated_document_router.py:115](#) Formats et téléchargement.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- AI Rust gère les configurations versionnées et les mandats de ressources ; IAM décide partage et propriété. Les ressources référencent des éditions autorisées.
- AI conserve une version de plan immuable ; Publisher reste seul propriétaire des publications de données déclenchées par un plan.
- AI Rust orchestre des tours bornés, annulation et budgets ; messages et citations portent la version de contexte et les éditions effectivement lues.
- Flows/AI/Embeddings utilisent des runs bornés. Writer prépare un FILESET et des EMBEDDINGS immuables ; Publisher publie sous WAP avant acquittement de la page source.
- EMBEDDINGS Iceberg conserve vecteurs, hash du contenu, édition source, découpage, modèle et dimension. L'index Milvus ou son remplaçant devient une projection reconstruisible par édition approuvée.
- AI/Runtimes exécutent les outils sous mandat et budgets ; Files contrôle les pièces jointes. Un EXPORT est un effet à reçu, un artefact durable peut être un FILESET publié via Writer/Publisher.

Plan de données proposé

- Ingestion et embedding bornés sous Writer ; recherche sur index construit depuis une édition approuvée ; conversation avec tours et outils bornés.

Découpage du core Rust

- ai-contexts, ai-conversations, ai-tools, corpus-runs et projections embeddings ; calcul Python utilisateur isolé derrière Runtimes.

Identité, autorisation et isolation

- Faits IAM émis sous indicateur ; sujets historiques compatibles Keycloak à migrer.
- Les documents externes et résultats d'outils sont des données non fiables ; jamais une autorité pour élargir le mandat.
- Un index ne doit pas exposer textes ou nombres de candidats interdits.

Exploitation et reprise

- Migrations séparées du démarrage web, hook de préparation et mode migrate ; Alembic hors enforcement migrations racine.
- Boucles sources/documents, leases, outbox et replay à reprendre ; tests Docker non exécutés.
- AGENTS signale un décalage du paquet cleyrop_common publié et des imports monorepo ; ne pas présumer le démarrage qualifié.

Séquence de migration

- Importer UUID, propriétaires et références, reconstruire les faits IAM puis comparer les accès avant bascule.
- Reprendre versions et diffs ; conserver une distinction de contrat et de vocabulaire entre configuration publiée et édition de données approuvée.
- Migrer historiques et identifiants ; qualifier la compatibilité des événements streaming et les reprises avant bascule des clients.
- Importer les documents et leurs provenances comme éditions auditées ; reprendre les travaux en attente avec correspondance explicite entre ancien curseur et plage source.
- Écrire une génération candidate, auditer dimensions et droits, publier puis construire l'index privé avant bascule de lecture ; ne pas assimiler l'alias d'index à la vérité publiée.
- Migrer métadonnées et objets avec hashes ; qualifier isolation Python, droits d'outils, formats et suppression des artefacts avant transfert.

Écarts et décisions encore ouverts

- Compatibilité complète des outils Python, documents et formats à mesurer ; isolation non qualifiée ici.
- La profondeur de ce lot final est ciblée et ne remplace pas un audit ligne par ligne de chaque outil.
- Qualifier les dépendances des images Milvus et du bus pour ne conserver aucune JVM.

Références de l'analyse

[monorepo](#) · [services/ai-gen-assistant/app/src/context/application/router/context_router.py:103](#) Création et modèle de configuration.

[monorepo](#) · [services/ai-gen-assistant/app/src/context/application/router/context_version_router.py:148](#) Comparaison des versions.

[monorepo](#) · [services/ai-gen-assistant/app/src/assistant/application/router/conversation_router.py:207](#) Création et streaming.

[monorepo](#) · [services/ai-gen-assistant/app/src/corpus/application/router/corpus_router.py:111](#) Surface de corpus et contrôle projet.

[monorepo](#) · [services/ai-gen-assistant/app/src/corpus/infra/adapter/storage/chunk_adapter.py:308](#) Ordre écriture/nettoyage et absence de transaction.

[monorepo](#) · [services/ai-gen-assistant/app/src/assistant/application/router/toolbox_router.py:211](#) Variables de toolbox utilisateur.

SERVICE ACTUEL · MCP-SERVER-DATASET

Outils MCP d'analyse des datasets

5 capacités analysées

services/mcp-server-dataset · Python/FastMCP, clients HTTP, SQLGlot et orchestration LLM ; dépendance actuelle data-serve/Trino.

Découvrir schémas et métadonnées puis répondre en SQL contrôlé à des questions en langage naturel.

Lecture de code

Pas de qualification runtime nouvelle

Domaines cible : MCP, Query, Catalog, IAM, AI, Audit.

Périmètre de lecture

- Analyses existantes 11-ml-embeddings et 14-front-api reprises comme contexte ; vérifications ciblées des routes, outils et implémentations citées ci-dessous.
- Instructions racine et AGENTS.md du composant lorsqu'il existe ; aucun binaire propriétaire inspecté.
- Lecture statique ciblée pour finalisation ; les branches non exécutées et le déploiement restent non qualifiés.

Objets et cycle de vie

- Schémas, métadonnées, questions, SQL, résultats et historiques d'essais.

Dépendances du service

- Data-serve, modèle de génération, IAM et délégation MCP.

Fonctionnalités et compatibilité à préserver

MCPD-01 Cible non qualifiée

Découvrir les datasets et sélectionner les structures pertinentes

COMPORTEMENT OBSERVÉ

Les outils listent les datasets, recherchent leurs structures et proposent métadonnées et schéma pour un dataset précis. La découverte lit l'API data-serve paginée. Les noms ou identifiants sont résolus dans le scope autorisé ; absence de données, credential absent et refus backend ont des formes d'erreur distinctes.

UTILISATEURS ET RÔLES

Agents d'analyse et utilisateurs.

INTERFACES DANS LE CODE

list_datasets_tool ; dataset_structure_research_tool ; get_dataset_metadata_tool ; get_dataset_schema_tool.

DONNÉES ET ÉTATS

Catalogue temporaire de schémas, exemples et métadonnées.

DÉPENDANCES ET ÉCHANGES

Data-serve ; scopes MCP et identité.

COMPORTEMENT CIBLE PROPOSÉ

MCP/Catalog Rust sert découverte et schémas avec édition et droits de découverte explicites.

MIGRATION DE CETTE CAPACITÉ

Adapter chaque endpoint metadata/structured et préserver la pagination ; introduire pins d'édition dans le résultat.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un dataset hors scope n'apparaît ni dans le choix ni dans les suggestions ; toutes les pages utilisent la même édition.

Références de l'analyse

[monorepo](#) · [services/mcp-server-dataset/src/tools/dataset_structure_researcher.py:688](#) Découverte des datasets.

[monorepo](#) · [services/mcp-server-dataset/src/tools/dataset_structure_researcher.py:1050](#) Outil de schéma ciblé.

MCPD-02 **Cible non qualifiée**

Transformer une question en SQL de lecture vérifié

COMPORTEMENT OBSERVÉ

database_research_tool récupère des schémas et exemples puis appelle un générateur SQL. La chaîne travaille actuellement en dialecte Trino/Cleyrop, vérifie que la requête est en lecture seule et détecte les colonnes absentes avant EXPLAIN et exécution. Le SQL généré est donc une proposition soumise à plusieurs contrôles et non une autorité sur les droits.

UTILISATEURS ET RÔLES

Analystes et agents conversationnels.

INTERFACES DANS LE CODE

MCP database_research_tool ; data-serve /datasets/sql/query ; EXPLAIN.

DONNÉES ET ÉTATS

Question, schémas récupérés, SQL et résultat structuré.

DÉPENDANCES ET ÉCHANGES

LLM, SQLGlot/dialecte Cleyrop, Trino via data-serve.

COMPORTEMENT CIBLE PROPOSÉ

MCP/Query Rust remplace Trino ; le générateur reçoit le catalogue de capacités réel du moteur sans JVM et des entrées épinglées.

MIGRATION DE CETTE CAPACITÉ

Établir un corpus NL/SQL et requalifier fonctions, types, agrégats et erreurs ; aucune parité Trino implicite.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une requête d'écriture ou colonne hors schéma est refusée avant exécution ; une lecture rejoint exactement les éditions déclarées.

Références de l'analyse

[monorepo](#) · [services/mcp-server-dataset/src/tools/database_researcher.py:509](#) Chaîne génération, contrôle de schéma et exécution.

[monorepo](#) · [services/mcp-server-dataset/src/tools/database_researcher.py:1479](#) Contrôle de lecture seule.

MCPD-03 Cible non qualifiée

Réparer les erreurs SQL avec un budget et une mémoire de session

COMPORTEMENT OBSERVÉ

Une erreur transitoire rejoue le même SQL sans nouveau LLM ; une erreur déterministe peut provoquer une régénération. Les essais rejetés sont transmis au générateur et conservés en mémoire de session liée au caller et à la conversation. La répétition sans progrès arrête la boucle. Un budget partagé borne les réparations de parsing et les nouvelles générations ; les échecs connus reviennent sous forme structurée.

UTILISATEURS ET RÔLES

Agents autonomes et opérateurs de coûts.

INTERFACES DANS LE CODE

`_validate_and_execute_sql` ; `_run_with_transient_replay` ; `session_memory`.

DONNÉES ET ÉTATS

Journal d'essais, classe d'erreur et mémoire de session ; pas de checkpoint de données.

DÉPENDANCES ET ÉCHANGES

Backend SQL, générateur LLM et configuration de budgets.

COMPORTEMENT CIBLE PROPOSÉ

MCP Rust garde un budget global temps/tokens/exécutions et une mémoire isolée par mandat ; Audit corrèle les tentatives.

MIGRATION DE CETTE CAPACITÉ

Rejouer erreurs temporaires, syntaxiques et répétitives ; mesurer appels au LLM et arrêter les boucles improductives.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une panne réseau ne déclenche pas une régénération ; deux échecs identiques arrêtent selon la règle ; une autre identité ne reçoit pas les essais de la session.

Références de l'analyse

[monorepo](#) · [services/mcp-server-dataset/src/tools/database_researcher.py:388](#) Replay transitoire.

[monorepo](#) · [services/mcp-server-dataset/src/tools/database_researcher.py:509](#) Budgets partagés et mémoire des essais.

MCPD-04 Cible non qualifiée

Restituer les résultats SQL et expliquer les réponses vides

COMPORTEMENT OBSERVÉ

Les résultats ont une représentation structurée, Markdown et CSV, avec erreurs typées et rendu de null. Le chemin de recherche peut signaler un résultat vide, effectuer un comptage et rechercher des correspondances proches. Les endpoints de schéma et métadonnées ont une gestion d'erreur commune ; un

échec partiel ne doit pas être pris pour un dataset absent.

UTILISATEURS ET RÔLES

Agents et interfaces de résultats.

INTERFACES DANS LE CODE

SqlQueryResult.to_markdown/to_csv ; _signal_empty_result ; get_dataset_structure/get_dataset_metadata.

DONNÉES ET ÉTATS

Lignes, colonnes, détails d'erreur, tentatives et suggestions.

DÉPENDANCES ET ÉCHANGES

Data-serve et outils de rapprochement textuel.

COMPORTEMENT CIBLE PROPOSÉ

Query/MCP Rust fournit réponses paginées et exports avec même pin d'édition ; EXPORT est un effet avec reçu quand conservé.

MIGRATION DE CETTE CAPACITÉ

Comparer CSV, nulls, résultats vides et limites ; qualifier les probes de suggestions sans exposer de données hors droit.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un agrégat nul n'est pas confondu avec zéro ligne ; pagination et export gardent les mêmes données et autorisations.

Références de l'analyse

[monorepo](#) · [services/mcp-server-dataset/src/dataserve/data_server.py:263](#) Contrat de résultat et rendus.

[monorepo](#) · [services/mcp-server-dataset/src/tools/database_researcher.py:823](#) Signalement des réponses vides.

MCPD-05 **Cible non qualifiée**

Isoler le credential et le périmètre de lecture de chaque appel

COMPORTEMENT OBSERVÉ

Le credential entrant accepte le Bearer, sans fallback vers les anciens headers API-token. Une délégation exige fresh_headers et utilise un backend interne plutôt que le parcours public normal. Les outils vérifient le scope des datasets ; le serveur monte les outils de recherche SQL et de structure sur MCP HTTP et propage les traces.

UTILISATEURS ET RÔLES

Orchestrateur, IAM et clients MCP.

INTERFACES DANS LE CODE

UserCredential ; get_user_credential ; FastMCP server.

DONNÉES ET ÉTATS

Mandat/credential par requête ; contextes de transport et scope.

DÉPENDANCES ET ÉCHANGES

cleyrop_common.mcp_read, IAM et data-serve interne.

COMPORTEMENT CIBLE PROPOSÉ

MCP/IAM Rust limite chaque mandat à lecture des éditions nommées ; le moteur SQL n'obtient aucun droit générique table.modify.

MIGRATION DE CETTE CAPACITÉ

Migrer Bearer et délégation avec tests croisés d'identité, expiration et scopes.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un ancien header seul ne donne aucun droit ; une délégation révoquée bloque metadata, EXPLAIN et exécution.

Références de l'analyse

[monorepo](#) · [services/mcp-server-dataset/src/utils/user_token.py:28](#) Headers frais et endpoint délégué.

[monorepo](#) · [services/mcp-server-dataset/src/utils/user_token.py:58](#) Bearers admis et absence de fallback.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- MCP/Catalog Rust sert découverte et schémas avec édition et droits de découverte explicites.
- MCP/Query Rust remplace Trino ; le générateur reçoit le catalogue de capacités réel du moteur sans JVM et des entrées épinglées.
- MCP Rust garde un budget global temps/tokens/exécutions et une mémoire isolée par mandat ; Audit corréle les tentatives.
- Query/MCP Rust fournit réponses paginées et exports avec même pin d'édition ; EXPORT est un effet avec reçu quand conservé.
- MCP/IAM Rust limite chaque mandat à lecture des éditions nommées ; le moteur SQL n'obtient aucun droit générique table.modify.

Plan de données proposé

- Découverte et requêtes de lecture bornées ; exécution dans Query autorisé.

Découpage du core Rust

- mcp-dataset, sql-planner-guard, attempt-budget et result-rendering.

Identité, autorisation et isolation

- SQL de lecture et preflight de schéma ; contrôle serveur d'accès toujours nécessaire.
- Credentials sans repr et scope fixé indépendamment des suggestions du LLM.

Exploitation et reprise

- Budgets de génération/exécution et erreurs structurées ; mémoire locale de session non assimilée à journal durable.

- Tests de dialecte, erreurs et délégation présents, non exécutés.

Séquence de migration

- Adapter chaque endpoint metadata/structured et préserver la pagination ; introduire pins d'édition dans le résultat.
- Établir un corpus NL/SQL et requalifier fonctions, types, agrégats et erreurs ; aucune parité Trino implicite.
- Rejouer erreurs temporaires, syntaxiques et répétitives ; mesurer appels au LLM et arrêter les boucles improductives.
- Comparer CSV, nulls, résultats vides et limites ; qualifier les probes de suggestions sans exposer de données hors droit.
- Migrer Bearer et délégation avec tests croisés d'identité, expiration et scopes.

Écarts et décisions encore ouverts

- Le dialecte et les fonctions Trino doivent être requalifiés sur le moteur sans JVM.
- Le contrat actuel n'établit pas un pin d'édition Iceberg ni une reprise de résultat après panne de serveur.

Références de l'analyse

[monorepo](#) · [services/mcp-server-dataset/src/tools/dataset_structure_researcher.py:688](#) Découverte des datasets.

[monorepo](#) · [services/mcp-server-dataset/src/tools/database_researcher.py:509](#) Chaîne génération, contrôle de schéma et exécution.

[monorepo](#) · [services/mcp-server-dataset/src/tools/database_researcher.py:388](#) Replay transitoire.

[monorepo](#) · [services/mcp-server-dataset/src/dataserve/data_server.py:263](#) Contrat de résultat et rendus.

[monorepo](#) · [services/mcp-server-dataset/src/utils/user_token.py:28](#) Headers frais et endpoint délégué.

SERVICE ACTUEL · MCP-SERVER-CORPUS

Outils MCP de recherche de corpus

4 capacités analysées

services/mcp-server-corpus · Python/FastMCP, aiohttp, Pydantic et OpenTelemetry.

Offrir découverte et recherche documentaire autorisées aux agents sans dupliquer le backend RAG.

Lecture de code Pas de qualification runtime nouvelle

Domaines cible : MCP, AI, Embeddings, IAM, Audit.

Périmètre de lecture

- Analyses existantes 11-ml-embeddings et 14-front-api reprises comme contexte ; vérifications ciblées des routes, outils et implémentations citées ci-dessous.
- Instructions racine et AGENTS.md du composant lorsqu'il existe ; aucun binaire propriétaire inspecté.
- Lecture statique ciblée pour finalisation ; les branches non exécutées et le déploiement restent non qualifiés.

Objets et cycle de vie

- CorpusListResult et CorpusSearchResult ; mandat de lecture et scope par requête.

Dépendances du service

- ai-gen-assistant public/interne ; cleyrop_common.mcp_read ; IAM via délégation.

Fonctionnalités et compatibilité à préserver

MCP-01 Cible non qualifiée

Découvrir les corpus accessibles dans le périmètre de l'agent

COMPORTEMENT OBSERVÉ

list_corpora_tool appelle l'API de corpus pour l'utilisateur courant, puis filtre selon les noms techniques imposés au contexte MCP. Le résultat contient UUID, nom, description, nombre de documents et total ; des erreurs structurées distinguent refus d'accès et défaillance.

UTILISATEURS ET RÔLES

Agents MCP et utilisateurs RAG.

INTERFACES DANS LE CODE

MCP list_corpora_tool.

DONNÉES ET ÉTATS

Résultat de découverte éphémère, sans catalogue local autoritaire.

DÉPENDANCES ET ÉCHANGES

API publique ai-gen-assistant et mandat de lecture.

COMPORTEMENT CIBLE PROPOSÉ

MCP Rust expose uniquement les corpus autorisés et épinglés ; Catalog/Embeddings restent les autorités.

MIGRATION DE CETTE CAPACITÉ

Conserver schéma et noms des outils ; remplacer le backend par un adaptateur aux éditions publiées.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un contexte limité à A ne voit ni nom ni total de B ; IAM indisponible ne devient pas une liste complète.

Références de l'analyse

[monorepo](#) · [services/mcp-server-corpus/src/tool/corpus_searcher.py:226](#) Découverte et filtrage du périmètre.

MCPC-02 Cible non qualifiée

Chercher sémantiquement dans un ou plusieurs corpus

COMPORTEMENT OBSERVÉ

L'outil accepte une question et des noms techniques. Le scope de contexte remplace l'argument du modèle. Il résout les noms en UUID parmi les corpus accessibles, refuse les inconnus puis appelle search_corpora. Sans sélection, il cherche dans les corpus accessibles. Le résultat contient documents et métadonnées ; les champs de texte sont nettoyés des caractères invisibles et espaces exotiques.

UTILISATEURS ET RÔLES

Agents et utilisateurs qui interrogent leurs connaissances.

INTERFACES DANS LE CODE

MCP corpus_search_tool ; client POST /corpora/search.

DONNÉES ET ÉTATS

Documents retournés, total et erreur ; aucune écriture de vecteurs ici.

DÉPENDANCES ET ÉCHANGES

Backend de recherche assistant, droits IAM et données de corpus.

COMPORTEMENT CIBLE PROPOSÉ

MCP restitue citations, hashes et édition servie ; Embeddings vérifie les droits avant texte et avant envoi au LLM.

MIGRATION DE CETTE CAPACITÉ

Préserver nettoyage et erreurs ; ajouter provenance d'édition et limites de résultats sans inventer un checkpoint dans MCP.

CRITÈRE D'ACCEPTATION À EXÉCUTER

La question ne peut élargir le scope fixé ; une citation reste résoluble sur l'édition lue et disparaît de la restitution après révocation.

Références de l'analyse

[monorepo](#) · [services/mcp-server-corpus/src/tool/corpus_searcher.py:286](#) Résolution de noms et recherche.

[monorepo](#) · [services/mcp-server-corpus/src/tool/corpus_searcher.py:188](#) Normalisation du texte.

MCPC-03 Cible non qualifiée

Propager une identité ou un mandat de lecture délégué

COMPORTEMENT OBSERVÉ

L'identité vient du Bearer entrant. Le client sait aussi porter DelegatedRead, appelle son autorisation et obtient des headers frais pour le backend interne. Les erreurs MCPReadAuthenticationError exposent code et possibilité de retry. La session HTTP partagée désactive la persistance des cookies pour éviter qu'un cookie reçu pour un utilisateur soit renvoyé pour un autre.

UTILISATEURS ET RÔLES

Orchestrateur d'assistant, IAM et utilisateurs.

INTERFACES DANS LE CODE

get_user_access_token ; DelegatedRead ; CorpusApiClient.

DONNÉES ET ÉTATS

Credential par requête, session HTTP mutualisée sans cookies utilisateur.

DÉPENDANCES ET ÉCHANGES

cleyrop_common.mcp_read, API interne et API publique.

COMPORTEMENT CIBLE PROPOSÉ

IAM/MCP Rust émet un mandat court lié au sujet, tenant, ressources et action de lecture ; jamais de credential générique d'écriture.

MIGRATION DE CETTE CAPACITÉ

Qualifier expiration, renouvellement et révocation déléguée ; conserver la différence entre mandat et Bearer direct.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Deux utilisateurs partageant le pool HTTP ne partagent aucune session backend ; un mandat expiré est refusé avant lecture.

Références de l'analyse

[monorepo](#) · [services/mcp-server-corpus/src/utils/user_token.py:17](#) Lecture du Bearer et délégation.

[monorepo](#) · [services/mcp-server-corpus/src/corpus_api/corpus_api_client.py:56](#) Session commune et cookies.

MCPC-04 Cible non qualifiée

Servir MCP HTTP avec progression, erreurs et traces

COMPORTEMENT OBSERVÉ

Le serveur monte les deux outils sur un transport HTTP streamable, fournit /health et instrumente les requêtes ASGI et les appels aiohttp pour propager les traces. Les outils émettent des informations et une progression. Le lifespan ferme la session HTTP ; aucun stockage local durable de recherche n'est montré.

UTILISATEURS ET RÔLES

Clients MCP et opérateurs.

INTERFACES DANS LE CODE

{CONTEXT_PATH}/mcp ; GET /health ; événements progress/info/error.

DONNÉES ET ÉTATS

Sessions et traces de transport.

DÉPENDANCES ET ÉCHANGES

FastMCP, aiohttp et OpenTelemetry.

COMPORTEMENT CIBLE PROPOSÉ

MCP Rust conserve schémas, annulation, progression et corrélation ; la santé distingue process et dépendances.

MIGRATION DE CETTE CAPACITÉ

Tests de protocole MCP avec clients existants, erreurs backend et arrêt en cours d'appel.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une annulation libère les connexions ; une erreur backend conserve un code stable et les logs n'exposent pas le credential.

Références de l'analyse

[monorepo](#) · [services/mcp-server-corpus/src/server.py:84](#) Fermeture de session.

[monorepo](#) · [services/mcp-server-corpus/src/server.py:128](#) Transport et chemin MCP.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- MCP Rust expose uniquement les corpus autorisés et épinglés ; Catalog/Embeddings restent les autorités.
- MCP restitue citations, hashes et édition servie ; Embeddings vérifie les droits avant texte et avant envoi au LLM.
- IAM/MCP Rust émet un mandat court lié au sujet, tenant, ressources et action de lecture ; jamais de credential générique d'écriture.
- MCP Rust conserve schémas, annulation, progression et corrélation ; la santé distingue process et dépendances.

Plan de données proposé

- Recherche en lecture seule et restitution de documents bornée.

Découpage du core Rust

- mcp-corpus, delegated-read et adaptateur Embeddings.

Identité, autorisation et isolation

- Le scope serveur prime l'argument du modèle ; aucune autorité tirée du texte retrouvé.

- Session HTTP sans cookies ; refus d'auth structurés.

Exploitation et reprise

- Service stateless pour les données ; pool HTTP et traces ; tests non exécutés.

Séquence de migration

- Conserver schéma et noms des outils ; remplacer le backend par un adaptateur aux éditions publiées.
- Préserver nettoyage et erreurs ; ajouter provenance d'édition et limites de résultats sans inventer un checkpoint dans MCP.
- Qualifier expiration, renouvellement et révocation déléguée ; conserver la différence entre mandat et Bearer direct.
- Tests de protocole MCP avec clients existants, erreurs backend et arrêt en cours d'appel.

Écarts et décisions encore ouverts

- Le backend doit ajouter une édition explicite ; le résultat actuel seul ne prouve pas un pin stable.

Références de l'analyse

[monorepo](#) · [services/mcp-server-corpus/src/tool/corpus_searcher.py:226](#) Découverte et filtrage du périmètre.

[monorepo](#) · [services/mcp-server-corpus/src/tool/corpus_searcher.py:286](#) Résolution de noms et recherche.

[monorepo](#) · [services/mcp-server-corpus/src/utils/user_token.py:17](#) Lecture du Bearer et délégation.

[monorepo](#) · [services/mcp-server-corpus/src/server.py:84](#) Fermeture de session.

SERVICE ACTUEL · MCP-SERVER-INTERNET

Outils MCP de recherche internet

4 capacités analysées

services/mcp-server-internet · Python/FastMCP, aiohttp, providers SearXNG/Staan et parsing de pages.

Chercher puis extraire des sources web pour les agents, avec protections de destination et garde de consommation.

Lecture de code Pas de qualification runtime nouvelle

Domaines cible : MCP, Egress, AI, Audit.

Périmètre de lecture

- Analyses existantes 11-ml-embeddings et 14-front-api reprises comme contexte ; vérifications ciblées des routes, outils et implémentations citées ci-dessous.
- Instructions racine et AGENTS.md du composant lorsqu'il existe ; aucun binaire propriétaire inspecté.
- Lecture statique ciblée pour finalisation ; les branches non exécutées et le déploiement restent non qualifiés.

Objets et cycle de vie

- Requête web, résultat de moteur, page extraite et décision de quota.

Dépendances du service

- Fournisseurs externes, DNS/HTTP/TLS et configuration de quota.

Fonctionnalités et compatibilité à préserver

MCPI-01 Cible non qualifiée

Rechercher sur le web avec un fournisseur configurable

COMPORTEMENT OBSERVÉ

L'outil web_researcher_and_scraper reçoit question, nombre de résultats, sites suggérés, catégories, période et langue. Une factory choisit SearXNG ou Staан selon configuration. Le résultat conserve URL, titre, description, contenu, rang, score et erreur. La description de l'outil annonce une borne de dix résultats ; sa compatibilité exacte reste à tester.

UTILISATEURS ET RÔLES

Agents ayant un mandat de recherche externe.

INTERFACES DANS LE CODE

MCP web_researcher_and_scraper.

DONNÉES ET ÉTATS

SearchRequest/SearchResult et résultat agrégé éphémère.

DÉPENDANCES ET ÉCHANGES

SearXNG ou Staan, configuration et réseau.

COMPORTEMENT CIBLE PROPOSÉ

MCP/Egress Rust autorise les fournisseurs, borne budget et sources et conserve des références citables.

MIGRATION DE CETTE CAPACITÉ

Comparer les paramètres supportés par chaque fournisseur ; annoncer les écarts au lieu d'une recherche uniforme supposée.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Même requête conserve ses paramètres et limites ; un fournisseur indisponible retourne une erreur explicite sans inventer des sources.

Références de l'analyse

[monorepo](#) · [services/mcp-server-internet/src/tools/web_researcher.py:76](#) Paramètres et résultat.

[monorepo](#) · [services/mcp-server-internet/src/search/search_provider_factory.py:54](#) Sélection de fournisseur.

MCPI-02 **Cible non qualifiée****Extraire les pages et conserver les succès partiels****COMPORTEMENT OBSERVÉ**

Le pipeline réutilise le texte fourni par le moteur quand il existe ; il scrape seulement les résultats qui en ont besoin, et certains restent à leur snippet. Les extractions sont rassemblées avec leurs erreurs individuelles pour conserver les succès partiels. Le scraper applique robots.txt à chaque redirection et extrait un texte nettoyé. Les sources retournées et modes d'extraction sont journalisés.

UTILISATEURS ET RÔLES

Agents de recherche et opérateurs.

INTERFACES DANS LE CODE

web_scraper ; WebResearcherResult ; SearchResult.needs_scraping.

DONNÉES ET ÉTATS

Contenu, statut, erreur et durée par page.

DÉPENDANCES ET ÉCHANGES

HTTP externe, parsing HTML et règles robots.

COMPORTEMENT CIBLE PROPOSÉ

Egress/MCP Rust fournit extraction bornée et provenance URL/horodatage/hash ; le contenu externe reste non fiable pour les actions de l'agent.

MIGRATION DE CETTE CAPACITÉ

Rejouer pages sans texte, redirections et extraction partielle ; préserver distinction snippet/contenu complet.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une page bloquée ne fait pas perdre les autres résultats ; la réponse indique les limites de chaque source.

Références de l'analyse

[monorepo](#) · [services/mcp-server-internet/src/tools/web_researcher.py:141](#) Assemblage des pages avec erreurs.

[monorepo](#) · [services/mcp-server-internet/src/scraper/web_scraper.py:108](#) Extraction et robots.

MCPI-03 Cible non qualifiée

Empêcher les téléchargements vers des adresses non publiques

COMPORTEMENT OBSERVÉ

Le guard valide les URL et adresses IP. Un résolveur DNS contrôle les adresses obtenues et remet ces adresses au connecteur, limitant la fenêtre de re-résolution. Les redirections sont suivies manuellement, revalidées à chaque étape et limitées à cinq. Les IP littérales, destinations privées et cas multicast ont des règles explicites. Cette lecture établit des protections présentes, pas une preuve de résistance à tous les scénarios réseau.

UTILISATEURS ET RÔLES

Opérateurs de sécurité et utilisateurs de recherche externe.

INTERFACES DANS LE CODE

PublicAddressResolver ; ensure_public_url ; open_public_url.

DONNÉES ET ÉTATS

Décision de destination par connexion et compteur de redirections.

DÉPENDANCES ET ÉCHANGES

aiohttp, DNS, TLS et guard partagé.

COMPORTEMENT CIBLE PROPOSÉ

Egress Rust centralise la même politique avec quotas octets/temps et contrôle des proxys ; les outils n'obtiennent aucun accès au réseau interne.

MIGRATION DE CETTE CAPACITÉ

Porter le corpus SSRF/DNS/redirection existant et qualifier dans le réseau cible.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une redirection publique vers une adresse privée ou un résultat DNS mixte est refusé avant téléchargement ; cinq sauts respectent la borne.

Références de l'analyse

[monorepo](#) · [services/mcp-server-internet/src/utils/address_guard.py:104](#) Contrôle des adresses DNS utilisées.

[monorepo](#) · [services/mcp-server-internet/src/utils/address_guard.py:149](#) Validation de chaque redirection.

MCPI-04 Cible non qualifiée

Borner les recherches payantes et exposer leur état

COMPORTEMENT OBSERVÉ

Le quota de recherche est un token bucket en mémoire par pod, avec débit horaire, capacité de burst et horloge monotone. Une exhaustion renvoie un délai de retry. Le code précise que plusieurs replicas multiplient le plafond et qu'un redémarrage réinitialise les compteurs ; ce n'est pas un budget mensuel durable. Le serveur

expose /health et instrumentation de traces.

UTILISATEURS ET RÔLES

Opérateurs et responsables de consommation.

INTERFACES DANS LE CODE

SearchQuotaGuard ; InMemorySearchQuotaGuard.try_consume ; GET /health.

DONNÉES ET ÉTATS

Compteurs locaux et décision allow/deny avec retry_after.

DÉPENDANCES ET ÉCHANGES

Factory de fournisseur, horloge et OpenTelemetry.

COMPORTEMENT CIBLE PROPOSÉ

AI/Egress Rust ajoute une réservation de budget tenant persistante si un plafond global est requis ; garde local conservé pour les bursts.

MIGRATION DE CETTE CAPACITÉ

Fixer politique de coût et cardinalité tenant ; comparer quotas à un et plusieurs replicas avant migration.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une boucle d'agent est freinée ; un redémarrage n'efface pas le budget global cible et ne divulgue aucun secret de fournisseur.

Références de l'analyse

[monorepo](#) · [services/mcp-server-internet/src/search/in_memory_search_quota_guard.py:25](#) Token bucket et limites par pod.

[monorepo](#) · [services/mcp-server-internet/src/server.py:46](#) Santé du serveur.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- MCP/Egress Rust autorise les fournisseurs, borne budget et sources et conserve des références citables.
- Egress/MCP Rust fournit extraction bornée et provenance URL/horodatage/hash ; le contenu externe reste non fiable pour les actions de l'agent.
- Egress Rust centralise la même politique avec quotas octets/temps et contrôle des proxys ; les outils n'obtiennent aucun accès au réseau interne.
- AI/Egress Rust ajoute une réservation de budget tenant persistante si un plafond global est requis ; garde local conservé pour les bursts.

Plan de données proposé

- Recherche et extraction HTTP bornées, sans publication de dataset implicite.

Découpage du core Rust

- mcp-web-research, provider-adapters, public-address-guard et quota-ledger.

Identité, autorisation et isolation

- Adresses publiques, guard DNS et redirections explicites ; tests non exécutés.
- Contenu des pages non fiable ; accès externe accordé par mandat et données de prompt minimisées.

Exploitation et reprise

- Quota local par pod, réinitialisé au redémarrage ; traces et health.

Séquence de migration

- Comparer les paramètres supportés par chaque fournisseur ; annoncer les écarts au lieu d'une recherche uniforme supposée.
- Rejouer pages sans texte, redirections et extraction partielle ; préserver distinction snippet/contenu complet.
- Porter le corpus SSRF/DNS/redirection existant et qualifier dans le réseau cible.
- Fixer politique de coût et cardinalité tenant ; comparer quotas à un et plusieurs replicas avant migration.

Écarts et décisions encore ouverts

- Authentification et autorisation d'exposition à confirmer au niveau gateway/deployment ; aucune garantie déduite du seul serveur MCP.
- Les plafonds octets, durées et catégories exactes doivent être qualifiés par fournisseur.

Références de l'analyse

[monorepo](#) · [services/mcp-server-internet/src/tools/web_researcher.py:76](#) Paramètres et résultat.

[monorepo](#) · [services/mcp-server-internet/src/tools/web_researcher.py:141](#) Assemblage des pages avec erreurs.

[monorepo](#) · [services/mcp-server-internet/src/utils/address_guard.py:104](#) Contrôle des adresses DNS utilisées.

[monorepo](#) · [services/mcp-server-internet/src/search/in_memory_search_quota_guard.py:25](#) Token bucket et limites par pod.

SERVICE ACTUEL · DATASPHERE-CLEYROP

Labels Datasphere, clearance et exports chiffrés

4 capacités analysées

services/datasphere-cleyrop · Python/FastAPI, SQLAlchemy/Alembic, binaire .NET Linux x86-64 propriétaire, Spark/JVM et S3.

Générer les labels, interroger un PDP de clearance et exporter les tables sous forme de fichiers chiffrés.

Lecture de code

Pas de qualification runtime nouvelle

Domaines cible : IAM, Audit, Files, Query, Egress, Flows.

Périmètre de lecture

- Analyses existantes 11-ml-embeddings et 14-front-api reprises comme contexte ; vérifications ciblées des routes, outils et implémentations citées ci-dessous.
- Instructions racine et AGENTS.md du composant lorsqu'il existe ; aucun binaire propriétaire inspecté.
- Lecture statique ciblée pour finalisation ; les branches non exécutées et le déploiement restent non qualifiés.

Objets et cycle de vie

- Label policy/classification/category, représentation JSON/base64 ; requête de clearance ; EncryptJob et fichier chiffré.

Dépendances du service

- Binaire Datasphere non inspecté ; PDP clearance ; PostgreSQL ; Spark/Iceberg ; S3.

Fonctionnalités et compatibilité à préserver

DSPC-01 Cible non qualifiée

Construire et conserver un label de sensibilité

COMPORTEMENT OBSERVÉ

PUT /labels reçoit UUID dataset, policy, classification et category, retourne 202 et lance une BackgroundTask. Le handler appelle le binaire Datasphere pour générer un JSON, encode ce JSON en base64 puis upsert le label en base. Une erreur de tâche est journalisée ; le 202 ne constitue pas une preuve de génération terminée. Le binaire propriétaire n'a pas été inspecté.

UTILISATEURS ET RÔLES

Gestionnaires de labels et service data-model.

INTERFACES DANS LE CODE

PUT /labels ; BuildLabelCommandHandler ; DataspheresLabelServiceImpl.build_label.

DONNÉES ET ÉTATS

Label avec paramètres, JSON, base64 et dates PostgreSQL.

DÉPENDANCES ET ÉCHANGES

Binaire Datasphere .NET Linux x86-64, configuration/SPIF et base.

COMPORTEMENT CIBLE PROPOSÉ

IAM/Audit Rust contrôle la politique et un job durable génère un label versionné lié à une édition ; adaptateur propriétaire à remplacer ou qualifier explicitement.

MIGRATION DE CETTE CAPACITÉ

Importer labels et politiques, comparer byte/hash et décisions ; remplacer BackgroundTask par travail reprenable avec statut.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un 202 fournit un job consultable ; redémarrer après acceptation ne perd pas la génération ; paramètres identiques conservent le résultat attendu.

Références de l'analyse

[monorepo](#) · [services/datasphere-cleyrop/app/src/datasphere/application/router/label_router.py:43](#) 202 et tâche de fond.

[monorepo](#) · [services/datasphere-cleyrop/app/src/datasphere/domain/handler/build_label_command_handler.py:27](#) Création et upsert.

[monorepo](#) · [services/datasphere-cleyrop/app/src/datasphere/infra/service/label/datasphere_label_service_impl.py:31](#) Appel binaire et formats.

DSPC-02 **Cible non qualifiée****Relire le label et demander les décisions de clearance****COMPORTEMENT OBSERVÉ**

GET /labels trouve le label par UUID. POST /access/check reçoit des ressources et leurs sujets, valide les UUID, charge tous les labels et refuse les labels absents. Il construit pour chacun un metadata_binding base64 puis appelle le PDP /decide/by-id. L'adaptateur vérifie que la réponse est une liste de même cardinalité et la traduit en décisions par sujet ; réseau, TLS et retries sont configurables.

UTILISATEURS ET RÔLES

IAM, services consommateurs et utilisateurs soumis aux clearances.

INTERFACES DANS LE CODE

GET /labels?id=... ; POST /access/check.

DONNÉES ET ÉTATS

Labels stockés et décisions retournées, sans pouvoir de publication.

DÉPENDANCES ET ÉCHANGES

Clearance PDP HTTP/TLS et repository de labels.

COMPORTEMENT CIBLE PROPOSÉ

IAM Rust compose droits ordinaires et classification ; décisions versionnées avec politique, label et sujet, refus si indisponible selon contrat.

MIGRATION DE CETTE CAPACITÉ

Rejouer matrice politique/classification/sujets ; comparer réponses et erreurs avant changement de PDP ou binaire.

CRITÈRE D'ACCEPTATION À EXÉCUTER

UUID invalide et label absent ne donnent aucun ALLOW ; une réponse partielle du PDP est refusée, avec traçabilité de la version de politique.

Références de l'analyse

[monorepo](#) · [services/datasphere-cleyrop/app/src/datasphere/domain/handler/check_access_command_handler.py:32](#)

Construction des requêtes et validations.

[monorepo](#) · [services/datasphere-cleyrop/app/src/datasphere/infra/service/clearance/clearance_service_impl.py:151](#) Appel PDP et validation de réponse.

DSPC-03 **Cible non qualifiée****Extraire une table et produire un export chiffré ZTDF****COMPOTEMENT OBSERVÉ**

POST /labels/encrypt charge le label et télécharge la table par son tech_name via Spark. Le service écrit un seul fichier Parquet local en coalesçant, puis refuse un fichier vide ou supérieur à 2 Gio et le nettoie. Le binaire marque le fichier puis le chiffre en .ztdf ; le fichier est téléversé sur S3 et le répertoire temporaire nettoyé. Le code lit la table par nom sans pin d'édition explicite dans ce chemin.

UTILISATEURS ET RÔLES

Utilisateurs autorisés à exporter et opérateurs de données classifiées.

INTERFACES DANS LE CODE

POST /labels/encrypt ; SparkFileServiceImpl.download_file ; LabelService.encrypt.

DONNÉES ET ÉTATS

Parquet temporaire, label, artefact .ztdf et objet S3.

DÉPENDANCES ET ÉCHANGES

Spark/JVM, Iceberg accessible au catalogue Spark, S3 et binaire Datasphere.

COMPOTEMENT CIBLE PROPOSÉ

Query/Files Rust extrait une édition approuvée épinglée ; EXPORT reste un effet à reçu avec hash, politique et destinataire. Remplacer Spark et ses outils JVM.

MIGRATION DE CETTE CAPACITÉ

Qualifier lecteur Arrow/Parquet sans JVM et compatibilité ZTDF ; borner les flux avant matérialisation et conserver preuves de label/édition.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Exporter une édition donnée malgré une publication concurrente ; rejouer retrouve le même reçu ; aucun artefact privé n'est exposé avant contrôle de sortie.

Références de l'analyse

[monorepo](#) · [services/datasphere-cleyrop/app/src/datasphere/domain/handler/encrypt_command_handler.py:90](#) Limite de taille et mise en queue.

[monorepo](#) · [services/datasphere-cleyrop/app/src/datasphere/infra/service/file/spark_file_service_impl.py:250](#) Extraction Spark en un fichier.

[monorepo](#) · [services/datasphere-cleyrop/app/src/datasphere/infra/service/label/datasphere_label_service_impl.py:63](#) Marquage puis chiffrement.

DSPC-04 **Cible non qualifiée**

Ordonner les travaux de chiffrement et surveiller les dépendances

COMPORTEMENT OBSERVÉ

EncryptQueue est une `asyncio.Queue` en mémoire avec un worker démarré dans le `lifespan`. Le worker traite les jobs, journalise les exceptions puis marque la tâche terminée. Les instructions indiquent que les singletons Spark et queue sont créés au démarrage ; une panne d'initialisation Spark est seulement loggée et laisse l'encryption indisponible. Alembic est exécuté au démarrage ou via le mode `migrate`. Le 202 ne donne pas de reçu de livraison durable dans les routes lues.

UTILISATEURS ET RÔLES

SRE et opérateurs d'export.

INTERFACES DANS LE CODE

`EncryptQueue.start/stop/enqueue/_worker` ; `lifespan` et mode `migrate`.

DONNÉES ET ÉTATS

File de jobs locale au processus et fichiers temporaires.

DÉPENDANCES ET ÉCHANGES

PostgreSQL, Spark, binaire et S3.

COMPORTEMENT CIBLE PROPOSÉ

Flows/Files Rust possède une queue durable, leases, limites de stockage et reçus d'effets ; `health` reflète chaque dépendance requise.

MIGRATION DE CETTE CAPACITÉ

Drainer les travaux connus, passer à une file persistée et tester reprises/cleanup avant retrait du runtime Spark.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Arrêt après extraction puis reprise n'abandonne pas de fichier sensible ; un Spark absent ou son remplaçant indisponible empêche une `readiness` d'export trompeuse.

Références de l'analyse

[monorepo](#) · [services/datasphere-cleyrop/app/src/datasphere/domain/handler/encrypt_command_handler.py:31](#) Queue mémoire, worker et erreurs.

[monorepo](#) · [services/datasphere-cleyrop/AGENTS.md:59](#) Comportement de démarrage et dépendances.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- IAM/Audit Rust contrôle la politique et un job durable génère un label versionné lié à une édition ; adaptateur propriétaire à remplacer ou qualifier explicitement.
- IAM Rust compose droits ordinaires et classification ; décisions versionnées avec politique, label et sujet, refus si indisponible selon contrat.
- Query/Files Rust extrait une édition approuvée épinglée ; EXPORT reste un effet à reçu avec hash, politique et destinataire. Remplacer Spark et ses outils JVM.

- Flows/Files Rust possède une queue durable, leases, limites de stockage et reçus d'effets ; health reflète chaque dépendance requise.

Plan de données proposé

- Lecture d'une édition autorisée puis production d'un export chiffré borné, avec reçu PostgreSQL ; aucune écriture directe Iceberg.

Découpage du core Rust

- label-registry, clearance-adapter, classified-export et durable-export-jobs.

Identité, autorisation et isolation

- Le label est un input de décision, pas une autorisation autonome d'export.
- La protection route/gateway et le mandat sur tech_name doivent être vérifiés de bout en bout ; non prouvés par les handlers seuls.

Exploitation et reprise

- Binaire testé seulement sur Linux ; tests d'intégration PDP exclus par défaut selon AGENTS, aucun lancé.
- File mémoire et BackgroundTasks ; migrations hors enforcement Alembic racine.

Séquence de migration

- Importer labels et politiques, comparer byte/hash et décisions ; remplacer BackgroundTask par travail reprenable avec statut.
- Rejouer matrice politique/classification/sujets ; comparer réponses et erreurs avant changement de PDP ou binaire.
- Qualifier lecteur Arrow/Parquet sans JVM et compatibilité ZTDF ; borner les flux avant matérialisation et conserver preuves de label/édition.
- Drainer les travaux connus, passer à une file persistée et tester reprises/cleanup avant retrait du runtime Spark.

Écarts et décisions encore ouverts

- Le contrat propriétaire ZTDF et le remplacement du runtime .NET doivent être décidés ; encapsulation temporaire non assimilée à core Rust.
- Le lien entre UUID label et tech_name de table doit être vérifié au contrôle d'entrée.

- Distinguer succès d'acceptation, de chiffrement, d'upload et remise au destinataire.

Références de l'analyse

[monorepo](#) · [services/datasphere-cleyrop/app/src/datasphere/application/router/label_router.py:43](#) 202 et tâche de fond.

[monorepo](#) · [services/datasphere-cleyrop/app/src/datasphere/domain/handler/check_access_command_handler.py:32](#) Construction des requêtes et validations.

[monorepo](#) · [services/datasphere-cleyrop/app/src/datasphere/domain/handler/encrypt_command_handler.py:90](#) Limite de taille et mise en queue.

[monorepo](#) · [services/datasphere-cleyrop/app/src/datasphere/domain/handler/encrypt_command_handler.py:31](#) Queue mémoire, worker et erreurs.

SERVICE ACTUEL · DATASPHERE-FRONT

Interface de composition des labels SPIF

4 capacités analysées

services/datasphere-front · TypeScript/React/Vite, Chakra UI, Axios, XML DOMParser et keycloak-js.

Éditer et prévisualiser les classifications et catégories de sécurité associées à un dataset.

Lecture de code Pas de qualification runtime nouvelle

Domaines cible : Frontends, IAM, Catalog, Audit.

Périmètre de lecture

- Analyses existantes 11-ml-embeddings et 14-front-api reprises comme contexte ; vérifications ciblées des routes, outils et implémentations citées ci-dessous.
- Instructions racine et AGENTS.md du composant lorsqu'il existe ; aucun binaire propriétaire inspecté.
- Lecture statique ciblée pour finalisation ; les branches non exécutées et le déploiement restent non qualifiés.

Objets et cycle de vie

- Politiques SPIF, sélections, erreurs de validation, dataset et policy sauvegardée.

Dépendances du service

- Data-model et son proxy SPIF ; OIDC actuel Keycloak.

Fonctionnalités et compatibilité à préserver

DSPF-01 Cible non qualifiée

Charger la configuration SPIF et le dataset à labelliser

COMPORTEMENT OBSERVÉ

L'application extrait un UUID de dataset de l'URL /datasets/{id}, charge son nom et les politiques. Le chargement de configuration passe par un proxy de data-model /spif-proxy/files ; il parse un XML de configuration pour récupérer les URLs SPIF puis charge et parse chaque politique. Les erreurs de chargement empêchent l'affichage du formulaire.

UTILISATEURS ET RÔLES

Gestionnaires de labels.

INTERFACES DANS LE CODE

Route navigateur /datasets/{id} ; fetchConfig ; data-model /spif-proxy/files et /datasets/{id}.

DONNÉES ET ÉTATS

Dataset et politiques chargés dans l'état React.

DÉPENDANCES ET ÉCHANGES

Data-model, proxy de fichiers XML et parseur SPIF.

COMPORTEMENT CIBLE PROPOSÉ

Frontends TypeScript conserve ce parcours ; IAM/Catalog fournit une politique versionnée et une édition dataset explicite.

MIGRATION DE CETTE CAPACITÉ

Préserver formats XML et erreurs, migrer URLs backend et inclure version de politique dans la sauvegarde.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une politique XML invalide donne une erreur visible ; le nom du dataset et la politique proviennent du contexte autorisé.

Références de l'analyse

[monorepo](#) · [services/datasphere-front/app/src/App.tsx:9](#) Contexte dataset et chargement.

[monorepo](#) · [services/datasphere-front/app/src/lib/fetch-config.ts:29](#) Chargement XML via proxy.

DSPF-02 **Cible non qualifiée**

Composer une classification, catégories et marques de confidentialité**COMPORTEMENT OBSERVÉ**

Le formulaire choisit une politique et réinitialise les sélections au changement. Il restaure la policy enregistrée du dataset en retrouvant classification et catégories visibles. Les sélecteurs traitent classification, contexte, ensembles de catégories, texte libre, releasability et privacy marks. Les catégories cachées ou obsolètes influent sur les options affichées.

UTILISATEURS ET RÔLES

Auteurs de labels SPIF.

INTERFACES DANS LE CODE

SpifSelector ; ClassificationSelector ; ContextSelector ; CategorySelector ; ReleasabilitySelector ; PrivacyMarkSelector.

DONNÉES ET ÉTATS

LabelSelections : classificationLacv, categorySelections, freeTextSelections et privacyMarkSelections.

DÉPENDANCES ET ÉCHANGES

Modèle SPIF et policy actuelle du dataset.

COMPORTEMENT CIBLE PROPOSÉ

Frontend TS conserve la composition ; la décision serveur se fonde sur la même version de politique et des identifiants stables.

MIGRATION DE CETTE CAPACITÉ

Créer fixtures de politiques réelles anonymisées et comparer restauration des labels et changements de politique.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une sélection enregistrée est restituée à l'identique ; un changement de politique retire les sélections incompatibles sans sauvegarde implicite.

Références de l'analyse

[monorepo](#) · [services/datasphere-front/app/src/components/SpifSelector.tsx:33](#) Restauration depuis le label courant.

[monorepo](#) · [services/datasphere-front/app/src/components/SpifSelector.tsx:88](#) Réinitialisation et sélecteurs.

DSPF-03 **Cible non qualifiée****Appliquer les règles SPIF et prévisualiser le label****COMPORTEMENT OBSERVÉ**

spif-rules calcule exclusions, classifications admises, catégories requises et sélections effectives ; il détecte les conflits de contexte puis valide l'ensemble. Le formulaire calcule les issues et ne génère le label que sans erreur. Le preview affiche notamment les privacy marks et sérialise les groupes de catégories dans la syntaxe attendue par le backend. Cette validation navigateur n'est pas une autorisation serveur.

UTILISATEURS ET RÔLES

Auteurs de labels et opérateurs de politiques.

INTERFACES DANS LE CODE

buildEffectiveSelections ; validateSelections ; generateLabel ; LabelPreview.

DONNÉES ET ÉTATS

Sélections calculées, issues error/warning et LabelOutput.

DÉPENDANCES ET ÉCHANGES

Parseur SPIF, règles locales et backend de label.

COMPORTEMENT CIBLE PROPOSÉ

Frontends partage un contrat de validation versionné avec IAM ; le serveur Rust revalide chaque commande et trace le résultat.

MIGRATION DE CETTE CAPACITÉ

Comparer règles client et serveur sur incompatibilités, catégories implicites et cardinalités ; n'accepter aucune divergence silencieuse.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un conflit bloque l'aperçu sauvegardable ; contourner le JavaScript ne permet pas de persister un label invalide.

Références de l'analyse

[monorepo](#) · [services/datasphere-front/app/src/lib/spif-rules.ts:229](#) Validation des sélections.

[monorepo](#) · [services/datasphere-front/app/src/components/SpifSelector.tsx:121](#) Blocage de génération en cas d'erreur.

[monorepo](#) · [services/datasphere-front/app/src/components/LabelPreview.tsx:20](#) Sérialisation des catégories.

DSPF-04 **Cible non qualifiée**

Enregistrer la policy avec identité renouvelée et retour utilisateur

COMPORTEMENT OBSERVÉ

Le bouton de sauvegarde est désactivé sans UUID valide, politique ou classification. Il envoie un PUT de policy au data-model puis affiche succès ou erreur. Axios extrait response.data et un interceptor vérifie/renouvelle le token Keycloak avant d'ajouter le Bearer ; l'option withCredentials est configurée. Le message de succès correspond à la requête de sauvegarde, pas à la preuve qu'un export chiffré a été produit.

UTILISATEURS ET RÔLES

Utilisateurs authentifiés et opérateurs.

INTERFACES DANS LE CODE

PUT /datasets/{datasetId}/label/policy ; saveDatasetPolicy ; interceptor Axios.

DONNÉES ET ÉTATS

État isSaving, policy envoyée et retour toast ; token de session.

DÉPENDANCES ET ÉCHANGES

data-model, keycloak-js et configuration runtime.

COMPORTEMENT CIBLE PROPOSÉ

Frontend TS remplace l'adaptateur Keycloak par OIDC sans JVM et affiche le statut durable de génération de label.

MIGRATION DE CETTE CAPACITÉ

Préserver renouvellement, gestion d'erreur et accessibilité ; relier le toast à un reçu de commande plutôt qu'à une publication supposée.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un token expiré est renouvelé ou l'action refusée ; double-clic ne crée pas deux jobs ; succès de label et succès d'export restent distincts.

Références de l'analyse

[monorepo · services/datasphere-front/app/src/types/dataset.ts:23](#) Contrat de sauvegarde.

[monorepo · services/datasphere-front/app/src/config/axios.ts:10](#) Renouvellement et Bearer.

[monorepo · services/datasphere-front/app/src/components/LabelPreview.tsx:146](#) Conditions du bouton.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- Frontends TypeScript conserve ce parcours ; IAM/Catalog fournit une politique versionnée et une édition dataset explicite.
- Frontend TS conserve la composition ; la décision serveur se fonde sur la même version de politique et des identifiants stables.
- Frontends partage un contrat de validation versionné avec IAM ; le serveur Rust revalide chaque commande et trace le résultat.
- Frontend TS remplace l'adaptateur Keycloak par OIDC sans JVM et affiche le statut durable de génération de label.

Plan de données proposé

- Aucun moteur de données dans le navigateur ; lecture des politiques et commandes de sauvegarde autorisées.

Découpage du core Rust

- Aucun frontend Rust requis ; cœur serveur label/validation en Rust, contrat TS généré et adaptateur OIDC.

Identité, autorisation et isolation

- Le navigateur ne tranche ni clearance ni droit d'export ; serveur autoritaire.
- Validation et chargement XML ne doivent pas autoriser une URL interne arbitraire via le proxy.

Exploitation et reprise

- Application Vite/bun ; état local React et configuration runtime ; aucun test UI exécuté.

Séquence de migration

- Préserver formats XML et erreurs, migrer URLs backend et inclure version de politique dans la sauvegarde.
- Créer fixtures de politiques réelles anonymisées et comparer restauration des labels et changements de politique.
- Comparer règles client et serveur sur incompatibilités, catégories implicites et cardinalités ; n'accepter aucune divergence silencieuse.
- Préserver renouvellement, gestion d'erreur et accessibilité ; relier le toast à un reçu de commande plutôt qu'à une publication supposée.

Écarts et décisions encore ouverts

- L'UI présente des privacy marks, mais la policy envoyée contient policy/classification/category : caractériser leur représentation attendue dans le contrat serveur.
- Version de politique et édition dataset non explicites dans le parcours lu ; à ajouter.

Références de l'analyse

[monorepo](#) · [services/datasphere-front/app/src/App.tsx:9](#) Contexte dataset et chargement.

[monorepo](#) · [services/datasphere-front/app/src/components/SpifSelector.tsx:33](#) Restauration depuis le label courant.

[monorepo](#) · [services/datasphere-front/app/src/lib/spif-rules.ts:229](#) Validation des sélections.

[monorepo](#) · [services/datasphere-front/app/src/types/dataset.ts:23](#) Contrat de sauvegarde.

SERVICE ACTUEL · CLEYROP-FRONT-END

Interface de conception, catalogue et administration

8 capacités analysées

services/cleyrop-front-end · TypeScript/React, shell Next.js pages router, SPA React Router sous /app, Chakra UI 2, TanStack Query, Redux pour état client, bun, Keycloak JS, Socket.io et SSE/XHR.

Interface principale des développeurs, responsables de données et administrateurs : projets, sources, datasets, flows, exécutions, IA, applications et sécurité.

Lecture de code

Pas de qualification runtime nouvelle

Domaines cible : Frontends, IAM, Catalog, Datasets, Flows, Query, Connectors, Runtimes, Applications, AI, Notifications.

Périmètre de lecture

- Rapport 14-front-api, routes racine/projet/admin/factory et builders d'API.
- Clients datasets, SQL, autorisation, token, SSE et Socket.io.
- Instructions composant et conventions de cache, formulaires, traductions et runtime config.

Objets et cycle de vie

- Projets/espaces, datasets/branches/historique, sources, graphes/versions/MR, exécutions/clusters.
- Applications/instances/images/dépôts/devspaces, agents/corpus, notifications et politiques IAM.
- Caches serveur TanStack Query, état client Redux, formulaires et config publique.

Dépendances du service

- Gateway et services IAM, Catalog/Datasets, Flows, Query, Connectors, Runtimes, Applications, AI, Notifications.
- Keycloak JS et transport HTTP/SSE/Socket.io ; runtime Next/bun.

Fonctionnalités et compatibilité à préserver

FE-01 Cible non qualifiée

Connexion, permissions et navigation autorisée

COMPORTEMENT OBSERVÉ

Le shell Next charge une SPA cliente sous /app. Les routes distinguent interface complète, applications accessibles, profil, clés SSH et écrans d'administration. Les clients HTTP nommés utilisent un token renouvelé ; les permissions viennent d'IAM avec traitement distinct d'un refus et d'une réponse dégradée. Les gates de route et navigation doivent poser la même question. Les routes admin séparent accès organisation et capacités exclusivement plateforme. La dépendance Keycloak JS est encore explicite.

UTILISATEURS ET RÔLES

Tous les utilisateurs et administrateurs.

INTERFACES DANS LE CODE

AppRoutes ; getFreshToken ; effectivePermissionsOptions ; AdminRoutes.

DONNÉES ET ÉTATS

Session navigateur, snapshot de permissions, caches TanStack Query et contexte de navigation.

DÉPENDANCES ET ÉCHANGES

IAM, IdP Keycloak et gateway.

COMPORTEMENT CIBLE PROPOSÉ

Frontends TypeScript reste en place ; un adaptateur OIDC remplace Keycloak JS pour l'IdP sans JVM. Les décisions sont serveur ; refus et indisponibilité restent distincts.

MIGRATION DE CETTE CAPACITÉ

Conserver liens et routes, qualifier refresh/logout et révocation. Remplacer les noms/rôles locaux comme source de droit par les capacités canoniques.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une panne IAM affiche une reprise et aucun écran autorisé par défaut. La saisie directe d'une route et son menu appliquent la même décision ; une révocation termine les accès suivants.

Références de l'analyse

[monorepo](#) · [services/cleyrop-front-end/app/src/modules/AppRoutes.tsx:48](#) Racine de la SPA et route groups.

[monorepo](#) · [services/cleyrop-front-end/app/src/entities/authz/authz.service.ts:66](#) Réponse IAM et état dégradé.

[monorepo](#) · [services/cleyrop-front-end/app/src/utils/http/auth.ts:29](#) Renouvellement partagé.

[monorepo](#) · [services/cleyrop-front-end/app/src/modules/admin/AdminRoutes.tsx:46](#) Séparation des capacités administratives.

FE-02 **Cible non qualifiée****Projets, espaces personnels et outils de développement****COMPORTEMENT OBSERVÉ**

Les routes projet réunissent bibliothèque de données, flows, applications, notebooks, SQL et fonctions IA. Les identifiants de projet sont passés aux clients des services. Les pages disposent d'une structure commune de navigation, formulaires, états de chargement et erreurs. Redux garde l'état client de projet/flow tandis que les lectures serveur utilisent TanStack Query. Les notebooks et devspaces ouvrent des environnements provisionnés par les services applicatifs.

UTILISATEURS ET RÔLES

Développeur et membre de projet.

INTERFACES DANS LE CODE

ProjectRoutes ; ProjectLibraryRoutes ; ProjectDataflowRoutes ; ProjectAIRoutes ; DevspaceRoutes.

DONNÉES ET ÉTATS

Projet sélectionné, personnel space, formulaires/drafts locaux et réponses serveur.

DÉPENDANCES ET ÉCHANGES

IAM, data-dev, app-management, services IA et Query actuel.

COMPOTEMENT CIBLE PROPOSÉ

Frontends inclut projet et environnement de données dans le contexte ; les objets exécutés portent leur édition/révision. Applications/Runtimes Rust fournit les outils sans JVM.

MIGRATION DE CETTE CAPACITÉ

Préserver bookmarks et espaces personnels, introduire des liens explicites vers l'environnement sans assimiler une branche Git à une édition publiée.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Passer de projet ou d'environnement annule les requêtes obsolètes et ne réaffiche pas une réponse tardive du contexte précédent.

Références de l'analyse

[monorepo](#) · [services/cleyrop-front-end/app/src/modules/project/ProjectRoutes.tsx:23](#) Composition des parcours projet.

[monorepo](#) · [services/cleyrop-front-end/app/src/modules/devspace/DevspaceRoutes.tsx:6](#) Parcours devspace.

[monorepo](#) · [services/cleyrop-front-end/AGENTS.md:44](#) Séparation ressources et données serveur.

FE-03 **Cible non qualifiée****Sources, connecteurs et préparation de collecte****COMPOTEMENT OBSERVÉ**

Les builders d'API exposent connecteurs, datasources, droits de datasource, datasets d'une source et accès SFTP. Le client dataset prépare un import par preview de configuration, lecture d'en-têtes, feuilles de classeur et vérification d'en-têtes. Ces parcours permettent de régler le format avant la création/exécution côté serveurs. L'interface ne doit pas interpréter un aperçu comme preuve de publication.

UTILISATEURS ET RÔLES

Intégrateur de données et administrateur de sources.

INTERFACES DANS LE CODE

COLLECT_CONNECTORS_URL ; COLLECT_DATASOURCES_URL ; getDatasetConfigPreview ; getDatasetSheetNames ; checkDatasetHeaders ; DataOriginsRoutes.

DONNÉES ET ÉTATS

Formulaire de source, configuration d'import, échantillon, schéma proposé et cache.

DÉPENDANCES ET ÉCHANGES

Connecteurs/data-collect, Files, SFTP et IAM.

COMPOTEMENT CIBLE PROPOSÉ

Frontends consomme Connectors/Flows Rust avec jobs bornés de découverte et secrets référencés. L'écran montre source, epoch/range, candidat, drift et dernier acquittement séparément.

MIGRATION DE CETTE CAPACITÉ

Conserver options de format et erreurs ; mapper chaque ancien formulaire à un contrat versionné. Rendre les fonctions JVM non prises en charge visibles avant activation.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un drift de schéma ouvre le contrat et une nouvelle décision ; il ne relance pas automatiquement un import destructif. Une preview ne change ni le checkpoint ni l'édition approuvée.

Références de l'analyse

[monorepo](#) · [services/cleyrop-front-end/app/src/utls/endpoints/endpoints.ts:66](#) API sources et connecteurs.

[monorepo](#) · [services/cleyrop-front-end/app/src/entities/dataset/dataset.service.ts:63](#) Préparation et preview de configuration.

[monorepo](#) · [services/cleyrop-front-end/app/src/entities/dataset/dataset.service.ts:90](#) Vérification des en-têtes.

FE-04 Cible non qualifiée

Catalogue, aperçu, schéma, qualité et historique des datasets

COMPORTEMENT OBSERVÉ

Le client dessert catalogue global/projet, détail, permissions de téléchargement, preview par branche, historique, schéma, métadonnées, règles et politiques de labels/chiffrement. Le détail est rafraîchi périodiquement ; sa clé actuelle utilise dataset et projet. Les clés ne portent pas systématiquement environnement et édition, ce qui constitue un contrat à étendre sans conclure à une fuite déjà prouvée. Les actions de qualité et chiffrement sont des appels serveur distincts.

UTILISATEURS ET RÔLES

Data steward, analyste et développeur.

INTERFACES DANS LE CODE

getDatasetsCatalog ; getDatasetDetailsOptions ; getDatasetPreview ; updateDatasetSchema ; getDatasetPolicies ; encryptDataset.

DONNÉES ET ÉTATS

Cache dataset, page de catalogue, version historique, aperçu et état des traitements.

DÉPENDANCES ET ÉCHANGES

Catalog/data-model, data-serve, IAM et services de gouvernance.

COMPORTEMENT CIBLE PROPOSÉ

Catalog/Datasets Rust retournent une édition approuvée explicite. Preview, schéma, statistiques, pagination et export épinglent la même édition. L'historique approuvé reste distinct de tous les snapshots techniques.

MIGRATION DE CETTE CAPACITÉ

Ajouter identité/projet/environnement/édition/version sémantique aux clés ; annuler les lectures anciennes lors d'un changement. Conserver les champs métiers et vues FILESET/MODEL/EMBEDDINGS adaptées.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une publication pendant la pagination ne mélange pas deux éditions. Un snapshot candidat refusé n'apparaît pas dans le sélecteur public ; une révocation masque aussi totaux et métadonnées.

Références de l'analyse

[monorepo](#) · [services/cleyrop-front-end/app/src/entities/dataset/dataset.service.ts:54](#) Identité actuelle du cache.

[monorepo](#) · [services/cleyrop-front-end/app/src/entities/dataset/dataset.service.ts:274](#) Catalogue et polling.

[monorepo](#) · [services/cleyrop-front-end/app/src/entities/dataset/dataset.service.ts:308](#) Politique du dataset.

[monorepo](#) · [services/cleyrop-front-end/app/src/entities/dataset/dataset.service.ts:301](#) Contrat de changement de schéma.

FE-05 Cible non qualifiée

Éditeur de flows, branches, propositions et publication

COMPORTEMENT OBSERVÉ

Les routes et builders desservent graphes, validation/génération de schéma, branches, merge requests, versions et modules internes. Le front porte un état local de flow et des formulaires de transformation ; le serveur data-dev contrôle la sauvegarde et les conflits. La publication actuelle de définition ne garantit pas à elle seule une édition de données publiée. Les chemins de run ciblent une version précise du flow.

UTILISATEURS ET RÔLES

Auteur de flow et relecteur.

INTERFACES DANS LE CODE

ProjectDataflowRoutes ; DEV_DATAFLOW_VALIDATE_URL ; DEV_DATAFLOW_GENERATE_URL ; DATA_DEV_DATAFLOW_MERGE_REQUESTS_URL ; DATA_DEV_MODULES_URL.

DONNÉES ET ÉTATS

Draft de graphe, branche, révision, proposition et paramètres d'exécution.

DÉPENDANCES ET ÉCHANGES

Flows/data-dev, Runtimes/process-data, schémas datasets et modules.

COMPORTEMENT CIBLE PROPOSÉ

Frontends distingue enregistrer une révision, approuver un plan et publier une édition. L'approbation porte hash du plan, bases, manifeste et politique ; une modification l'invalide. Aucun bouton de publication forcée ne contourne un veto.

MIGRATION DE CETTE CAPACITÉ

Préserver les opérateurs et graphes ; exposer préconditions de sauvegarde et incompatibilités sans JVM. Les kinds sont TABLE, INCREMENTAL, VIEW, FILESET, MODEL, EMBEDDINGS, FEATURESET, STREAM ; EXPORT reste un effet.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Deux sauvegardes concurrentes montrent un conflit sans perte de draft. Après audit refusé, aucune édition n'est visible ; modifier le plan requiert nouvelle approbation et nouvel audit.

Références de l'analyse

[monorepo · services/cleyrop-front-end/app/src/modules/project/dataflow/ProjectDataflowRoutes.tsx:10](#) Parcours de conception.

[monorepo · services/cleyrop-front-end/app/src/utils/endpoints/endpoints.ts:25](#) Validation/génération du graphe.

[monorepo · services/cleyrop-front-end/app/src/utils/endpoints/endpoints.ts:31](#) Branches et propositions.

[monorepo · services/cleyrop-front-end/app/src/utils/endpoints/endpoints.ts:38](#) Bibliothèque de modules.

FE-06 Cible non qualifiée

Exécutions, exploration SQL et mises à jour temps réel

COMPORTEMENT OBSERVÉ

Les clients appellent exécutions, versions et clusters, et envoient une requête SQL avec projet et branche optionnelle. Le helper SSE emploie XHR pour joindre un bearer renouvelé. Socket.io utilise uniquement WebSocket, connexion explicite et token dans auth ; il dispose de namespaces général, collect et admin ainsi que de réinscription aux rooms. Le helper SSE lu ne fournit pas le protocole de replay durable de l'édition. Un changement de build vide caches et localStorage avant rechargement.

UTILISATEURS ET RÔLES

Développeur, analyste et opérateur.

INTERFACES DANS LE CODE

execSQLQuery ; PROCESS_EXECUTIONS_URL ; XhrSource ; socket/collectSocket/adminSocket ; handleNewBuild.

DONNÉES ET ÉTATS

Résultats SQL, run, logs, état affiché et caches navigateur éphémères.

DÉPENDANCES ET ÉCHANGES

Query/data-serve, Runtimes, Notifications et gateway.

COMPORTEMENT CIBLE PROPOSÉ

Frontends expose préparation, audit, conflit, publication et ACK_PENDING séparément. Les événements ont séquence/curseur ; les éditions sont relues depuis Catalog. Une réponse perdue récupère le reçu idempotent au lieu de recommencer la publication.

MIGRATION DE CETTE CAPACITÉ

Ajouter pins d'entrées, annulation de requête et curseur de reprise. Les drafts qui doivent survivre à un déploiement sont persistés côté serveur, pas seulement dans localStorage.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Reconnecter sur un autre pod après une publication récupère le reçu sans régression d'état. SQL, export et pagination utilisent les mêmes éditions ; ack en retard ne devient pas ingestion échouée.

Références de l'analyse

[monorepo · services/cleyrop-front-end/app/src/entities/sql/sql.service.ts:6](#) Requête SQL avec contexte projet/branche.

[monorepo · services/cleyrop-front-end/app/src/utils/xhrSource.ts:34](#) Transport SSE authentifié.

[monorepo · services/cleyrop-front-end/app/src/socket/socket.ts:37](#) Transport WebSocket et authentification.

[monorepo · services/cleyrop-front-end/app/src/modules/index.tsx:9](#) Invalidation au changement de build.

FE-07 **Cible non qualifiée****Factory, images, instances, dépôts et applications publiées****COMPORTEMENT OBSERVÉ**

La Factory possède création/modification d'application, création/modification d'instance, création d'image et gestion des dépôts. Un groupe de routes distinct conserve les détails d'application, instance et image en lecture dans l'interface Applications restreinte. Les pages de devspace et applications s'appuient sur app-management ; les commandes et états de build/déploiement restent serveur.

UTILISATEURS ET RÔLES

Développeur applicatif et utilisateur d'une application partagée.

INTERFACES DANS LE CODE

FactoryRoutes ; FactoryReadRoutes ; FactoryRepositoriesRoutes ; ApplicationsRoutes ; DevspaceRoutes.

DONNÉES ET ÉTATS

Formulaires applicatifs, image, instance, dépôt et statut de déploiement.

DÉPENDANCES ET ÉCHANGES

Applications/app-management, IAM et événements de build.

COMPOTEMENT CIBLE PROPOSÉ

Frontends conserve la Factory sur Applications/Runtimes Rust. Chaque lancement montre révision de configuration, digest qualifié et état opérationnel ; partage et exposition portent une autorisation serveur.

MIGRATION DE CETTE CAPACITÉ

Maintenir les routes de détail partagé ; qualifier les droits lecture/écriture séparément et les états longs de build avant remplacement des endpoints.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un lecteur d'application partagée peut ouvrir le détail autorisé sans obtenir les boutons ni l'API de déploiement. Une image reconstruite affiche son nouveau digest sans changer l'historique.

Références de l'analyse

[monorepo](#) · [services/cleyrop-front-end/app/src/modules/factory/FactoryRoutes.tsx:29](#) Parcours de construction et déploiement.

[monorepo](#) · [services/cleyrop-front-end/app/src/modules/factory/FactoryRoutes.tsx:43](#) Détails accessibles dans interface restreinte.

FE-08 **Cible non qualifiée****IA, gouvernance administrative, notifications et configuration UI****COMPOTEMENT OBSERVÉ**

Les routes comprennent agents/corpus du projet, applications IA, notifications, utilisateurs, projets, clusters, clés de modèle, monitoring, audit, permissions et sécurité. Les builders couvrent sessions utilisateur, teams/memberships, comptes de service, fournisseurs d'identité et politiques de mot de passe/session. La configuration navigateur est injectée au runtime ; les traductions couvrent français, anglais et allemand. Ces groupes partagent design system, formulaires et contrôle de permissions, mais gardent des services métier distincts.

UTILISATEURS ET RÔLES

Administrateur plateforme/organisation, auteur d'agent et utilisateur.

INTERFACES DANS LE CODE

ProjectAIRoutes ; AdminRoutes ; NOTIFICATIONS_URL ; IAM_ADMIN_* ; publicRuntimeConfig.

DONNÉES ET ÉTATS

Formulaires et projections des politiques, notifications et ressources IA ; paramètres publics de déploiement.

DÉPENDANCES ET ÉCHANGES

IAM, AI, Embeddings, MCP, Notifications et Applications.

COMPOTEMENT CIBLE PROPOSÉ

Frontends garde les parcours TypeScript en appelant les domaines Rust dédiés. La découverte des noms, relations et erreurs exige un droit explicite ; l'indisponibilité des décisions n'accorde jamais un accès.

MIGRATION DE CETTE CAPACITÉ

Garder les trois langues, focus et clavier ; vérifier l'équivalence des formulaires et erreurs, puis retirer les contrats Keycloak/JVM exposés à l'utilisateur.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Chaque persona accède uniquement aux pages et métadonnées autorisées. Le changement de politique a un reçu serveur ; les traductions et navigation clavier restent utilisables après migration.

Références de l'analyse

[monorepo](#) · [services/cleyrop-front-end/app/src/modules/admin/AdminRoutes.tsx:46](#) Groupes administratifs et capacités.

[monorepo](#) · [services/cleyrop-front-end/app/src/utils/endpoints/endpoints.ts:125](#) Parcours IA projet et corpus.

[monorepo](#) · [services/cleyrop-front-end/app/src/utils/endpoints/endpoints.ts:203](#) Politiques et audit.

[monorepo](#) · [services/cleyrop-front-end/AGENTS.md:52](#) Configuration publique injectée.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- Interfaces TypeScript conservées, DTO versionnés et adaptateurs de chemins legacy.
- Pins d'édition, manifestes approuvés et reçus de publication visibles ; aucune autorité métier dans le cache.

Plan de données proposé

- Rendu, téléchargement autorisé et flux événementiels ; aucun commit catalogue effectué par navigateur.

Découpage du core Rust

- Aucun portage Rust de la SPA proposé. Les APIs métier migrent vers les domaines Rust et publient leurs schémas de contrat.

Identité, autorisation et isolation

- Permissions de découverte couvrent noms, schémas, compteurs, erreurs et lignage.
- Refus et indisponibilité distincts ; menu masqué ne remplace pas contrôle serveur.
- Changer identité/projet/environnement invalide le contexte et annule les requêtes anciennes.

Exploitation et reprise

- Runtime config synchronisée entre manifeste frontend, Next et chart.
- Changement de build efface caches/localStorage : conserver les drafts durables côté serveur.
- Tests UI, accessibilité, trois langues et contrats à exécuter avant bascule ; aucun résultat revendiqué ici.

Séquence de migration

- Extraire adaptateur OIDC et maintenir guards.
- Versionner DTO et ajouter pins édition/environnement dans URLs et caches.
- Séparer sauvegarde, audit, publication, ack et release multi-sorties.

- Basculer les parcours par projet puis retirer les contrats Spark/Keycloak/JVM exposés.

Écarts et décisions encore ouverts

- Les clés actuelles ne portent pas toutes une édition ; l'étendue des changements doit être mesurée par parcours.
- Reprise SSE durable et cohérence multi-sorties ne sont pas établies par le code client actuel.
- L'étude antérieure décrit des écrans cible ; leur qualification n'est pas un test de l'interface existante.

Références de l'analyse

[monorepo](#) · [services/cleyrop-front-end/app/src/modules/AppRoutes.tsx:54](#) Composition réelle du frontend.

[monorepo](#) · [services/cleyrop-front-end/app/src/entities/dataset/dataset.service.ts:54](#) Contrat de cache.

[monorepo](#) · [services/cleyrop-front-end/app/src/utils/http/auth.ts:29](#) Authentification client.

SERVICE ACTUEL · BUSINESS-FRONT-END

Interface métier de conversations, agents et applications

6 capacités analysées

services/business-front-end · TypeScript, React, Vite, Chakra UI 3, TanStack Query et contextes React, bun, Keycloak JS ; pas de Next ni Redux.

SPA destinée aux utilisateurs métier pour dialoguer avec des agents, configurer leur contexte et ouvrir les applications disponibles.

Lecture de code

Pas de qualification runtime nouvelle

Domaines cible : Frontends, IAM, AI, Embeddings, MCP, Applications, Files, Query.

Périmètre de lecture

- Rapport 14 et instructions du composant.
- AuthProvider, users.access, clients conversations/agents/applications/fileSessions/corpora/toolboxes/datasets.
- Routes chat, agents et applications ; configuration runtime documentée.

Objets et cycle de vie

- Utilisateur et studio IAM, décision d'accès, agents/versions, conversations/messages.
- Documents/versions, sessions de fichiers, corpus, toolboxes/variables et applications.

Dépendances du service

- IAM/IdP, AI, Embeddings/corpus, MCP/tools, Applications, Files et Query.
- TanStack Query, React context et SSE/XHR.

Fonctionnalités et compatibilité à préserver

BFE-01 Cible non qualifiée

Admission à l'interface métier et contexte utilisateur

COMPORTEMENT OBSERVÉ

AuthProvider attend l'initialisation Keycloak et l'authentification puis charge utilisateur, permissions et espace studio. Seul system.canAccessFullInterface booléen vrai et non dégradé autorise l'application. Valeur absente/mal formée, erreur ou header de dégradation affiche une page de reprise ; le refus explicite possède une variante restreinte. L'ID studio est directement l'ID projet IAM. La langue utilisateur alimente i18n. La décision est rafraîchie périodiquement sans maintien permissif d'une dernière bonne réponse.

UTILISATEURS ET RÔLES

Utilisateur métier.

INTERFACES DANS LE CODE

AuthProvider ; getBusinessAppAccess ; readBusinessAppAccess ; accessView.

DONNÉES ET ÉTATS

AuthContext, utilisateur, studiold et décision allowed/restricted/unavailable.

DÉPENDANCES ET ÉCHANGES

IAM, IdP Keycloak et clients HTTP.

COMPOTEMENT CIBLE PROPOSÉ

Frontends TypeScript consomme IAM Rust via un adaptateur OIDC indépendant du fournisseur. Le contrat à trois états reste identique.

MIGRATION DE CETTE CAPACITÉ

Préserver la sémantique des IDs et la dégradation ; migrer les endpoints sans extraire un ID depuis le nom du projet.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un champ non booléen ou un IAM indisponible n'ouvre pas l'application. Après révocation, le prochain rafraîchissement ferme l'accès avec le bon message.

Références de l'analyse

[monorepo · services/business-front-end/app/src/providers/AuthProvider.tsx:17](#) Chargement identité, studio et admission.

[monorepo · services/business-front-end/app/src/entities/users/users.access.ts:10](#) Validation stricte et dégradation.

[monorepo · services/business-front-end/app/src/entities/users/users.access.ts:41](#) Trois états explicites.

BFE-02 **Cible non qualifiée**

Conversation avec un agent et historique**COMPOTEMENT OBSERVÉ**

Le client liste l'historique global ou par agent, charge une conversation et crée/continue une conversation par SSE/XHR en POST/PUT. La création peut cibler une version de contexte. Le détail utilise staleTime infini et une rétention de cache prolongée pendant le streaming, afin qu'une navigation hors du chat conserve l'historique déjà chargé. Renommage et suppression sont des commandes HTTP dédiées.

UTILISATEURS ET RÔLES

Utilisateur dialoguant avec un agent.

INTERFACES DANS LE CODE

getConversationsHistory ; getConversation ; createConversation ; continueConversation ; updateConversation ; deleteConversation.

DONNÉES ET ÉTATS

Conversation, messages, titre, agent/contextVersion et buffer client durant le flux.

DÉPENDANCES ET ÉCHANGES

AI, sessions conversationnelles, IAM et SSE.

COMPOTEMENT CIBLE PROPOSÉ

Frontends conserve le chat sur AI Rust, avec IDs de message/run, provenance des entrées et reprise de flux. Les effets outillés suivent leurs contrats d'approbation serveur.

MIGRATION DE CETTE CAPACITÉ

Conserver l'historique et la version de contexte choisie ; ajouter curseur durable et déduplication des fragments sans supposer qu'un flux client vaut un résultat final.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Quitter puis revenir pendant une réponse ne duplique ni prompt ni message. Reconnecter récupère l'état autorisé ; une génération ne peut agir avec des droits issus du texte du prompt.

Références de l'analyse

[monorepo · services/business-front-end/app/src/entities/conversations/conversations.service.ts:30](#) Cache du détail pendant streaming.

[monorepo · services/business-front-end/app/src/entities/conversations/conversations.service.ts:43](#) Création avec version et continuation.

[monorepo · services/business-front-end/app/src/entities/conversations/conversations.service.ts:55](#) Renommage et suppression.

BFE-03 Cible non qualifiée

Configuration des agents, corpus et outils

COMPORTEMENT OBSERVÉ

Le client liste les agents, lit un agent avec currentVersion optionnelle et crée/modifie/supprime des agents dans un projet. Les corpus de projet peuvent être filtrés par IDs. Les toolboxes sont listées et détaillées, et leurs variables sont lues puis enregistrées par PATCH. La configuration d'un agent rassemble ainsi instructions et ressources venant de plusieurs services ; les validations d'accès doivent rester serveur.

UTILISATEURS ET RÔLES

Auteur d'agent dans son espace ou projet.

INTERFACES DANS LE CODE

getAgents ; getAgent ; createProjectAgent ; updateAgent ; deleteAgent ; getProjectCorpora ; getToolboxVars ; saveToolboxVars.

DONNÉES ET ÉTATS

Agent/version, AgentForm, corpus référencés et variables d'outils.

DÉPENDANCES ET ÉCHANGES

AI, corpus/Embeddings, MCP/toolboxes, IAM et projets.

COMPORTEMENT CIBLE PROPOSÉ

AI/MCP/Embeddings Rust publient une révision de contexte qualifiée, référencent les éditions de corpus et bornent les outils. Une variable secrète n'est pas un paramètre réaffichable librement.

MIGRATION DE CETTE CAPACITÉ

Importer références et versions puis qualifier les permissions de chaque corpus/outil. Conserver brouillon et version publiée séparés.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Modifier un brouillon n'altère pas une conversation épinglée à une version. Un corpus ou outil refusé ne peut être ajouté par manipulation du payload.

Références de l'analyse

[monorepo · services/business-front-end/app/src/entities/agents/agents.service.ts:20](#) Version de contexte et CRUD par projet.

[monorepo · services/business-front-end/app/src/entities/corpora/corpora.service.ts:8](#) Sélection des corpus.

monorepo · services/business-front-end/app/src/entities/toolboxes/toolboxes.service.ts:38 Variables des outils.

BFE-04 **Cible non qualifiée**

Pièces jointes et sessions de fichiers

COMPORTEMENT OBSERVÉ

Le service client crée une session puis téléverse un fichier en multipart avec AbortSignal optionnel. Le même contrat est utilisé par les pièces jointes du chat et les formulaires d'agent. Une route DELETE retire un fichier de la session par nom. Le navigateur ne démontre pas à lui seul le scan, les limites ni la rétention, qui relèvent des serveurs.

UTILISATEURS ET RÔLES

Utilisateur attachant un document et auteur d'agent.

INTERFACES DANS LE CODE

createFileSession ; uploadSessionFile ; deleteSessionFile.

DONNÉES ET ÉTATS

Session de fichier, upload local, nom et annulation de requête.

DÉPENDANCES ET ÉCHANGES

AI sessions, Files et stockage serveur.

COMPORTEMENT CIBLE PROPOSÉ

Files/AI Rust fournissent upload borné, quarantaine et références immuables ; les corpus publiés suivent WAP. Le navigateur distingue transfert, validation et disponibilité pour le modèle.

MIGRATION DE CETTE CAPACITÉ

Conserver le parcours multipart et les erreurs, puis ajouter IDs stables de fichier/édition et état de validation. Ne pas assimiler annulation HTTP à suppression de l'objet déjà reçu.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Annuler au milieu puis réessayer ne laisse pas de document utilisé sans validation. Un fichier d'une autre session reste inaccessible même si son nom est connu.

Références de l'analyse

monorepo · services/business-front-end/app/src/entities/fileSessions/fileSessions.service.ts:4 Session et upload.

monorepo · services/business-front-end/app/src/entities/fileSessions/fileSessions.service.ts:14 Suppression partagée entre chat et formulaire.

BFE-05 **Cible non qualifiée**

Documents générés, graphiques et export de données

COMPORTEMENT OBSERVÉ

Le client consulte un document généré avec version optionnelle, demande les formats disponibles et télécharge le document. Il peut télécharger un graphique lié à un message. Un autre service envoie une requête d'export dataset avec options de séparateur/quoting et reçoit un blob. Les datasets du projet sont listés pour les usages métier. Le contrat actuel doit être enrichi pour prouver l'édition qui a servi au résultat.

UTILISATEURS ET RÔLES

Utilisateur métier qui consulte ou exporte une réponse.

INTERFACES DANS LE CODE

getConversationDocument ; getConversationDocumentFormats ; exportConversationDocument ;
downloadConversationChart ; downloadDataset.

DONNÉES ET ÉTATS

Document/version, graphique de message, blob téléchargé et requête d'export.

DÉPENDANCES ET ÉCHANGES

AI documents, Query/data-serve, Files et IAM.

COMPORTEMENT CIBLE PROPOSÉ

AI/Query/Files Rust lient chaque résultat à ses éditions d'entrée et à son reçu. EXPORT est un effet audité et autorisé ; il ne crée pas un neuvième kind de dataset.

MIGRATION DE CETTE CAPACITÉ

Préserver formats et noms d'usage, ajouter provenance et reprise du reçu, reconstruire les droits au moment d'export.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Après révocation, ni un ancien lien ni un cache ne permet un nouvel export. Une réponse et son export pointent vers la même version documentaire et les mêmes éditions de données.

Références de l'analyse

[monorepo · services/business-front-end/app/src/entities/conversations/conversations.service.ts:77](#) Document et version.

[monorepo · services/business-front-end/app/src/entities/conversations/conversations.service.ts:90](#) Formats et téléchargement.

[monorepo · services/business-front-end/app/src/entities/datasets/datasets.service.ts:10](#) Export de requête et paramètres.

BFE-06 **Cible non qualifiée**

Catalogue d'applications et shell métier au runtime**COMPORTEMENT OBSERVÉ**

Le client charge les applications agents, conserve uniquement les statuts DEPLOYED ou WARNING puis les trie par nom. Le détail d'application possède sa propre clé de cache. Le router compose chat, agents et applications. La configuration est injectée via window avant fallback Vite, et la couche HTTP déballe déjà response.data. Le thème, les composants Chakra 3 et les contextes sont propres à cette SPA.

UTILISATEURS ET RÔLES

Utilisateur ouvrant une application publiée et opérateur de l'interface.

INTERFACES DANS LE CODE

getApplications ; getApp ; AppRouter ; config/env.ts.

DONNÉES ET ÉTATS

Liste d'applications disponibles, détail, configuration publique et cache.

DÉPENDANCES ET ÉCHANGES

Applications/app-management, IAM et assets publics.

COMPORTEMENT CIBLE PROPOSÉ

Frontends reste TypeScript avec Applications Rust. Disponibilité opérationnelle, droit de découverte et droit d'ouverture sont des propriétés séparées ; une application déployée n'est pas automatiquement visible.

MIGRATION DE CETTE CAPACITÉ

Conserver filtrage/tri et compatibilité runtime config ; qualifier liens, états de chargement, langues et accessibilité. Retirer le couplage IdP JVM sans changer les usages.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une application supprimée ou révoquée disparaît après invalidation et son lien direct est refusé côté serveur. Le même artefact frontend fonctionne dans deux environnements avec configuration publique injectée.

Références de l'analyse

[monorepo](#) · [services/business-front-end/app/src/entities/applications/applications.service.ts:20](#) Filtrage de statut et tri.

[monorepo](#) · [services/business-front-end/app/src/entities/applications/applications.service.ts:36](#) Détail.

[monorepo](#) · [services/business-front-end/app/src/AppRouter.tsx:8](#) Composition des routes.

[monorepo](#) · [services/business-front-end/AGENTS.md:44](#) Configuration publique au runtime.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- SPA TypeScript conservée ; contexte/version d'agent, droits et provenance visibles.
- Le cache navigateur ne valide ni publication ni effet d'un outil.

Plan de données proposé

- Chat streamé, pièces jointes, rendu documentaire et exports bornés côté serveur.

Découpage du core Rust

- Aucun core métier navigateur Rust ; consommateurs TypeScript des APIs Rust AI/IAM/Files/Applications.

Identité, autorisation et isolation

- Admission stricte booléenne non dégradée, sans maintien permissif.
- Contrôler à nouveau l'accès au document, corpus, outil et export sur serveur.
- Ne pas exposer variables secrètes ou résultats de sessions non autorisées dans les caches.

Exploitation et reprise

- Config window au runtime avant fallback Vite, build bun et Chakra 3.
- Les conversations en cache pendant streaming requièrent invalidation correcte au logout/changement de contexte.
- Aucun parcours UI ou test en navigateur exécuté dans cet audit.

Séquence de migration

- Garder le contrat d'admission et remplacer l'adaptateur Keycloak.
- Versionner agents et provenance des corpus.
- Ajouter reprise du stream, reçus d'export et état d'upload validé.
- Qualifier personas, langues, focus/clavier et navigation entre chat et applications.

Écarts et décisions encore ouverts

- Reprise exacte d'un flux de conversation après perte réseau à qualifier.
- La protection et rétention des fichiers relèvent des services ; elles ne se déduisent pas des fonctions upload.
- Les effets d'outils doivent être classés selon leur niveau d'approbation ; le prompt utilisateur ne suffit pas à autoriser toute action.

Références de l'analyse

[monorepo](#) · [services/business-front-end/app/src/providers/AuthProvider.tsx:17](#) Admission et contexte.

[monorepo](#) · [services/business-front-end/app/src/entities/conversations/conversations.service.ts:30](#) Conversation et durée de cache.

SERVICE ACTUEL · NOTIFICATION-CENTER

Notifications, abonnements et diffusion temps réel

13 capacités analysées

services/notification-center · Kotlin/Spring Boot, modules domaine et infrastructure ; PostgreSQL/JPA/Flyway ; Kafka/Protobuf pour les événements externes et JSON pour la livraison interne ; JavaMail/SMTP, FreeMarker ; netty-socketio et JWT. Toutes les briques JVM doivent disparaître dans la cible.

Maintenir les destinataires et leurs abonnements, transformer les événements métier en notices regroupées, servir une boîte personnelle, remettre les emails prévus et diffuser les changements de collecte, flow et cluster aux interfaces connectées.

Lecture de code

Pas de qualification runtime nouvelle

Domaines cible : Notifications, IAM, Audit, Egress, Flows, Datasets, Runtimes, Frontends, Migration.

Périmètre de lecture

- AGENTS.md du monorepo et du composant, règles Helm et contrat d'authentification des sockets.
- Les deux controllers NotificationController et SubscriptionController, tous leurs DTO, les agrégats et handlers de notification/souscription, les repositories et mappings persistants.
- NotificationConsumer, lamUserEventConsumer, DataDevEventConsumer, SendNotificationEventConsumer et SynchronousCommunicationConsumer ; convertisseurs Protobuf et messages JSON internes.
- Regroupement des accumulations, lamDeliveryAuthorization, rendus mail et leurs trois templates, émetteur INAPP, TokenAuthService, WsAuthorizationService, SocketIoService et gate du protocole.
- Outbox, idempotence, dead-letter, gestion d'erreur Kafka, santé des listeners, sécurité HTTP, migrations SQL et de code, propriétés applicatives et dépendances Gradle.
- Chart : deployment, service, réseau, secrets référencés, probes, volumes et autoscaling ; inventaire des tests de contrats et tests wire, sans exécution.

Objets et cycle de vie

- Subscriber : UUID compatible avec le sujet JWT historique, nom/prénom, locale, médias et abonnements ; désactivation logique et réactivation possibles. IAM est la source d'identité, le service en maintient un miroir.
- Media : INAPP ou EMAIL et destination ; les identifiants de médias sont conservés lors des modifications d'abonné.
- Subscription : association abonné/entité avec filtres d'actions et médias. EntityActionTypeDAO est global par nom d'action ; la portée réelle de préférences médias différentes doit être caractérisée.
- Accumulation : action, ressource origine, ressource notifiée, destinataire, contenus JSON/mail, date et UUID de message optionnel ; supprimée après réduction du groupe.
- Notification : résultat regroupé avec UUID, destinataire, ensemble d'entités, action, dates de création/lecture, contenus et canaux. La boîte INAPP conserve ce résultat en PostgreSQL.
- OutboxEvent : payload et topic à transmettre, date d'envoi et tentatives ; ProcessedMessage : clé de groupe/UUID ; DeadLetterEvent : payload, origine transport et cause d'échec.
- Session Socket.IO : sujet authentifié, token expirant, namespaces et rooms en mémoire ; la connexion et ses messages ne sont pas un historique durable.

- Migration de code : tâche enregistrée en base et exécutée au démarrage ; distincte de l'historique Flyway.

Dépendances du service

- IAM et IdP : JWT, sujets historiques, événements utilisateurs et broker can_view ; les headers HTTP de proxy ne sont fiables que dans le mode explicitement configuré.
- Producteurs métier : contrats InitNotificationEvent et SynchronousCommunicationEvent de packages/proto ; data-collect publie notamment demandes d'accès, échecs de collecte et résultats CHECK_FILE.
- Consommateurs des événements de souscription : changements réémis via outbox en Protobuf ; les migrations de code rejouent certaines souscriptions.
- Kafka : transport entrant, livraison interne en deux groupes par canal et communication synchrone ; le bus JVM actuel ne satisfait pas la cible finale.
- PostgreSQL : abonnés, préférences, accumulations, boîte, outbox, marqueurs et quarantaine ; sauvegarde et restauration doivent couvrir cet ensemble.
- SMTP : remise d'emails HTML ; le nom SendinBlue de la classe ne prouve pas une intégration API fournisseur.
- APISIX et frontends : routes HTTP et Socket.IO ; la route socket laisse l'authentification au service ; rooms et noms d'événements constituent des contrats clients.
- DataDevEventConsumer décode aussi des changements de projet et de membres dont les handlers lus journalisent les données sans modifier les abonnements ; ne pas leur attribuer une synchronisation absente.

Fonctionnalités et compatibilité à préserver

NC-01 **Cible non qualifiée**

Synchroniser les abonnés et leurs coordonnées depuis l'identité

COMPORTEMENT OBSERVÉ

Un abonné est identifié par le sujet utilisateur et conserve nom, prénom, langue, médias et abonnements. Les événements historiques Subscriber créent, modifient ou désactivent ce miroir. Le nouveau consumer IAM exige un messageUid pour la déduplication ; il utilise le keycloakId comme identifiant compatible avec les sujets JWT. Il peut créer, mettre à jour, réactiver ou désactiver un abonné selon active. La suppression utilise un indicateur séparé et les messages d'offboarding sont ignorés car ce service ne possède pas les ressources à réassigner. Le consumer IAM démarre seulement si sa feature est activée. Les langues sont FR, EN et DE, avec normalisation des sous-tags et repli EN. La mise à jour conserve les identifiants de médias.

UTILISATEURS ET RÔLES

IAM, utilisateurs destinataires, opérateurs de migration d'identité.

INTERFACES DANS LE CODE

NotificationConsumer : branche SUBSCRIBER ; iamUserEventConsumer ;
SubscriberCreateOrUpdateCommandHandler ; SubscriberDeleteCommandHandler.

DONNÉES ET ÉTATS

subscriber avec suppression logique ; medias EMAIL/INAPP ; noms et locale ; processed_message pour le nouveau consumer IAM.

DÉPENDANCES ET ÉCHANGES

Événements utilisateurs historiques et IAM, Protobuf, PostgreSQL ; sujets JWT partagés avec le frontend.

COMPORTEMENT CIBLE PROPOSÉ

Notifications maintient un miroir Rust minimal de IAM avec sujet canonique, tenant, locale et préférences. IAM reste autoritaire pour l'activation. Les anciens sujets deviennent des alias explicites et les suppressions ont une politique de rétention distincte des reçus de livraison.

MIGRATION DE CETTE CAPACITÉ

Rejouer les utilisateurs IAM depuis l'origine dans un consumer isolé, comparer les UUID et les adresses, puis choisir une source unique. Réconcilier les abonnés soft-deleted avant d'activer les suppressions. Remplacer la référence Keycloak en même temps que l'IdP JVM.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Le replay de la même création conserve l'abonné et les médias. Une mise à jour fr-CA devient FR, une langue inconnue EN ; une désactivation retire les futures livraisons et une réactivation restaure le sujet sans dupliquer ses abonnements.

Références de l'analyse

[monorepo · services/notification-center/app/notification-center-domain/src/main/kotlin/com/cleyrop/domain/subscription/command/SubscriberCreateOrUpdateCommandHandler.kt:27](#) Mise à jour et réactivation du miroir.

[monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/common/consumer/IamUserEventConsumer.kt:60](#) Transaction, UUID et types d'événements.

[monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/common/consumer/IamUserEventConsumer.kt:124](#) Suppression sous indicateur et choix du keycloakId.

[monorepo · services/notification-center/app/notification-center-domain/src/main/kotlin/com/cleyrop/domain/subscription/aggregate/Subscriber.kt:25](#) Langues, champs et conservation des médias.

NC-02 **Cible non qualifiée**

Créer les abonnements personnels aux échecs de dataset et de flow**COMPORTEMENT OBSERVÉ**

La création REST s'applique à l'utilisateur authentifié, sans identifiant de destinataire fourni par le client. Le DTO exige une entité UUID, un type DATASET ou DATAFLOW et une liste non vide d'actions cohérentes avec ce type. Les trois actions exposées sont DATAFLOW_EXECUTION_FAILED, DATASET_COLLECT_FAILED et DATASET_QUALITY_FAILED. Les médias acceptés sont INAPP et EMAIL, sélectionnés tous deux par défaut. Le domaine ajoute les filtres absents et émet un événement de création d'abonnement. Un abonné absent entraîne un résultat nul puis une erreur 500 du controller. Le mapping persistant des médias est porté par EntityActionTypeDAO identifié globalement par le nom d'action ; la capacité d'avoir des préférences différentes selon deux abonnés doit donc être caractérisée par un test ciblé.

UTILISATEURS ET RÔLES

Utilisateurs de l'interface et clients authentifiés qui surveillent datasets et flows.

INTERFACES DANS LE CODE

POST /subscriptions ; SubscriptionCreateDTO ; EntityActionTypeDTO ; EntityTypeDTO.

DONNÉES ET ÉTATS

Subscriber, Subscription(entityId), EntityActionType et mediaTypes ; association entre souscription et type d'action.

DÉPENDANCES ET ÉCHANGES

Principal issu du JWT ou du proxy de confiance ; contrôle de route IAM à l'entrée ; outbox d'événements de souscription.

COMPOTEMENT CIBLE PROPOSÉ

Notifications expose un catalogue versionné des actions réellement livrables et des canaux par action. Les préférences sont stockées par tenant, sujet, ressource, action et canal. IAM contrôle le droit de s'abonner à la ressource et le droit de recevoir son contenu.

MIGRATION DE CETTE CAPACITÉ

Importer les abonnements avec un relevé des médias réellement persistés. Définir la valeur cible des préférences ambiguës avant conversion. Garder les trois actions historiques, annoncer séparément les nouvelles possibilités.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Deux utilisateurs choisissent des canaux différents pour la même action sans se modifier mutuellement. Une action de flow sur une entité DATASET est refusée. Un utilisateur sans miroir obtient une erreur métier définie plutôt qu'un abonnement fantôme.

Références de l'analyse

monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/subscription/controller/SubscriptionController.kt:49 Création limitée au principal courant.

monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/subscription/controller/dto/SubscriptionCreateDTO.kt:11 Validation du type et médias par défaut.

monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/subscription/controller/dto/EntityActionTypeDTO.kt:7 Exactement trois actions REST.

monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/subscription/repository/dao/EntityActionTypeDAO.kt:17 Clé d'action globale et colonne média.

NC-03 **Cible non qualifiée**

Recevoir les actions métier et les abonnements créés par les producteurs

COMPOTEMENT OBSERVÉ

Le consumer externe decode InitNotificationEvent et distingue ACTION_RESOURCE, SUBSCRIPTION et SUBSCRIBER. Les commandes d'abonnement produites par d'autres services peuvent nommer un ou plusieurs utilisateurs, ressources, actions et médias ; la conversion combine les ressources et les actions. La suppression événementielle retire les abonnements à l'entité pour l'utilisateur indiqué, ou pour tous les utilisateurs selon le message. Le contrat Protobuf définit 27 actions actives couvrant datasets, projets, datasources, flows, applications et demandes Git ; cette énumération ne prouve ni un producteur vivant ni un template email pour chaque action. Les créations et suppressions d'abonnements sont réémises via outbox en Protobuf. Certaines migrations retirent des anciennes souscriptions, notamment GIT_MR_REQUESTED, bien que ce nom reste au contrat.

UTILISATEURS ET RÔLES

Services producteurs de demandes d'accès, collectes, flows, projets et applications ; utilisateurs destinataires choisis par ces services.

INTERFACES DANS LE CODE

InitNotificationEvent : ACTION_RESOURCE/SUBSCRIPTION/SUBSCRIBER ; SubscriptionEventHandlerKafkaImpl ; EnumEntityTypeActionType.

DONNÉES ET ÉTATS

Abonnements automatiques et événements

SubscriberCreateSubscriptionEvent/SubscriberDeleteSubscriptionEvent ; noms d'action conservés en texte dans le domaine.

DÉPENDANCES ET ÉCHANGES

packages/proto ; bus Kafka ; services producteurs dont data-collect et data-dev ; migrations Flyway et migrations de code.

COMPORTEMENT CIBLE PROPOSÉ

Notifications maintient un registre d'actions, leur schéma de payload, les acteurs producteurs autorisés et les canaux disponibles. Une commande automatique indique sa raison, son origine et son idempotence ; elle ne permet pas à un producteur non autorisé d'abonner arbitrairement un autre tenant.

MIGRATION DE CETTE CAPACITÉ

Recenser les producteurs effectifs pour les 27 actions avant de retirer les noms obsolètes. Convertir les contrats Protobuf sans réutiliser leurs numéros réservés. Conserver les événements de souscription nécessaires aux services qui en maintiennent une projection.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Rejouer une création automatique puis sa suppression ciblée sans toucher aux autres utilisateurs. Un ancien événement connu reste décodable ; une action retirée suit une règle explicite de rejet ou d'archivage et ne disparaît pas silencieusement.

Références de l'analyse

[monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/common/consumer/NotificationConsumer.kt:48](#)
Dispatch des trois familles d'événements.

[monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/common/consumer/convertter/InitNotificationEventProtoConverter.kt:58](#) Conversion des abonnements et médias.

[monorepo · packages/proto/src/main/proto/notificationcenter/entity_action_type.proto:8](#) Catalogue complet des actions et numéros réservés.

[monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/subscription/service/producer/SubscriptionEventHandlerKafkaImpl.kt:18](#) Publication des changements de souscription via outbox.

NC-04 Cible non qualifiée

Lister, vérifier et supprimer les abonnements personnels

COMPORTEMENT OBSERVÉ

Les routes de lecture renvoient les UUID d'entités pour une action ou regroupent les abonnements par liste d'actions. La vérification en lot transforme les requêtes en Set puis renvoie une liste de booléens ; deux requêtes identiques peuvent donc ne produire qu'un seul résultat. La suppression REST sépare la liste de couples reçus en une liste d'entités et une liste d'actions ; le filtre du domaine retire tout abonnement dont

l'entité appartient à la première et l'action à la seconde. Cette lecture établit une sémantique de produit cartésien à caractériser pour un lot hétérogène, sans conclure à un incident observé. Les lectures sont toujours attachées au principal courant.

UTILISATEURS ET RÔLES

Interfaces de suivi, tableaux d'abonnement et utilisateurs authentifiés.

INTERFACES DANS LE CODE

GET /subscriptions/entities?entityActionType=... ; GET /subscriptions?entityActionType=... ; POST /subscriptions/check_bulk ; DELETE /subscriptions.

DONNÉES ET ÉTATS

Subscriptions du sujet courant ; requêtes bulk dédoublées ; modifications persistées puis événements de suppression.

DÉPENDANCES ET ÉCHANGES

Repository des abonnés et outbox ; frontend qui associe potentiellement les réponses bulk par position.

COMPORTEMENT CIBLE PROPOSÉ

Notifications spécifie une correspondance explicite requête/résultat par clé de corrélation et une suppression par couples exacts. Les opérations par ensemble ou par produit cartésien, si nécessaires, deviennent des commandes nommées distinctes.

MIGRATION DE CETTE CAPACITÉ

Capturer le comportement attendu des clients pour les doublons et les lots croisés. Ajouter des routes versionnées si la correction change le contrat visible. Dédupliquer les données historiques avec un rapport avant réimport.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un lot contenant deux fois le même couple retourne deux résultats corrélés si le contrat conserve les positions. Supprimer A/collect et B/quality ne retire pas A/quality. Les réponses ne révèlent pas les abonnements d'un autre sujet.

Références de l'analyse

monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/subscription/controller/SubscriptionController.kt:93 Séparation des entités/actions pour suppression.

monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/subscription/controller/SubscriptionController.kt:156 Conversion en Set du bulk.

monorepo · services/notification-center/app/notification-center-domain/src/main/kotlin/com/cleyrop/domain/subscription/aggregate/Subscriber.kt:99 Prédicat de suppression par appartenance aux deux listes.

monorepo · services/notification-center/app/notification-center-domain/src/main/kotlin/com/cleyrop/domain/subscription/query/FindSubscriptionBulkQueryHandler.kt:19 Résultats de vérification par requête distincte.

NC-05 **Cible non qualifiée**

Choisir les destinataires et contrôler leur accès au contenu

COMPORTEMENT OBSERVÉ

Une action métier porte une ressource d'origine et une ressource à notifier. Le handler recherche les abonnés correspondant à l'une ou à l'autre pour cette action et les déduplique par sujet. Sans abonné, il ne crée rien et le consumer compte le rejet. Pour les trois actions librement souscriptibles, `IamDeliveryAuthorization` vérifie `can_view` sur les ressources auxquelles l'utilisateur est effectivement abonné ; au moins une réponse `ALLOW` suffit. Les modes `OFF`, `SHADOW` et `ENFORCE` respectivement désactivent, observent ou imposent ce contrôle ; en `ENFORCE`, une panne IAM retient la notification. Les autres familles sont laissées à la sélection du producteur. Le messageUuid entrant, quand il est valide, exclut les abonnés qui ont déjà une accumulation portant cet UUID ; les messages historiques sans UUID restent acceptés.

UTILISATEURS ET RÔLES

Services producteurs, abonnés, IAM et opérateurs du déploiement progressif des droits.

INTERFACES DANS LE CODE

`AccumulationCreateCommandHandler` ; `IamDeliveryAuthorization.mayReceive` ; `NotificationConsumer.consume`.

DONNÉES ET ÉTATS

Une Accumulation par destinataire éligible, action et message ; messageUuid optionnel ; compteurs sans abonné et décisions IAM.

DÉPENDANCES ET ÉCHANGES

Broker IAM `can_view` pour `DATASET/DATAFLOW` ; configuration `ws.authz.mode` ; abonnements à la ressource origine ou notification.

COMPORTEMENT CIBLE PROPOSÉ

Notifications contrôle toutes les familles selon une politique explicite. Le contrôle avant effet prend la décision IAM courante et masque les champs que le destinataire ne peut plus lire. La clé `source/message/destinataire` reste durable au-delà de la durée d'une accumulation.

MIGRATION DE CETTE CAPACITÉ

Passer les décisions en shadow sur toutes les familles, analyser les destinataires divergents, puis imposer les politiques. Attribuer un message ID aux producteurs historiques. Définir la gestion des notifications de suppression dont la ressource n'est plus consultable.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un abonné privé d'accès avant émission ne reçoit pas de contenu protégé. Une indisponibilité IAM en `ENFORCE` garde le travail rejouable. Le replay d'un message après vidage des accumulations reste sans seconde notification dans la cible.

Références de l'analyse

[monorepo · services/notification-center/app/notification-center-domain/src/main/kotlin/com/cleyrop/domain/notification/command/AccumulationCreateCommandHandler.kt:56](#) Sélection origine/notification et absence de destinataire.

[monorepo · services/notification-center/app/notification-center-domain/src/main/kotlin/com/cleyrop/domain/notification/command/AccumulationCreateCommandHandler.kt:72](#) Autorisation puis déduplication par accumulations existantes.

[monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/notification/service/IamDeliveryAuthorization.kt:38](#) Modes et comportement en erreur.

[monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/notification/service/IamDeliveryAuthorization.kt:103](#) Périmètre limité aux trois familles REST.

NC-06 **Cible non qualifiée**

Regrouper les événements proches en une notification

COMPORTEMENT OBSERVÉ

L'orchestrateur traite périodiquement les groupes d'accumulation dont le dernier événement est assez ancien. Le délai du scheduler est borné à cinq secondes au minimum ; la requête d'éligibilité considère la fenêtre de silence configurée. Le groupe associe action, ressource d'origine, abonné et valeur `trigger_lastname` utilisée comme identité de déclencheur. Le service fusionne les `ActionObjects` JSON distincts, garde le premier sujet mail et ajoute les noms du destinataire aux objets réservés au mail. Pour `mailContent`, il prend le premier contenu non vide. Il recharge les abonnés et leurs abonnements pour calculer les canaux actuels. Après publication de la notification, il supprime les accumulations du groupe ; un groupe en erreur est conservé et un destinataire disparu est écarté. Le parcours lu n'expose pas de revendication verrouillée ni de transaction englobant sélection, publication et suppression, ce qui appelle un scénario de concurrence et de crash.

UTILISATEURS ET RÔLES

Destinataires recevant des événements répétés et opérateurs qui règlent le regroupement.

INTERFACES DANS LE CODE

`AccumulationOrchestrator` ; `DecreaseAccumulationCommandHandler` ; `AccumulatorServiceImpl` ; `AccumulationJPA.findAvailable`.

DONNÉES ET ÉTATS

accumulation, vue `accumulation_grouped`, Notification nouvellement identifiée ; groupes supprimés après succès.

DÉPENDANCES ET ÉCHANGES

Horloge, PostgreSQL, snapshots actuels des abonnés et outbox interne de livraison.

COMPORTEMENT CIBLE PROPOSÉ

Notifications possède un moteur Rust de regroupement avec fenêtre et clé documentées, claim à bail, identifiant de groupe stable et sortie atomique vers l'outbox. Les payloads sont versionnés et bornés en taille ; l'identité du déclencheur est un champ typé.

MIGRATION DE CETTE CAPACITÉ

Reproduire les regroupements existants sur un corpus anonymisé. Décider si le premier contenu mail est le comportement voulu. Reprendre les accumulations en attente en préservant la frontière temporelle de chaque groupe.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Deux workers traitent le même groupe sans produire deux identifiants de notification. Une panne entre création et suppression reprend le même résultat. Des actions ou déclencheurs distincts ne se mélangent pas et un événement continu suit une borne de latence définie.

Références de l'analyse

[monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/notification/service/orchestrator/AccumulationOrchestrator.kt:13](#) Cadence de traitement.

[monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/notification/service/AccumulatorServiceImpl.kt:21](#) Clé de regroupement et fusion des contenus.

[monorepo · services/notification-center/app/notification-center-domain/src/main/kotlin/com/cleyrop/domain/notification/command/DecreaseAccumulationCommandHandler.kt:31](#) Traitement, succès/suppression et conservation des erreurs.

monorepo · services/notification-center/app/notification-center-
 infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/notification/repository/jpa/AccumulationJPA.kt:2
 4 Sélection des groupes éligibles.

NC-07 **Cible non qualifiée**

Acheminer séparément les livraisons INAPP et EMAIL

COMPORTEMENT OBSERVÉ

Une notification préparée est sérialisée en message JSON interne, avec son ID, les médias calculés et un snapshot de l'abonné. Deux groupes de consommation distincts assurent EMAIL et INAPP. Chaque branche vérifie le canal, puis un marqueur processed_message propre au groupe et à l'ID de notification. Le handler SMTP peut être désactivé globalement ; dans ce cas il retourne avant le marquage. La branche INAPP persiste la notification et émet un événement de communication synchrone via outbox. Les méthodes de consommation sont transactionnelles pour les écritures PostgreSQL. L'appel SMTP reste un effet externe à cette transaction : le code ne démontre pas un envoi exactement une fois après crash. L'événement de lecture d'une notification est seulement journalisé par le handler d'événements.

UTILISATEURS ET RÔLES

Destinataires, interfaces connectées et opérateurs SMTP.

INTERFACES DANS LE CODE

SendNotificationEventConsumer.consumeEmail/consumeInApp ; NotificationKafkaMessage ;
 InAppNotificationEmitterImpl ; NotificationEventHandlerKafkaImpl.

DONNÉES ET ÉTATS

processed_message indexé par groupe et messageUuid ; notification persistée pour INAPP ; outbox de communication synchrone ; snapshot d'abonné embarqué.

DÉPENDANCES ET ÉCHANGES

Bus Kafka interne JSON, PostgreSQL, serveur SMTP, Socket.IO indirect via événements Protobuf.

COMPORTEMENT CIBLE PROPOSÉ

Notifications orchestre des effets Rust par canal avec reçu durable, état prepared/sending/accepted/failed/unknown et tentative corrélée. PostgreSQL est autoritaire pour ces reçus. La notification référence le run et l'édition approuvée quand elle annonce une publication ; elle ne devient pas la preuve de publication.

MIGRATION DE CETTE CAPACITÉ

Importer les notifications, marqueurs et travaux non livrés. Remplacer les snapshots trop anciens par des références réévaluées avant effet. Définir un traitement manuel ou idempotent des envois SMTP dont l'acceptation est incertaine.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Rejouer le même message produit une seule entrée de boîte INAPP. Une panne après acceptation SMTP et avant commit donne un état incertain explicite dans la cible. Couper SMTP ne bloque pas INAPP et ne fait pas croire qu'un email a été remis.

Références de l'analyse

monorepo · services/notification-center/app/notification-center-
 infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/common/consumer/SendNotificationEventConsumer.kt
 :35 Transactions, sélection des canaux et marqueurs.

monorepo · services/notification-center/app/notification-center-
 infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/notification/service/notification/inapp/notificationemitter/InAppNotificationEmitterImpl.kt:35 Persistance puis émission de l'événement temps réel.

[monorepo](#) · [services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/notification/service/producer/NotificationEventHandlerKafkaImpl.kt:19](#) Outbox des nouvelles notifications et lecture uniquement journalisée.

NC-08 **Cible non qualifiée**

Consulter sa boîte de notifications et marquer la lecture

COMPORTEMENT OBSERVÉ

GET /notifications retourne les notifications du principal courant, paginées et triées ; le tri par défaut est createdAt décroissant. Un abonné inexistant obtient une page vide. PUT /notifications/read exige une liste d'UUID non vide et reçoit un readAt fourni par le client, par défaut l'instant courant. Le repository filtre les UUID par leur propriétaire avant de modifier les dates de lecture, donc des UUID étrangers ne suffisent pas à toucher la boîte d'autrui. Le DTO expose l'action, les dates, le propriétaire/déclencheur et un payload structuré par type d'objet. Il retire les objets réservés au mail et certains noms techniques du destinataire. La suppression HTTP de notification n'est pas exposée.

UTILISATEURS ET RÔLES

Utilisateurs connectés, cloche de notification et centre de notifications frontend.

INTERFACES DANS LE CODE

GET /notifications?page=...&size=...&sort=... ; PUT /notifications/read.

DONNÉES ET ÉTATS

Notification : UUID, subscriberId, entityIds, action, createdAt, readAt, contenus JSON/mail, médias ; suppression logique présente dans le mapping DAO.

DÉPENDANCES ET ÉCHANGES

Principal utilisateur ; PostgreSQL ; objets de présentation partagés avec les événements Socket.IO.

COMPORTEMENT CIBLE PROPOSÉ

Notifications sert une boîte Rust par tenant/sujet avec pagination stable, curseur et total non lu. La date serveur fait foi pour la lecture ; une date client éventuelle reste séparée. Une politique de rétention contrôle archivage et suppression sans effacer les reçus d'effets requis.

MIGRATION DE CETTE CAPACITÉ

Conserver les UUID, horodatages et payloads lisibles, documenter les tris acceptés et la conversion des propriétaires. Caractériser les cas readAt fourni et notifications supprimées avant de figer le contrat cible.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une lecture paginée ne mélange pas les utilisateurs. Un lot comprenant un UUID étranger ne le modifie pas. Marquer deux fois le même ensemble produit l'état défini sans doublon d'événement et les champs réservés au mail restent absents du DTO.

Références de l'analyse

[monorepo](#) · [services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/notification/controller/NotificationController.kt:29](#) Routes personnelles, pagination et lecture.

[monorepo](#) · [services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/notification/controller/dto/MarkNotificationAsReadDTO.kt:7](#) Liste non vide et date de lecture.

[monorepo](#) · [services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/notification/repository/NotificationRepositoryJPATImpl.kt:35](#) Filtrage par abonné pour page et mutation.

[monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/notification/controller/dto/PayloadDTO.kt:20](#)
Projection et exclusions de payload.

NC-09 **Cible non qualifiée**

Rendre les trois emails métiers localisés et les remettre au SMTP

COMPORTEMENT OBSERVÉ

L'implémentation nommée EmailSendinBlueImpl utilise JavaMailSender et SMTP ; le code lu n'appelle pas une API Sendinblue. MessageBuilder construit un message HTML avec enveloppe générique et template FreeMarker, puis localise sujet et corps en FR, EN ou DE. Il ne résout que PROJECT_USER_MEMBERSHIP_ADDED, DATAFLOW_EXECUTION_FAILED et DATASET_QUALITY_FAILED. Pour toute autre action, il retourne null et l'émetteur saute l'envoi. DATASET_COLLECT_FAILED est donc souscriptible en EMAIL dans l'API mais n'a pas de template résolu ici. Le modèle est construit depuis les ActionObjects JSON, avec alias de noms et contexte de client ; le champ mailContent accumulé n'est pas utilisé comme corps par ce builder.

UTILISATEURS ET RÔLES

Utilisateurs destinataires d'invitations projet, échecs de flow et erreurs de qualité ; opérateurs mail.

INTERFACES DANS LE CODE

EmailSendinBlueImpl ; MessageBuilder.build/resolveTemplate ; EmailArgumentMapper ; templates/mail/*.ftlh.

DONNÉES ET ÉTATS

Locale du snapshot abonné ; destinataires EMAIL ; modèles et messages localisés en ressources applicatives ; configuration d'expéditeur.

DÉPENDANCES ET ÉCHANGES

Serveur SMTP, JavaMail, FreeMarker et bundle de traductions ; URLs fournies dans le payload métier.

COMPORTEMENT CIBLE PROPOSÉ

Notifications possède un renderer Rust à templates versionnés et des adaptateurs d'effet mail. Le catalogue affiche les canaux réellement disponibles. Les liens, champs obligatoires et échappements HTML sont validés ; Egress contrôle la destination SMTP et les références externes.

MIGRATION DE CETTE CAPACITÉ

Reprendre les trois modèles et leurs traductions sous tests de rendu. Décider explicitement de créer un email d'échec de collecte ou de retirer ce canal de cette action. Distinguer absence de template, canal désactivé et remise au fournisseur dans les reçus.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Chaque combinaison des trois actions et trois langues produit un sujet et un corps attendus sans champs manquants. Une action sans template donne un statut unsupported observable. Un nom contenant du HTML est échappé et un refus SMTP laisse une tentative rejouable.

Références de l'analyse

[monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/notification/service/notification/email/MessageBuilder.kt:30](#) Absence de template retourne null.

[monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/notification/service/notification/email/MessageBuilder.kt:78](#) Modèle depuis ActionObjects.

[monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/notification/service/notification/email/MessageBuilder.kt:115](#) Trois actions et langues résolues.

[monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/notification/service/notification/email/sendinblue/EmailSendinBlueImpl.kt:11](#) Adaptateur JavaMailSender et saut des messages sans template.

NC-10 **Cible non qualifiée**

Authentifier les connexions Socket.IO et borner leur durée

COMPOTEMENT OBSERVÉ

Le transport temps réel est Socket.IO/Engine.IO, pas un WebSocket JSON interchangeable. Le service authentifie lui-même ses connexions : payload auth, header Authorization Bearer, puis cookie legacy si l'option le permet. Les paramètres de query ne servent pas de jeton. Avec le fallback cookie désactivé, une connexion peut attendre le paquet CONNECT pour son payload auth, mais aucune action de room n'est admise avant authentification. Le token doit être actif, décodable et porteur d'un sub. Une gate de protocole refuse les CONNECT invalides avant leur acquittement ; un délai d'authentification ferme les connexions incomplètes et l'expiration du JWT provoque aussi la fermeture même au repos. Les frames mal formées sont fermées et les données client non authentifiées ne sont pas écrites dans les logs ordinaires.

UTILISATEURS ET RÔLES

Frontends et clients Socket.IO authentifiés.

INTERFACES DANS LE CODE

Handshake Engine.IO ; paquet Socket.IO CONNECT avec auth ; TokenAuthService ; SocketIoConnectGate ; événements join-room/leave-room.

DONNÉES ET ÉTATS

Sessions en mémoire, sujet et JWT validé, échéances d'authentification et d'expiration ; aucune session temps réel durable.

DÉPENDANCES ET ÉCHANGES

JWT decoder/IdP, configuration cookie fallback, APISIX sur une route sans authentification de gateway, netty-socketio.

COMPOTEMENT CIBLE PROPOSÉ

Un serveur Rust compatible avec le protocole réellement utilisé, ou une transition de clients vers un protocole explicite, reprend les mêmes refus et échéances. IAM émet les tokens sans Keycloak JVM ; la durée de socket ne dépasse pas celle du credential.

MIGRATION DE CETTE CAPACITÉ

Exécuter le corpus de tests wire contre le serveur candidat avec les versions de clients déployées. Retirer le cookie legacy après migration des clients et fournir un renouvellement/reconnect documenté. Préserver la séparation entre authentification du transport et autorisation des rooms.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un jeton forgé ou expiré n'obtient aucun CONNECT réussi ; un jeton en query est ignoré. Un client sans auth ferme à échéance. Un socket valide devient inutilisable à l'expiration même sans nouveau message, et le frontend peut se reconnecter avec un token renouvelé.

Références de l'analyse

[monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/synchronousCommunication/service/TokenAuthService.kt:28](#) Sources de credentials et ordre de résolution.

monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/synchronousCommunication/service/SocketIoService.kt:292 Délai d'authentification et échéance du JWT.

monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/synchronousCommunication/service/SocketIoService.kt:574 Refus du CONNECT portant un payload invalide.

monorepo · services/notification-center/AGENTS.md:1 Contrat de gateway et tests wire à préserver.

NC-11 Cible non qualifiée

Autoriser les namespaces et les rooms avant chaque diffusion

COMPORTEMENT OBSERVÉ

Le service prévoit OFF, SHADOW et ENFORCE pour l'autorisation Socket.IO, avec OFF comme valeur par défaut dans les propriétés. En ENFORCE, une réponse IAM manquante refuse l'action. Le namespace racine donne la room personnelle correspondant au sujet ; /admin exige le droit administrateur système. Un namespace projet doit contenir un UUID canonique et ses rooms dataflow-exec-* sont soumises au can_view du projet. /collect accepte des rooms collect-* contrôlées sur la datasource. Les familles inconnues sont refusées en mode imposé. Le serveur revalide le credential et l'autorisation pour chaque client avant diffusion et retire les clients refusés de la room. Les namespaces projets dynamiques ont un plafond et une éviction après inactivité.

UTILISATEURS ET RÔLES

Utilisateurs de projets, administrateurs de clusters, créateurs de sources et opérateurs IAM.

INTERFACES DANS LE CODE

WsAuthorizationService.authorizeConnect/authorizeJoin ; SocketIoService.onJoinRoom/publish ; namespaces /, /admin, /collect et /projects:{projectId}.

DONNÉES ET ÉTATS

Appartenance aux rooms en mémoire ; réservations de namespaces dynamiques, échéances d'inactivité et cache éventuel du client IAM.

DÉPENDANCES ET ÉCHANGES

IAM can_view pour projet/datasource et privilège système administrateur ; WsAuthzProperties.

COMPORTEMENT CIBLE PROPOSÉ

Notifications impose une ACL par famille de message et ressource canonique. Un événement de flow prouve son appartenance au projet demandé. Les décisions sont revérifiées à la livraison avec une politique de cache bornée ; Audit trace les refus sans exposer les payloads.

MIGRATION DE CETTE CAPACITÉ

Comparer les décisions shadow aux routes réellement utilisées, résoudre les UUID legacy, puis activer ENFORCE. Reprendre les limites de namespaces et définir la fréquence maximale de revalidation IAM pour la charge cible.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Après révocation, un client connecté ne reçoit plus les messages protégés. Une room d'exécution d'un autre projet est refusée même si l'utilisateur connaît son ID. Une rafale de namespaces invalides ne dépasse ni le plafond mémoire ni le délai de nettoyage.

Références de l'analyse

monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/synchronousCommunication/service/WsAuthorizationService.kt:101 Plans de droits par namespace et room.

`monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/synchronousCommunication/service/SocketIoService.kt:218` Contrôle de chaque destinataire avant émission.

`monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/synchronousCommunication/service/SocketIoService.kt:629` Création bornée des namespaces projets et éviction.

`monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/configuration/WsAuthzProperties.kt:16` Modes d'autorisation et valeurs par défaut.

NC-12 Cible non qualifiée

Diffuser les six familles de messages de supervision et d'interaction

COMPORTEMENT OBSERVÉ

Le consumer de communication synchrone convertit six familles Protobuf. NOTIFICATION vise la room utilisateur à la racine, sous le nom notification. CLUSTER_SESSION_EXEC, CLUSTERS_SESSION_EXEC_LIST et CLUSTER_MONITORING visent /admin avec respectivement cluster-{id}, cluster-all et cluster-monitoring. DATAFLOW_REFRESH vise le namespace /projects:{projectId} et dataflow-exec-{executionId}. CHECK_FILE vise /collect et collect-{datasourceId}, avec résultat d'analyse de schéma. La publication utilise les clients actuellement présents dans les namespaces locaux. Ce chemin ne montre ni curseur client, ni journal de rattrapage, ni acquittement métier. Le consumer utilise un groupe partagé ; la portée de diffusion entre pods doit être testée puisque les sockets sont maintenues en mémoire locale et le chart autorise le scale.

UTILISATEURS ET RÔLES

Centre de notifications frontend, pages d'exécution des flows, administration de clusters et assistants d'import de fichiers.

INTERFACES DANS LE CODE

SynchronousCommunicationConsumer ; PublishMessageCommandHandler ;
EnumSynchronousCommunicationType ; noms de rooms et événements ci-dessus.

DONNÉES ET ÉTATS

Messages éphémères issus du bus ; sessions/rooms en mémoire ; l'historique de notifications INAPP existe séparément en PostgreSQL.

DÉPENDANCES ET ÉCHANGES

Événements produits par data-collect/data-dev et autres services de runtime ; contrats Protobuf ; clients Socket.IO.

COMPORTEMENT CIBLE PROPOSÉ

Notifications transporte des projections de runs, de contrôles de fichiers et de supervision. Un identifiant, un numéro de séquence et une URL de resynchronisation permettent au frontend de rattraper une interruption. Les messages liés aux données référencent l'édition approuvée ; ils ne font pas foi comme checkpoint publié.

MIGRATION DE CETTE CAPACITÉ

Préserver les six payloads pendant la migration des clients. Définir une diffusion entre instances ou un routage garantissant l'accès aux sockets du destinataire. Séparer les flux de supervision volumineux des notices durables et leur rétention.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Avec deux instances et des clients répartis, chaque destinataire autorisé reçoit le message attendu. Après déconnexion, un frontend restaure l'état courant et détecte les séquences manquantes. Un CHECK_FILE ne révèle pas le schéma à un utilisateur sans droit sur sa source.

Références de l'analyse

`monorepo · services/notification-center/app/notification-center-domain/src/main/kotlin/com/cleyrop/domain/synchronousCommunication/command/PublishMessageCommandHandler.kt:29` Routage des six familles vers namespaces/rooms/events.

`monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/common/consumer/SynchronousCommunicationConsumer.kt:15` Consumer et groupe Kafka.

`monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/common/consumer/convert/SynchronousCommunicationEventProtoConverter.kt:16` Conversion des payloads.

`monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/synchronousCommunication/service/SocketIoService.kt:192` Diffusion sur les clients locaux présents.

NC-13 **Cible non qualifiée**

Exploiter la messagerie avec outbox, quarantaine et migrations

COMPOTEMENT OBSERVÉ

Le relais d'outbox sélectionne les messages non envoyés avec FOR UPDATE SKIP LOCKED, attend la confirmation Kafka puis renseigne `sentAt` ; erreurs et tentatives restent persistées. Le nettoyage ne retire que les messages envoyés au-delà de la rétention. Les consommateurs utilisent un gestionnaire d'erreur avec reprises puis DLT ; une DLT n'est jamais republiée récursivement en DLT. Le consumer terminal persiste les coordonnées et l'erreur, avec un UUID stable dérivé du topic/partition/offset de la DLT, et réémet l'erreur si PostgreSQL échoue. Un indicateur de santé expose les listeners arrêtés de manière anormale. Les migrations Flyway coexistent avec des migrations de code jouant des abonnements ; les instructions du composant signalent l'absence de verrou global des migrations de code et leur sortie de processus en cas d'échec.

UTILISATEURS ET RÔLES

SRE, opérateurs d'upgrade, support des événements rejetés et responsables de continuité de service.

INTERFACES DANS LE CODE

`OutboxRelay` ; `OutboxEventJPA.claimUnsentBatch` ; `DeadLetterEventConsumer` ; `KafkaListenersHealthIndicator` ; `CleyropMigrationRunner`.

DONNÉES ET ÉTATS

`outbox_event`, `dead_letter_event`, `processed_message`, historique Flyway et `code_migration` ; marqueurs de livraison et tentatives.

DÉPENDANCES ET ÉCHANGES

PostgreSQL, Kafka, chart Kubernetes, probes et metrics ; migrations de contrat de souscription.

COMPOTEMENT CIBLE PROPOSÉ

Un socle Rust commun fournit outbox, replay, dead-letter, health et métriques par backlog/âge. Le bus final est sans JVM et conserve les garanties nécessaires de rétention et d'ordre. Les migrations ont un verrou explicite, un statut durable et une procédure de rollback Expand/Migrate/Contract.

MIGRATION DE CETTE CAPACITÉ

Reprendre les messages non envoyés, les marqueurs et la quarantaine avant bascule de consumer. Établir le plan de replay et les clés d'idempotence. Transférer les migrations d'abonnements dans des jobs bornés reprenables, puis retirer Spring, Kafka JVM et leurs images.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une interruption entre acquittement du bus et commit provoque au plus un replay absorbé. Une DLT illisible ou une panne DB n'avance pas l'offset terminal. Un upgrade concurrent n'exécute pas deux fois une migration et une restauration remet en cohérence inbox/outbox/reçus.

Références de l'analyse

monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/common/outbox/OutboxEventJPA.kt:14 Claim verrouillé de l'outbox.

monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/common/outbox/OutboxRelay.kt:23 Acquittement puis état envoyé.

monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/common/deadletter/DeadLetterEventConsumer.kt:26 Persistance terminale et comportement en erreur.

monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/codemigration/migrations/SubscriptionMigration.kt:20 Replay des souscriptions par migration de code.

Catalogue des actions de notification

Les actions réservées APPLICATION_USER_MEMBERSHIP_ADDED et APPLICATION_DATASET_LINK_FAILED ne sont pas des valeurs actives. INAPP dépend de la présence d'un événement, d'un abonné, d'un canal sélectionné et des décisions d'autorisation ; le tableau ne promet pas une émission pour chaque nom.

Action	Déclarée en protobuf	Création via DTO abonnement	Template email résolu
DATASET_COLLECT_FAILED	Oui	Oui	Non
DATASET_DATAFLOW_FAILED	Oui	Non	Non
DATASET_DELETED	Oui	Non	Non
DATASET_CATALOG_PUBLISHED	Oui	Non	Non
DATASET_MANUALLY_REFRESHED	Oui	Non	Non
DATASET_QUALITY_FAILED	Oui	Oui	Oui
DATASET_OWNER_PROMOTED	Oui	Non	Non
PROJECT_USER_MEMBERSHIP_ADDED	Oui	Non	Oui
PROJECT_DATASET_REQUESTED	Oui	Non	Non
PROJECT_DATASET_ACCEPTED	Oui	Non	Non
PROJECT_DATASET_REFUSED	Oui	Non	Non

Action	Déclarée en protobuf	Création via DTO abonnement	Template email résolu
PROJECT_DATASET_DELETE D	Oui	Non	Non
PROJECT_DATASOURCE_RE QUESTED	Oui	Non	Non
PROJECT_DATASOURCE_AP PROVED	Oui	Non	Non
PROJECT_DATASOURCE_REF USED	Oui	Non	Non
PROJECT_OWNER_PROMOT ED	Oui	Non	Non
DATAFLOW_MR_REQUESTED	Oui	Non	Non
DATAFLOW_MR_ACCEPTED	Oui	Non	Non
DATAFLOW_MR_REFUSED	Oui	Non	Non
DATAFLOW_EXECUTION_FAI LED	Oui	Oui	Oui
DATAFLOW_OWNER_PROMO TED	Oui	Non	Non
APPLICATION_GROUP_MEMB ERSHIP_ADDED	Oui	Non	Non
GIT_MR_REQUESTED	Oui	Non	Non
GIT_MR_ACCEPTED	Oui	Non	Non
GIT_MR_REFUSED	Oui	Non	Non
DATASOURCE_DELETED	Oui	Non	Non
DATASOURCE_OWNER_PRO MOTED	Oui	Non	Non

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- Notifications Rust possède le registre des actions, les schémas de payload, les canaux réellement supportés, les préférences par sujet et la boîte personnelle. PostgreSQL reste autoritaire pour ces commandes et états d'effets.
- IAM décide l'accès à la ressource lors de la souscription et avant effet. Les notices de suppression et de demandes d'accès ont une politique dédiée, sans supposer que la ressource reste consultable.
- Les événements annonçant une donnée publiée référencent l'édition Iceberg approuvée, le run et la publication. Une réussite de transport, de collecte ou de notification n'est pas un acquittement de publication.
- Audit corréle événement source, regroupement, décision IAM, rendu versionné, canal, tentatives et reçu. Une livraison incertaine reste représentée comme telle.

- Les templates, canaux et locales sont versionnés ; le frontend n'offre que les combinaisons supportées. Les actions automatiques mentionnent leur producteur et leur mandat de destinataire.

Plan de données proposé

- Workers Rust bornés pour fanout, regroupement, rendu et effets ; taille des payloads, nombre de destinataires, débit et délai bornés par politique tenant.
- Adaptateur SMTP ou fournisseur encapsulé avec identifiant d'effet stable et traitement de l'incertitude ; aucun engagement implicite d'exactly-once SMTP.
- Service temps réel sans JVM avec compatibilité Socket.IO validée ou migration coordonnée des clients ; authentification à échéance, droits de room et diffusion entre instances.
- Bus durable sans JVM, outbox et consommateurs idempotents. Les messages de supervision peuvent être compacts, mais un état durable consultable permet la resynchronisation après une coupure.
- Le transport des notifications ne publie aucune table Iceberg. Les références de données pointent sur les publications approuvées et les lecteurs restent épinglés à leur édition.

Découpage du core Rust

- notifications-contracts : registre action/version, schémas, ressources canoniques, canaux, templates et validation des payloads.
- notifications-preferences : miroir IAM, abonnements personnels/automatiques, préférences par canal et commandes idempotentes.
- notifications-aggregation : groupes déterministes, fenêtres bornées, claim concurrent et publication atomique du résultat vers l'outbox.
- notifications-delivery : vérification IAM avant effet, rendu HTML localisé, adaptateurs d'envoi et machine d'état des reçus.
- notifications-inbox : boîte personnelle, pagination, lecture et rétention ; effets séparés de la projection temps réel.
- notifications-realtime : protocole client, authentification, ACL de rooms, gestion de l'expiration, limites de namespaces, diffusion entre instances et resynchronisation.
- Socle partagé d'outbox, processed-message durable, dead-letter, métriques, replay et migrations ; remplacement du runtime JVM et du broker JVM coordonné avec les producteurs.

Identité, autorisation et isolation

- HTTP : principal courant pour boîte et souscriptions ; confiance de X-Userinfo conditionnée au mode gateway et au proxy déclaré. Les routes nouvelles doivent passer le garde de couverture du composant.
- Socket : auth payload/header Bearer, cookie legacy sous option, jamais de token en query. La gate CONNECT, le timeout d'auth et l'échéance du token sont des garanties distinctes à conserver.
- Autorisation de livraison et de rooms sous OFF/SHADOW/ENFORCE ; défaut OFF dans les propriétés lues. Ne pas déduire le mode réellement déployé du seul chart ou du code.
- Le contrôle de livraison vise actuellement les trois actions souscriptibles par REST et s'effectue avant l'accumulation. La cible doit trancher la révocation entre cette étape et l'effet SMTP ou INAPP.

- Le contrôle projet des rooms d'exécution ne constitue pas à lui seul une preuve que l'exécution appartient au projet. Un scénario de croisement des identifiants doit caractériser la protection de bout en bout.
- Les emails et payloads de notification peuvent porter des noms et liens métier ; minimiser les snapshots, borner la rétention des dead-letter et restreindre les accès aux journaux.
- Les namespaces dynamiques sont validés, plafonnés et évincés ; les journaux ordinaires excluent les valeurs client non authentifiées. Le rendu HTML doit conserver l'échappement et les contrôles de liens.

Exploitation et reprise

- Deux modules Gradle, démarrage Spring, Flyway et migrations de code ; le runner de migrations de code ne prend pas de verrou global et les instructions signalent un arrêt avec code 0 en cas d'échec.
- Outbox réessayée et purgée seulement après envoi ; un crash après confirmation Kafka peut entraîner une rediffusion. Les consommateurs doivent conserver une idempotence compatible avec la rétention de replay.
- DLT terminale persistée avant l'acquiescement ; déduplication stable par coordonnées de DLT dans notification-center. Rejouer une DLT doit garder l'identité source et enregistrer l'opérateur.
- Indicateur de santé des listeners Kafka ; compléter par âge du plus ancien travail, taille de backlog, échecs IAM, rendus sans template, remises SMTP incertaines et clients temps réel rejetés.
- Le chart applique utilisateur non root, probes, références de secrets et options de volume. Le NetworkPolicy lu restreint l'entrée mais ne définit pas une restriction d'egress pour SMTP/IAM/bus.
- L'autoscaling est conditionné par la configuration de persistance ; la diffusion aux sockets et la réduction des accumulations doivent être qualifiées à plusieurs instances avant d'en déduire une scalabilité fonctionnelle.
- Les tests d'infrastructure demandent Docker et des conteneurs PostgreSQL/Kafka. Les tests existants couvrent notamment les canaux, duplications, templates, contrôle IAM et auth wire ; ils n'ont pas été exécutés dans cet audit.
- Detekt est configuré pour réécrire les fichiers : il n'a pas été lancé lors de cette lecture seule. Le garde des routes est présent dans la CI locale du service mais les instructions indiquent qu'il n'est pas inclus par la pipeline racine.

Séquence de migration

1. Exporter un inventaire anonymisé des abonnés, abonnements, médias, langues, actions utilisées et canaux effectivement livrés ; comparer aux 27 actions déclarées et aux trois templates.
2. Figurer les contrats HTTP, Protobuf, JSON interne et Socket.IO, avec fixtures et noms d'événements. Caractériser les lots bulk, suppression croisée et préférences médias sans supposer une correction compatible.
3. Rejouer le miroir IAM en parallèle sous indicateurs, réconcilier les identifiants, puis choisir une seule autorité de mise à jour et préparer la sortie de Keycloak.
4. Déployer le noyau Rust des préférences et de la boîte sur schéma étendu, migrer les UUID et dates, comparer les lectures puis basculer les commandes.
5. Migrer accumulations, outbox, marqueurs et quarantaines avec clés stables. Valider concurrence, crashes et replays avant bascule du regroupement et des effets.
6. Reprendre les rendus FR/EN/DE et qualifier les trois actions SMTP. Décider explicitement du cas EMAIL pour DATASET_COLLECT_FAILED et des autres actions sans template.

- 7. Qualifier le serveur temps réel et la diffusion entre instances avec les clients réels ; migrer les clients si le protocole change et imposer les ACL après observation shadow.
- 8. Retirer le bus et les composants JVM quand tous les producteurs/consommateurs sont basculés. Appliquer Expand/Migrate/Contract séparément, restaurer un backup complet en exercice et retirer les anciens secrets seulement après vérification.

Écarts et décisions encore ouverts

- Le mapping EntityActionTypeDAO partage mediaTypes par nom d'action. Reproduire deux abonnés avec canaux différents avant de conclure au comportement persistant et de choisir la conversion cible.
- La suppression par listes séparées et la déduplication du bulk changent la correspondance des couples fournis. Aucun incident runtime n'est établi ici ; caractériser les attentes des clients et les lots croisés.
- La déduplication d>ActionResource s'appuie sur les accumulations encore présentes. Quel horizon de replay est garanti après leur suppression ? Prévoir un registre durable dans la cible.
- Le réducteur ne montre pas un claim ou une transaction englobant sélection/publication/suppression. Confirmer par un scénario à deux instances et crash après outbox avant d'affirmer une duplication réelle.
- Le consumer de communication synchrone partage un groupe alors que les sockets sont locales. Vérifier les mécanismes de routage/broadcast effectivement déployés et la livraison sur deux pods.
- MessageBuilder ne rend pas DATASET_COLLECT_FAILED malgré la sélection EMAIL possible. Faut-il créer un template, corriger l'offre de canal ou conserver temporairement ce comportement ?
- NotificationDAO déclare une suppression SQL visant subscriber alors que son entité correspond à notification. Aucune route de suppression n'est exposée dans les controllers lus ; vérifier les chemins réellement appelés avant de traiter cette observation comme un défaut runtime.
- Quels producteurs émettent encore les actions Git et les actions de merge request de flow ? Le Protobuf les conserve mais une migration retire GIT_MR_REQUESTED des abonnements.
- Le consumer historique Subscriber et le nouveau consumer IAM peuvent coexister. Définir l'ordre, la source autoritaire et la résolution d'une adresse ou activation contradictoire.
- Quel délai maximal est accepté entre retrait d'un droit et un email déjà préparé ? Le contrôle actuel est placé avant accumulation et non juste avant l'appel SMTP.
- La sortie finale de JVM inclut le serveur Socket.IO, le bus Kafka, FreeMarker, JavaMail et Keycloak, pas uniquement les handlers Kotlin. Aucun substitut n'est déclaré qualifié par cet audit.

Références de l'analyse

[monorepo · services/notification-center/AGENTS.md:28](#) Contrats d'auth, indicateurs, outbox et contraintes d'exploitation.

[monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/notification/controller/NotificationController.kt:29](#) Surface de boîte personnelle.

[monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/subscription/controller/SubscriptionController.kt:49](#) Surface de souscription.

[monorepo · services/notification-center/chart/templates/deployment.yaml:50](#) Conteneur, probes, secrets et volumes.

[monorepo · services/notification-center/chart/templates/networkpolicy.yaml:23](#) Contrôle réseau entrant.

[monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/notification/repository/dao/NotificationDAO.kt:35](#) Mapping et annotation de suppression à caractériser.

SERVICE ACTUEL · MIGRATION-ORCHESTRATOR

Format et validation des migrations transverses

5 capacités analysées

services/migration-orchestrator · Python, Pydantic, PyYAML et CLI cleyrop-migrate-lint. Aucun exécuteur, API HTTP, schéma de runs, Dockerfile ou chart dans l'état documenté et le périmètre de fichiers lu.

Définir et vérifier un corpus de migrations de cluster hors schémas propres aux services. Le dépôt contient des outils de validation utilisables ; il ne livre pas encore de service d'exécution.

Lecture de code

Pas de qualification runtime nouvelle

Domaines cible : Migration, Audit, Maintenance, IAM.

Périmètre de lecture

- README, pyproject, corpus/schema.py, loader.py, targeting.py et lint/manifest.py lus.
- Absence de migration livrée indiquée par le composant ; aucune commande d'application exécutée.

Objets et cycle de vie

- Migration possède nom/version/ciblage et liste d'étapes ordonnées ; Step décrit precheck/apply/verify.
- LoadedMigration conserve le dossier source ; CorpusError et LintIssue localisent une erreur. Aucun objet Run durable n'est constaté.

Dépendances du service

- Pydantic et PyYAML pour les outils présents.
- Les exécuteurs SQL/Kubernetes/shell sont seulement des valeurs du schéma et une cible de conception, pas des adaptateurs exécutables livrés.

Fonctionnalités et compatibilité à préserver

MO-01 Cible non qualifiée

Valider la structure d'un manifeste de migration

COMPORTEMENT OBSERVÉ

Le modèle impose apiVersion cleyrop.tech/v1, kind Migration, un nom non vide, une version entière positive et au moins une étape. Chaque étape possède nom, executor sql/kubectl/shell, precheck et verify non vides ; apply est optionnel. Les noms d'étape doivent être uniques et les champs supplémentaires sont refusés. Cette validation décrit un contrat ; elle n'exécute ni SQL ni commande système.

UTILISATEURS ET RÔLES

Auteurs de migrations et relecteurs.

INTERFACES DANS LE CODE

corpus.schema.Migration ; Step ; Targets ; Migration.model_validate.

DONNÉES ET ÉTATS

Manifestes YAML et objets Pydantic en mémoire ; aucun run persistant.

DÉPENDANCES ET ÉCHANGES

Pydantic et format YAML du corpus.

COMPOTEMENT CIBLE PROPOSÉ

Migration Rust porte le même contrat versionné avec actions typées, droits et empreinte du manifeste. Le runner futur distingue observation, application et vérification, sans remplacer les migrations de schéma possédées par chaque service.

MIGRATION DE CETTE CAPACITÉ

Porter les fixtures valides et invalides vers le validateur Rust. Conserver les champs historiques ou définir une conversion explicite ; refuser les commandes non couvertes par le contrat.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un manifeste sans verify, avec étape dupliquée ou champ inconnu est refusé avant toute action. Une validation réussie ne crée aucun effet externe.

Références de l'analyse

[monorepo · services/migration-orchestrator/src/migration_orchestrator/corpus/schema.py:57](#) Contrat des étapes et blocs requis.

[monorepo · services/migration-orchestrator/src/migration_orchestrator/corpus/schema.py:112](#) Version, types et unicité des étapes.

MO-02 **Cible non qualifiée****Charger un corpus dans un ordre déterministe et refuser les doublons****COMPOTEMENT OBSERVÉ**

Le chargeur parcourt les sous-répertoires dans l'ordre lexicographique, ignore fichiers racine et dossiers commençant par point ou underscore, et exige manifest.yaml dans les autres dossiers. Il utilise un chargement YAML sûr, exige un mapping racine puis valide le modèle. Les erreurs conservent le chemin source. load garde le répertoire d'origine de chaque migration et refuse deux migrations portant le même nom, même dans deux dossiers distincts.

UTILISATEURS ET RÔLES

Auteurs du corpus et opérateurs.

INTERFACES DANS LE CODE

corpus.loader.discover ; load_manifest ; load ; CorpusError ; LoadedMigration.

DONNÉES ET ÉTATS

Liste ordonnée de manifestes validés et chemins de ressources relatives.

DÉPENDANCES ET ÉCHANGES

Système de fichiers du corpus et validateur de schéma.

COMPOTEMENT CIBLE PROPOSÉ

Le chargeur Rust fige un corpus et son digest avant exécution. Le runner persiste nom, version, hash, ordre et références de fichiers nécessaires pour qu'une reprise ne lise pas un corpus modifié silencieusement.

MIGRATION DE CETTE CAPACITÉ

Conserver l'ordre des préfixes et les erreurs localisées. Ajouter une politique de chemin pour les ressources de commande avant de permettre un exécuteur `sql_file`.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un dossier sans manifeste ou un nom dupliqué bloque le chargement. Une reprise avec le même nom/version mais un contenu différent est refusée tant qu'un changement de plan n'est pas approuvé.

Références de l'analyse

[monorepo · services/migration-orchestrator/src/migration_orchestrator/corpus/loader.py:64](#) Découverte déterministe et dossiers ignorés.

[monorepo · services/migration-orchestrator/src/migration_orchestrator/corpus/loader.py:131](#) Rejet des noms de migration dupliqués.

MO-03 **Cible non qualifiée****Déterminer si une migration cible le cluster****COMPORTEMENT OBSERVÉ**

`targets.clusters` comporte `include` et `exclude`. La valeur `all` dans `include` cible tous les clusters ; `exclude` prend toujours le dessus. Une liste `include` vide ne cible rien. Le matching est une comparaison des noms fournis, sans découverte de labels Kubernetes. La fonction retourne uniquement un booléen ; aucun saut ou `run` n'est encore persisté.

UTILISATEURS ET RÔLES

Exploitation multi-clusters.

INTERFACES DANS LE CODE

`corpus.targeting.applies_to` ; `corpus.schema.Targets`.

DONNÉES ET ÉTATS

Nom de cluster et listes de ciblage du manifeste.

DÉPENDANCES ET ÉCHANGES

Nom de cluster fourni par la configuration de l'outil.

COMPORTEMENT CIBLE PROPOSÉ

Migration Rust fige la décision de ciblage avec identité stable du cluster, version de plan et motif appliqué/ignoré. Une migration ne doit pas toucher un cluster hors scope même si son nom ou sa configuration dérive en cours de `run`.

MIGRATION DE CETTE CAPACITÉ

Reprendre les tests de la matrice `include/all/exclude`. Définir comment les noms historiques correspondent à une identité de cluster avant adoption du runner.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un cluster présent à la fois dans `include` et `exclude` est ignoré. `include` vide ne lance rien. La preuve de ciblage reste attachée au `run` repris.

Références de l'analyse

[monorepo · services/migration-orchestrator/src/migration_orchestrator/corpus/targeting.py:27](#) Priorité de `exclude` et sémantique de `all`.

MO-04 **Cible non qualifiée****Détecter les incohérences entre exécuteur et blocs d'action****COMPORTEMENT OBSERVÉ**

Le linter complète le schéma par des contrôles sémantiques. Il vérifie que precheck, apply et verify ne contiennent que les clés admises et que chaque bloc contient une entrée adaptée à executor. `sql_file` est admis à la place de `sql` uniquement pour `apply`. `target_db` vide est refusé pour SQL ; `target_db` est interdit aux autres types. Un ciblage dont `include` et `exclude` sont tous deux vides est signalé. Les contrôles portent la structure, pas la sûreté ni l'idempotence du SQL ou du shell.

UTILISATEURS ET RÔLES

Auteurs et reviewers de manifests.

INTERFACES DANS LE CODE

`lint.manifest._lint_migration` ; `_lint_step` ; `_lint_block` ; `LintIssue`.

DONNÉES ET ÉTATS

Liste de problèmes localisés par manifeste et étape.

DÉPENDANCES ET ÉCHANGES

Schéma partagé et chargeur YAML.

COMPORTEMENT CIBLE PROPOSÉ

Un validateur Rust conserve ces règles puis ajoute politique d'exécution, analyse des chemins, périmètre des credentials et préconditions de schéma. Le résultat lint reste distinct d'une preuve de migration réussie.

MIGRATION DE CETTE CAPACITÉ

Porter les diagnostics et leurs fixtures. Fermer l'écart du linter global qui charge chaque manifeste individuellement et ne passe pas par la vérification des noms dupliqués de load.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un bloc `kubectl` sous `executor sql` est refusé. `sql_file` dans `verify` est refusé. Deux noms identiques sont aussi détectés par la commande lint finale, au même titre que par le chargeur.

Références de l'analyse

[monorepo · services/migration-orchestrator/src/migration_orchestrator/lint/manifest.py:79](#) Cohérence `target_db` et blocs.

[monorepo · services/migration-orchestrator/src/migration_orchestrator/lint/manifest.py:138](#) Clés autorisées et correspondance avec `executor`.

[monorepo · services/migration-orchestrator/src/migration_orchestrator/lint/manifest.py:36](#) Parcours individuel qui ne vérifie pas les noms au niveau corpus.

MO-05 **Cible non qualifiée****Fournir une commande de validation avec codes de sortie exploitables****COMPORTEMENT OBSERVÉ**

Le paquet expose `cleyrop-migrate-lint` et un point d'entrée python `-m migration_orchestrator.lint`. La commande reçoit un répertoire, collecte les problèmes, les écrit sur `stderr` avec chemin et étape puis retourne 1. Si aucun problème n'est trouvé, elle annonce le nombre de migrations et retourne 0 ; un corpus vide peut donc réussir. Aucun `cleyrop-migrate apply`, API, worker d'exécution ou table de runs n'est livré dans le code lu. Le README précise aussi l'absence d'intégration CI/hook à cette étape.

UTILISATEURS ET RÔLES

Développeurs, CI et futurs opérateurs de migration.

INTERFACES DANS LE CODE

```
cleyrop-migrate-lint <corpus_path> ; python -m migration_orchestrator.lint <corpus_path> ;
lint.manifest.main.
```

DONNÉES ET ÉTATS

Sortie de commande et diagnostics ; pas d'historique d'exécution durable.

DÉPENDANCES ET ÉCHANGES

Packaging Python actuel ; futur runner Rust et API Kubernetes/SQL.

COMPORTEMENT CIBLE PROPOSÉ

La CLI Rust sépare lint, plan, apply, inspect et retry. Un runner futur prend un verrou et une époque, exécute des runs bornés, conserve preuves precheck/apply/verify et rend un reçu durable. L'autorité des commandes de migration reste PostgreSQL ; elle ne remplace pas les checkpoints publiés des datasets.

MIGRATION DE CETTE CAPACITÉ

Livrer d'abord le validateur compatible et son branchement CI. Construire ensuite un runner avec tests de crash et restauration avant toute première migration transverse réelle ; ne pas présenter les commandes futures comme existantes.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une validation invalide retourne non-zéro avec un chemin exploitable. Un corpus vide est rendu explicitement. Le futur apply interrompu entre action et verify reprend par precheck sans doubler l'effet.

Références de l'analyse

[monorepo · services/migration-orchestrator/src/migration_orchestrator/lint/manifest.py:183](#) Contrat CLI et codes de sortie.

[monorepo · services/migration-orchestrator/src/migration_orchestrator/lint/__main__.py:6](#) Point d'entrée module.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- Corpus immuable identifié par hash, plans ciblés, approbations concrètes, états durables et preuves d'application.
- Les migrations de schéma locales restent détenues par leur service et suivent Expand/Migrate/Contract.

Plan de données proposé

- Aucun plan d'exécution livré aujourd'hui. La cible lance des jobs bornés avec credentials propres au cluster et à l'action, puis vérifie le résultat.

Découpage du core Rust

- Validateur et chargeur compatibles ; planificateur de ciblage ; runner et magasin de reçus à construire.

- Adaptateurs SQL et Kubernetes étroitement typés ; shell uniquement dans un périmètre déclaré et auditable.

Identité, autorisation et isolation

- Un manifeste validé n'est pas une autorisation d'exécuter son contenu.
- Le runner doit borner namespace, base et permissions, conserver l'acteur et ne pas inclure de credentials dans ses manifestes ou rapports.

Exploitation et reprise

- Le composant actuel est une bibliothèque/CLI de validation, sans service à déployer ni corpus de migration rempli.
- L'exploitation cible exige exclusion concurrente, reprise, délais, observabilité et restauration avant adoption d'une première action réelle.

Séquence de migration

- Porter les contrats et fixtures de validation sans créer de faux parcours d'application.
- Brancher lint sur la CI et vérifier noms uniques au niveau corpus.
- Construire stockage de runs, verrou/époque et exécuteurs ; qualifier une migration canari réexécutable.
- Ajouter déploiement et intégration des mises à niveau seulement après preuve de reprise.

Écarts et décisions encore ouverts

- Le runner, ses credentials, la table de runs et l'API ne sont pas implémentés dans le périmètre constaté.
- La politique de résolution de sql_file et d'immutabilité du corpus doit précéder son exécution.
- Le commentaire du linter suggère un include vide explicite pour une migration garée, mais le code signale tout ciblage où include et exclude sont vides ; ce comportement doit être clarifié.
- Aucun lint ni test du composant n'a été exécuté dans ce lot.

Références de l'analyse

[monorepo](#) · [services/migration-orchestrator/README.md:21](#) État actuel, outils présents et absence du runner/déploiement.

[monorepo](#) · [services/migration-orchestrator/pyproject.toml:16](#) Unique commande livrée cleyrop-migrate-lint.

ÉTUDE DE DOMAINE

Les 34 composants de support : bibliothèques, charts et images

Le retrait des services Kotlin ne suffit pas à produire une plateforme sans JVM. Le contrat porte aussi sur les bibliothèques, générations de clients, runtimes, opérateurs, jobs et outils de secours.

packages : responsabilités et trajectoires

- Les lignes décrivent la disposition proposée de chaque dossier recensé. Les métadonnées du planner et les fichiers d'entrée étayent son rôle et ses dépendances ; elles ne constituent pas un audit ligne par ligne de tout le composant. Les gates ci-dessous restent à exécuter.

Composant	Rôle	Trajectoire proposée	Gate de retrait ou maintien
packages/cleyrop-common	Utilitaires Python communs	Conserver les adaptateurs utilisateurs ; transférer politique et invariants dans des crates Rust partagées.	Parité des erreurs, identifiants et contexte tenant.
packages/cleyrop-sdk	SDK de développement	Préserver Python/SQL et ajouter source(), manifests, pins, run et receipts.	Notebook existant migré sans écriture directe sur main.
packages/cleyrop-spring-auth	Authentification Spring et intégration FGA	Retirer la bibliothèque JVM ; extraire contrats et fixtures vers le middleware Rust.	Même décision sur une matrice de sujets/actions/ressources.
packages/cleyrop-testing	Fixtures et aides Python	Réutiliser les jeux de contrat entre anciens et nouveaux services.	Tests clients identiques contre API de compatibilité puis Rust.
packages/client-python	Client de plateforme	Générer un client versionné avec types de pins, erreurs et opérations asynchrones.	Pagination, cancellation et retries ne dupliquent pas les effets.
packages/downloads-site	Distribution des clients	Conserver le site et signer les distributions des clients migrés.	Liens, checksums et compatibilité des versions contrôlés.
packages/error-lib	Contrat d'erreurs JVM	Retirer le JAR ; garder codes stables et corrélation dans Rust et clients.	Erreurs machine stables, aucun détail de ressource interdite.
packages/keycloak-plugins	Extensions Keycloak	Reproduire les comportements nécessaires via IdP natif et IAM Rust.	Recette de fédération, claims, sessions et révocation.
packages/keycloak-theme	Écrans et packaging Keycloak	Porter les écrans vers l'IdP retenu, supprimer le JAR.	Connexion, récupération, MFA, accessibilité et langues.
packages/name-lib	Conventions de nommage JVM	Porter les règles utiles dans une crate sans dépendance JVM.	Corpus de noms existants et collisions conservé.

Composant	Rôle	Trajectoire proposée	Gate de retrait ou maintien
packages/proto	Contrats protobuf et génération	Préserver les contrats nécessaires ; remplacer Gradle par génération sans JVM.	Wire compatibility, champs réservés et clients mixtes.
packages/pyaicleyrop	Bibliothèque IA Python	Conserver côté utilisateur ; médiation des appels par core Rust et proxy gouverné.	Identité, quotas, isolation et provenance des artefacts.

Références de l'analyse

- [monorepo · packages/cleyrop-common/README.md:1](#) Point d'entrée et inventaire du composant packages/cleyrop-common
- [monorepo · packages/cleyrop-sdk/README.md:1](#) Point d'entrée et inventaire du composant packages/cleyrop-sdk
- [monorepo · packages/cleyrop-spring-auth/README.md:1](#) Point d'entrée et inventaire du composant packages/cleyrop-spring-auth
- [monorepo · packages/cleyrop-testing/README.md:1](#) Point d'entrée et inventaire du composant packages/cleyrop-testing
- [monorepo · packages/client-python/README.md:1](#) Point d'entrée et inventaire du composant packages/client-python
- [monorepo · packages/downloads-site/README.md:1](#) Point d'entrée et inventaire du composant packages/downloads-site
- [monorepo · packages/error-lib/README.md:1](#) Point d'entrée et inventaire du composant packages/error-lib
- [monorepo · packages/keycloak-plugins/README.md:1](#) Point d'entrée et inventaire du composant packages/keycloak-plugins
- [monorepo · packages/keycloak-theme/Dockerfile:1](#) Point d'entrée et inventaire du composant packages/keycloak-theme
- [monorepo · packages/name-lib/README.md:1](#) Point d'entrée et inventaire du composant packages/name-lib
- [monorepo · packages/proto/README.md:1](#) Point d'entrée et inventaire du composant packages/proto
- [monorepo · packages/pyaicleyrop/README.md:1](#) Point d'entrée et inventaire du composant packages/pyaicleyrop

charts : responsabilités et trajectoires

- Les lignes décrivent la disposition proposée de chaque dossier recensé. Les métadonnées du planner et les fichiers d'entrée étayent son rôle et ses dépendances ; elles ne constituent pas un audit ligne par ligne de tout le composant. Les gates ci-dessous restent à exécuter.

Composant	Rôle	Trajectoire proposée	Gate de retrait ou maintien
charts/cleyrop-backup	Sauvegarde plateforme	Étendre le manifeste de restauration à toutes les autorités et journaux.	DR-1 à DR-8, aucun effet relancé par restauration.
charts/cleyrop-common	Bibliothèque Helm	Réutiliser avec profils contrôle/données, capacités et service accounts séparés.	Lint et rendu : pas de secret partagé ni JVM transitive.
charts/cnpg-operator	Opérateur PostgreSQL	Conserver et qualifier sauvegarde/restauration des autorités de contrôle.	PITR réel, bascule et compatibilité des extensions.
charts/gitea	Versionnement des projets	Conserver pour code et drafts, distinguer branches Git et éditions de données.	Webhook rejoué une seule fois, accès projet et restauration.
charts/jupyterhub	Sessions de développement	Conserver le service utilisateur après changement des images et credentials.	Notebook sans JRE, lecture épinglée et écriture via Flows.

Composant	Rôle	Trajectoire proposée	Gate de retrait ou maintien
charts/kafka-operator	Déploiement du journal Kafka	Retirer Kafka/Strimzi ; qualifier un journal durable sans JVM.	Offsets, replay, rétention, panne disque/nœud et protocoles clients.
charts/keycloak-operator	Cycle de vie Keycloak	Retirer après migration de l'IdP et de toutes les fédérations.	Plus aucun realm, SPI ou initContainer JVM résiduel.
charts/postgres	Clusters PostgreSQL	Conserver les autorités nécessaires ; différencier caches reconstructibles et contrôle durable.	Chaîne WAL réellement restaurable et restauration coordonnée.
charts/registry	Distribution des images	Conserver, signer et inventorier les images finales par digest.	Images applicatives, jobs, hooks et initContainers sans JRE/JAR.
charts/sftp	Entrée fichiers SFTP	Conserver l'entrée protocolaire ; publier des FILESET après intégrité et scan.	Upload incomplet invisible ; doublon et retry sans double édition.
charts/trino	SQL distribué JVM	Retirer au terme de la qualification workload de Serving Rust et du moteur distribué.	Parité types, SQL, ACL, charge et annulation sur corpus client.
charts/valkey	Cache et échanges rapides	Conserver pour données dérivées ; aucune autorité de checkpoint source.	Perte totale du cache sans perte d'édition ni effet dupliqué.

Références de l'analyse

- [monorepo · charts/cleyrop-backup/README.md:1](#) Point d'entrée et inventaire du composant charts/cleyrop-backup
- [monorepo · charts/cleyrop-common/Chart.yaml:1](#) Point d'entrée et inventaire du composant charts/cleyrop-common
- [monorepo · charts/cnpg-operator/AGENTS.md:1](#) Point d'entrée et inventaire du composant charts/cnpg-operator
- [monorepo · charts/gitea/AGENTS.md:1](#) Point d'entrée et inventaire du composant charts/gitea
- [monorepo · charts/jupyterhub/Dockerfile:1](#) Point d'entrée et inventaire du composant charts/jupyterhub
- [monorepo · charts/kafka-operator/README.md:1](#) Point d'entrée et inventaire du composant charts/kafka-operator
- [monorepo · charts/keycloak-operator/README.md:1](#) Point d'entrée et inventaire du composant charts/keycloak-operator
- [monorepo · charts/postgres/AGENTS.md:1](#) Point d'entrée et inventaire du composant charts/postgres
- [monorepo · charts/registry/Chart.yaml:1](#) Point d'entrée et inventaire du composant charts/registry
- [monorepo · charts/sftp/AGENTS.md:1](#) Point d'entrée et inventaire du composant charts/sftp
- [monorepo · charts/trino/README.md:1](#) Point d'entrée et inventaire du composant charts/trino
- [monorepo · charts/valkey/AGENTS.md:1](#) Point d'entrée et inventaire du composant charts/valkey

hub : responsabilités et trajectoires

- Les lignes décrivent la disposition proposée de chaque dossier recensé. Les métadonnées du planner et les fichiers d'entrée étayent son rôle et ses dépendances ; elles ne constituent pas un audit ligne par ligne de tout le composant. Les gates ci-dessous restent à exécuter.

Composant	Rôle	Trajectoire proposée	Gate de retrait ou maintien
hub/ci	Construction des runtimes	Adapter matrice, SBOM et tests aux images sans JVM.	Aucune étape Gradle/Maven/java requise dans la chaîne finale.
hub/e2e	Acceptation des runtimes	Remplacer les attentes Spark par scénarios SDK/Flight/SQL qualifiés.	Même jeu utilisateur exécuté dans chaque image publiée.
hub/jupyterlab	Image notebook incluant Spark/Java	Reconstruire avec Python, Arrow et SDK ; moteurs via contrats explicites.	Import, dataframes, dépendances et kernels sans JRE.
hub/python	Image Python utilisateur	Conserver langage et isolation ; injecter seulement des capacités limitées.	Exécution bornée, secrets, egress et reprise contrôlés.
hub/shared	Ressources communes des runtimes	Revoir bootstrap, configuration et dépendances partagées.	Aucun chemin réintroduisant Spark, JAR ou droits généraux.
hub/spark	Runtime Spark	Retirer ; migrer les notebooks et jobs par corpus, pas par renommage d'image.	SQL, UDF, gros joins, skew, mémoire et délais mesurés.
hub/spark-base	Base Java/Spark	Retirer après extinction de toutes les images filles.	Graphe de dépendances et SBOM sans descendants JVM.

Références de l'analyse

- [monorepo](#) · [ci/components.json:1](#) Point d'entrée et inventaire du composant hub/ci
- [monorepo](#) · [ci/components.json:1](#) Point d'entrée et inventaire du composant hub/e2e
- [monorepo](#) · [hub/jupyterlab/Dockerfile:1](#) Point d'entrée et inventaire du composant hub/jupyterlab
- [monorepo](#) · [hub/python/Dockerfile:1](#) Point d'entrée et inventaire du composant hub/python
- [monorepo](#) · [ci/components.json:1](#) Point d'entrée et inventaire du composant hub/shared
- [monorepo](#) · [hub/spark/Dockerfile:1](#) Point d'entrée et inventaire du composant hub/spark
- [monorepo](#) · [hub/spark-base/Dockerfile:1](#) Point d'entrée et inventaire du composant hub/spark-base

images : responsabilités et trajectoires

- Les lignes décrivent la disposition proposée de chaque dossier recensé. Les métadonnées du planner et les fichiers d'entrée étayent son rôle et ses dépendances ; elles ne constituent pas un audit ligne par ligne de tout le composant. Les gates ci-dessous restent à exécuter.

Composant	Rôle	Trajectoire proposée	Gate de retrait ou maintien
images/alpine-tools	Outils opérationnels	Conserver après inventaire des scripts et capacités.	Jobs d'installation, secours et maintenance sans JVM.
images/cleyrop-devspace	Environnement de développement	Adapter aux clients natifs, génération protobuf et tests de contrat.	Parcours développeur complet sans Java.
images/trivy-with-curl	Image de contrôle sécurité	Conserver et intégrer les contrôles de composition des images.	Rapport par digest et politique de traitement des écarts.

Références de l'analyse

[monorepo](#) · [images/alpine-tools/Dockerfile:1](#) Point d'entrée et inventaire du composant images/alpine-tools

[monorepo](#) · [images/cleyrop-devspace/Dockerfile:1](#) Point d'entrée et inventaire du composant images/cleyrop-devspace

[monorepo](#) · [images/trivy-with-curl/Dockerfile:1](#) Point d'entrée et inventaire du composant images/trivy-with-curl

ÉTUDE DE DOMAINE

Icegres : bibliothèque de calcul, protocoles et writer de confiance

L'intégration reprend les composants de l'arbre local inspecté. Le service actuel n'est pas annoncé conforme aux invariants Hemera. Lecture gouvernée, écriture et maintenance ont des gates distincts.

Lectures, métadonnées et droits

- La cible expose Flight SQL et pgwire au travers du même résolveur de contexte. Celui-ci fixe tenant, projet, environnement, UUID de table, snapshot publié, version de politique et identité du lecteur avant toute ouverture de fichier. Une édition doit rester identique pendant pagination, statistiques, export et requête longue. Le résultat rend ses pins visibles au client.
- Flight reçoit des en-têtes de snapshot, timestamp et branche mais les refuse sur le chemin observé. Construire le moteur sur main ne suffit donc pas à servir les lectures épinglées envoyées par Eidos. Les listings Flight SQL doivent aussi filtrer tables et schémas avant de renvoyer noms ou cardinalités. Le filtre de tail.Tables ne prouve pas que toutes les interfaces appliquent la même politique.
- Le cache se partitionne par identité de ressource, édition, schéma et politique. Une révocation invalide les capacités d'accès et ne repose pas seulement sur la fermeture de la session frontend. Les identifiants Basic et jetons locaux observés sont à remplacer sur le chemin plateforme par un contexte authentifié issu de la gateway.

Capacité	Contrat proposé	Recette
Résolution SQL	Pins explicites et stables ; erreur si édition inaccessible	Changer main pendant une pagination ne change ni lignes ni schéma.
Découverte	Filtrage avant GetTables/GetSchemas et erreurs	Deux utilisateurs voient uniquement leurs ressources, y compris les noms.
Accès S3	Credentials limités aux objets autorisés	Le lecteur ne peut télécharger le staging par une autre API.
Caches et sessions	Clés comprenant politique et édition	Révocation et changement d'environnement ne réutilisent pas un résultat privé.

Références de l'analyse

- [icegres · icegres/src/flight.rs:528](#) Pins Eidos refusés sur le chemin Flight inspecté.
- [icegres · icegres/src/flight.rs:1719](#) Construction des métadonnées Flight SQL.
- [icegres · icegres/src/flight.rs:2156](#) Construction du moteur et référence main.
- [icegres · icegres/src/authz.rs:195](#) Adaptateur d'autorisation local, intégration OpenFGA à construire.

Écriture par opération et non par promesse globale

- Le core DML connaît un snapshot_id, mais le protocole Flight renvoie un nombre de lignes sur le chemin observé. Hemera exige un PreparedWrite et un reçu de publication. Une réponse SQL réussie ne devient pas à elle seule une preuve d'édition approuvée.
- Les préconditions d'UUID, tête et ascendance présentes dans Icegres sont à conserver. Elles doivent être étendues au contexte du publisher, à la génération de tentative et à la preuve d'audit du candidat exact. Des extra_properties de table ne remplacent pas le summary origin.* du snapshot.

- Les restrictions sur format v3, partitions et delete manifests dans certains chemins COW, bulk et compaction doivent rester explicites. La qualification est une matrice par opération, version de format, type de delete, partitionnement, évolution de schéma et concurrence. Aucun résultat de SELECT ne valide MERGE ou compaction.

Opération	Trajectoire	Gate de sortie
SELECT / scans	Réutiliser après intégration des pins et du PEP	v2, deletes, types et isolation par édition.
INSERT simple	Unifier le provider et le writer de confiance	Même contrat que bulk ; aucune publication directe.
UPDATE / DELETE / COW	Adapter DmlOutcome en candidat auditable	Conflit concurrent, suppression, nulls et évolution de schéma.
Bulk append	Conserver le streaming Arrow borné	Summary origin.*, limite mémoire, abort et objets orphelins.
Compaction	Nouvelle édition équivalente sous WAP	Équivalence logique, provenance et lecteurs actifs.
Expiration / GC	Plan approuvé et journal de suppression	Protéger releases, sauvegardes, staging et lecteurs ; reprise partielle.

Références de l'analyse

[icegres · icegres/src/flight.rs:1016](#) Réponse DML du protocole.

[icegres · icegres/src/overwrite.rs:216](#) DmlOutcome porte le snapshot.

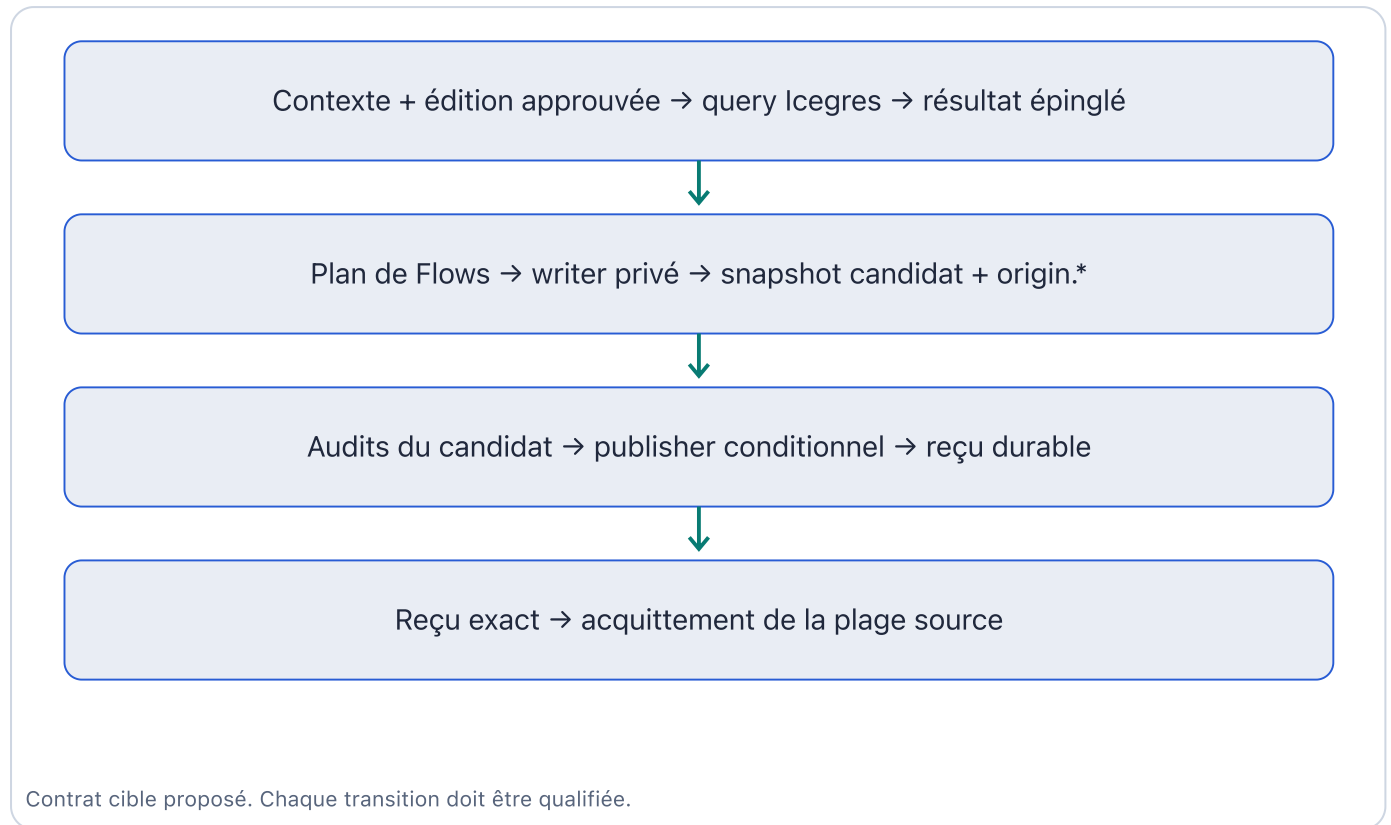
[icegres · icegres/src/overwrite.rs:1630](#) Préconditions de commit.

[icegres · icegres/src/overwrite.rs:2324](#) Construction du summary et propriétés.

[icegres · icegres/src/overwrite.rs:1915](#) Restrictions des chemins d'écriture.

Séparer capture rapide et vérité publiée

- Le buffer tail peut acquitter avant commit et superposer des données non publiées. Ce comportement n'est pas utilisable comme vérité gouvernée. Sa reprise par maximum de propriété de table et sidecar ne satisfait pas origin.cursor sur la dernière édition publiée.
- Si cette capture est conservée, elle devient un journal privé avec contrat de durabilité explicite. Flows en découpe des plages bornées, fait préparer un candidat, attend ses audits et sa publication, puis acquitte la plage source couverte. Les requêtes produit ne mélangent pas overlays tail et éditions approuvées.
- Le découpage proposé isole les crates de résolution/catalogue, politique, planification SQL, exécution, writer Iceberg et maintenance. Le service query ne possède pas le droit de publier. Un worker générique envoie des données ou un plan autorisé au writer de confiance, sans token Lakekeeper modify ni clés générales du bucket.



Références de l'analyse

- [icegres](#) · [icegres/src/buffer.rs:3](#) Sémantique de buffer et acquittement.
- [icegres](#) · [icegres/src/buffer.rs:1810](#) Reprise des checkpoints du buffer.
- [icegres](#) · [icegres/src/compact.rs:613](#) Commit du chemin de compaction.
- [icegres](#) · [icegres/src/maintain.rs:143](#) Protections existantes du GC à préserver.

ÉTUDE DE DOMAINE

Eidos : sémantique gouvernée et actions préparées

Eidos apporte des définitions métier, un compilateur, des propositions d'action et des projections. L'intégration exige de relier chacune de ces fonctions aux identités, éditions et publications Hemera.

Modèles, compilation et lecture métier

- La version publiée du modèle sémantique devient un bundle immuable référencé par une édition Iceberg de manifeste. Les drafts, demandes de changement et commandes demeurent dans un stockage de contrôle durable. L'état PostgreSQL actif n'est pas implicitement une publication approuvée.
- Les bindings doivent identifier les UUID de tables et les IDs de champs, puis fixer les éditions compatibles. Des noms de tables et de colonnes seuls ne résistent pas aux renommages, créations et dérives de schéma. Un plan conserve la version du compilateur, le bundle sémantique et les pins de données.
- Le chemin `authorized_view` contrôle OpenFGA, mais d'autres routes de registry, export ou mutation de branche demandent une revue homogène. L'existence de ce helper ne prouve pas une protection globale du serveur. Les identités de développement fondées sur `x-eidos-user` et les claims vides ne doivent pas devenir l'identité de production.

Fonction	Contrat cible	Acceptation
Registry et branches	Draft versionné, approbation puis édition sémantique	Un update du draft ne modifie pas la version utilisée par un lecteur épinglé.
Compiler / bindings	UUID, field IDs, contrat de types et version du bundle	Renommage, suppression et création détectés avant exécution.
Query / counts / exports	Même pin et même politique sur toutes les routes	Counts, liste et export correspondent à la même édition.
Recherche métier	Filtrage avant résultat, statistiques et citations	Aucune fuite entre instances ou tenants, même via une erreur.

Références de l'analyse

`eidos` · `crates/eidos-server/src/state.rs:197` Vue autorisée et contrôle OpenFGA.

`eidos` · `crates/eidos-server/src/api/registry.rs:19` Entrées du registre à protéger.

`eidos` · `crates/eidos-server/src/api/exports.rs:17` Chemin export à aligner sur la vue autorisée.

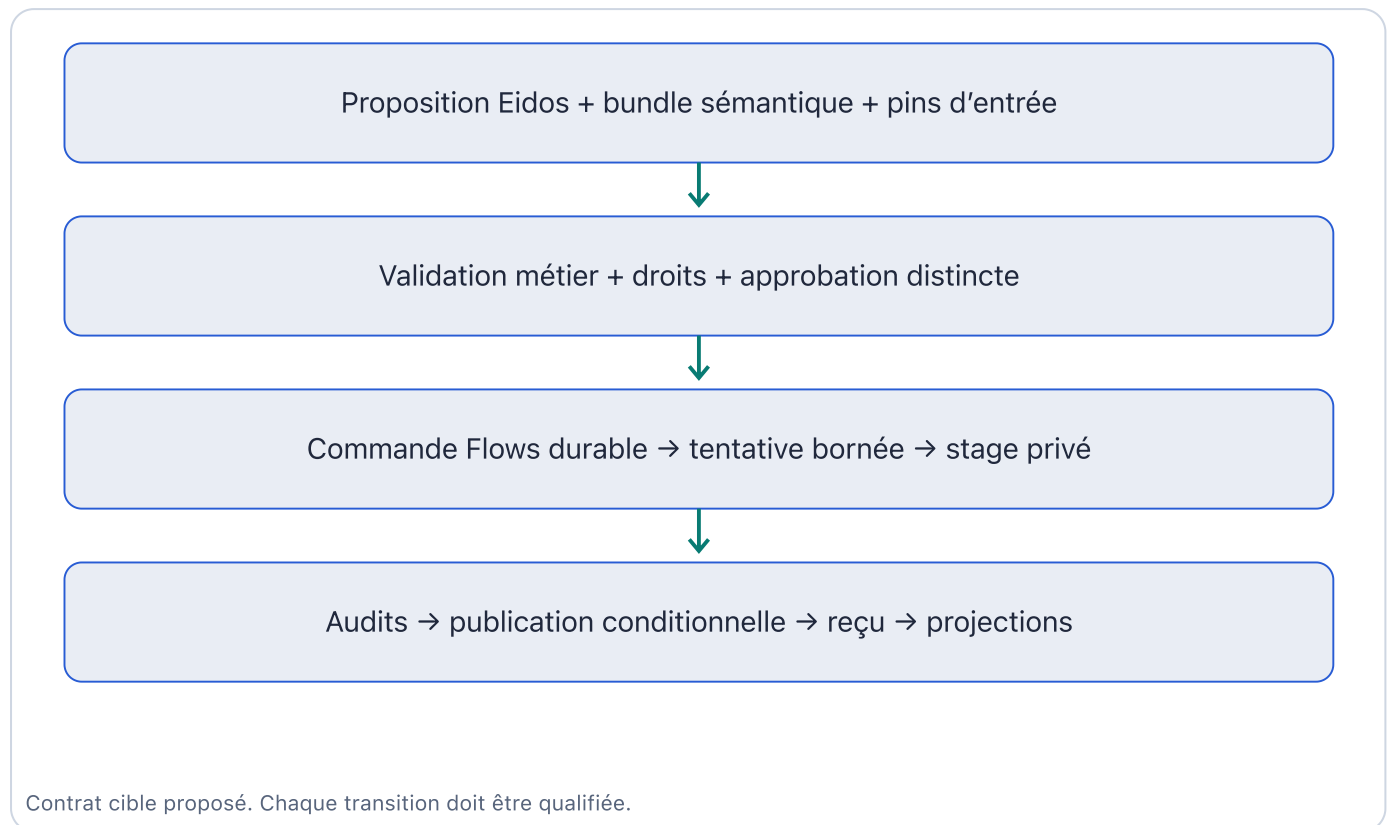
`eidos` · `crates/eidos-compiler/src/model.rs:380` Représentation des bindings compilés.

`eidos` · `crates/eidos-core/src/pg.rs:325` Persistance et activation du registre.

De la proposition à une édition publiée

- La validation d'une proposition et le contrôle d'approbateur constituent des points de départ utiles. Cependant, plusieurs statements appliqués séparément, un snapshot de départ éventuellement absent et un undo qui lit puis écrit sans CAS ne forment pas une transaction Hemera.
- Une action autorisée produit un plan immuable. Flows vérifie les pins, les droits actualisés, les limites et la séparation demandeur/approbateur, puis prépare les écritures sur staging. Le publisher valide toutes les préconditions de publication. Une action touchant plusieurs sorties publie une release autoritaire dont les lecteurs résolvent les membres ensemble.

- Undo devient une compensation : une nouvelle action contre l'état courant, avec nouvelles préconditions et nouvel audit. Il ne rétracte pas un export, une notification ou un ack source déjà réalisé. Les reçus de ces effets ont un journal propre.



Références de l'analyse

- eidos · [crates/eidos-core/src/functions/proposal.rs:16](#) Validation des propositions.
- eidos · [crates/eidos-server/src/engine/apply.rs:47](#) Contrôles lors de l'application.
- eidos · [crates/eidos-server/src/engine/apply.rs:129](#) Application des statements.
- eidos · [crates/eidos-server/src/engine/apply.rs:267](#) Gestion d'un snapshot absent.
- eidos · [crates/eidos-server/src/engine/undo.rs:69](#) Compensation et vérifications de snapshot.

Automatisations, embeddings et read models

- Les projections conservent l'édition source, la version du modèle, le watermark et la version de politique. Le choix du snapshot au plus grand committed_at est à remplacer par la résolution d'une édition autorisée. Lire les counts puis résoudre un snapshot peut produire un affichage incohérent si main a changé entre les deux.
- Les embeddings sont reconstruits depuis une entrée épinglée et un modèle versionné. Une collection nommée par type et un backfill qui relit l'ancre courante à chaque page ne suffisent pas. Le proxy sert seulement la collection compatible avec l'édition demandée et applique l'autorisation par instance.
- Le couple événement/outbox/progression transactionnel de l'automator est réutilisable. Les boucles de polling doivent recevoir leases, déduplication par moniteur/édition et fencing au point d'effet. Rejouer une projection ne relance pas une action métier déjà confirmée.

Références de l'analyse

- eidos · [crates/eidos-core/src/flight/exec.rs:305](#) Sélection par committed_at.
- eidos · [crates/eidos-core/src/flight/headers.rs:86](#) Pins envoyés au protocole Flight.
- eidos · [crates/eidos-core/src/readmodel/refresh.rs:30](#) Lecture des counts et résolution de snapshot.
- eidos · [crates/eidos-server/src/semantic.rs:419](#) Parcours de backfill.

[eidos](#) · [crates/eidos-core/src/automate/automator.rs:183](#) Outbox et progression.

[eidos](#) · [crates/eidos-server/src/loops.rs:19](#) Boucles à rendre fongibles et reprises idempotentes.

ÉTUDE DE DOMAINE

Architecture cible : domaines Rust, contrats et états

Ce découpage représente des responsabilités, pas une obligation de déployer dix-huit microservices dès le premier lot. Plusieurs cores peuvent partager un binaire initial tout en séparant leurs autorisations, données et interfaces.

Responsabilités de contrôle et de données

- Chaque domaine métier possède un core Rust testable sans réseau, entouré d'adaptateurs HTTP/gRPC, catalogue, stockage et ordonnanceur. Les frontends TypeScript et les workloads Python/SQL restent des interfaces et des exécutants. Un tiers est remplaçable derrière le contrat du domaine ; il ne choisit ni droits, ni édition autoritaire, ni checkpoint.
- Les appels de contrôle acceptent des clés d'idempotence et renvoient une opération consultable. Les événements transportent des identifiants et versions ; un consommateur vérifie l'autorité avant de reconstruire sa projection. La compatibilité API peut préserver les routes existantes pendant que leur propriétaire change.

Domaine proposé	Plan de contrôle	Plan de données	Autorité / contrat
Gateway / contexte	Sessions et API compatibles ; contexte signé	Routage, quotas et limites de requête	Identité, projet, environnement, corrélation ; fail-closed
IAM / politique	Organisations, projets, membres, invitations, comptes et droits	Réconciliation IdP/OpenFGA, contrôle avant accès	Journal de commandes et politiques versionnées
Connections / egress	Connecteurs, secrets référencés, découverte et validation	Tests bornés et proxy sortant autorisé	Pas de secret dans manifeste, log ou édition
Flows	Manifests, graphes, versions, schedules, leases, cancellations	Placement et supervision de runs bornés	Commandes durables ; checkpoint lu dans édition publiée
Capture / journal	Configuration des protocoles et rétention	Capture durable mutualisée ; découpage de pages	Époque, partitions, ordre et preuve de durabilité
Trusted writer	Validation du plan, contexte et capacités	Écriture privée, fichiers immuables, candidat estampillé	PreparedWrite ; aucune capacité modify donnée au worker
Audits / publisher	Règles, approbations, préconditions et décisions	Audit du snapshot exact, publication et reçu	UUID, heads, ascendance, génération et preuve immuable
Catalog / lineage	Identité dataset, descriptions, labels, contrats, éditions	Indexation dérivée des publications autorisées	Découverte filtrée, provenance et pins
Serving / Icegres	Sessions, requêtes, profils de placement et quotas	Flight SQL, pgwire, Arrow, exécution native	ReadContext épinglé ; aucun droit implicite de publier
Files	Réservations, métadonnées, ACL et versions	Transfert, scan, hash, manifeste FILESET	URL limitée, objets privés avant publication

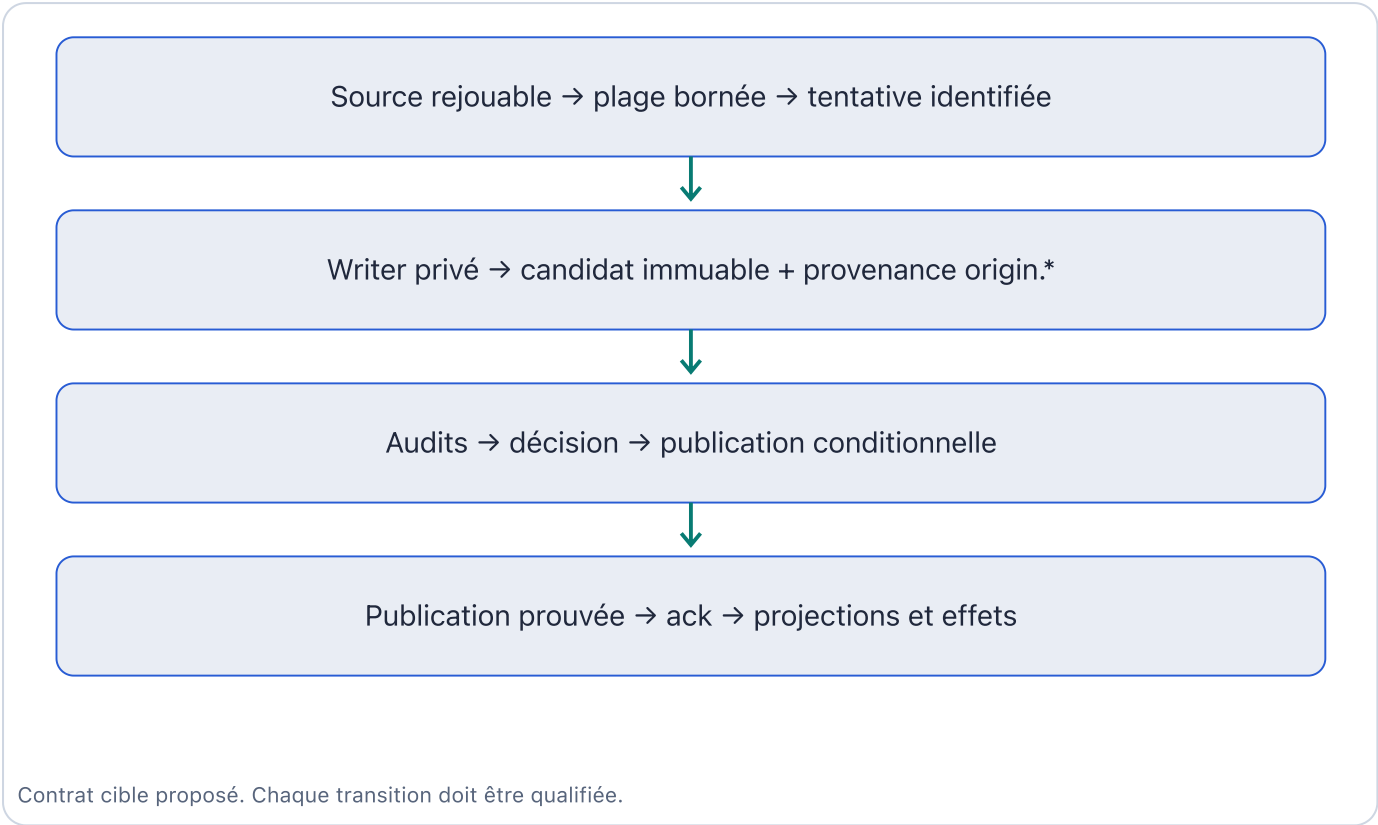
Domaine proposé	Plan de contrôle	Plan de données	Autorité / contrat
Eidos / semantic	Drafts, bundles, types métier et propositions	Compilation, projections, actions via Flows	Plan immuable, bundle et pins, aucune mutation main directe
Apps / devspaces	Spécifications, versions, routage, quotas et états désirés	Pods, sessions, logs et déploiements	Journal de contrôle indépendant du dataset
AI gateway / assistant	Conversations, budgets, outils, modèles et politiques	Appels inférence et outils sous identité déléguée	Consentement/approbation des effets et provenance des réponses
Embeddings / features	Spécifications, modèles et fraîcheur	Calcul et projection des éditions offline	Index et store online reconstruisibles, watermarks explicites
MCP / exposure	Outils publiés, schémas, permissions et consommateurs	Adaptateurs requête/recherche/action	Autorisation par ressource ; sources externes non fiables
Exports / delivery	Plan de livraison, format, destination et droits	Transfert d'une édition épinglée	Journal d'effets, clé idempotence et reçu externe
Notifications	Abonnements, templates, préférences et langues	Outbox, envoi et reprise des canaux	Déduplication et suppression d'informations privées
Operations / migration	Capacités, backups, migrations et rétention	Restore, compaction, jobs et purge planifiée	Plan approuvé, journal d'application et preuves de reprise

Références de l'analyse

[study](#) · [docs/hemera-v2/08-plateforme-rust-sans-jvm.html:50](#) Architecture consolidée, responsabilités et autorité des éditions.

Contrat commun de publication

- Les noms ci-dessous sont des propositions d'interface. Ils désignent des contenus obligatoires et doivent être versionnés dans les contrats partagés avant implémentation. Aucun champ isolé ne suffit à prouver une publication.
- La preuve d'audit est produite après le candidat et liée durablement à celui-ci. Elle ne peut pas être ajoutée rétroactivement par un simple fast-forward du snapshot. Le registre des publications et ses références retenues permettent de distinguer un snapshot approuvé d'un stage, y compris après une réponse réseau perdue.



Message	Contenu	Règle de validation
ReadContext	Sujet, tenant, projet, environnement, table UUID, snapshot ou release, politique	Résolu une fois, filtré puis transmis sans substitution silencieuse.
SourceRange	Source stable, époque, partitions, début/fin, transactions complètes	La plage couverte et le curseur publié restent cohérents après crash.
PreparedWrite	operation/attempt IDs, génération, table UUID, base heads, candidat, schéma/spec, summary origin.*, hashes	Les fichiers audités deviennent immuables pour le worker.
AuditDecision	Candidat exact, version des règles, résultats, approbateur, hash de preuve	Veto interdit la publication ; toute modification crée un nouveau candidat.
PublishReceipt	Publication autorisée, snapshot/release, candidat audité, opération, plage couverte	Trouvable après perte de réponse ; pas déduit du listing brut des snapshots.
AckReceipt	Système source, plage réellement acquittée, reçu de publication, statut observé	ACK_PENDING se réconcilie sans republier les données.
Release	Édition de manifeste, participants, pins membres, checkpoint commun	Lecteur voit la release ancienne ou nouvelle, sans mélange de membres.

Références de l'analyse

[study](#) · [docs/hemera-v2/09-decision-el4-wap.html](#):25 Frontière EL-4 et protocole writer/publisher.

Huit kinds, une autorité de publication

Kind	Contenu autoritaire	Contrat spécifique
TABLE	Données tabulaires	Snapshot, schéma, pins d'entrée et audits.
INCREMENTAL	Mise à jour bornée	Clés, stratégie de merge, plage source et idempotence.
STREAM	Raw rejouable publié par lots	Journal durable, ordre, rétention et audits avant ack.
VIEW	Définition publiée	Manifeste versionné et contrat de résolution des entrées.
FILESET	Ensemble d'objets	Manifeste Iceberg, objets immuables, hashes, ACL et rétention.
MODEL	Artefact de modèle	Manifeste, poids, signature, code, jeux d'entraînement et évaluations.
EMBEDDINGS	Vecteurs publiés	Modèle, inputs épinglés, vecteurs ; index dérivé.
FEATURESET	Features offline	Édition source ; store online dérivé avec watermark.

Références de l'analyse

[study](#) · [docs/hemera-v2/visuels/content.py:195](#) Taxonomie corrigée et distinctions EXPORT/EXPOSURE.

Manifeste, outils tiers et EL-4

- `source()` déclare le type de source, son identité stable, sa classe de durabilité, ses références de secrets, le contrat de schéma, la stratégie de curseur, la fenêtre bornée et ses audits. Les objets `SOURCE`, `OP` et `TEST` forment le graphe du flow ; `EXPORT` est un effet externe et `EXPOSURE` une déclaration de consommateur.
- OLake reste un extracteur Go encapsulé. Le sidecar writer Java ne fait pas partie de la cible. Si le writer Rust direct n'est pas qualifié pour une source, l'extraction aboutit à un journal privé puis à une édition raw audité. `STREAM` ne dispense jamais d'audit avant publication.
- `dbt`, `SQLMesh`, `dlt` et les workloads Python reçoivent des entrées épinglées et une sortie staging. Leur state interne peut accélérer l'exécution, mais la reprise source repart du checkpoint publié. Les scénarios de qualification incluent bootstrap/CTID, transactions multi-tables, `TRUNCATE`, absence de lignes mais avancement de curseur, slot partagé, crash et réponse de commit perdue.
- DataFusion est un core de calcul, pas une preuve de moteur distribué. Le placement interactif Icegres, les jobs Python/DuckDB et le calcul distribué ont des matrices distinctes. Ballista et Sail restent des candidats du dossier 08, à qualifier par version et workload ; leurs possibilités ne doivent pas être fusionnées en une promesse unique.

Références de l'analyse

[study](#) · [docs/hemera-v2/08-plateforme-rust-sans-jvm.html:159](#) Placement et limites des candidats de calcul.

[study](#) · [docs/hemera-v2/09-decision-el4-wap.html:25](#) Décision EL-4 sans writer Java.

ÉTUDE DE DOMAINE

Exploitation, restauration et retrait progressif des JVM

La migration avance par cohortes de workloads acceptés. Une durée globale ou un niveau de performance ne sont pas chiffrés sans volumétrie, corpus client et mesure.

Ordre de migration et critères de sortie

Lot	Périmètre	Preuve exigée
M0 · Contrats	Inventaire, compatibilité API, jeux de données et autorité des états	Toutes les capacités de la matrice ont un propriétaire et une recette priorisée.
M1 · Confiance	IAM/PEP, contexte, staging privé, writer/publisher et journal	Veto, fuite staging, zombie/failover et crash publish/ack testés.
M2 · Lecture	Catalogue, Icegres read-only, front/SDK et Eidos projections	Pins stables, découverte filtrée, golden SQL et révocation.
M3 · Ingestion	Une cohorte de connecteurs et FILESET	Bootstrap, incrémental, delete, replay et restauration validés.
M4 · Transformation	Flows, data-dev/process et mutations Eidos	Runs bornés, audit, release multi-sorties et équivalence workload.
M5 · Usages	BI, apps, devspaces, assistant, MCP, ML et exports	Parcours utilisateurs complets et effets externes idempotents.
M6 · Extinction	IdP natif, journal natif, retrait Spark/Trino et chaînes JVM	Inventaire images/processus/jobs/CI sans JVM, restore et exploitation acceptés.

Références de l'analyse

[study](#) · [docs/hemera-v2/08-plateforme-rust-sans-jvm.html:71](#) Périmètre du retrait JVM et gates associés.

Durabilité et reprise

- Le manifeste de sauvegarde coordonne catalogue, objets, UUID/éditions, politique OpenFGA, IAM, clés, drafts, commandes, annulations et reçus d'effets. Une date identique sur plusieurs backups n'établit pas leur compatibilité. Avant réouverture, le plan de données vérifie effectivement manifests, data files, delete files et artefacts.
- Le chart PostgreSQL inspecté prévoit des snapshots quotidiens et l'archivage WAL désactivé par défaut ; le profil micro désactive les sauvegardes. Une configuration n'est pas une preuve de RPO. La rétention doit calculer l'âge réel du point restaurable le plus ancien, en tenant compte des sauvegardes manquées.
- Un rollback de lecture ne déplace pas le checkpoint actif. Restaurer avant des acks irréversibles exige le journal couvrant l'écart. On ne peut ni supposer les messages encore présents ni copier un curseur avancé sur des données anciennes. Les seuils RPO/RTO doivent être contractés par profil puis mesurés.
- Les métriques suivent délai capture/publication/ack, lag des projections, échecs d'audit, conflits, zombies refusés, backlog, quotas, coût par workload et couverture de rétention. Les logs corrént opération/tentative/édition sans contenir secrets ou données sensibles. Le diagnostic distingue timeout, veto, conflit, ack incertain et incident de politique.

Références de l'analyse

[monorepo · charts/postgres/values.yaml:245](#) Paramètres de sauvegarde du chart inspecté.

[monorepo · charts/postgres/templates/retention-cronjob.yaml:103](#) Logique de rétention à qualifier.

[icegres · icegres/src/overwrite.rs:1196](#) Racines d'expiration actuelles à étendre.

Huit campagnes de restauration à rendre exécutables

- Ces scénarios sont des critères proposés, pas des résultats annoncés. Ils complètent I1–I12 et les tests propres à chaque fonctionnalité.

Campagne	Injection	Résultat attendu
DR-1	Perdre uniquement les projections	Reconstruire éditions et statuts sans nouvelle publication ni effet.
DR-2	Perdre la réponse de publication avant ack	Retrouver le reçu exact, puis réconcilier l'ack sans doublon.
DR-3	Restaurer le catalogue derrière les acks	Prouver la couverture du journal, sinon bloquer et rebootstrap sous nouvelle époque.
DR-4	Manquer plusieurs sauvegardes puis lancer la rétention	Conserver toute chaîne WAL nécessaire à un point encore annoncé restaurable.
DR-5	Restaurer IAM et données à des points différents	Rester fermé tant que cohérence, révocations et suppressions ne sont pas réconciliées.
DR-6	Basculer avec une ancienne tentative vivante	Refuser la publication zombie au point de commit, même si la tête n'a pas changé.
DR-7	Interrompre une release de plusieurs sorties	Lecteurs sur ancienne ou nouvelle release complète ; checkpoint commun.
DR-8	Purger avec lecteurs, staging et backups actifs	Aucun objet requis supprimé ; journal de purge repris sans perdre ses racines.

Références de l'analyse

[study · docs/hemera-v2/08-plateforme-rust-sans-jvm.html:223](#) Résilience et migration de la cible.

ÉTUDE DE DOMAINE

Parcours utilisateurs et recette de compatibilité

Le dossier relie les fiches techniques aux usages. Les critères ci-dessous valent pour le frontend, le SDK et les interfaces automatisées lorsqu'ils exposent la même opération.

Parcours à conserver de bout en bout

Persona	Séquence	Critère visible
Administrateur	Créer projet, inviter, déléguer, configurer secrets et egress	Droits effectifs expliqués ; révocation vérifiée sur données, listings et URLs déjà émises.
Data engineer	Connexion → source → bootstrap → incrémental → reprise	Plage observée, plage préparée, édition publiée et ack affichés séparément.
Analyste	Trouver dataset → prévisualiser → SQL → exporter	Même édition sur pagination, statistiques, requête et export ; provenance consultable.
Développeur	Dev zéro-copie → draft → run → audits → promotion	Branches Git, branches métier et refs Iceberg identifiées ; promotion conditionnelle.
Steward / approubateur	Décrire → classifier → auditer → approuver ou veto	Preuve du candidat exact ; aucun bouton pour forcer un veto.
Data scientist	Notebook → features → entraînement → modèle → app	Données, code, artefacts et évaluations liés ; routage app dans son contrôle durable.
Utilisateur métier	Explorer Eidos → proposition → approbation → résultat	Action passe par Flows ; undo devient compensation sur le nouvel état.
Utilisateur assistant	Question → outils → citations → action éventuelle	Permissions du sujet conservées ; approbation explicite d'effet, coûts et sources visibles.
Exploitant	Incident → diagnostic → restauration → réouverture	Statuts et journaux cohérents ; aucun effet rejoué silencieusement.

Références de l'analyse

[study](#) · [docs/hemera-v2/03-design-frontend.html:1](#) Parcours historiques dont les exceptions sont corrigées par les addenda.

Contrat d'interface pour une édition stable

- Le frontend conserve le contexte identité/projet/environnement/édition/version sémantique dans ses clés de cache et ses requêtes. Une bascule d'environnement invalide les appels pendants. Un tableau paginé doit afficher le pin de lecture, pas une valeur latest recalculée entre deux pages.
- Les événements de progression utilisent une séquence rejouable ou imposent un rechargement complet. Un événement SSE perdu ne change pas l'autorité d'une publication. L'interface distingue PREPARED, AUDITING, VETOED, CONFLICT, PUBLISHED, ACK_PENDING et résultat de commit inconnu à réconcilier.

- Les clients reçoivent des erreurs versionnées avec `operation_id` et informations suffisantes pour décider d'un retry. Un retry automatique n'est permis que lorsque l'idempotence du contrat le couvre. Annuler une tentative publiée ne promet pas de rétracter un ack ou une livraison externe.

Références de l'analyse

[monorepo](#) · [services/cleyrop-front-end/app/src/entities/dataset/dataset.service.ts:49](#) Requêtes et polling des datasets.

[monorepo](#) · [services/cleyrop-front-end/app/src/utils/xhrSource.ts:3](#) Transport SSE actuel.

[monorepo](#) · [services/cleyrop-front-end/app/src/entities/authz/authz.service.ts:44](#) Appels d'autorisation frontend.

Recette transverse minimale

Famille	Scénario	Acceptation
Concurrence	Deux approbations contre la même base	Une publication valide ; l'autre conflit explicite, aucune perte silencieuse.
Confidentialité	Reader tente les octets staging et une ressource voisine	Refus avant/après veto, via SQL, catalogue, URL et cache.
Panne	Arrêt worker avant/après publication et avant/après ack	Reprise documentée, reçu exact, ni perte ni double effet.
Schéma	Renommage, ajout, type incompatible, changement spec	Compatibilité validée ou blocage lisible ; pas de binding par nom erroné.
UI	Révocation et changement de contexte durant pagination/export	Pas de fuite ni mélange d'éditions.
Événements	SSE perdu, doublé, désordonné puis reconnexion	Projection réparée et commandes non répétées.
Accessibilité	Clavier, focus, contraste, langues et écran mobile	Toutes les actions critiques utilisables, statuts non fondés sur la couleur seule.
Performance	Gros scan/join, skew, mémoire, quotas, cancellation	Mesures reproductibles par workload ; placement refusé s'il ne tient pas le contrat.

Références de l'analyse

[study](#) · [docs/hemera-v2/10-preuves-et-campagnes.html:20](#) Distinction entre résultats du banc et gates encore ouverts.

ÉTUDE DE DOMAINE

Comparaison avec iceberg-data-platform et intégration dans Cleyrop

La référence clonée au commit 4506d8a82833bef3cd8edd2d613a2285352521b3 est un laboratoire local éducatif. Elle fournit des parcours utiles à reprendre, mais ne démontre ni multi-tenant Cleyrop, ni WAP confidentiel, ni exploitation souveraine.

Ce que Cleyrop reprend et ce qui change

Domaine	Référence observée	Différence cible Cleyrop	Réutilisation proposée
Catalogue	Polaris, schémas, snapshots et branches visibles	Conserver Lakekeeper ; exposer uniquement les éditions autorisées et leur provenance.	Détail dataset clair, navigation schéma/partitions/historique.
Identité	Keycloak, utilisateurs, équipes et rôles	IAM/OpenFGA et IdP sans JVM ; fédérations, délégation et révocation à qualifier.	Écran de connexion et droits effectifs compréhensibles.
SQL	DuckDB et connexions depuis outils	Icgres/Serving gouverné avec pins, quotas et moteurs qualifiés.	Instructions de connexion par environnement et client.
Sémantique	Parcours métier et navigation du portail	Eidos comme compiler/bundle/projections ; actions via Flows.	Navigation objet/relation reliée aux données publiées.
Partage	Parcours de partage du laboratoire	Shares réversibles, versionnés et soumis aux droits Cleyrop.	Prévisualiser ce que le destinataire verra réellement.
Développement	Notebooks marimo et environnement local	Runtime optionnel derrière Apps/Devspaces, sans credential général.	Onboarding rapide et exemples exécutables sur données autorisées.
MCP	Accès agent aux ressources du laboratoire	Outils bornés, identité déléguée, audit et contrôle des effets.	Découverte d'outils et sources avec permissions explicites.
Stockage	RustFS/S3 en Compose	Fournisseur S3 qualifié selon déploiement client ; aucun remplacement obligatoire.	Configuration lisible et validation de connexion.
Exploitation	Démarrage local et provisioning de démo	Kubernetes, backups coordonnés, journal, quotas et reprise ; zéro JVM final.	Documentation reproductible, sans Docker socket ni secrets admin dans l'usage produit.

Références de l'analyse

- [reference](#) · [README.md:1](#) Présentation et assemblage de la référence.
- [reference](#) · [docs/architecture.md:1](#) Architecture du laboratoire.
- [reference](#) · [SECURITY.md:1](#) Périmètre et limites de sécurité.
- [reference](#) · [user_portal/README.md:1](#) Parcours du portail.
- [reference](#) · [user_portal/public/catalog.js:113](#) Navigation catalogue inspectée.

Lots d'intégration à faible couplage

- Lot R1 : reprendre la présentation des connexions et le détail dataset dans le frontend existant, avec l'API de compatibilité et les éditions publiées. Acceptation : aucun accès staging, connexion vérifiée pour chaque rôle et environnement.
- Lot R2 : intégrer la navigation sémantique Eidos et les usages SQL sur Icegres après fermeture des gates de pins et d'autorisation. Acceptation : une même édition explique schéma, résultats, counts et citations.
- Lot R3 : proposer un runtime notebook optionnel et des outils MCP gouvernés. Acceptation : credentials limités, runs bornés, provenance conservée et action soumise à l'approbation requise.
- Le clonage ne justifie pas de remplacer Lakekeeper par Polaris, de reprendre Keycloak dans la cible finale, ni de transporter la topologie Compose en production. Les contrats Cleyrop et la matrice des services constituent les critères d'intégration.

Références de l'analyse

[study](#) · [docs/hemera-v2/08-plateforme-rust-sans-jvm.html:16](#) Comparaison consolidée avec la référence clonée.

Annexes de navigation et de preuve

Les fichiers complémentaires conservent la matrice machine, les déclarations d'interfaces et les preuves de lecture.

- [Matrice des fonctionnalités en CSV](#)
- [Déclarations statiques en CSV](#)
- [Sources et empreintes des fichiers consultés](#)
- [Couverture des services et composants](#)